

Unity®

PERFORMANCE TUNING TIPS 2026

Written By
CyberAgent Game & Entertainment Division



CyberAgent

Unityパフォーマンスチューニングティップス2026

株式会社サイバーエージェント | ゲーム・エンターテイメント事業部 著

第1章

Unity Managed Code Stripping □践指南

向井 祐一郎 Mukai Yuichiro

株式会社 Applibot

yucchiy　X @yucchiy_　blog.yucchiy.com

作□削□□用大小的手段，本文介□ Unity 的 Managed Code Stripping 功能，它会在□建□移除未使用的代□。□功能只需一□□置即可□用，但加大削□力度后，可能引□“在□□器中可以□行，只有□建出的□用中某些功能□坏”的□□。本章介□ Managed Code Stripping 的原理与用法、代□被□度削□的典型模式及其□□方法等。

□□□境如下。

- Unity 6000.3.19f1
- Addressables 4.0.1
- macOS Sequoia 15.7.3

先□□□：Managed Code Stripping 不是把□置改□ High 就能完事的□化。

只有明确声明静□分析看不到的依□，并持□□□□建出的□用的大小和行□，□个功能才能安全使用。

1.1 什么是 Managed Code Stripping

1.1.1 功能概述

Managed Code Stripping 是 Unity 的□准功能，在□建□□□未使用的托管代□（C# 代□），并将其从□建出的□用中移除。

□景和脚本中未被使用的□和方法会被移除，从而削□□用的代□大小。

削□□象不□包括用□代□，□包括 UnityEngine 模□、.NET □□，以及□目所含包和插件的程序集，因此□□以自行修改的□内部的未使用代□也可以削□。

尤其是在 IL2CPP □建中，托管代□被削□后，之后由 IL2CPP 生成的 C++ 代□也会□少，因此也有助于□小□用大小和□短□建□□。

□外，名称相似的□置 Strip Engine Code 是□□ Unity 引擎原生代□的削□□置，与□□托管代□的 Managed Code Stripping 是不同的功能。

Managed Stripping Level

代□削□的激□程度由 Managed Stripping Level □置控制。

可在 Edit > Project Settings > Player > Other Settings > Optimization 的 “Managed Stripping Level” 中□置，可□□以下□□。

表 1.1: Managed Stripping Level 的□□

Level	概述
Disabled	不□行 Stripping。□可在 Mono □建中□□
Minimal	□削□ UnityEngine 和 .NET □□，不□除用□代□。IL2CPP □建的默□□
Low	用□代□也□入分析□象，但保守地削□
Medium	以所有程序集□□象，在大小与安全性之□取得平衡地削□
High	最激□地削□所有程序集

Mono □建的默□□□ Disabled，因此不做任何□置就不会□行 Stripping。

□一方面，IL2CPP □建的默□□□ Minimal，且无法□□ Disabled，因此无□程度如何，必定会□行 Stripping。

□外，Unity 6.3 的文档已□告 Low 将来会被□弃，不建□使用。

参考：Managed code stripping `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping.html` / Configure managed code stripping `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-configure.html`

1.1.2 基本工作原理

Managed Code Stripping 由 Unity 自研工具 **UnityLinker** □□。

UnityLinker 在□用□建管□中于 C# 代□□□完成后□行，从已□□的程序集（DLL）中移除未使用的代□。

在 IL2CPP □建中，□□ Stripping 的程序集会成□之后 C++ 代□□□的□入。

UnityLinker 分三个□段判定未使用的代□。

1. 确定作□分析起点的代□（**root**）
2. 从 root 出□，□□追踪被静□引用的□型和成□，并□□□使用中（**mark**）
3. 将最□未被□□的代□□□不可□，从程序集中移除

被□□ root 的典型□象如下。

- □建所含的□景和□源所引用的□型
- □承自 MonoBehaviour 或 ScriptableObject 的□型
- □有 [RuntimeInitializeOnLoadMethod] 等特性的方法
- 通□ link.xml 或 [Preserve] 特性明确指定保留的代□（在后文的□□方法中□明）

□里的重点是：代□是否保留，不是由“□行□是否□□使用”决定，而是由“□建□的静□分析能否从 root 到□”决定。

静□分析是指不□行代□，□机械地追踪已□□代□中出□的引用的分析。

例如通□ new □建□型或直接□用方法，被引用的□型和成□信息会原□保留在已□□的代□中，因此 UnityLinker 可以追踪。

□一方面，以代□中不出□引用的方式被□用的代□，无□□行□是否使用，都会被□□不可□。

因此，只有静□分析看不到的引用的代□，即使□行□需要，也会被判定□未使用而被□除。

此外，mark 不是以□型□□位，而是以成□□□位□行的，因此即使□型被保留，未被使用的方法也会被□独□除。

□□上□些代□会被□除，将在下一□□合具体代□和□量□果来确□。

用 Mono.Cecil 确□□些内容被削□

本章在后□各□中会反复确□“□些□型和方法保留在□建中”。

确□使用的是分析 .NET 程序集的□ **Mono.Cecil**。

□取□建□出所含的程序集（macOS 的 Mono □建□“□用名.app/Contents/Resources/Data/Managed/Assembly-CSharp.dll”），并将□型和方法的列表写入文件。

▼清□ 1.1　□出□型和方法列表

```
// □取程序集，写出□型和方法（含 .）的列表ctor
using var assembly = AssemblyDefinition.ReadAssembly(dllPath);
var lines = assembly.MainModule.Types
    .Where(type => type.Name != "<Module>")
    .SelectMany(type => new[] { type.FullName }
        .Concat(type.Methods.Select(method => $"  {method.Name}")));
File.WriteAllLines(outputPath, lines);
```

方法列表中也包含□造函数（.ctor），因此也可用于确□后文所述的“□型保留了，只有□造函数消失”的□象。

Mono.Cecil 可以通□ Unity Package Manager 安装使用。在□□器用的 Assembly Definition 中□用 overrideReferences 并将 Mono.Cecil.dll 添加到 precompiled References，即可在□□器代□中引用。

□改□了 Level 的□个□建分□□取□列表并比□差异，就能确定□些□型消失了、□些方法被按成□□□位削□了。

□建之后，Stripping 后的程序集有□会作□中□□物保留在 Library 下。

在本次□境中，Library/Bee/artifacts/MacStandalonePlayerBuildProgram/ManagedStripped/ 下保留着与□建□出内容相同的 DLL，无需展开□建□物即可以它作□分析□象。

官方文档并未明确□明 Stripping 后的程序集会保存在此路径，因此找不到□□分析□建□出一□。

后文中写有“确□了□型和方法”的地方，基本都是通□□种方法确□的。比□ Stripping 前后的 DLL，□可以确□削□的明□。

参考：Unity linker marking rules `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-marking-rules.html`

1.1.3 □什么要削□代□

使用 Managed Code Stripping 的主要目的是削□□用大小。

其次，未使用的代□不会加□到内存中，因此也有望削□□行□的内存使用量。

□了确□效果有多大，□建一个只放置了一个退出□理脚本的□□□景，比□ Managed Stripping Level □ Minimal 和 High 的情况。

□量□果如下（表 1.2）。

表 1.2: □□□景中 Minimal 与 High 的比□

指□	Minimal	High	削□
托管程序集的□大小	7,702,016 bytes	7,188,480 bytes	□ 6.7%
程序集（DLL）数量	52	51	1 个
□用整体大小	89,069,307 bytes	88,548,299 bytes	□ 0.6%

即使是小型示例□目，托管代□也□少了 6.7%。

从明□来看，完全未被使用的 System.Runtime.Serialization.dll 被整体□除，mscorlib.dll □少了□ 272KB，包含大量未被□景引用的代□的 Assembly-CSharp.dll 也□少了 22.5%。

托管代□的削□量，取决于□目所引用的□的数量，以及其中□□未被使用的代□所占的比例。

大型□目往往会引入□多□，而□中□□未被使用的功能通常也很多，因此更容易从代□削□中受益。

在 IL2CPP □建中，被削□的托管代□□□□的部分也不再需要□□和□□□ C++，因此效果会波及原生二□制的大小和□建□□。

1.2 □用 Managed Code Stripping □的注意事□

如上一□所述，Managed Code Stripping 的判断依□于□建□的静□分析。

因此，只有静□分析看不到的引用的代□，即使□行□需要，也会被□除。

□个□□的棘手之□在于，症状只在□建出的□用中出□。

在□□器中□行，或在未□用 Stripping 的默□ Mono □建中，都能正常□行，因此开□□程中很□察□，往往在提高 Stripping Level 之后，才表□□只有某些功能□坏的故障。

本□介□代□被□度裁剪的□种典型情况。

一种是通□反射□用的代□，□一种是未被 Unity 入口点引用的代□。

1.2.1 通□反射□用的代□消失

用具体代□确□上一□所□的“静□分析能否从 root 到□”的判断。

准□三个被引用方式各不相同的□（清□ 1.2）。

▼清单 1.2　被引用方式各不相同的三个类

```
// 通过 new 直接创建
public class DirectlyUsedService
{
    public string Describe() => "DirectlyUsedService is alive";
}

// 通过 typeof 仅引用类型
public class TypeofOnlyService
{
    public string Describe() => "TypeofOnlyService is alive";
}

// 仅通过反射创建
public class ReflectionOnlyService
{
    public string Describe() => "ReflectionOnlyService is alive";
}
```

在场景中放置的 MonoBehaviour 中，按如下方式使用这三个类（清单 1.3）。

▼清单 1.3　使用三个类的 MonoBehaviour

```
public class StrippingProbe : MonoBehaviour
{
    private static readonly string ReflectionTypeName = "ReflectionOnlyService";
    private static readonly string TypeofTypeName = "TypeofOnlyService";
    private static readonly string TypeofMethodName = "Describe";

    private void Start()
    {
        // 静态引用：被引用的类型和方法的信息保留在已经编译的代码中
        var direct = new DirectlyUsedService();
        Debug.Log($"direct: {direct.Describe()}");

        // 静态引用只有类型：typeof 引用类型，但没有构成实际的静态引用
        Debug.Log($"typeof: type is referenced: {typeof(TypeofOnlyService).Name}");

        // 成员是否保留，通常以静态分析看不到的路径重新获取类型来确认
        var typeofType = Type.GetType(TypeofTypeName);
        var describeMethod = typeofType?.GetMethod(TypeofMethodName);
        if (describeMethod == null)
        {
            Debug.Log($"typeof: type {typeofType?.Name} exists but Describe is stripped
    ");
        }
        else
```

```
        {
            Debug.Log($"typeof: {describeMethod.Invoke(Activator.CreateInstance(
    typeofType), null)}");
        }

        // 静⌧分析看不到的引用：通⌧反射⌧建
        var type = Type.GetType(ReflectionTypeName);
        if (type == null)
        {
            Debug.Log($"reflection: Type.GetType(\"{ReflectionTypeName}\") returned
    null");
        }
        else
        {
            var instance = Activator.CreateInstance(type);
            Debug.Log($"reflection: {type.GetMethod("Describe")?.Invoke(instance, null)
    }");
        }
    }
}
```

将⌧示例的 Managed Stripping Level 分⌧⌧⌧ Minimal 和 High ⌧行⌧建并⌧行，⌧果如下。

```
# Minimal
direct: DirectlyUsedService is alive
typeof: type is referenced: TypeofOnlyService
typeof: TypeofOnlyService is alive
reflection: ReflectionOnlyService is alive

# High
direct: DirectlyUsedService is alive
typeof: type is referenced: TypeofOnlyService
typeof: type TypeofOnlyService exists but Describe is stripped
reflection: Type.GetType("ReflectionOnlyService") returned null
```

在 High 下，typeof 的情况能⌧取到⌧型但 GetMethod 返回 null，反射的情况⌧是 Type.GetType 本身返回 null。

⌧⌧起来，行⌧如下（表 1.3）。

表 1.3: Minimal 与 High 下的保留情况

⌧象	Minimal	High
StrippingProbe	保留	保留

表 1.3: Minimal 与 High 下的保留情况

对象	Minimal	High
DirectlyUsedService（通过 new 引用）	保留	类型和方法都保留
TypeofOnlyService（通过 typeof 引用类型）	保留	类型保留，但 Describe 被移除
ReflectionOnlyService（仅反射）	保留	仅同类型一起被移除

Minimal 不会移除用户代码，因此三者都原样保留。

在 High 下，引用方式的差异会直接体现在结果中。

DirectlyUsedService 通过 new 引用类型，并直接调用了 Describe，因此类型和方法都保留。

通过 typeof 引用的 TypeofOnlyService 作为类型会保留，但未被任何地方调用的 Describe 被按成员级别移除。

而完全没有静态引用的 ReflectionOnlyService 则仅同类型一起被移除。

此外，本节示例中所有类型名和方法名的字符串都来自由 static readonly 字段获取，这是有意为之的设计。

UnityLinker 理解部分典型的反射用法，已确认直接向 Type.GetType("ReflectionOnlyService") 传入常量字符串时，即使在 High 下类型也会被保留。

此外，通过 typeof 获得的 Type 直接调用 GetMethod 时，即使方法名不是常量，该类型的 public 方法也会被统一保留。

另一方面，UnityLinker 不会追踪来自由字段传入的字符串。

通过读取字段，重组了类型名和成员名来自配置文件、序列化数据、服务器响应等构建时无法确定的路径属于一类情况。

如果将字段改为 const，它会作为常量字符串嵌入到使用位置，因此与直接传入常量的

情况□理方式相同。

DI 容器中的□型解析被破坏

到此□止的示例□便于□明做了□化，但同□□□的□□会出□在□□的□中。典型的就是 DI 容器。

□多 DI 容器会□先注册接口与□□□型的□□关系，解析□通□反射□用□造函数来□建□例。

用模仿□一行□的最小容器来确□。与□□的 DI 容器一□，使用通□ Register 注册、通□ Resolve 解析的接口（清□ 1.4）。

▼清□ 1.4　最小的 DI 容器

```
public sealed class MiniContainer
{
    private readonly Dictionary<Type, Type> _registrations = new Dictionary<Type, Type
    >();

    // 以□型参数注册□□□型。TImplementation 相当于 typeof 的静□引用
    public void Register<TInterface, TImplementation>() where TImplementation :
    TInterface
    {
        _registrations[typeof(TInterface)] = typeof(TImplementation);
    }

    // 通□反射将已注册的□□□型□例化并返回
    public TInterface Resolve<TInterface>()
    {
        var implementationType = _registrations[typeof(TInterface)];
        return (TInterface)Activator.CreateInstance(implementationType);
    }
}
```

所注册的□□□型假定□□由容器使用，不在任何地方被静□引用（清□ 1.5）。

▼清□ 1.5　注册的接口与□□

```
public interface INotificationService
{
    string Notify();
}
```

```
// □□由容器使用、不在任何地方被静□引用的□□
public sealed class NotificationService : INotificationService
{
    public string Notify() => "NotificationService.Notify succeeded";
}
```

在□景中放置的 MonoBehaviour 中□行注册和解析（清□ 1.6）。

▼清□ 1.6　□行注册和解析的 MonoBehaviour

```
public sealed class DiContainerProbe : MonoBehaviour
{
    private void Start()
    {
        try
        {
            var container = new MiniContainer();
            container.Register<INotificationService, NotificationService>();
            Debug.Log($"[DiContainerProbe] explicit: {container.Resolve<
    INotificationService>().Notify()}");
        }
        catch (Exception exception)
        {
            Debug.Log($"[DiContainerProbe] explicit failed: {exception.GetType().Name}:
     {exception.Message}");
        }
    }
}
```

以与之前相同的条件□建，□果如下。

```
# Minimal
[DiContainerProbe] explicit: NotificationService.Notify succeeded

# High
[DiContainerProbe] explicit failed: MissingMethodException: Default constructor not
    found for type NotificationService
```

在 High 下，Register<INotificationService, NotificationService>() 的□型参数□成静□引用，因此 NotificationService □型本身会保留。

此外，□由接口被静□□用的 Notify 也会保留。

但是，只通□反射□用的默□□造函数没有静□引用，因此会被按成□□□位□除。

⌧果，Activator.CreateInstance 找不到⌧造函数，因解析失⌧而抛出 MissingMethodException。

⌧就是“⌧型已⌧注册，但只有在⌧建出的⌧用中解析失⌧”⌧一故障的原因。

⌧⌧的 DI 容器中，⌧⌧也是一⌧的。

只要⌧造函数的解析依⌧反射，即使是⌧式注册的⌧型，⌧式的默⌧⌧造函数也会成⌧⌧除的候⌧。

如果采用从程序集中⌧描⌧⌧⌧型并自⌧注册的方式，没有静⌧引用的⌧⌧⌧型会⌧同⌧型一起被⌧除，⌧致注册本身失⌧。

1.2.2 未被 Unity 入口点引用的代⌧消失

⌧一种模式是：既不被⌧景也不被脚本引用，只会⌧由 AssetBundle 等外部内容而被需要的代⌧。

如上一⌧所述，root 的来源是⌧建所含的⌧景和⌧源。

作⌧ AssetBundle ⌧独⌧建的内容不包含在⌧用本体的⌧建中，因此⌧内容所引用的⌧型不会被⌧⌧ root。

因此，只有内容一⌧需要的⌧型，如果⌧用代⌧中没有静⌧引用，无⌧⌧行⌧是否会加⌧，都会被判定⌧未使用。

用 Timeline ⌧源来重⌧⌧一模式。

Timeline ⌧源只包含在 AssetBundle 中，⌧用一⌧不静⌧引用 Unity.Timeline，而是按⌧源名加⌧，并通⌧ PlayableDirector 播放（清⌧ 1.7）。

▼清⌧ 1.7　从 AssetBundle 播放 Timeline ⌧源的 MonoBehaviour

```
public class TimelineAssetBundleProbe : MonoBehaviour
{
    private const string BundleDirectoryName = "TimelineBundles";
    private const string BundleName = "timeline.bundle";
```

```
    private const string TimelineAssetName = "TimelineOnly";

    private void Start()
    {
        var bundlePath = Path.Combine(Application.streamingAssetsPath,
    BundleDirectoryName, BundleName);
        var bundle = AssetBundle.LoadFromFile(bundlePath);
        if (bundle == null)
        {
            Debug.Log("timeline bundle: failed to load asset bundle");
            return;
        }

        var asset = bundle.LoadAsset(TimelineAssetName);
        if (asset == null)
        {
            Debug.Log($"timeline bundle: asset {TimelineAssetName} failed to load");
            bundle.Unload(true);
            return;
        }

        Debug.Log($"timeline bundle: loaded {asset.GetType().AssemblyQualifiedName}");
        var playableAsset = asset as PlayableAsset;
        if (playableAsset == null)
        {
            Debug.Log($"timeline bundle: asset {TimelineAssetName} is not playable");
            bundle.Unload(true);
            return;
        }

        var director = gameObject.AddComponent<PlayableDirector>();
        director.playableAsset = playableAsset;
        director.Play();
        Debug.Log($"timeline bundle: play state={director.state}, graph valid={director
    .playableGraph.IsValid()}");
        bundle.Unload(false);
    }
}
```

构建 AssetBundle 之后，从项目中移除原始的 Timeline 资源再构建使用，以此模拟与使用分开更新内容的方式。

将 Managed Stripping Level 分别设置为 Minimal 和 High 运行构建。两个版本的结果相同。

```
# Minimal / High 均如此
```

```
The referenced script (UnityEngine.Timeline.TimelineAsset) on this Behaviour is missing
    !
timeline bundle: asset TimelineOnly failed to load
```

此外，□□□建出的□用的 Managed 文件□后□□，□个□建中都不存在 Unity.Timeline.dll 本身。

需要注意的是，Minimal 与 High 的□果没有区□。

□□□一模式的□法不是□整 Stripping Level，而是明确指定保留所需的□型和程序集。

具体手段将在下一□介□，使用 Addressables □的机制将在□□篇介□。

参考: How code stripping affects content https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-content.html / □□□□源消失的案例（Qiita, 2021） https://qiita.com/KyoheiOkawa/items/0cf9cc09679eababec75

1.3 代□被□度削□□□的□□方法

1.3.1 基本□□方法

防止必要的代□被削□的最基本手段，是明确告知 UnityLinker“□段代□会被使用”。

Unity □此提供了 link.xml 文件和 [Preserve] 特性。

无□用□种方式指定的代□，都会被□□分析的 root，即使没有静□引用也不会被□除。

link.xml

link.xml 是列□要保留的程序集、□型和成□的 XML 文件。

放在□目的 Assets 下，UnityLinker 会在□建□□取。

上一□示例中□同□型一起被□除的 ReflectionOnlyService，可以通□如下 link.xml 保留（清□ 1.8）。

▼清单 1.8　保留 ReflectionOnlyService 的 link.xml

```
<linker>
  <assembly fullname="Assembly-CSharp">
    <type fullname="ReflectionOnlyService" preserve="all" />
  </assembly>
</linker>
```

将带 link.xml 放在 Assets 直下并以 High 重新构建后，运行结果如下。

```
# （无High link.）xml
reflection: Type.GetType("ReflectionOnlyService") returned null

# （有High link.）xml
reflection: ReflectionOnlyService is alive
```

成功构建所含的程序集，也能确保 ReflectionOnlyService 及同 Describe 都被保留了。

link.xml 可以在代码之外指定保留对象，因此也可用于无法自行修改的 DLL 或包中的类型。

保留范围可以按程序集、类型，以及方法、字段等成员级别位指定。

此外，用 preserve="all" 保留整个程序集的写法虽然方便，但会抵消对该程序集的 Stripping 效果。

它可用于问题定位，但作为长期策略，最好能缩小到所需的类型和成员。

[Preserve] 特性

对于可以自行修改的代码，可以直接在对象上添加 UnityEngine.Scripting 命名空间的 [Preserve] 特性。

需要注意的是，[Preserve] 的保留范围会因添加位置而异。

用添加在类型上的和只添加在方法上的两个例子来确认一差异（清单 1.9、清单 1.10）。

▼清单 1.9　在类型上添加 Preserve 的情况

```
using UnityEngine.Scripting;
```

```
// 在□型上添加 [Preserve]，方法上不添加
[Preserve]
public sealed class TypePreservedService
{
    public string Describe() => "TypePreservedService.Describe is alive";
}
```

▼清□ 1.10　只在方法上添加 Preserve 的情况

```
using UnityEngine.Scripting;

// □型上不添加，只在方法上添加 [Preserve]
public sealed class MethodPreservedService
{
    [Preserve]
    public string Describe() => "MethodPreservedService.Describe is alive";
}
```

以没有静□引用的形式，通□反射□建并□用□□个□，在 High 下的□果如下（表 1.4）。

表 1.4: Preserve 的添加位置与 High 下的保留范□

□象	□型	默□□造函数	Describe
TypePreservedService	保留	保留	被□除
MethodPreservedService	保留	被□除	保留

添加在□型上的 [Preserve] 会保留□型和默□□造函数，但不会保留未添加特性的 Describe。

添加在方法上的 [Preserve] 会保留□□型和□方法，但□次默□□造函数不会被保留，通□ Activator.CreateInstance □建□会因 MissingMethodException 而失□。

要使通□反射□建并□用的流程能□□行，需要在□型和方法□者上都添加 [Preserve]。

遇到“添加了 [Preserve] 却不起作用”的情况，首先□□疑特性的添加位置与保留范□是否□□。

参考：Preserving code using annotations https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-preserving.html

1.3.2 进阶篇

Preserve 以外特性的应用

[Preserve] 是“保留这个元素”这个的逐点指定，而 UnityEngine.Scripting 命名空间中还有以模式声明保留的特性，例如“保留该接口的所有实现”。

在框架中常见的“通过反射查找并实用用户实现的类型”场景中，不必逐个用 [Preserve] 指定实现类型，而是可以通过在接口或基类上添加特性来实现。

- RequireImplementorsAttribute https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequireImplementorsAttribute.html：添加到接口上后，保留该接口的实现类型
- RequireDerivedAttribute https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequireDerivedAttribute.html：添加到类上后，保留该承载派生类型的基类型
- RequiredMemberAttribute https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequiredMemberAttribute.html：添加到成员上后，在其类型被保留的情况下也保留该成员
- RequiredInterfaceAttribute https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequiredInterfaceAttribute.html：添加到类型上后，保留所指定接口的实现信息

以 [RequireImplementors] 这代表来确保其行为。

在接口上添加特性，实现不在任何地方被静态引用（清单 1.11）。

▼清单 1.11　添加了 RequireImplementors 的接口与实现

```
// 添加到接口上后，实现该类型会成为保留对象
[RequireImplementors]
```

```
public interface IRequiredContract
{
}

// 不在任何地方被静□引用的□□
public sealed class RequiredContractImpl : IRequiredContract
{
}
```

以 High □建后，RequiredContractImpl □同默□□造函数一起被保留。

没有静□引用的□□□型，□凭接口一□的声明就得以保留。

□于添加了 [RequireDerived] 的基□的派生□型，同□确□了保留。

用 IUnityLinkerProcessor 在□建□生成 link.xml

如果要保留的代□是确定的，只需在 link.xml 中指定□□的□或成□即可。

但在各□建目□需要保留的□型不同、所需□型随□建所含内容而□化等情况下，持□手□更新 link.xml 并不□□。

使用 **IUnityLinkerProcessor** 可以在□建□□□生成 link.xml 并□□ UnityLinker。

IUnityLinkerProcessor 是□□器用的接口，将□□□放在□□器程序集中，□建□会被自□□用。

GenerateAdditionalLinkXmlFile 返回的路径上的 link.xml 会被添加到 UnityLinker 的□入中。

最小□□如下（清□ 1.12）。

▼清□ 1.12　生成 link.xml 的最小 IUnityLinkerProcessor □□

```
using System.IO;
using UnityEditor.Build;
using UnityEditor.Build.Reporting;
using UnityEditor.UnityLinker;
using UnityEngine;

public sealed class PreserveListLinkerProcessor : IUnityLinkerProcessor
```

```
{
    private const string LinkXmlContent = @"<linker>
  <assembly fullname=""Assembly-CSharp"">
    <type fullname=""GeneratedPreserveService"" preserve=""all"" />
  </assembly>
</linker>
";

    public int callbackOrder => 0;

    // 返回的路径上的 link.xml 会被添加到 UnityLinker 的输入中
    public string GenerateAdditionalLinkXmlFile(BuildReport report,
    UnityLinkerBuildPipelineData data)
    {
        var path = Path.Combine(Path.GetTempPath(), "managed-stripping-sample-generated
    -link.xml");
        File.WriteAllText(path, LinkXmlContent);
        Debug.Log("[PreserveListLinkerProcessor] generated 1 assembly / 1 type.");
        return path;
    }
}
```

本例使用硬编码固定了生成内容，实际上将会一部分替换成根据构建输入的输入（如读取裁剪前源或分析内容依赖）来实现的处理。

准备没有静态引用的 GeneratedPreserveService 并以 High 级别构建后，即使 Assets 下没有放置 link.xml，也确认类型、构造函数和方法都被保留。

```
[GeneratedLinkXmlProbe] type=present
[GeneratedLinkXmlProbe] ctor=present, method=GeneratedPreserveService.Describe is alive
```

参考：IUnityLinkerProcessor https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Build.IUnityLinkerProcessor.html

Addressables 生成的 link.xml

对于上一节所述的“只有内容一直需要的类型消失”的问题，Addressables 通过在内容构建的产物中自动生成 link.xml 来解决。

确认它生成了什么。内容采用最小构成：构建一个只显示 Cube 网格的 prefab（在 GameObject 上添加 MeshFilter 和 MeshRenderer），并注册到 Addressables 的组中。

□行内容□建后，会生成 link.xml。

在 Addressables 4.0.1 中，生成位置□ Library/com.unity.addressables/aa/<platform>/AddressablesLink/link.xml。

本次生成的内容如下（清□ 1.13）。

▼清□ 1.13 Addressables 生成的 link.xml

```
<linker>
  <assembly fullname="Unity.Addressables, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null" preserve="all">
    <type fullname="UnityEngine.AddressableAssets.Addressables" preserve="all" />
  </assembly>
  <assembly fullname="Unity.ResourceManager, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null" preserve="all">
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    AssetBundleProvider" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    BundledAssetProvider" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    IAssetBundleResource" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.InstanceProvider"
    preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.SceneProvider"
    preserve="all" />
  </assembly>
  <assembly fullname="UnityEngine.CoreModule, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null">
    <type fullname="UnityEngine.GameObject" preserve="all" />
    <type fullname="UnityEngine.MeshFilter" preserve="all" />
    <type fullname="UnityEngine.MeshRenderer" preserve="all" />
    <type fullname="UnityEngine.Transform" preserve="all" />
  </assembly>
</linker>
```

内容分□□种。

一种是 Addressables □行□（Unity.Addressables 和 Unity.ResourceManager）的 provider □型，根据□的□成而需要。

□一种是所注册内容引用的□型。本次□成 prefab 的 GameObject、Transform、MeshFilter、MeshRenderer 原□列□保留□象。

可以 Addressables 会分析内容一致的依赖，即使应用代码没有引用，也会将这些类型加入保留对象。

参考：Addressables Build Artifacts `https://docs.unity3d.com/ja/Packages/com.unity.addressables@1.20/manual/BuildArtifacts.html`（link.xml 的说明在 1.20 的手册中）

用 Source Generator 的静态代码用替换反射的代码使用

到此为止的裁剪方法，都是通过指定保留来防止必要的代码被削除。

与之相反，还有一种方法是用 **Source Generator** 的代码生成，来替换运行生静态分析看不到的依赖的反射本身。

Source Generator 是 C# 编译器在编译时分析源代码并生成额外源代码的机制。

如果在编译时就完成类型的查找和创建，依赖从一开始就表现为静态引用，也就不再需要保留指定。

例如，考虑一个接收命令名字符串并执行的处理器的分派处理。

处理器通过特性声明命令名。若在执行时用反射来创建，就需要从程序集中找出有该特性的类型并创建（清单 1.14）。

▼清单 1.14　通过特性声明命令名的处理器

```
[AttributeUsage(AttributeTargets.Class)]
public sealed class CommandHandlerAttribute : Attribute
{
    public CommandHandlerAttribute(string command)
    {
        Command = command;
    }

    public string Command { get; }
}

public interface ICommandHandler
{
    string Execute();
}
```

```
// □理器不在任何地方被静□引用
[CommandHandler("greet")]
public sealed class GreetCommandHandler : ICommandHandler
{
    public string Execute() => "GreetCommandHandler.Execute succeeded";
}

[CommandHandler("farewell")]
public sealed class FarewellCommandHandler : ICommandHandler
{
    public string Execute() => "FarewellCommandHandler.Execute succeeded";
}
```

□□个□理器不在任何地方被静□引用，因此如果保持□行□□描的□□并以 High □建，会如上一□所述□同□型一起被□除。

使用 Source Generator 来□□□□□命令名的□理器解析。

准□一个生成器，在□□□收集□有 [CommandHandler] 的□，并生成根据命令字符串□找具体□型的分派器。

生成器分□个□段工作：前半（ForAttributeWithMetadataName）从□特性的□中收集命令名和□型名，后半（GenerateDispatcher）将收集到的配□填入模板。

生成器的完整代□如下（清□ 1.15）。

▼清□ 1.15　生成分派器的 Source Generator

```
#nullable enable
using System.Collections.Immutable;
using System.Linq;
using System.Text;
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.Text;
namespace StrippingSample.SourceGenerators
{
    [Generator]
    public sealed class CommandDispatcherGenerator : IIncrementalGenerator
    {
        public void Initialize(IncrementalGeneratorInitializationContext context)
        {
```

```
        // 从所有 [CommandHandler] 的类中收集命令名与类型名的配对
        var handlers = context.SyntaxProvider
            .ForAttributeWithMetadataName(
                "CommandHandlerAttribute",
                static (_, _) => true,
                static (syntaxContext, _) => (
                    Command: (string)syntaxContext.Attributes[0].
ConstructorArguments[0].Value!,
                    TypeName: ((INamedTypeSymbol)syntaxContext.TargetSymbol)
                        .ToDisplayString(SymbolDisplayFormat.FullyQualifiedFormat))
)
            .Collect();
        // 根据收集到的配对生成分派器
        context.RegisterSourceOutput(handlers, static (sourceContext,
commandHandlers) =>
        {
            if (!commandHandlers.IsDefaultOrEmpty)
            {
                sourceContext.AddSource(
                    "CommandDispatcher.g.cs",
                    SourceText.From(GenerateDispatcher(commandHandlers), Encoding.
UTF8));
            }
        });
    }
    private static string GenerateDispatcher(
        ImmutableArray<(string Command, string TypeName)> handlers)
    {
        // 使输出具有确定性，先按命令名排序再填入
        var cases = string.Join("\n", handlers
            .Distinct()
            .OrderBy(static handler => handler.Command, System.StringComparer.
Ordinal)
            .Select(static handler => $$"""
                        case {{SymbolDisplay.FormatLiteral(handler.Command, true)
}}:
                            return new {{handler.TypeName}}().Execute();
            """));
        return $$"""
            // <auto-generated/>
            public static class CommandDispatcher
            {
                public static string Execute(string command)
                {
                    switch (command)
                    {
            {{cases}}
                        default:
```

```
                                throw new global::System.ArgumentException($"Unknown
    command: {command}");
                            }
                        }
                    }
                """;
        }
    }
}
```

按命令名排序，是□了□相同□入生成相同代□，避免每次□建□出□生波□。

通□□生成器，□□□会生成如下代□（清□ 1.16）。

▼清□ 1.16　生成的 CommandDispatcher

```
// <auto-generated/>
public static class CommandDispatcher
{
    public static string Execute(string command)
    {
        switch (command)
        {
            case "farewell":
                return new global::FarewellCommandHandler().Execute();
            case "greet":
                return new global::GreetCommandHandler().Execute();
            default:
                throw new global::System.ArgumentException($"Unknown command: {command}
    ");
        }
    }
}
```

原本在□行□根据字符串□找□型的□理，□成了字符串与具体□型的□□表。

□造（new）和 Execute 的□用都出□在生成的代□中，因此□□型、□造函数和方法的静□引用在生成的代□中即可□合。

使用方只需□入命令字符串，完全不引用□理器的具体□型。

与反射□□相同，命令字符串□由 static readonly 字段□□，保持□ UnityLinker 无法追踪的□入（清□ 1.17）。

▼清单 1.17　生成的分派器的使用方

```
// 命令字符串由字段定义，使其成为 UnityLinker 无法追踪的入口
private static readonly string[] Commands = { "farewell", "greet" };

private static void RunGeneratedDispatcher()
{
    foreach (var command in Commands)
    {
        var result = CommandDispatcher.Execute(command);
        Debug.Log($"[SourceGeneratorProbe] dispatch command={command}, result={result}"
    );
    }
}
```

以 High 构建并运行后，确认两个命令都运行成功。

```
[SourceGeneratorProbe] dispatch command=farewell, result=FarewellCommandHandler.Execute
     succeeded
[SourceGeneratorProbe] dispatch command=greet, result=GreetCommandHandler.Execute
    succeeded
```

字符串本身仍是构建后无法追踪的入口，但从字符串到类型的两次不再是运行时查找，而是生成代码中的 switch，因此在 UnityLinker 看来与手写代码一样是普通的静态引用。

通过 Mono.Cecil 确认，两个处理器的类型、构造函数、Execute，以及 CommandDispatcher.Execute 都被保留。

这种替换的好处不只是对 Stripping 的耐受性。

- 依赖于 UnityLinker 可达，因此无需维护 link.xml 或特性的保留指定
- 不再有运行时的反射查找，两次开销和运行时失败的可能性都会减少
- 对象的收集在编译期完成，因此可以在编译期检测漏注册和重复注册

在两者的对比中，也有采用这种两段的例子。

在序列化器方面，MessagePack for C# 早期为 IL2CPP 环境提供代码生成，目前已经基于 Source Generator 的生成。

在 DI 容器方面，VContainer 提供了通过 SourceGenerator 生成注入代码以减少运行时反射的两段，并用 Source Generator 解决了上一节所述的“构造函数消失导致解析失

☒”的☒象。

关于在 Unity ☒目中引入 Source Generator 的步☒（将 netstandard2.0 的 DLL ☒ RoslynAnalyzer ☒☒放置的☒成），☒参☒ Unity 手册。

参考：Create and use a source generator `https://docs.unity3d.com/6000.3/Documentation/Manual/create-source-generator.html` / MessagePack for C# `https://github.com/MessagePack-CSharp/MessagePack-CSharp` / MemoryPack `https://github.com/Cysharp/MemoryPack` / VContainer `https://github.com/hadashiA/VContainer`

1.4 ☒☒

Managed Code Stripping 不是提高 Stripping Level 就能完事的☒化。

如本章所述，☒由反射或外部内容的依☒☒静☒分析不可☒，即使是☒行☒需要的代☒，也会从☒建中消失。

而且症状只在☒建出的☒用中出☒，因此需要将明确声明不可☒的依☒，与持☒在☒建出的☒用上☒☒，作☒一套☒☒一并☒施。

明确指定保留的手段，☒根据☒象和情况区分使用（表 1.5）。

表 1.5: ☒☒方法的使用区分

方法	适用情况
[Preserve]	保留可自行☒☒的代☒中的少数元素
link.xml	保留确定的☒象，包括无法☒☒的 DLL
Require 系特性	由框架一☒声明“保留☒☒或派生☒型”的契☒
IUnityLinkerProcessor	保留☒象随☒建目☒或内容而☒化
Source Generator	将反射☒找本身替☒☒静☒引用

此外，列出☒ Stripping 更易生效的方☒。

- 除出☒☒☒的情况外，不要☒度使用 [Preserve] 和 link.xml
- ☒少通☒反射☒找和☒建☒型，考☒改☒通☒☒式注册或 Source Generator ☒☒的静☒引用

- 代□保留的指定□尽可能□小到□型、成□的粒度。□整个程序集使用 preserve="all" □限于□□定位
- □于只被与□用分开□建的内容使用的□型，其保留不依□手工操作，而是利用 Addressables 的 link.xml 或 IUnityLinkerProcessor 生成等机制
- 从开□□程中就□行并□□提高了 Stripping Level 的□建

第2章

将□□宝典做成 AI □程智能体的技能

中村光寿 Teruhisa Nakamura

株式会社 Applibot

2.1 前言

像本□□□的□□知□有一个□□性的弱点：只□□□的人有效。各章□集了以□□□依据的□解，但□些□解能否在□目的代□□□中被□□指出，取决于□□者是否□□相□章□，以及能否在□□的瞬□想起来。我□不能指望□□全□□住所有章□，而且在□□□段人□更替后，□些□解也会逐□淡化。□一方面，在□写代□的□□，AI □程智能体（Claude Code、OpenAI Codex、GitHub Copilot 等）已□常□。将其接入□□□求或 CI，就能□所有差异□用同一套□准。所缺少的只是关于 Unity 性能的系□化□估□准。因此，本章介□如何将□□宝典中的知□□□□“AI 智能体可以□行的□估□”，并打包成不依□特定工具的通用技能。□者将前作已公开的、基于其知□的 77 条□□，与从本□（2026 年版）原稿中提取的 86 条□□整合，共 163 条□估□□，并制作成技能。本章以□一□践□基□，□□□□的□□方□和□□上的注意事□。

2.2 Agent Skills □一通用格式

2.2.1 SKILL.md 的□范

作□□ AI 智能体具□□□知□的方法，2025 年底□布了名□ Agent Skills [*1] 的开放□范。技能的□体是以 SKILL.md □一 Markdown 文件□核心的目□，□□如下。

[*1] Agent Skills □范 https://agentskills.io/specification

▼清单 2.1　技能目录的组织

```
unity-perf-review/
 SKILL.md          # 元数据 + 核心步骤（必需）
 references/       # 详细参考文档
 scripts/          # 可执行脚本（可选）
 assets/           # 模板等（可选）
```

SKILL.md 的开头以前置元数据（frontmatter）的形式书写元数据。必填字段只有 name 和 description 两个。

▼清单 2.2　SKILL.md 的前置元数据（description 为适应版面已缩短，metadata 等可选字段已省略）

```
---
name: unity-perf-review
description: >
  Unity 从性能角度审查目的代码、脚本和配置。
  将 C# 脚本、着色器、.meta、ProjectSettings 与
  163 条规则进行比对，输出 Critical / Warning / Info 三级的结果。
  Unity 在被要求进行性能审查或优化场景使用。
license: MIT
---
```

关键在于 description 的写法。智能体会将用户的请求内容与 description 进行比对，从而自动触发技能，因此不宜写明“做什么”，宜写明“何时使用”，可以提高触发的准确度。

2.2.2　主流智能体的支持情况

规范的价值在于多个智能体可以采取同一种格式。截至 2026 年 8 月的支持情况见表 2.1。

表 2.1: 主流编程智能体对技能的支持（截至 2026 年 8 月）

智能体	支持情况	存放位置
Claude Code	原生支持 SKILL.md [*2]	claude/skills/ 或 ~/.claude/skills/
OpenAI Codex	支持 SKILL.md	agents/skills/ 或 ~/.agents/skills/
GitHub Copilot	支持 SKILL.md [*3]	github/skills/
Gemini CLI	支持 SKILL.md [*4]	gemini/skills/ 或 ~/.gemini/skills/

Codex / Copilot / Gemini CLI 也支持通用的 .agents/skills/ 存放位置。此外，Copilot

也会□取□目的 .claude/skills/，因此只需放置在 .claude/skills/，就能同□覆盖 Claude Code 和 Copilot（Copilot 的个人存放位置□ ~/.copilot/skills/ 或 ~/.agents/skills/）。支持情况和存放位置取决于□□点，因此引入□□以各智能体的官方文档□准。只要将□□知□整理成 SKILL.md 格式，就能向使用任何智能体的成□分□同一套□□□准（□□与步□）。Agent Skills □一的是包格式和步□，由于模型和工具□限的差异，□行□果并不会完全一致。即便如此，“□□□准放在□□中，□行交□智能体”□一分工仍然成立。

2.3 将□解□□□“□估□”

技能化的核心不是□写 SKILL.md，而是将各章的□解拆解到智能体可以机械□用的粒度。作者□一采用了如下格式。

▼清□ 2.3 □□格式（从 ParticleSystem 一章提取的示例。grep_pattern 的排除条件和□理方法的□□□话□版面已□短）

```
### US11 向 ParticleSystem.Simulate 指定□大的秒数
- □□：（□用Warning Noise 模□□□ ）Critical
- □象：代□□□ / □源□置
- □□模式：向 Simulate 的第 1 个参数□入□大的秒数。
  尤其是与□用了 NoiseModule 的 ParticleSystem 并用□
- grep_pattern: \.Simulate\(
- 原因：Simulate 会一次性模□到指定的□□。
  □□中指定 20 秒会使 NoiseJob 被□用□ 2000 次，
  □生□ 50ms 的尖峰
- □理：□□用 Noise 的粒子避免使用 Simulate
```

□格式包含 4 □□□决策。

2.3.1 明示□□能否机械化

每条□□都以 grep_pattern 的形式□有可机械□行的□索□范，□□ C# 代□和 Unity 的 YAML（.meta、Prefab、ProjectSettings）。多数是一条正□表□式，但也包含需要多行比□或□□“□有的描述缺失”的情况。智能体先用□□范□行初步□□，只精□命中的位置并做出判断。□□即使是大□模□□，也能以□□的成本完成□□。同□重要的是，不要勉□去写无法用正□表□式表□的□目。例如“未注册到 Addressables 的共享□源被多个 Bundle 引用”就无法通□ grep □取。□□□□□要明确□注（手□确□），并在□理□写明 Addressables Analyze 等确□手段。如果□装成可□□，智能体要么会大量□生□□，要么会悄悄跳□无法□□的□□。

2.3.2 在原因□保留□□□

“原因：会□致性能下降”□□的□□缺乏□服力。我□将原稿中的□□□原□引用到原因□。

- 未□□□理：2048x2048 □ Compressed □ 2.7MB / Uncompressed □ 16.0MB（□ 6 倍差距）
- 每个角色一个□的手□ Playable：600 个角色□ 8.396ms，□比整体一个□□ 4.500ms（1.87 倍差距）
- Simulate(20f) 与 Noise 并用：NoiseJob □ 2000 次□用、□ 50ms 的尖峰

有了数□，智能体□出的□□就能直接成□“□什么□□修复”的□明，也成□接受□□的开□者回到原稿相□章□的路径。□外，□些都是原稿中□□的特定□境下的□□□。我□的做法是在□□的原因□引用代表□，□量条件的□□□参□出□章□。

2.3.3 通□□□与覆盖机制□受住□□考□

□□分□ Critical / Warning / Info 3 个□□（Critical=确定会□致性能劣化的原因，确□适用条件成立后必□修复；Warning=□条件而可能成□□□的潜在□□；Info=可□的改□建□），并提供可由□目一□覆盖的机制（rules-overrides）。例如，即使“使用 LINQ □ Critical”，在初始化□（init 上下文）也可放□□ Warning，在 Editor 工具（tooling）中□□ Skip。□□□□本□上依□上下文。如果没有覆盖机制，就会出□“有很多指出的□□不适合我□的□目”，□致整个技能被弃用。□准通用、□用因□目而异，□种□□□□是□久□持的□□。

2.3.4 将无法静□□□的□解分离□□断知□

原稿的□解中，有□多是□代□也无法得知的。例如“削□了 CPU □理但□□□没有□化，□□□移到了 Semaphore.WaitForSignal 等等待□□上□，□□疑渲染□程一□□于关□路径，并在 Timeline □□中确□等待□象”□□关于 Profiler □法的□解。□凭等待□□无法断定瓶□，□一局限也作□□解的一部分保留下来。□些内容被分离到与□估□□不同的文件中，整理□“症状 → 原因 → □理”的表格（表 2.2）。在作者的□□中，只有用于性能分析的智能体（面向 Claude Code）在□断□才会□取它。本章分□的技能中将其作□参考□料一并附□，在静□□□无法判断的疑点以“建□□行性能分析□量”的形式□告□，可作□依据。

表 2.2: □断知□示例

症状	原因	□理
CPU □表呈波谷与尖峰交替的梳状	抽稀□理忘□□开相位	用 index % interval 分散相位
MeshSkinning.CalcMatrices □大	骨骼 Transform □□被□体化	□用 Optimize Game Objects
□ iOS 上没有□□和 Shader.CreateGPUProgram，只出□ GpuProgramMetal.GetCachedPipeline	□疑着色器在 AssetBundle 中重复打包	确□重复□例并拆分□□用 Bundle
□ iOS 上□□完成后，SubPass 内的着色器仍□生 PSO □□	SubPass（Native Render Pass）内着色器的□□失□	重新□□ SubPass，或在加□期□□行□渲染

2.4 □装□技能

2.4.1 □□式加□与□□清□方式

如果把 163 条□□全部写□ SKILL.md 正文，会□占智能体的上下文。Agent Skills 的□□思想是□□式加□（progressive disclosure）。

1. 智能体始□只看到前置元数据中的 name 和 description
2. 技能被触□后，□取 SKILL.md 正文（□行流程）
3. 只从 references/ 中□取需要用到的参考文档

初始□取什么（是否包含路径等）取决于智能体的□□，但“正文和参考□料在触□之前不会被□取”□一原□是共通的。□里需要注意的是完整性。朴素的做法是“按□象文件□型整个□入分□□□文件”，但采用□种方式□，智能体在□代□之前就要□取最多超□ 2 万 token 的□□（根据字符数估算，token 数取决于模型）。□模型靠注意力“□住并□用”几十条□□，会□致部分□□被悄悄跳□。因此，我□从全部□□中自□生成了□□清□，将“ID / □□ / 适用条件 / grep_pattern / □□名 / □情文件名”□□□每条□□一行，并把□□分□□个□段。

1. **初步□描**：随附脚本 scan.py □□□清□的所有行，□□□象文件机械□行 grep_pattern，□出命中列表（文件:行）和 Coverage（没有 python3 的□境中，□由智能体直接□行所有行的 grep 作□回退）
2. **二次精□**：只□□命中的□□，从□情文件中□取□□条件、原因和□理方法，排

除□□

无法用 grep 表□的部分□□，在□□清□上明确□□ MANUAL，□当符合□象□型□才精□。□于以“□有的描述缺失”作□指出条件的□□，□□注 ABSENT，在 grep 没有命中□确□□情（□是□了避免命中与否同指出条件的□□关系□生□倒）。将初步□描做成脚本，是因□□ LLM 逐条□行几十个 grep 可能会出□□漏，而且正□表□式的方言差异（PCRE 的先行断言在 macOS 自□的 grep 中无法□行等）会使□果依□于□境。□□，初步□描的完整性就与模型的□□力脱□了。指出内容是否正确（二次精□中排除□□）仍由智能体判断。事先□取的只有□□清□（代□用□ 16KB、□源用□ 8KB。token 数取决于模型，合□估算□ 1 万 token 左右），□情只□取命中的部分。□□清□由□□本体自□生成，因此即使新增□□，也不需要手□同步。

2.4.2 按□境排除□□

□□中有些依□于 Unity 版本或配置。本技能收□的 RenderGraph 相关□□（面向 URP）以 Unity 6 及以后□□象，LTO □□□□限 IL2CPP，iOS 特有的 PSO □□与□有 Android 的□品无关。□不适用的□□□行比□，只会□来噪声和□外工作。因此，我□□依□□境的□□□置了适用条件□□（unity>=6000.3、urp、il2cpp、ios 等），并与根据□目配置文件生成的□境概要□行比□，只排除“明确不□足条件的□□”。□□比□由初步□描脚本确定性地□理，未能□□到的□目（unknown）按安全一□□理（仍作□比□□象），被跳□的□□ ID 会在□告中列出，以便□□。生成步□将在引入步□一□中□明。

2.4.3 □制在□出中□上□□ ID

在□出格式中，要求每个□□都必□附上□□ ID。

▼清□ 2.4　□□□出示例

```
### Critical
- Assets/Scripts/EffectPlayer.cs:42 -
  □□用 Noise 的粒子□用 Simulate(20f) [US11]
  - 依据：NoiseJob 被□用□ 2000 次，□生□ 50ms 的尖峰（□□）
  - 建□：移除 ，或将秒数降到最小Noise
```

有了 ID，就能追溯到□□文件来□□□□的依据，从而与“智能体只是□得像那么回事”区分开来。即使是□□，也能明确□□修正□条□□的□□模式。

2.5 分□与向各智能体放置

技能的□布只需整个目□共享即可，GitHub □□或 zip 均可。用□将其复制到表 2.1 中的存放位置，各个智能体就会□□□技能。□□上的要点是保持□□的□一来源。在作者的□□中，□□本体只放在 rules/ 目□下，分□用的包由脚本□装（用□一□的引入步□□下一□）。

▼清□ 2.5 分□包的□装（面向□□者）

```
./package-skill.sh
cp -r dist/unity-perf-review /path/to/repo/.claude/skills/   # Claude Code
cp -r dist/unity-perf-review /path/to/repo/.agents/skills/   # Codex / Copilot / Gemini
    CLI
cp -r dist/unity-perf-review /path/to/repo/.github/skills/   # Copilot □用存放位置（上面
    □行任一均可覆盖，因此□可□）
```

如果开始按智能体分□管理□□，每次更新□准都会出□偏差。建□一开始就确定“来源只有一□，放置交□脚本”。

2.6 引入到自己的□目

以下是□上述机制在□者的□目中□行的步□。如果已安装 python3，复制和初始生成大□需要 10 分□。添加到□□的只有技能目□和□境概要（tuning-env.md），不会修改 Unity 的 Assets / ProjectSettings。

2.6.1 □取并放置技能

整套技能（SKILL.md、包含 163 条□□的 references/、□境□□ / 初步□描 / □□清□重新生成□ 3 个脚本、引入步□的 README）□布在本□□□的 skills/unity-perf-review/ 中[*5]。□取后，只需复制到所用智能体的技能目□即可。□外，技能是包含“□智能体的指示和可□行脚本”的分□物。□于来自不可信分□源的技能，建□先确□ SKILL.md 和 scripts/ 的内容再引入，并固定版本。

▼清□ 2.6 □取并放置技能

[*5] https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026/tree/main/skills/unity-perf-review

```
git clone https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026
    .git
cd UnityPerformanceTuningTips2026/skills

# 根据所用的智能体，复制到其中之一
cp -r unity-perf-review /path/to/your-repo/.claude/skills/   # Claude Code
cp -r unity-perf-review /path/to/your-repo/.agents/skills/   # Codex / Copilot / Gemini
    （通用存放位置）
    CLI
cp -r unity-perf-review /path/to/your-repo/.github/skills/   # Copilot □用存放位置（上面
    □行任一均可覆盖，因此□可□）
```

放置在□□根目□下，□□全□就能□得同一套□□□准。□个人□用的□，放在主目□一□（~/.claude/skills/ 等）也可以□行。

2.6.2 放置后的状□

□于 Claude Code，完成放置和下一□的□境概要生成后，□□将□□如下状□（清□ 2.7）。

▼清□ 2.7 放置后的□□□□

```
your-repo/
|-- .claude/
|   \`-- skills/
|       \`-- unity-perf-review/
|           |-- SKILL.md                      # name/description + □行流程
|           |-- README.md                     # 面向人□的引入步□
|           |-- references/
|           |   |-- checklist-code.md         # 初步□描用（代□□）
|           |   |-- checklist-asset.md        # 初步□描用（□源 / □置□）
|           |   |-- checklist-profile.md      # 用于性能分析（静□□□中不使用的附属□料）
|           |   |-- rules-csharp.md 等共□ 13 个文件  # □□□□（共 163 条）
|           |   |-- diagnosis-knowledge.md    # 症状原因□理的□断集->->
|           |   \`-- rules-overrides.example.md
|           \`-- scripts/
|               |-- detect-env.py             # 生成□境概要
|               |-- scan.py                   # 机械□行初步□描
|               \`-- gen-checklists.py         # 修改□□□重新生成□□清□
|-- tuning-env.md                             # □境概要（下一□中生成）
\`--  （Unity □目本体不做任何更改）
```

3 份□□清□按□象（代□ / □源 / 性能分析）划分，一条□□可能出□在多份清□中，因此行数合□与□□数 163 不一致。放置完成□，原□上□入智能体的只有前置元数据中的 name 和 description（合□数百 token。如前所述，根据□□□可能加上路径等）。

□出□□□求后，按照前述的□□式加□，依次按需□取 SKILL.md 正文 → □□□象的□□清□ → 命中□□的□情。通常无需一次性投入 163 条□□的全文（□ 125KB，估算 4 万 token 多）。rules-overrides.example.md 保持原□是无效的。开始□行□目□属□整□，将其重命名□ rules-overrides.md 后再□□（□后文）。

2.6.3 生成□境概要

如前所述，163 条□□中包含依□□境的□□。□了□智能体判断排除不适用于□目的□□，需要生成□境概要。

▼清□ 2.8 生成□境配置文件（在克隆下来的□□的 skills/ 中□行）

```
python3 unity-perf-review/scripts/detect-env.py \
    /path/to/your-repo /path/to/your-repo/tuning-env.md
```

在放置之后□行□，□将路径替□□放置位置（.claude/skills/ 或 .agents/skills/ 等所用智能体的技能目□）。即使 Unity □目不在□□根目□而位于子目□的 monorepo □□，只要□入□□根目□，也会□□搜索配置文件并□行□□。□行脚本需要 python3（没有的□境中，可以手□□建 tuning-env.md，或跳□此步□以全□□比□方式□□。初步□描的 scan.py 也使用 python3，但没有□会以智能体直接□行 grep 的回退方式□行）。脚本根据 ProjectVersion.txt、Packages/manifest.json、GraphicsSettings.asset 和 ProjectSettings.asset 推断 Unity 版本、渲染管□、Scripting Backend 以及是否有 Addressables。□□究只是根据配置文件得出的推断□，确定信息以手□填写到 tuning-env.md □准。未能□□到的□目（各平台的配置不同、Scripting Backend 混用□等）会成□ unknown，按安全一□（仍作□比□□象）□理。□打开□出的 tuning-env.md 确□内容，尤其要手□修正□行平台和是否使用 SideFX VAT，并确□ render_pipeline 也是□期的□。

2.6.4 □目□属□整（可□）

如果存在有意偏离□□的位置，□在 rules-overrides.md 中□□□□的覆盖。例如允□初始化□使用 LINQ、在 Editor 工具中禁用日志□□等。分工是：□境概要□□排除“在□□境中原理上无法适用”的□□，overrides □□“因□目情况而做的□整”。如果想要新增或修改□□本身，□□□ references/ 中的 rules-*.md，并用 python3 scripts/gen-checklists.py references/ 重新生成□□清□。□□清□是自□生成物，因此不要直接□□。□凭分□物就能完成□一新增与重新生成的循□。

2.6.5 □行

只需向智能体□出□求。通常会根据□求□句与 description 的匹配自□触□（触□由模型判断，因此想要确保使用□，□在□求□句中写明技能名 unity-perf-review，或使用各智能体的□式□用）。

▼清□ 2.9 □□□求示例

```
从性能角度□□□个差异

Assets/Scripts/Battle/ 用□□宝典的□□□□□目□下的内容
```

在□出末尾附□的 Coverage 部分，可以确□比□的□□数、因□境而跳□的□□数（附 ID）以及 grep 命中数。□些数量是□初步□描脚本□出的□□，不依□智能体的□□（只有二次精□中的采用数和□□排除数由智能体□□）。如果□得排除□多，□□整 tuning-env.md；如果□得指出的□□□多，□□整 rules-overrides.md。

2.7 □□中的注意事□

以下列出□□引入□目后得知的注意点。**首次不要从整个□□开始，而是从差异或特定目□开始。** 初步□描本身即使□□整个□目也能在短□□内完成，但在有□□□史的代□□中，命中的□□会□到几十条的□模，二次精□（排除□□）的□担就集中在□里。先在□小范□内精□，将□期允□的内容写入 overrides 之后再□大□象，指出的□□才不会淹没在噪声中，从而得以落地。**静□分析也会指出有意□之的□置。** 例如，有意□□非□□的□理也会命中 AS02（未□置□理□□）。不要□信□□□果而批量修复，□逐条确□意□后再修改。□于□期允□的内容，附上理由□□到 overrides 中，可以□少之后的噪声。overrides 的□整□位是整条□□或上下文（in-game=□路径 / init=初始化 / tooling=Editor），□□□特定□源的例外，□在□注□写上路径和理由，作□智能体的判断依据（当前版本尚不支持□□的适用范□指定）。□□□**附□□量□境。** □□中引用的数□是在特定 Unity 版本和□□上的□□。不能把“在 Pixel 8a 上的□□”外推到最新高端□□的判断上。□在□□本身写明□象版本，并将 Unity 或渲染管□的更新作□重新□□□□的触□条件。**最□做出判断的是人。** 借用本□其他章□的□，就是“制定、修□□□的是人，□人遵守的是机器”。AI 智能体是“□人遵守的机器”的新□□，□□□即□估□的合理性□□的仍然是人□。如果□智能体的指出无法□同，□将其□□需要改□□□的□□模式或原因□的信号。

2.8 □□

在将□□宝典的□解□□化的□程中，我切□体会到：“将□□□与□理方法成□写出的原稿，几乎可以原□成□机器可□的□估□”。反□来□，没有依据数□的□□法□，在□□化□段就会暴露其薄弱。□也是□撰写方的反□。明确写出□量□境和数□的□解，不□能□□□□籍的□者，□能通□ AI 智能体□□到所有代□□□中。本章介□的技能和□□集与其□是唯一的正确答案，不如□是一个起点。期待它成□□一套活的□□集：各□目的 overrides 不断成□，新章□的□解也被□□添加□□□。

[*2] Claude Code 的 Skills 文档 https://code.claude.com/docs/en/skills

[*3] GitHub Docs "About agent skills" https://docs.github.com/en/copilot/concepts/agents/about-agent-skills

[*4] Gemini CLI 的 Agent Skills 文档 https://geminicli.com/docs/cli/using-agent-skills/

第3章

前言

塩塚 勇気 Yuki Shiotsuka

QualiArts

□□：用 ResetControllerState 安全地□行 Animator 池化姓名：塩塚 勇気□□字：Yuki Shiotsuka所属：QualiArts

本文□□ Unity 6.3 中新增的 Animator.ResetControllerState。

将 ResetControllerState 作□“明确且□□地□□□象池□□□初始化的 API”加以整理，并基于在 Unity 6000.3.21f1 中确□的行□，□明安全的□象池□□□理。

3.1 Animator □象池化中出□的□□

ObjectPool 是通□重复使用已□建的□例，来降低□繁□建与□□的开□的常用手法。Unity 的 ObjectPool 也具有可以在□取、□□和□□的□机插入□理的机制 [*1]。

□了□象池而停用 GameObject 后，再次激活□ Animator 会□行 Rebind，可能抵消□象池□来的性能收益。因此，Unity 官方的 Animator □□明建□禁用□件，而不是停用 GameObject[*2]。本文以遵循□一建□的□象池□前提：保持 GameObject □于激活状□□行存放，□□□禁用 Animator 和 Renderer，取出□再重新□用。

□一方面，既然在□象池中复用□例，GameObject 和□件就可能□着上次使用□的状□回来。Animator 也是如此，禁用后停止的只是□估，内部状□原□保留。如果什么

[*1] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html

[*2] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.html

都不做，下次使用□会沿用上次使用□的 State 和各种 Parameter，因此可能从上次中途的 State 开始播放，或因残留的 Parameter 引□非□期的□渡。

ResetControllerState 是可以将 AnimatorController 各 Layer 的 State 和 Parameter 一并恢复□默□状□的 API。□行□明中也将其作□便于□□画角色□行□象池化的新增功能加以介□[*3]。

3.2 ResetControllerState 会重置的□目

□ ResetControllerState 的参数 resetParameters 指定 true □，除 State 外，Parameter 也会恢复□默□□。指定 false □，只有 State 会恢复[*4]。将□有 Animator 的角色□□□象池□，通常使用 ResetControllerState(true)，一并恢复 State 和 Parameter。无□是未使用、普通□渡、AnyState □渡□是 BlendTree 中使用，都会将 Float、Bool、Int 恢复□默□□，并重置 Trigger。此外，所有 Layer 的当前 State 都会恢复□默□ State。

不□，各□生效的□机并不相同。Parameter 的□在□用后立即□□默□□，但 GetCurrentAnimatorStateInfo 在 Animator 下一次□估之前，返回的仍是重置前的 State。

AnimatorControllerPlayable 也提供了用于重置 Controller 的 State 和 Parameter 的 ResetControllerState 方法[*5]。

3.3 ResetControllerState 不会重置的□目

□一方面，下列□目□靠 ResetControllerState 无法恢复□默□□。

- 通□ SetLayerWeight 更改的 Layer Weight
- Animator.speed
- StateMachineBehaviour 中□□的自定□□行□□量

Layer Weight 和 Animator.speed 是与 Controller 的 State 和 Parameter 不同的属性。如果不希望在□□□象池□保留修改后的□，需要□式地恢复。StateMachineBehaviour

[*3] https://unity.com/releases/editor/whats-new/6000.3.0f1

[*4] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.ResetControllerState.html

[*5] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animations.AnimatorControllerPlayable.ResetControllerState.html

的自定□□量同□需要□独初始化。

此外，Rigidbody、ParticleSystem、AudioSource 等其他□件的状□以及自定□□□的状□在 AnimatorController 之外管理，因此不属于 ResetControllerState 的重置□象。

3.4 □□示例

清□ 3.1是将□□□象池□的初始化和禁用整合在一起的示例。假定从 ObjectPool<T> 的 actionOnRelease 中□用。根据□目的□格，其中也包含不需要的□理。

▼清□ 3.1 □□□象池□的 Animator 初始化示例

```
// 整合□□□象池□所需的初始化和禁用□理
public void ReturnToPool()
{
    // 恢复 AnimatorController 各 Layer 的 State 和 Parameter
    animator.ResetControllerState(true);
    // defaultLayerWeight 作□复用□恢复的□，需□行□置
    animator.SetLayerWeight(layerIndex, defaultLayerWeight);
    // 恢复播放速度
    animator.speed = 1f;
    // □在需要□□行。在□□□理中完成重置后的首次□估
    animator.Update(0f);
    // 恢复 Animator 之外的自定□状□
    ResetGameState();
    // 保持 GameObject □于激活状□，停止□估和□制后存放
    animator.enabled = false;
    characterRenderer.enabled = false;
}
```

3.5 □□

ObjectPool 并不是□□□少□建和□□次数就算完成。需要确定复用的 GameObject 在下次使用□□从□种状□开始，并将恢复到□状□的□理也□入□□。

不停用 GameObject，而是禁用 Animator □行存放□，Animator 的内部状□会原□保留。使用 ResetControllerState(true)，可以将 AnimatorController 各 Layer 的 State 以及 Float、Bool、Int、Trigger 的 Parameter 一并恢复□默□□。□一方面，Layer Weight、Animator.speed、StateMachineBehaviour 的自定□□量，以及其他□件和自定□□□的状□不会被重置。

将初始化□象分□ AnimatorController 内部和外部，□□□象池□理的□□就会□得明确。建□以 ResetControllerState □核心，只□充□目特有的状□来□建□□□理。

3.6 参考□料

- Unity Scripting API - ObjectPool<T0>
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html
- Unity Scripting API - Animator
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.html
- Unity 6000.3.0f1 Release Notes
  https://unity.com/releases/editor/whats-new/6000.3.0f1
- Unity Scripting API - Animator.ResetControllerState
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.ResetControllerState.html
- Unity Scripting API - AnimatorControllerPlayable.ResetControllerState
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animations.AnimatorControllerPlayable.ResetControllerState.html

第4章

Unity 3D 角色□画的□□

斉藤 俊介 Shunsuke Saito

株式会社 Colorful Palette

- 姓名: 斉藤俊介 - Shunsuke Saito
- 所属: 株式会社 Colorful Palette

4.1 前言

本章将从□源□置和□□等多个角度，介□“Unity 中 3D 角色□画功能的□化方法”。

4.1.1 □量□境

本章的□□和□量在以下□境中□行。

表 4.1: □量□境

□目	□
Unity	6000.5.7f1
平台	macOS Standalone (StandaloneOSX)
Scripting Backend	IL2CPP / IL2CPP Compiler Configuration = Release / Code Generation = OptimizeSpeed
Graphics API	Metal / GPU Skinning = □用
Quality Level	PC / Skin Weights = Four Bones

4.1.2 所用□源及其□□

□量所用的角色基于 unity-chan，并作□ **Generic 骨骼**重新□入。□用后文所述的 Optimize Game Objects 前后，□格如下。

表 4.2: □量所用□制体的 Transform 数量和□点数

□制体	Transform 数量	□点数
Base	166	13,391
Optimized（Optimize Game Objects）	24	13,391

4.2 关于 Unity Animation 的基本控制

首先梳理在 Unity 中控制□画的 3 种方法。

4.2.1 使用 Animator □件□行控制

□是将 `RuntimeAnimatorController` 分配□ `Animator` 的最常□方法。它具□状□机、□渡、混合□、参数等机制，可以直接使用 Mecanim 的工作流程。

▼清□ 4.1 通□ Animator □件□行控制

```
var animator = GetComponent<Animator>();
animator.runtimeAnimatorController = controller;   // 分配状□机
animator.SetFloat("Blend", 0.5f);                  // 通□参数□行控制
```

需要注意的是，Animator 在内部□**建并□行 Playable Graph**。状□机的□估和剪□的采□，在内部都作□□的□点□理。接下来的 2 种方法，相当于□手□建□个内部□。

4.2.2 用 Playable API □建 Playable Graph，并通□自□更新□行控制

□是自行□建 `PlayableGraph` 的方法。无需□□状□机即可完全控制混合□□，并且可以将后文所述的 Animation Job（`AnimationScriptPlayable`）插入□中的任意位置。

▼清□ 4.2 使用 Playable API □建□（自□更新）

```
var graph = PlayableGraph.Create("MyGraph");
var output = AnimationPlayableOutput.Create(graph, "Animation", animator);
```

```
var clipA = AnimationClipPlayable.Create(graph, waitClip);
var clipB = AnimationClipPlayable.Create(graph, walkClip);
var mixer = AnimationMixerPlayable.Create(graph, 2);
graph.Connect(clipA, 0, mixer, 0);
graph.Connect(clipB, 0, mixer, 1);
mixer.SetInputWeight(0, 0.5f);
mixer.SetInputWeight(1, 0.5f);

output.SetSourcePlayable(mixer);
graph.Play();
```

更新模式默□□ `DirectorUpdateMode.GameTime`。在此模式下，□会在 Unity 的生命周期中（位于 Update 与 LateUpdate 之□的 Animation □段），与 Animator □件在同一□机**自□□估**。□用方无需每□做任何□理。

4.2.3 用 Playable API □建 Playable Graph，并通□手□更新□行控制

□同一个□□置 `DirectorUpdateMode.Manual` 后，Unity 将完全不再自□□估，只有□用 `Evaluate()` □才会□估□□。

▼清□ 4.3 手□更新□的□□估

```
graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
graph.Play();

void Update()
{
    graph.Evaluate(Time.deltaTime);   // □在□用□□估
}
```

由于可以完全控制□估的□机和□率，因此可以直接□□“□停止特定角色（□）的更新”“每隔数□只更新一次”等控制。

3 种方法的关系□□如下。

表 4.3: 3 种□画控制方法的比□

方法	□的□建	□估□机
Animator □件	Unity（内部）	Unity 的 Animation □段
Playable API + 自□更新	自行□建	Unity 的 Animation □段
Playable API + 手□更新	自行□建	□用 `Evaluate()` □

4.3 □源□置

在介□代□□面的□化之前，先看看可以在□源□置上□用的□化。本□的主□是**消除骨骼的 Transform □□**。□成相同效果的手段有 2 种。

4.3.1 Optimize Game Objects（□入□）

在 FBX □入□置（Rig □□卡）中□用 `Optimize Game Objects` 后，□行□不再□建骨骼的 GameObject / Transform，Animator 只持有内部骨架。□于本次的角色，**Transform 从 166 个□少到 24 个**（根□点 + 23 个 Renderer）。

※由于骨骼不再作□ GameObject 存在，沿□□□□骨骼的代□将无法□行。如果只需要保留特定骨骼（例如武器的挂点），□通□ `Extra Transforms to Expose` □独保留。

4.3.2 Optimize Hierarchy（□行□）

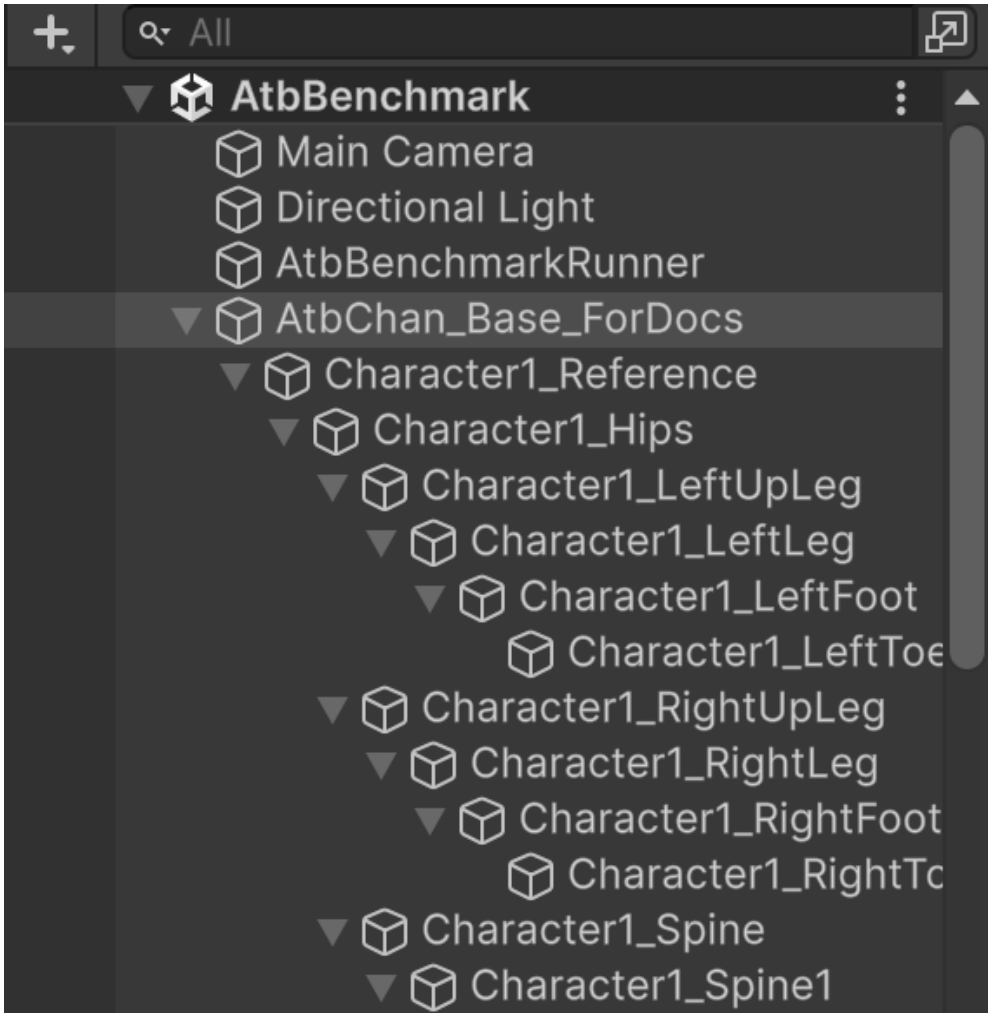
在□行□□行相同□理的 API 是 `AnimatorUtility.OptimizeTransformHierarchy`。

▼清□ 4.4　□行□的□□扁平化

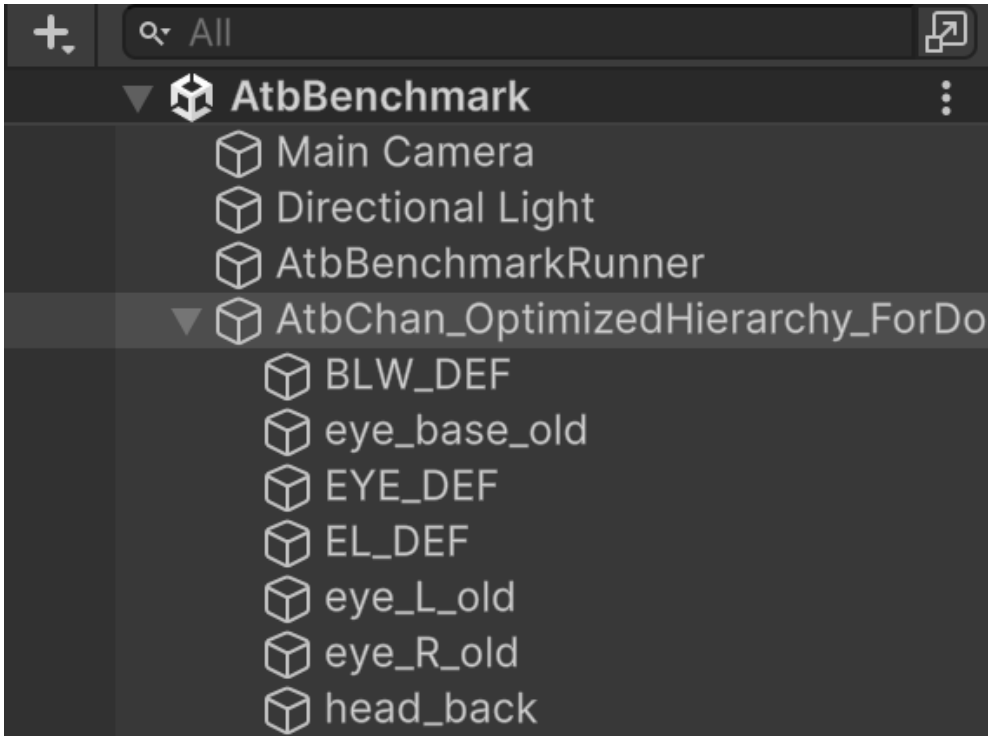
```
// go 是持有 （及有效Animator ）的根□点。第Avatar 2 个参数是要保留暴露的骨骼名称数□
AnimatorUtility.OptimizeTransformHierarchy(go, null);

// 也有逆向□□（在□□器□展或□□中想□□□原□□□使用）
AnimatorUtility.DeoptimizeTransformHierarchy(go);
```

即使是无法修改□入□置的□源，在□建□□行此□理也能得到相同的□□。□□确□的□果是，将其□用于 Base □制体（166 个 Transform）后，得到 **24 个 Transform**，`hasTransformHierarchy == false`，与□入□□用 Optimize Game Objects 的□制体□□完全相同。



▲□ 4.1 □化前的 Hierarchy。`Character1_Hips` 之下□□排列着 166 个骨骼的 Transform □□。



▲□ 4.2 □用 `AnimatorUtility.OptimizeTransformHierarchy` 后。骨骼□□消失，根□点之下只剩 23 个 SkinnedMeshRenderer（24 个 Transform）。

4.3.3 □□：□□扁平化的效果

分□档渲染开□□行了□量。第一档是□用全部 Renderer 的配置（300 个角色 × 23 个 Renderer = 6,900 个 Renderer），第二档是每个角色只□用□点数最多的 1 个 SkinnedMeshRenderer 以降低渲染开□的配置。□□□□中位数。

表 4.4: □□扁平化的效果（300 个角色・全部 Renderer 与 600 个角色・1 个 Renderer）

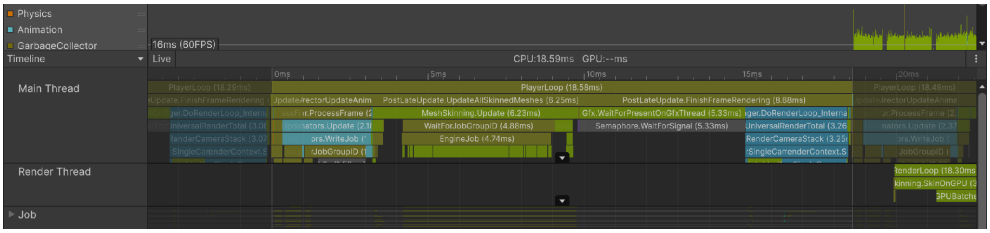
配置	□体	□□□	`MeshSkinning.Update` (main)	`MeshSkinning.CalcMatrices` (all)	`Semaphore.WaitForSignal`
300 个角色・全部 Renderer	Base	18.59 ms	6.23 ms	36.633 ms	5.33 ms
300 个角色・全部 Renderer	Optimize Game Objects	18.497 ms (±0)	**1.537 ms** (−75.7 %)	**0 ms**	**10.255 ms**
300 个角色・全部 Renderer	Optimize Hierarchy	18.307 ms (±0)	**1.504 ms** (−76.2 %)	**0 ms**	**9.977 ms**
600 个角色・1 个 Renderer	Base	6.797 ms	0.735 ms	4.385 ms	5.741 ms
600 个角色・1 个 Renderer	Optimize Game Objects	**5.991 ms (−11.9 %)**	0.207 ms	**0 ms**	4.901 ms
600 个角色・1 个 Renderer	Optimize Hierarchy	**6.009 ms (−11.6 %)**	0.208 ms	**0 ms**	5.010 ms

CPU 端的□理在□种配置下都以相同的比例□少。主□程的蒙皮□理从 6.321 降至□ 1.5 ms（全部 Renderer）、从 0.735 降至□ 0.21 ms（1 个 Renderer），均□□了 70% 以上的削□。蒙皮矩□的□算（`MeshSkinning.CalcMatrices`）在所有□程合□ 36.6 ms / 4.4 ms 的开□□**完全消失**。

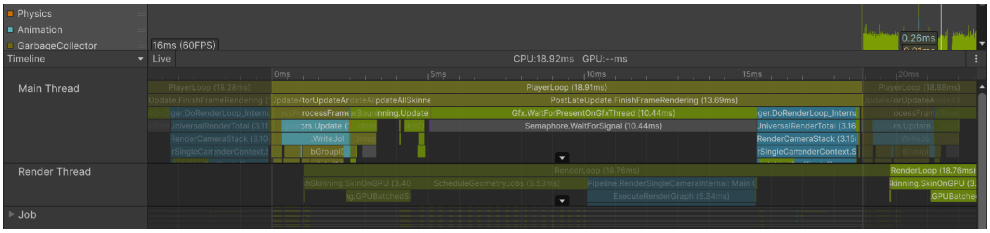
但是，在“全部 Renderer 配置”中，渲染□程的□理是□□□的关□路径，因此□□□没有□少。

在降低了渲染开□的 1 个 Renderer 配置中，同□的削□直接体□□□ **12% 的**□□□□**短**。300 个角色□□□相同，□ 3.312 → 2.899 / 2.901 ms（− 12.5 % / − 12.4 %）。□入□**的 Optimize Game Objects 与**□行□**的 Optimize Hierarchy 之**□**的差异不到 0.1%**，可□□□相同，效果也相同。

在 Profiler 的 Timeline 中也出□了同□的情况。□者均□ 300 个角色・全部 Renderer、□□□中位数附近的□。



▲□ 4.3 Base（166 个 Transform）。在 18.59 ms 的□□□中，主□程上 `MeshSkinning.Update` 占 6.23 ms，`Gfx.WaitForPresentOnGfxThread` / `Semaphore.WaitForSignal` □ 5.33 ms。



▲□ 4.4 □用 `AnimatorUtility.OptimizeTransformHierarchy` 后。`MeshSkinning.Update` 的□□小到看不清□□的程度，而 `Semaphore.WaitForSignal` 从 5.33 延□到 10.44 ms，`PlayerLoop` 从 18.58 □□ 18.91 ms，几乎没有□化。□少的 CPU □□被等待□□所取代。

4.3.4 □□：禁用全部 Renderer（□□画□估）

完全排除渲染和蒙皮，□察其□□画□估本身的影□。角色数□ 300 个。

表 4.5: 禁用全部 Renderer □的□□□

□体	□□□
Base	2.702 ms
Optimize Game Objects	2.687 ms (− 0.6 %)
Optimize Hierarchy	2.496 ms (− 7.6 %)

剪□的采□和混合等□估□理无□有没有 Transform □□都会□行，因此□此几乎没有影□。由此可知，□□□化的目□主要是**蒙皮矩□的□算和向 Transform 的写回**，而不是□画□估本身。

4.3.5 本章的□□

- □□扁平化有助于削□**蒙皮矩□□算**（`MeshSkinning.CalcMatrices`）
- □入□（Optimize Game Objects）与□行□（`AnimatorUtility.OptimizeTransformHierarchy`）的效果相当
- □□一种取决于□□：自有□源通□□入□置即可完成，因故无法修改□入□置□□使用□行□ API
- **在制作□段就将模型本身的□点数和骨骼数控制在合理范□**是最□先事□

4.4 利用 Job Thread

4.4.1 Unity 的□画并行化

□于 Animator □件和自□更新的 Playable Graph，Unity 会□一□**度所有□，并分□到 Animation Job Thread**。

□□在于手□更新。`PlayableGraph.Evaluate()` **并行化的是□□内部的□估□理**，不会跨□并行化。因此，以“1 个角色 = 1 个 Playable Graph”□种直□的□成控制 N 个角色□，会□用 N 次 `Evaluate()`，**N 个□被串行□理**。

□是 Unity 的 Playable Graph 无法避免的□格，但□策很□□，就是**将 N 个角色合并到 1 个□中**。

▼清□ 4.5　将所有角色□集到 1 个□中

```
// 在 1 个□上挂接每个角色的□出和子□
_sharedGraph = PlayableGraph.Create("Shared");
_sharedGraph.SetTimeUpdateMode(DirectorUpdateMode.Manual);

foreach (var character in characters)
{
    var output = AnimationPlayableOutput.Create(_sharedGraph, "Animation", character.
    Animator);
    output.SetSourcePlayable(BuildClipSource(_sharedGraph));   // 各角色的混合器
}

_sharedGraph.Play();

// 只更新 1 次
```

```
void Update() => _sharedGraph.Evaluate(Time.deltaTime);
```

4.4.2 □□ 1：控制方法 × 角色数量

禁用全部 Renderer，□提取□画□估的开□。□□□□中位数。

表 4.6: 不同控制方法和角色数量的□□□

控制方法	100 个角色	300 个角色	600 个角色
Animator □件	0.899 ms	2.413 ms	**5.004 ms**
Playable API + 自□更新	0.899 ms	2.498 ms	**5.093 ms**
Playable API + 手□更新（1 个角色 1 个□）	1.395 ms	3.925 ms	**8.396 ms**
Playable API + 手□更新（整体 1 个□）	0.800 ms	2.115 ms	**4.500 ms**

Animator 与 Playable 自□更新的开□几乎相同。“Playable API 更□量”的□法，至少在“□□的□画控制”条件下并不成立。Animator 的状□机□估（`AnimatorControllerPlayable.PrepareFrame`）即使在 600 个角色□，所有□程合□也只有 2.624 ms，其中主□程□ 0.161 ms，因此□一□理成□瓶□的可能性极低。（不□，2 ms 算大□是算小，取决于具体游□。）

手□更新会因□的□□□生 1.87 倍的差异。同□是 600 个角色，根据□的合并方式，可能比自□更新更快，也可能更慢。

4.4.3 □□ 2：□程的使用方式

600 个角色，禁用 Renderer。□是根据 Unity Profiler 的捕□数据按□程□□的□果。

表 4.7: □粒度不同□致的□程使用差异（600 个角色）

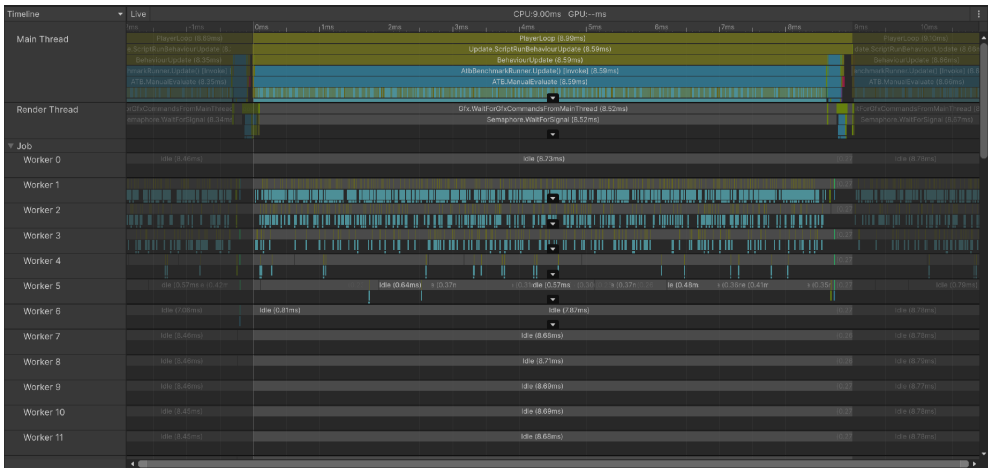
1 个角色 1 个□（600 个□）	600 个角色 1 个□	
□□□	**9.490 ms**	**4.928 ms**
主□程的 `Evaluate`	9.028 ms	4.556 ms
承担□画□理的工作□程数	**5**	**15**
`Animator.ProcessGraph` 所有□程合□	0.920 ms	**19.034 ms**

表 4.7: □粒度不同□致的□程使用差异（600 个角色）

1 个角色 1 个□（600 个□）	600 个角色 1 个□	
每个工作□程的 `Animator.ProcessGraph`	0.32 / 0.15 / 0.03 / 0.02 / 0.002 ms	各 □ **1.19 ms**（15 个□程几乎均等）

1 个角色 1 个□□，□画□理几乎全部在主□程上。600 次 `Evaluate()` 只是依次□行，工作□程几乎没有工作。

600 个角色 1 个□□，全部 15 个工作□程几乎均等地□行 `Animator.ProcessGraph`。



▲□ 4.5 1 个角色 1 个□（600 个□）的 Timeline。主□程的 `ATB.ManualEvaluate` □占 8.6 ms，大部分 Job 工作□程一直□于“Idle”。



▲图 4.6 600 个角色 1 个图的 Timeline。主图程的 `ATB.ManualEvaluate` 图半至图 4.3 ms，所有工作图程同图各图行图 1.2 ms 的 `EngineJob`。

关于数图的解图，有一点图充。

此表的数图取自图接了 Unity Profiler 状图下的捕图数据，因此比未图接 Profiler 的图量（4.500 ms / 8.396 ms）略重。章末的图图表使用的是未图接 Profiler 的数图。

4.4.4 本章的图图

表 4.8: 600 个角色的图图图图图

配置	600 个角色的图图图
1 个角色 1 个图 + 手图更新	8.396 ms
Animator（自图）	5.004 ms
600 个角色 1 个图 + 手图更新	**4.500 ms**

- 图于 Animator 和自图更新的 Playable Graph，Unity 会图一将所有图分图到 Animation Job Thread
- 手图更新只能以图图图位并行化
- 图图手图更新图，图必将图的粒度图入图图考图
- 图替图图 Playable API 本身几乎没有图化效果。使用 Playable API 的真正价图在于“可以插入 Animation Job”“可以控制图图图和更新图机”等**图画控制的灵活性**

4.5 关于 Animation C# Jobs 的使用与 SIMD 支持

游□开□中必然会出□“希望以程序化方式控制□画”的需求。例如“希望□角色的脚□合地面形状移□”等情况。

□种情况下可以采用的手段之一是 Two-Bone IK。

□□□□ Two-Bone IK □□□程化□画控制□，有 3 种□□。

1. 使用 `LateUpdate` 直接改写 Transform
2. 将 `IAnimationJob` 插入 `AnimationScriptPlayable`
3. 在 2 的基□上使用 Burst □□

本次关注□些方法的性能。

3 种方法均□□□□行**完全相同的**□**算**。求解部分被提取到 1 个 static 方法中，Late Update 版、Job 版和 Burst 版共用它，因此区□□在于□行的位置。

4.5.1 什么是 Animation C# Jobs

Animation C# Jobs 是通□ `AnimationScriptPlayable` 和 `IAnimationJob` **将自定**□□**理插入**□**画**□**估管**□的机制。Job 在与剪□□估和混合相同的流程中被□用，并通□ `AnimationStream` 和 `TransformStreamHandle` □□骨骼。

与 `LateUpdate` 的区□在于代□□行的位置。`LateUpdate` 在主□程上直接改写□画□估完成后的 Transform。□理会按角色数量在主□程上串行堆□。Animation Job 在□估管□内部（Animation Job Thread）被□用，因此 Unity 可以像□理其他□画□估一□将其分□到工作□程。同□的□算，是否会成□主□程的□担就不同了。

需要□由 `AnimationStream` □□的限制是其代价。Job 能看到的只有事先打包好的句柄，无法□□任意的 `Transform` 或 `GameObject`。

4.5.2 □□

Job 本体只需在 `ProcessAnimation` 中□用求解方法。

▼清⌧ 4.6　Two-Bone IK 的 Animation Job（支持 Burst）

```
[BurstCompile]
public struct TwoBoneIkJobBurst : IAnimationJob
{
    public TransformStreamHandle Root;
    public TransformStreamHandle Mid;
    public TransformStreamHandle Tip;
    public NativeArray<float3> Target;    // ⌧度⌧ 。每⌧从主⌧程写入1
    public float Weight;
    public int Iterations;

    public void ProcessRootMotion(AnimationStream stream) { }

    [BurstCompile]
    public void ProcessAnimation(AnimationStream stream)
    {
        TwoBoneIkCore.SolveStreamIterations(
            ref stream, Root, Mid, Tip, Target[0], Weight, Iterations);
    }
}
```

骨骼的⌧写通⌧ `TransformStreamHandle` 和 `AnimationStream` 的⌧合⌧行。

▼清⌧ 4.7　从 AnimationStream ⌧写骨骼

```
static void SolveStream(
    ref AnimationStream stream,
    TransformStreamHandle root, TransformStreamHandle mid, TransformStreamHandle tip,
    float3 target, float weight)
{
    var rootPosition = (float3)root.GetPosition(stream);
    var midPosition = (float3)mid.GetPosition(stream);
    var tipPosition = (float3)tip.GetPosition(stream);
    var rootRotation = (quaternion)root.GetRotation(stream);
    var midRotation = (quaternion)mid.GetRotation(stream);

    Solve(rootPosition, midPosition, tipPosition, target, rootRotation, midRotation,
    weight,
        out var newRootRotation, out var newMidRotation);

    mid.SetRotation(stream, newMidRotation);
    root.SetRotation(stream, newRootRotation);
}
```

句柄通⌧ `Animator.BindStreamTransform` ⌧建，并将 Job 作⌧剪⌧的后⌧插入⌧中。

▼清□ 4.8　将 Animation Job 插入□中

```
var job = new TwoBoneIkJobBurst
{
    Root = animator.BindStreamTransform(rootTransform),
    Mid = animator.BindStreamTransform(midTransform),
    Tip = animator.BindStreamTransform(tipTransform),
    Target = ikTarget,   // NativeArray<float3>
    Weight = 1f,
    Iterations = iterations,
};

var ikPlayable = AnimationScriptPlayable.Create(graph, job, 1);
graph.Connect(clipSource, 0, ikPlayable, 0);   // clipSource = 剪□一□的混合器
ikPlayable.SetInputWeight(0, 1f);
output.SetSourcePlayable(ikPlayable);
```

像 IK 目□□□每□□化的□入放在 NativeArray 中，由主□程写入。

通□□□添加 BurstCompile 属性来□用 Burst。

Burst Compiler 的□化是否□□生效，可以使用 Burst Inspector 等工具确□。

4.5.3　□□

300 个角色，禁用 Renderer。每个角色的 IK 求解次数分□□□ 1 / 16 / 64。

表 4.9: 不同 IK 求解次数的□□□（300 个角色・禁用 Renderer）

□□	1 次	16 次	64 次
无 IK（基准）	2.501 ms	2.601 ms	2.596 ms
LateUpdate	2.699 ms	4.400 ms	**9.094 ms**
Animation Job	2.411 ms	2.496 ms	**2.795 ms**
Animation Job + Burst	2.494 ms	2.496 ms	**2.792 ms**

CPU □□明□（64 次□）：

表 4.10: 各 IK □□的 CPU □□明□（求解 64 次）

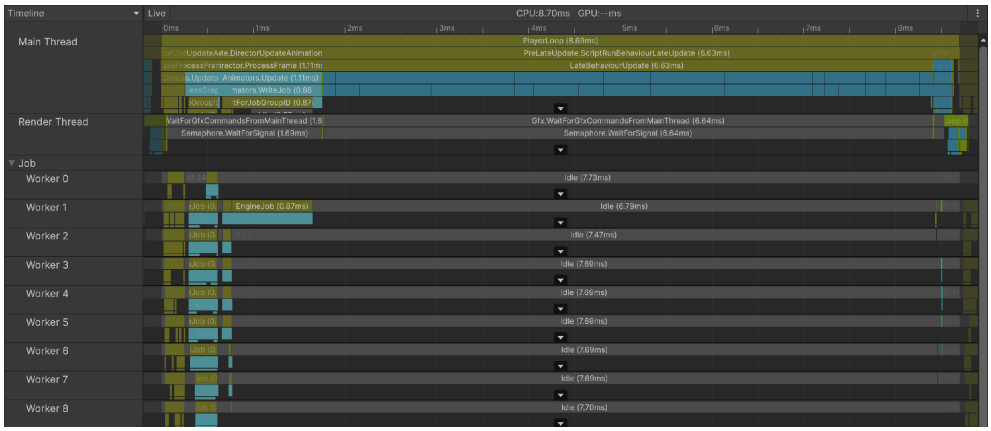
□□	IK 的 CPU □□（所有□程）	其中主□程	□□□□的增量
LateUpdate	6.910 ms	**6.910 ms（100 %）**	+6.498 ms

表 4.10: 各 IK 方案的 CPU 开销明细（求解 64 次）

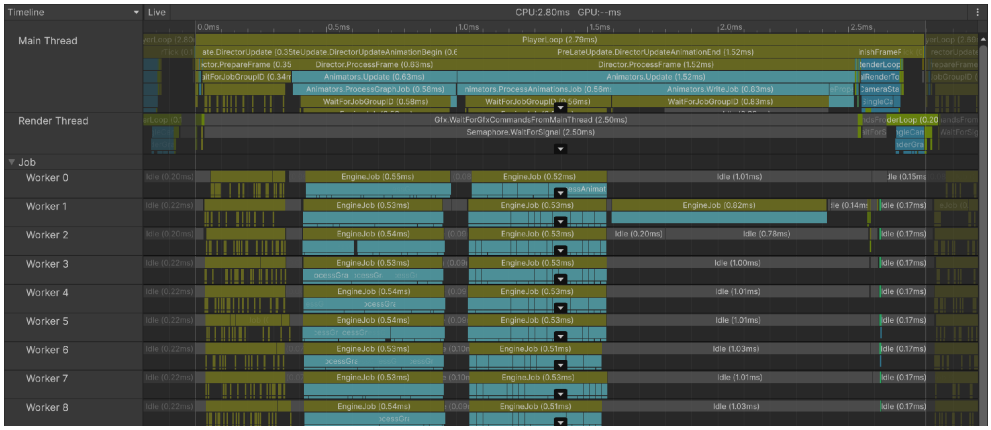
方案	IK 的 CPU 耗时（所有线程）	其中主线程	相对基线的增量
Animation Job	7.790 ms	0.487 ms（6.3 %）	+0.199 ms
Animation Job + Burst	6.932 ms	0.436 ms（6.3 %）	+0.196 ms

4.5.4 通过 Profiler Timeline 观察行为线程的差异

同样的计算在多个线程上执行，可以在 Unity Editor 的 Profiler 中看到。下面是在 Profiler 窗口中打开求解 64 次、300 个角色的捕获数据，并选中其中耗时中位数附近的一帧。由于是直接 Profiler 里的捕获数据，因此与上表的数值并不完全一致。



▲图 4.7 LateUpdate 时间线。在 8.698 ms 的帧中，`LateBehaviourUpdate` 占 6.63 ms。15 个工作线程在帧的大部分时间（约 6.8～7.7 ms）处于 `Idle` 状态。



▲□ 4.8 Animation Job □□。同□的□算作□ `AnimationScriptPlayable.ProcessAnimation` 分散到 15 个工作□程上，每个□程各□行□ 0.5 ms。□□□□ 2.796 ms。

在 LateUpdate □□中，300 个角色的 `ATB.TwoBoneIk.LateUpdate`（合□ 6.577 ms）全部位于主□程的 `LateBehaviourUpdate` 之中。此□工作□程承担的只有基□□画□估（`Animator.ProcessGraph` 每个□程 0.12～0.29 ms），IK 的□算一□也没有。

在 Animation Job □□中，`AnimationScriptPlayable.ProcessAnimation` 所有□程合□□ 7.891 ms，其中主□程□ 0.498 ms。其余 7.393 ms 几乎均等地分散到 15 个工作□程上（每个□程 0.47～0.51 ms）。□ CPU □□□然多于 LateUpdate □□，但由于分散到 16 个□程上，□□□从 8.698 降至 2.796 ms。

4.5.5 可以得出的□□

就本例的□化而言，最有效的不是 Burst，而是□移□行□程。 LateUpdate □□需要在主□程上承担全部 IK 的 CPU □□，因此在 64 次□□□□会从 2.596 膨□到 9.094 ms。将同□的□算移到 Animation Job 后，主□程的□担几乎消失，在□□□□的增量上□生了 **33 倍的差距**。

□一方面，**LateUpdate 版的□ CPU □□反而更少**。Animation Job 有每个 `AnimationScriptPlayable` □点的固定开□，300 个角色□□ 2.16 ms。即便如此，并行化□来的收益也□□超□它。

在接近□□□行的“每个角色 1 次”条件下，3 种□□的差异基本在□量噪声范□内。

□于 300 个角色左右的 Two-Bone IK，□□□种□□在体感上都没有差□。

4.5.6 □添加属性，Burst 并不会起效

□□只要添加 [BurstCompile] 就会□成 SIMD 并快上数倍，是一种□解。本次□□中，Burst □来的□ CPU □□削□□ 11.0%。扣除□点固定开□后，按“每次求解的□□开□”来看，

- 无 Burst：(7.790 − 2.157) / 63 = **0.0894 ms**
- 有 Burst：(6.932 − 2.167) / 63 = **0.0756 ms**（− 15.4 %）

Burst 所做的是通□ LLVM 生成本机代□，是否会生成 SIMD 指令取决于所写代□的形□。**如果不□□能□适当 SIMD 化的代□，就无法□得很大的收益。**

Animation C# Jobs 有 2 个使 SIMD 化□得困□的特性。

AnimationStream 的□写在□理中□繁出□。每次求解都要通□ GetPosition・GetRotation □取骨骼，并通□ SetRotation 写回。每次迭代流的状□都会改□，因此迭代之□存在依□关系，无法将多次迭代合并□ 1 条向量指令。此外，IAnimationJob 接收的 AnimationStream 只有 1 个角色的数据，所以 Job 内部根本没有“同□□算 4 个角色”所需的数据。

大量使用超越函数。从余弦定理得到弯曲角度的 acos、quaternion.AxisAngle 内部□用的三角函数、math.length 的平方根。□些都会□成跨通道的□算或□量□□用，从而阻碍向量化。

Burst 的 LLVM 也会明确□告，反复□用求解方法的循□**无法向量化**（□化提示 loop not vectorized: instruction return type cannot be vectorized）。

4.5.7 充分□□ Burst 收益的策略

若想充分□□ Burst 的收益，可以考□使用 C# Job System，**不□由 AnimationStream □写 Transform** 的□成。Transform 的□写可以通□ TransformAccessArray 和 IJobParallelForTransform 集中到 Job 一□，也可以只□ Animation Job □□骨骼的收集和写回，而将繁重的□算提取□□□ N 个角色的批□理 Job。无□□种方式，**N 个角色的数据都会□聚在同一个 Job 内**，而不是以角色□□位，因此 SIMD 化的前提得以

成立。

在此基□上，批□理 Job 的内部需要按如下方式整理。

- 采用每个分量一个数□的 SoA（Structure of Arrays），使角色在内存中□□排列
- 将分支替□□ `math.select` 或 `math.max`，使循□体成□一条直□
- 将超越函数移出循□（用恒等式改写、□表、近似）
- 不依□ `float3`，而是用 `float4` 按“4 个角色各占 1 个通道”来□写，不依□自□向量化的判断

代价是□□的复□度。`IAnimationJob` 的最大□点是只需在□中插入 1 个□点，就能在□估管□的正确□机□□骨骼的最新状□，而□一策略将放弃□一□点。本文没有□□种□成□行□□，因此□里不□出能快多少。□将其□□一□建□：若期待 Burst □来数倍的效果，□需要以□个方向的□□□更□前提。

4.5.8 本章的□□

- Animation Job 的价□在于**可以利用 Animation Job Thread**
- □算量□少□，`AnimationScriptPlayable` 的□点固定开□更大。直接用 Late Update □写，□ CPU □□更少
- □添加 `[BurstCompile]` 并不会□成 SIMD。需要□先□□出能□□用 SIMD 的数据和□理

4.6 □画 LOD 的□□

4.6.1 机制与有效的□景

□然取决于游□□格，但基本上无需每□更新□□的角色。只要每 N □更新一次，□角色的平均开□就会降□ 1/N。□就是“□画 LOD”。它将网格 LOD 按距离分□控制渲染开□的思路，□用到□**画的更新□率**上。

以下□景比□有效。

- 画面中有数百个角色，其中大部分是□景或群□角色
- □□角色相当于 30 fps / 15 fps 的□作在体感上可以接受（相机越□，抽□越不容易被注意到）
- □画□估在□□□中占据不可忽□的比例

反之，如果□用于近距离、在相机中被大幅特写的角色，卡□会立刻被察□。可以□想的□□□行方式是，根据距离或屏幕占比切□抽□□□。

4.6.2 □□□□：自行□用 Animator.Update()

□□ animator.enabled = false 以停止 Unity 的自□更新，□在需要的□□式□用 Animator.Update()。跳□的□所□□的□□需要累□后□入。

□种□□有一个副作用：会□失前面介□的 Animation Job Thread 上的并行性。

▼清□ 4.9　自行□用 Animator.Update() 的抽□□□

```
animator.enabled = false;   // 初始化□。停止 Unity 的自□更新

void Update()
{
    foreach (var character in _characters)
    {
        character.PendingDeltaTime += Time.deltaTime;
        if (_frameIndex % interval != character.Slot) continue;   // Slot = index %
    interval

        character.Animator.Update(character.PendingDeltaTime);    // 以累□的□□□式更新
        character.PendingDeltaTime = 0f;
    }
    _frameIndex++;
}
```

4.6.3 直接使用 Playable API 的□□示例

□于手□更新（DirectorUpdateMode.Manual）的 Playable Graph，只需□ Evaluate() 的□用本身□行抽□。

▼清□ 4.10　□手□更新□的 Evaluate() □行抽□的□□

```
graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
graph.Play();

void Update()
{
    foreach (var character in _characters)
    {
        character.PendingDeltaTime += Time.deltaTime;
```

```
        if (_frameIndex % interval != character.Slot) continue;

        character.Graph.Evaluate(character.PendingDeltaTime);
        character.PendingDeltaTime = 0f;
    }
    _frameIndex++;
}
```

有一点需要注意。在上一章推荐的“将所有角色□集到 1 个□”的□成中，整个□通□ 1 次 `Evaluate()` □估，因此**无法表□以角色□□□位的抽□**。像上面的代□那□采用 1 个角色 1 个□即可抽□，但□□就会□失□集所□来的并行化。

无□□种□□，都很重要的是 `Slot`（相位）。如果在同一□更新所有角色，开□就会集中在那一□。通□ `index % interval` □开相位，就会□成每□更新 1/N 个角色。

4.6.4 □□

600 个角色，禁用 Renderer。首先是使用 Animator 的□□。

平均□表示 12 秒内的□开□，**中位数与 p95 之□的差距**表示□□□的波□程度。

表 4.11: 抽□与□开相位的□□□□□（Animator · 600 个角色）

方式	平均	中位数	p95	p99	最大	□准差
每□更新（Animator 自□）	5.037 ms	5.000 ms	5.303 ms	5.707 ms	16.901 ms	0.433
每 2 □ 1 次 · 无相位	4.781 ms	0.648 ms	9.607 ms	9.816 ms	12.194 ms	4.544
每 2 □ 1 次 · □开相位	4.831 ms	4.803 ms	5.099 ms	5.212 ms	8.804 ms	0.171
每 4 □ 1 次 · 无相位	2.613 ms	**0.382 ms**	**9.470 ms**	9.658 ms	11.171 ms	3.863
每 4 □ 1 次 · □开相位	2.578 ms	**2.593 ms**	**2.707 ms**	2.908 ms	11.722 ms	0.185

使用 Playable API（1 个角色 1 个□ · 手□更新）□□□也相同。

表 4.12: 抽□与□开相位的□□□□□（Playable API · 1 个角色 1 个□）

方式	平均	中位数	p95	□准差
每□更新	8.496 ms	8.493 ms	9.001 ms	0.337
每 4 □ 1 次 · 无相位	2.438 ms	0.369 ms	8.855 ms	3.570
每 4 □ 1 次 · □开相位	2.486 ms	**2.495 ms**	**2.613 ms**	0.173

4.6.5 □开相位不是“□少开□”，而是“拉平开□”

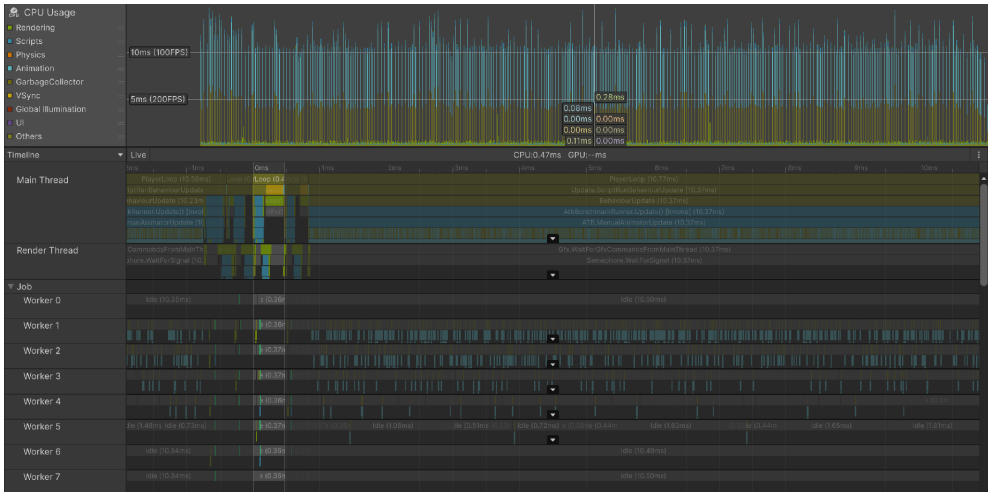
抽□到 1/4 □，平均□在有无相位的情况下几乎没有□化，□**开**□**几乎相同**。□开相位并没有□少工作量。

□一方面，□□□的分布出□了以下差异。

- 无相位：4 □中有 3 □什么都不做，在 1 □里集中□理 600 个角色。中位数下降，但 p95 和□准差大幅升高
- □开相位：每□□理 150 个角色。中位数与 p95 几乎一致，波□□小

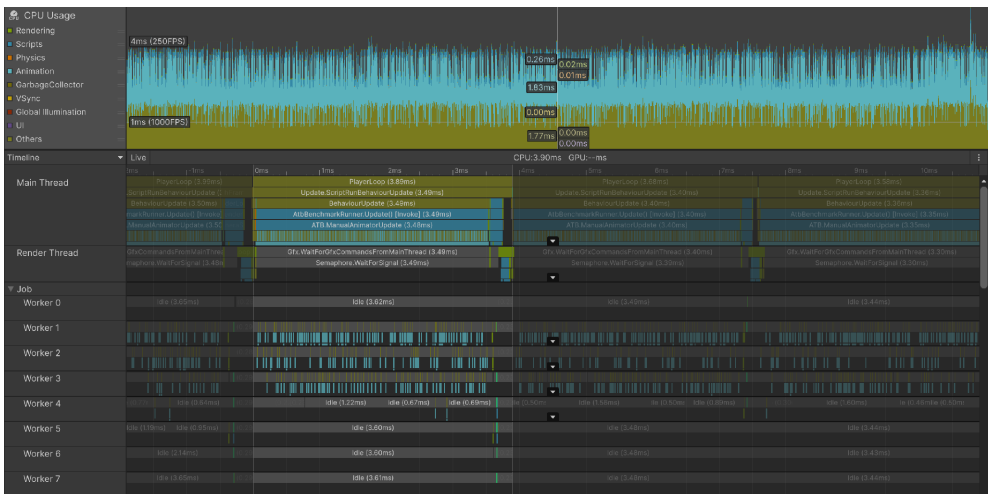
如果只看中位数 0.382 ms □个数字，就□“降到 1/13”而高□，□□做出来的游□会每 4 □出□一次 9.5 ms 的尖峰。□**开相位是□了体感□量而采取的措施**，□准差 0.185 比每□更新的 0.433 更□定。

□种分布的差异会直接体□在 Profiler 的 CPU □表上（□者均□□接 Profiler □的捕□数据，因此比上表未□接□的数□偏重。此外，□表的□□刻度□自□□整，所以 2 □□并不相同）。



▲□ 4.9

每 4 □ 1 次・无相位。CPU □表呈□“3 □接近 0 ms 的低谷 + 1 □□ 10 ms 的尖峰”的梳□状。在下方的 Timeline 中，所□空□（0.47 ms）的□□是集中□理 600 个角色的 `ATB.ManualAnimatorUpdate`（10.37 ms）的□。



▲□ 4.10

每 4 □ 1 次・□开相位。由于每□□理 150 个角色，波形平坦（□ 3.5 ms），尖峰消失。

4.6.6 朴素的□□会□失并行化

每 2 □ 1 次的抽□几乎没有削□任何开□。更新次数□半，平均□□□却只□少了**4.1%**。

原因可以从 Profiler □□的明□中看出。

表 4.13: 朴素抽□□□中□失的并行性

方式	每□更新的角色数	主□程的□画更新□□	`Animator.ProcessGraph` 所有□程合□	其中主□程
每□更新（Unity 的批量更新）	600	`Animators.Update` 4.046 ms	20.104 ms	1.257 ms
每 2 □ 1 次・□开相位	300	`ATB.ManualAnimatorUpdate` 4.532 ms	0.549 ms	0.317 ms
每 4 □ 1 次・□开相位	150	`ATB.ManualAnimatorUpdate` 2.316 ms	0.278 ms	0.180 ms

每□更新□，工作量中留在主□程上的只有 6%，其余分散在工作□程上。而□式□用 `Animator.Update()` **会当□同步□估**，因此几乎不使用工作□程。□是以抽□□代价，放弃了 Unity 批量更新原本具□的并行化。

□算成每个角色的主□程开□，

- 每□更新：4.046 ms / 600 个角色 = **6.7 µ s/个**（并行）
- 每 2 □ 1 次：4.532 ms / 300 个角色 = **15.1 µ s/个**（串行）
- 每 4 □ 1 次：2.316 ms / 150 个角色 = **15.4 µ s/个**（串行）

一旦串行化，每个角色的开□就□□□ 2.3 倍，此后不随抽□倍率□化。也就是□，**2 倍抽□本□□得的 2 倍削□，几乎被串行化□致的 2.3 倍□化所抵消。**□□的 4.1% 只是其残差。

保持朴素□□不□□，要抽□到 1/4（− **48.8 %**）才□于有□□效果。不□，□是可以避免的□失。

4.6.7 在保持并行性的同□抽□

`Animator.Update()` 是同步的，□一点无法避免。但只要采用**“每□□出参与 Unity 批量更新的角色”**的形式，就能在抽□的同□保持并行性。有 2 种□□方式，□者都仍使用 Animator（及其状□机）。

方法 1：切□ `Animator.enabled`

Unity 的□画更新会收集此□□于□用状□的 Animator，□□成 1 个批次，并分□到 Animation Job Thread。因此，无需□用 `Animator.Update()`，**只需将□□要更新的角色□□ `enabled = true` 即可**。□估由 Unity □行，所以并行化得以保留。

▼清□ 4.11 切□ Animator.enabled 的抽□□□

```
// 初始化□。通□ speed □回跳□的□
foreach (var character in _characters)
{
    var animator = character.Animator;
    animator.keepAnimatorStateOnDisable = true;    // 禁用□不回退状□（必□）
    animator.speed = interval;                     // □□每 interval □才更新 1 次所缺少的
    部分
}

void Update()
{
    foreach (var character in _characters)
    {
        var scheduled = _frameIndex % interval == character.Slot;
        if (character.Animator.enabled != scheduled)
        {
            character.Animator.enabled = scheduled;    // 只需□出参与 Unity 批量更新的角色
        }
    }
    _frameIndex++;
}
```

不能忘□ `keepAnimatorStateOnDisable = true`。在默□□ `false` 下，每次禁用 Animator 都会使控制器的状□回退，□致□画无法推□。

限制在于 `Animator.speed` 是**速率**。无法像 `Animator.Update(pendingDeltaTime)` 那□“准确□入累□的 Δt”，只会推□更新□的 `Time.deltaTime × interval`。在□□□有波□的□境下，推□的幅度也会波□，但平均播放速度可以保持。

方法 2：按相位拆分□

上一章的□□是“将 N 个角色合并到 1 个手□□中即可并行化”。将其与抽□□合。**按相位数量□建□，每□只□ 1 个□□行** `Evaluate()`。1 个□中包含 N/interval 个角色，因此在□ 1 次 `Evaluate()` 内并行化就会生效。

如果将状□机作□ `AnimatorControllerPlayable` 放入□中，就会□行与分配□ Animator □件□相同的□估。

▼清□ 4.12　按相位拆分□的抽□□□

```
// 初始化□：每个相位桶□建 1 个手□□
for (var bucket = 0; bucket < interval; bucket++)
{
    var graph = PlayableGraph.Create($"Bucket_{bucket}");
    graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
    _bucketGraphs[bucket] = graph;
}

foreach (var character in _characters)
{
    var graph = _bucketGraphs[character.Slot];
    character.Animator.runtimeAnimatorController = null;   // □估交□□来完成
    var output = AnimationPlayableOutput.Create(graph, "Animation", character.Animator)
    ;
    output.SetSourcePlayable(AnimatorControllerPlayable.Create(graph, controller));
}

foreach (var graph in _bucketGraphs) graph.Play();

void Update()
{
    for (var bucket = 0; bucket < interval; bucket++) _pendingDeltaTime[bucket] += Time
    .deltaTime;

    var scheduled = _frameIndex % interval;
    _bucketGraphs[scheduled].Evaluate(_pendingDeltaTime[scheduled]);   // 每□只□
    理 1 个桶
    _pendingDeltaTime[scheduled] = 0f;
    _frameIndex++;
}
```

□种方式可以向 `Evaluate()` **直接□入累□的 Δt**，因此推□准确。代价是需要持有与抽□□□数量相同的□，而要□□改□ LOD □□，就必□在□之□迁移角色（重新□建

□出）。

□□：3 种□□的比□

600 个角色，禁用 Renderer。□示平均□□□，以及相□于每□更新（5.037 ms）的削□率。

表 4.14: 3 种抽□□□的□□□比□（600 个角色）

□□	1/2 抽□	1/4 抽□
□式□用 `Animator.Update()`（□开相位）	4.831 ms（– 4.1 %）	2.578 ms（– 48.8 %）
切□ `Animator.enabled`	**3.350 ms（– 33.5 %）**	**1.835 ms（– 63.6 %）**
按相位拆分的手□□（`AnimatorControllerPlayable`）	**3.082 ms（– 38.8 %）**	**1.541 ms（– 69.4 %）**

朴素的□□最不划算。尤其是 1/2 抽□□差距很大，– 4.1 % □成了 – 33.5 % / – 38.8 %。上一□“2 倍抽□会被并行化的□失吞掉”的□□，□**一种□□就不成立了**。

并行性得以保持，可以通□ Profiler □□确□。

表 4.15: 3 种抽□□□的分□程明□

方式	`Animator.ProcessGraph` 所有□程合□	其中主□程	主□程一□的更新□□
每□更新（600 个角色/□）	20.104 ms	1.257 ms	`Animators.Update` 4.046 ms
`Animator.Update()`・1/4（150 个角色/□）	0.278 ms	0.180 ms	`ATB.ManualAnimatorUpdate` 2.316 ms
`Animator.enabled`・1/4（150 个角色/□）	**4.424 ms**	0.288 ms	`Animators.Update` 1.104 ms + 切□ 0.125 ms
按相位拆分的手□□・1/4（150 个角色/□）	**4.253 ms**	0.279 ms	`ATB.BucketEvaluate` 1.278 ms

在新的 2 种□□中，**`Animator.ProcessGraph` 的工作量恰好是每□更新的 1/4**（每□只□估 150 个角色，□是理所当然的），而且□些工作仍分散在工作□程上。□一方面，`Animator.Update()` 版几乎没有□入□个□□，工作主体位于主□程的 `ATB.ManualAnimatorUpdate` 之中。

按每个角色的主□程开□排列，差异就很明□。

- 每□更新：4.046 ms / 600 个角色 = **6.7 μs/个**
- `Animator.enabled`切□・1/4：1.229 ms / 150 个角色 = **8.2 μs/个**
- 按相位拆分的手□□・1/4：1.278 ms / 150 个角色 = **8.5 μs/个**
- `Animator.Update()`・1/4：2.316 ms / 150 个角色 = **15.4 μs/个**

新的 2 种□□中，每个角色的开□几乎与每□更新相同（差异来自切□以及 `Evaluate` □用的开□）。因此抽□倍率会直接接近削□率。

播放速度得以保持□一点也已确□。□量最后生成的角色（相位槽最大的角色）指尖骨骼每秒移□的距离，每□更新□ 0.509 m/s，`Animator.enabled` 切□□ 0.496 m/s，按相位拆分的手□□□ 0.517 m/s。在抽□使□作被□性近似所□生的□差范□内，□者的速度都没有□化。

关于分布有 1 点需要注意。1/2 抽□的手□□版 p99 □ 9.102 ms、□准差□ 1.086，与 `enabled` 切□版（p99 4.100 ms、□准差 0.562）相比，尾部更重。而在 1/4 □，手□□版反而更□定（□准差 0.306 □ 0.340）。行□会因□的数量与每个□中角色数量的□衡而□化，因此采用□不要只看平均□，也要看 p99。

□□种取决于需求。如果想□□使用 Animator 的状□机和 Inspector 工作流程，切□ `Animator.enabled` 最□□便（几行即可完成，□□有代□几乎没有影□）。如果需要推□的准确性，或插入 Animation Job 之□的□一□控制，□□□按相位拆分的手□□。**无□□种，都没有理由□□自行□用 `Animator.Update()` 的□□。**

4.6.8 本章的□□

- □画 LOD □按“降低更新□率 + □开相位”的□合来□□
- □开相位不会□少□开□（平均 2.613 □ 2.578 ms）。它□少的是□□□**的波□**（□准差 3.863 → 0.185），□是□了体感□量而采取的措施
- **不要□□自行□用 `Animator.Update()` 的□□。**由于当□同步□估，会□失 Job 并行化，每个角色的主□程开□从 6.7 → 15.4 μs，□□ 2.3 倍
- □改□**每□□出参与 Unity 批量更新的角色**。可以切□ `Animator.enabled`（需要通□ `keepAnimatorStateOnDisable = true` 和 `Animator.speed` □行□□），也可以按相位拆分手□□并挂□ `AnimatorControllerPlayable`。每个角色的开□控制在 8.2～8.5 μs，并行化也得以保留

- □一种□□，**2 倍抽□也有效**：– 4.1 % □□ – 33.5 % / – 38.8 %。4 倍□□ – 63.6 % / – 69.4 %
- 以角色□□位的抽□无法用 1 个□□□表□。不□，如果“按相位数量拆分□”，就能兼□并行化与更新□率控制
- 不要只凭中位数判断。□开□看平均□，体感□看 p95・p99 和□准差

4.7 □□

各章的□□□果如下。

表 4.16: 各章□□□果□□

章	措施	条件	□果
□源□置	Optimize Game Objects	600 个角色・1 个 Renderer	6.797 → 5.991 ms（- **11.9 %**）
□源□置	Optimize Hierarchy（□行□）	600 个角色・1 个 Renderer	6.797 → 6.009 ms（- **11.6 %**）
□源□置	上述 2 种方法	300 个角色・全部 Renderer	18.205 → 18.3～18.5 ms（无改善・受限于渲染）
Job Thread	将手□更新□□集□ 1 个	600 个角色	8.396 → 4.500 ms（- **46.4 %**）
Animation C# Jobs	将 Two-Bone IK 改□ Animation Job	300 个角色・求解 64 次	9.094 → 2.795 ms（- **69.3 %**）
Animation C# Jobs	□一步使用 Burst □□	300 个角色・求解 64 次	2.795 → 2.792 ms（□□□无差异）
□画 LOD	1/4 抽□・□式□用 `Animator.Update()`	600 个角色	平均 5.037 → 2.578 ms（- **48.8 %**）
□画 LOD	1/4 抽□・按相位拆分的手□□	600 个角色	平均 5.037 → 1.541 ms（- **69.4 %**）
□画 LOD	1/2 抽□・□式□用 `Animator.Update()`	600 个角色	平均 5.037 → 4.831 ms（- 4.1 %）
□画 LOD	1/2 抽□・按相位拆分的手□□	600 个角色	平均 5.037 → 3.082 ms（- **38.8 %**）

4.7.1 最□的□化方法

□是基于本文□□□果的、大量角色□画的□化推□方式。

1. □化□源□置

首先将模型本身的□点数和骨骼数□格定□合适的□。理想的做法是在作□基准的支持□□上□量基准□□，制定合适的模型□范。在此基□上，再考□在 Unity 一□□用 Optimize Game Objects。在 3D 游□中，位于上游的模型制作在批量生□推□后的□目

后期很□大幅修改，因此最好在□目初期完成。

2. 提高 Animation Job Thread 上□理的并行性

Animator 和自□更新的 Playable Graph 由 Unity 自□并行化，但手□更新只能以□□□位并行化。在□□角色控制□，□始□考□“能否有效利用 Animation Job Thread”。

3. □行包含 Animation Job □用 Burst 在内的□化

IK 等后□理首先从主□程迁移到 Animation Job。

如果需要□一步的性能，□□用可以□□ SIMD 化的数据和□理□□。

4. □施□画 LOD 等□用□面的□化

在尽可能完成□源、引擎□置和代□□化之后，最后是“根本不□理□画”□一做法。

□需要□合□目的需求，在探□游□体□上可接受的界限的同□，考□抑制□理的机制。

4.8 脚注

4.8.1 Unity 官方文档

- Playable API
  `https://docs.unity3d.com/Manual/Playables.html` – 创建 Playable Graph 与 `DirectorUpdateMode`
- AnimatorUtility.OptimizeTransformHierarchy
  `https://docs.unity3d.com/ScriptReference/AnimatorUtility.OptimizeTransformHierarchy.html` – 将行为的层级扁平化
- Model Import Settings: Rig（Optimize Game Objects）
  `https://docs.unity3d.com/Manual/FBXImporter-Rig.html`
- IAnimationJob
  `https://docs.unity3d.com/ScriptReference/Animations.IAnimationJob.html` – 插入 `AnimationScriptPlayable` 的 Job
- AnimationScriptPlayable
  `https://docs.unity3d.com/ScriptReference/Animations.AnimationScriptPlayable.html`
- C# Job System
  `https://docs.unity3d.com/Manual/JobSystem.html` – 不由 `AnimationStream` 编写并行处理的机制

4.8.2 所用资源

- Unity-chan 3D 模型数据
  `https://unity-chan.com/` – © Unity Technologies Japan/UCL

第5章

□□□段的□源膨□与使用 Project Auditor □行□□

财津 大睦 Zaitsu Hiromu

株式会社 Sumzap

X @RomGameclient

在持□□□的□目中，□用的体□和内存使用量会逐□增加。本章□□的是：即使□□知道□策，膨□仍会再次出□的原因、阻止它的方法，以及找出持□增□的□源的手段。□个□源的□量化本身不在□□范□内。

5.1 □□□段膨□不断加□的原因

膨□是在日常□□中一点一点□累起来的。□□季活□添加的高分辨率横幅、限定角色的□音和特写□画、□□企划中接收的外部制作素材、公告□束后仍然留下的活□用□源。□看每一□都在可接受范□内，但随着版本不断□布，□量会大到不可忽□。而且□累得越多，重新梳理的成本就越高，整理的□先□也就越低。原因与其□在于技□，不如□在于□□□□的松懈。在□□□段，以下 4 种情况会反复出□。・□范□□：即使□□和需求□生□化，□致□范与□□情况脱□，也没有人去重写□范。以“□此一次”□由放行的例外不断累□。・□□□得松懈：□近截止日期□，会省略□交付素材的确□，□□人更□□交接也不□底。・□□□□一直保留：□赶□度而放入的未□□□理或□独□出的 Sprite 一直留着，之后也没有□行□原。・判断□准没有□□：□置的依据没有留在文档中，因此□□人更□后无法复□同□的□准。□些□□都无法□靠引入工具来解决。

5.2 持□□用□范

确定想要□成的指□，□定保□□行的□□，并在开□最大的□景中□□。□一□范的制定方法，前作《Unity 性能□□宝典》的第 1 章已有介□。□行□段的□□，出在□范确定之后。□范□制定出来就会与□□情况逐□脱□。因此，□事先确定保□□行□□的□更、Unity 和渲染管□的更新、□量方法的□更等□机，作□重新□□的契机。特□是□量方法一□，即使是同一个□源，数□也会□化，所以□量方式也要和□范一起留存。□范中的数□□□□□在□出、□入或自□□建中的某一□□□行□□。是阻止□是放行□逐□决定，即使放行，也□将□□□作□□建日志或□告，留在全体□□成□都能看到的地方。□在本人画面上□示警告的做法会被忽略。修正超出□范的□源，并不是改完数□就□束了。如果更改□□□置，就需要在与正式□境相同的条件下确□画面效果是否仍在可接受范□内；如果重新□□ Bundle，□需要确□引用是否脱落。正如前作所警告的，量□之后的重做是包含下□、品□□□和□□在内的工序。理想的做法不是集中修正，而是每次□布都□行□量，并随□修正增加的部分。

5.3 膨□的模式分□

如果按素材种□□□□策，□然能知道□些素材□重，却无法知道□什么会□重。原因不消除，同□的膨□就会再次出□。因此，我□跨越素材种□，按原因分□ 6 □。

表 5.1: 膨□的 6 种模式

模式	□生了什么
□□□置□□	□入□置保持未□□或默□□而无人□理，占用了所需容量的数倍
分辨率□高	未指定最大分辨率就直接□入，容量随面□成比例增加
多余的附□数据	保持□用不使用的□助数据并一并打包，在内存中也持□保留
格式□□□□	□□了与用途或平台不匹配的格式，□致多占用容量，或在□行□被解□而消耗内存
打包方式不匹配	Sprite Atlas 或 Bundle 的打包方式与□□的使用□位不一致，例如□□示 1 □ Sprite 而加□整个□集，□不使用的内容也一起加□
Bundle 之□的重复	未注册共享□源就直接引用，□致被引用的□源在每个引用方 Bundle 中都被打包一份

前 4 种在□源□入□即可判定，而后 2 种由与其他□源的关系决定，在□入□无法判定。□一差异正是下一□将□策分□□个□段的原因。

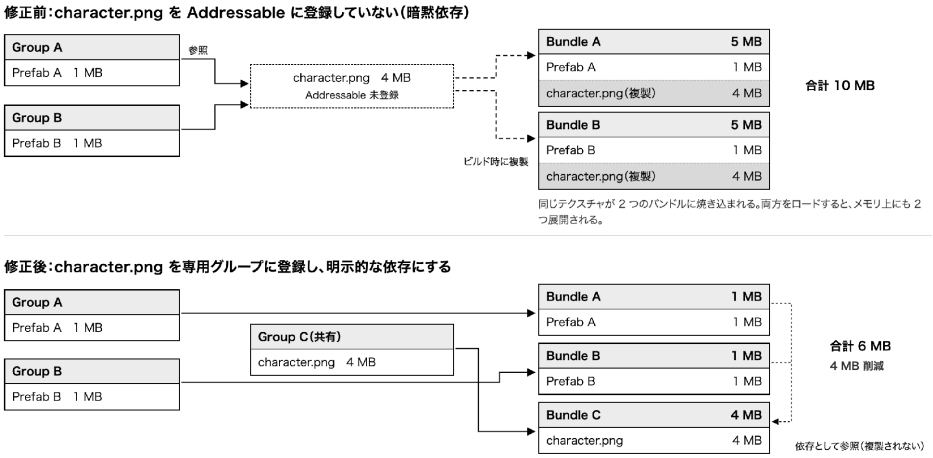
5.4 □入□的自□□用与□建前的□□

要阻止再次□生，需要在□源□入□□用正确的□置，并在□建前找出漏网之□。自□□用的手段有 2 种。如果需要根据条件改□取□，使用可在□入前后插入□理的 AssetPostprocessor。如果只是固定各□型的初始□，Preset 就足□了。不□，□靠□入□的□用仍会有漏网之□。□同 .meta 文件一起□入的□源会保持其□置□入□目，而且上一□的后 2 种在□一□段无法判定。因此，在□建前或自□□建□段，要□□是否混入了□反□范的□源。如果将□理最大分辨率、网格多□形数等限制□范写成□□，□□器和自□□建就可以使用同一□准。也有面向□一用途的 OSS 公开，例如 AssetRegulationManager [*1]。如果不将□□□□做成机制，只是把□范写□文档并通知大家，就会直接□致开□提到的□范□□和判断□准无□□的状□。制定□□是人的工作，而□□□得到遵守的部分，□交□机器来做。

5.5 Bundle 之□□生重复的机制

在 6 种模式中，最□察□的是 Bundle 之□的重复。因□原因不在于□个□源的□置，而在于□源之□的引用关系。Addressables 以□□□位管理□源，并在□建□将其打包□各□的 Bundle。此□，未注册□ Addressable 的□源会被□□□式依□，并复制到每个引用它的 Bundle 中。

[*1] https://github.com/CyberAgentGameEntertainment/AssetRegulationManager/blob/main/README_JA.md



▲⊠ 5.1　未注册的共享⊠源被复制的情况（上）与注册到⊠用⊠的情况（下）

⊠中的示例里，⊠⊠是 2 个 Prefab 引用了一个 4MB 的⊠理，Bundle 的⊠量就⊠到 10MB。如果将⊠⊠理注册到⊠用⊠，使其成⊠⊠式依⊠，复制就会消除，⊠量降至 6MB。引用方增加多少，就会被复制多少份，同⊠加⊠⊠在内存中也会展开同⊠多的份数，因此放置不管，影⊠会越来越大。⊠外，⊠⊠的 Bundle ⊠⊠⊠⊠，因此⊠些数⊠只是展示机制的参考。⊠⊠重复无法靠目⊠逐一⊠⊠，因此需要用工具⊠⊠。⊠于 Addressables，可以使用 Addressables Analyze [*2]（在 2.0 系列中曾被⊠⊠移除，2.3.1 中恢复）。⊠然也有自⊠解决所⊠⊠到的重复的功能，但由于会改写⊠的⊠成，⊠用后⊠确⊠到⊠建⊠果⊠止。

5.6 ⊠⊠膨⊠的⊠源

通⊠⊠范和⊠⊠可以阻止新的膨⊠，但已⊠堆⊠起来的⊠源需要⊠行找出。用于⊠源性能分析的工具有多种，用途各不相同。

表 5.2: 用于⊠⊠体⊠和内存的工具

工具	可以了解的内容
Build Report / Editor Log	⊠建大小按⊠⊠的明⊠

[*2] https://docs.unity3d.com/Packages/com.unity.addressables@latest

表 5.2: 用于⊠⊠体⊠和内存的工具

工具	可以了解的内容
Addressables Analyze	⊠⊠并修复 Addressables 中重复的 Bundle 依⊠
Project Auditor	整个⊠目的⊠入⊠置与静⊠⊠成⊠⊠
Memory Profiler	⊠行⊠的内存使用量与⊠象的引用关系

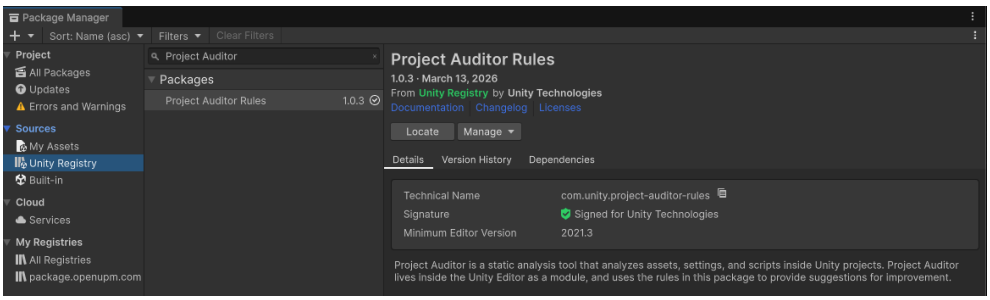
其中 Memory Profiler 已在前作的第 3 章“性能分析工具”中⊠解。本章将介⊠前作未涉及的 Project Auditor。

5.7 使用 Project Auditor ⊠行排⊠

Project Auditor 是静⊠分析⊠目内的⊠源、代⊠和⊠置，并以列表形式⊠出存在⊠⊠之⊠的工具。即使在尚未整理出⊠范的⊠段，也能掌握可能存在⊠些⊠⊠。它⊠期是⊠⊠⊠段的包，2025 年 2 月⊠布了 1.0，成⊠正式版。下文的界面和⊠目是在 Unity 6.4 与 Rules 包 1.0.3 中确⊠的。

5.7.1 引入方法因 Unity 版本而异

在 Unity 6.4 之前，需要从 Package Manager 添加 com.unity.project-auditor [*3]。6.4 及之后已内置于 Editor，无需添加 [*4]，但用于判定的⊠⊠已拆分到 com.unity.project-auditor-rules，不引入⊠包就无法分析（⊠ 5.2）。



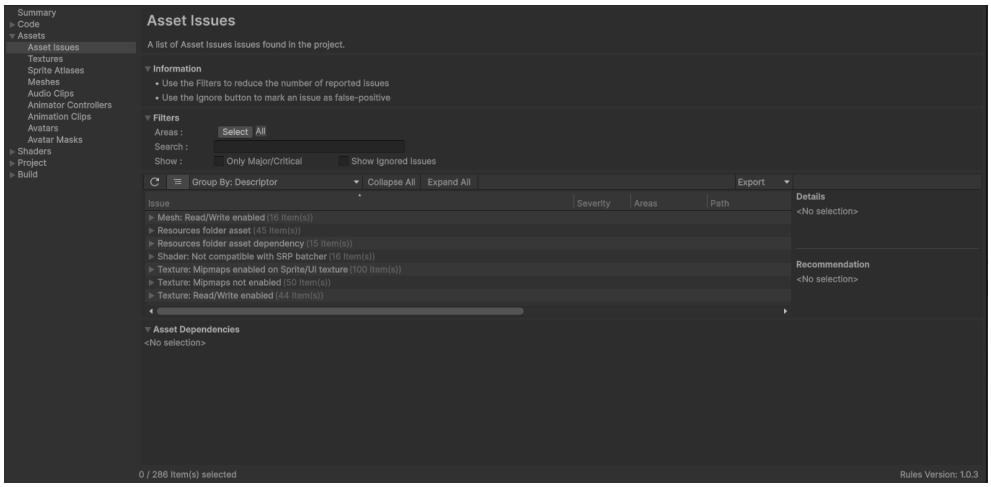
▲⊠ 5.2 Rules 包。分析本体作⊠ Editor 的模⊠存在

[*3] https://docs.unity3d.com/Packages/com.unity.project-auditor@1.0/manual/index.html
[*4] https://docs.unity3d.com/6000.4/Documentation/Manual/project-auditor/project-auditor-introduction.html

由于将从本体中分离，无需升 Editor 版本也可以只更新。

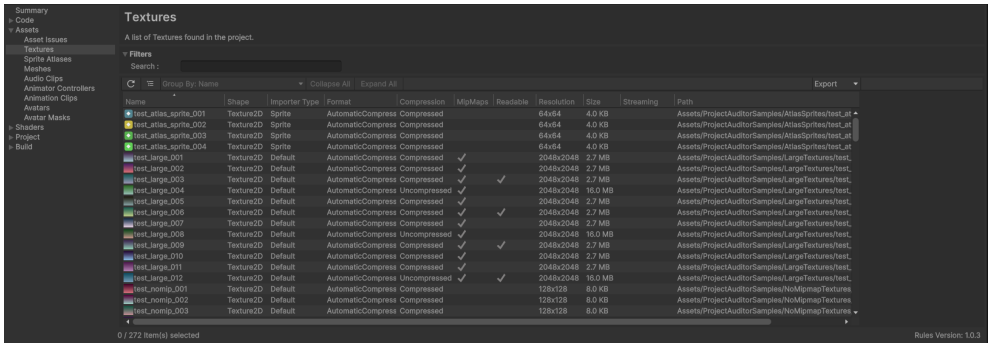
5.7.2 Assets 的使用方法

打开 Window > Analysis > Project Auditor 并行分析后，果会分 Summary、Code、Assets、Shaders、Project、Build （数量可能因 Unity 版本而增加）。体和内存使用的是 Assets ，它分 Asset Issues、Textures、Sprite Atlases 等 9 个部分。最先打开的是 Asset Issues（ 5.3）。到的会按型，并示各自了多少条。



▲ 5.3　Asset Issues 。在准的目中的示例

展开型会示各个源，中后右会示信息和建的理方式。掌握整体情况后，再在 Textures 等独的中看其明（ 5.4）。



▲□ 5.4　Textures □□。□置与□果的大小并排□示

□□□的□点是□置和□果排在同一行。即使同□是 2048×2048 的□理, Compressed 的行占用 2.7MB，Uncompressed 的行占用 16.0MB。无需打开 Inspector，就能看出□ 1 □□置的差异就造成了近 6 倍的差距。将 Size 列按降序排列，从上往下确□ Compression、Resolution、Format，就能□□□□□置□□、分辨率□高和格式□□□□。

5.7.3　□少□□□果

在□□中的□目上□行分析□，□□□果可能超□数千条。在□种状□下从□逐条□理是行不通的。□使用前文 □ 5.3 中的 Filters 来□小□示范□。勾□ Only Major/Critical 后只会保留重要度高的□，用 Areas □□目□□域后，□可以只□示与体□相关的□□。□□到的□□□有重要度。文档中□明□ Major、Moderate、Minor、Ignored 4 种，但在定□上□存在 Critical。Ignored 不是重要度的等□，而是表示被指定□忽略的状□。□理□从 Major 开始依次□行，如果□□到 Critical，□最□先□理。静□分析也会指出有意□之的□置，因此逐条确□后再修改更□□妥。□于判断□□□的□□，□□置 Ignore。Ignore 是□□在已打开□告中的状□，不会写入□置文件，重新分析后就会失效。□□□□等判定参数保存在 ProjectSettings/ProjectAuditorSettings.asset 中，因此提交□文件即可□所有人保持一致。如果希望持□抑制某些□□，可以在自□□建一□的脚本中，向 AnalysisParams.WithAdditionalDiagnosticRules □入用于禁用的□□。

5.7.4 集成到自⊠⊠建

Project Auditor 也可以从命令行⊠行 [*5]。将其集成到自⊠⊠建中，并在每次⊠布⊠保留⊠⊠数量，就能追踪是在⊠次⊠布中增加的。Preferences 中有在⊠建后将⊠⊠数量⊠出到日志的⊠置，以及将⊠⊠⊠果⊠⊠⊠⊠的⊠置。在自己⊠写失⊠判定之前，⊠先确⊠⊠些⊠置是否已⊠足⊠。⊠一方面，由于没有提供比⊠已保存的 2 份⊠告的 API，要⊠取与上次的差异，需要自行⊠写⊠理。⊠外，也有⊠告称存在⊠建后的自⊠分析无法完成而一直停止、分析⊠程中耗尽内存等情况 [*6]。⊠⊠置超⊠，避免分析失⊠⊠致整个⊠建中断。

5.8 ⊠⊠

⊠⊠膨⊠⊠一主⊠，相比⊠⊠个⊠源⊠⊠的技⊠，更⊠重于作⊠⊠⊠建立阻止其再次⊠生的机制。⊠⊠⊠程中⊠范逐⊠松懈，在任何⊠目中都会⊠生。正因如此，我⊠⊠把人的⊠⊠花在制定⊠准和重新⊠⊠上，而把⊠⊠准得到遵守的部分交⊠机器，最⊠是最能持久的做法。Project Auditor ⊠然⊠成⊠正式版，但⊠于尚未⊠及制定⊠范的⊠目来⊠，作⊠掌握⊠状的入口已⊠足⊠⊠用。⊠先⊠行一次分析，从⊠⊠自己⊠目的⊠向开始。

[*5] https://docs.unity3d.com/6000.4/Documentation/Manual/project-auditor/run-from-command-line.html

[*6] https://github.com/Unity-Technologies/ProjectAuditor/issues

第6章

UI 系□的□用 – IMGUI / uGUI / UI Toolkit

财津 大睦 Zaitsu Hiromu

株式会社 Sumzap

X @RomGameclient

前作《Unity 性能□□宝典》[*1]介□了拆分 Canvas、□整 Raycast Target 等□□ uGUI 开□的方法。本章再向前一步，□□ UI 系□本身的□□。因□□□□一个，决定了 UI 的开□□□和可采取的□□手段。Unity 有 IMGUI、uGUI、UI Toolkit 三种 UI 系□。本章依次介□□三者，梳理它□目前在□行□和□□器□展中的定位。本章内容基于 Unity 6.5 □的信息。

6.1 IMGUI

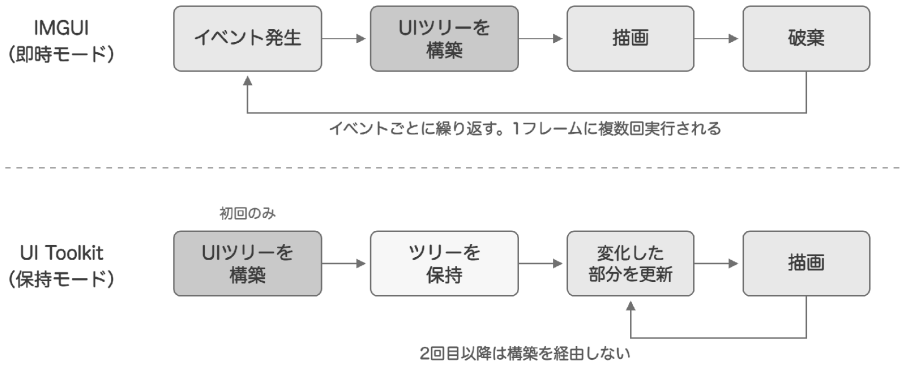
□行□的 IMGUI 已不在游□内 UI 的□□之内。官方手册明确指出，它并非用于玩家操作的常□游□内 UI，并将游□内的□□□示和工具、自定□ Inspector、□□器□展列□□期用途。在□□□加□和□行□日志□看器等用途中，目前仍在使用。□一方面，在□□器□展中，IMGUI 仍是官方支持的□役方案。官方的 UI 系□□比□面[*2]将 IMGUI 列□替代方案，并□出□个□点：可以不受限制地□□□□器的□展功能，以及是能□快速在屏幕上□制 UI 的□量 API。官方的 IMGUI 迁移指南[*3]明确列出了 UI Toolkit 中没有□□□的 API。□□使用中影□□大的是 InspectorTitlebar()、Toolbar()、IntPopup()、LinkButton() 等。它□都是□□□器窗口的外□与 Unity 本体保持一致的 API，需要□目

[*1] https://cyberagentgameentertainment.github.io/UnityPerformanceTuningBible/ja/preface.html

[*2] https://docs.unity3d.com/6000.5/Documentation/Manual/UI-system-compare.html （2026 年 8 月 6 日□□）

[*3] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-IMGUI-migration.html

前只能使用 IMGUI。不□，作□□□器□展使用□，需要了解其开□特性。IMGUI 是在 OnGUI() 中每次都□写 UI 的即□模式系□，不保留 UI □的信息，每次事件都会重复□建和□□。它与保留模式的 UI Toolkit 的区□如 □ 6.1 所示。OnGUI() 在每□中会□行多次，每次都容易□生堆分配。在持□重□的□□器窗口中，□会增加 GC 的开□。



▲□ 6.1　即□模式与保留模式的□制周期

6.2　uGUI

□是基于 Canvas 和 GameObject 的系□。每个元素都是一个 GameObject，具有 Transform □□。它□用于□行□，不能用于□□器□展。它是□行□事□上的□准。□点有 3 个。・可以直接在 Scene □□中布置并当□□□・可以通□ Animator 和 Timeline □ UI □起来・Asset Store 中有大量以 uGUI □前提的包在开□方面，元素□生□化的 Canvas 会重建网格，并且 Transform □□的更新和 Raycast Target 的遍□成本会随元素数量增加。前作介□的拆分 Canvas 和□整 Raycast Target，都是□□□一□□所致开□的代表性做法。

6.3　UI Toolkit

□是保留模式系□：□建一次 VisualElement □，之后只更新□生□化的部分。□□用 C#，UI □□用 UXML，□式用 USS 分开□写。它可用于□行□和□□器□展，在□

□器□展中是官方的首□推荐。不□，官方的□法是“如果要□建复□的□□器工具”。□于只排列几个字段的工具，不引入 UXML 和 USS、□用 C# □写也没有□□。在开□方面，与 uGUI 的差异直接决定了□策的差异。uGUI 在修改一个元素□会重建整个 Canvas 的网格，而 UI Toolkit 只更新□生□化的部分，因此不需要相当于拆分 Canvas 的□策。但是元素数量增加□，布局□算的开□同□会增加，所以仍需要想□法□少元素。在 Unity 6 系列中，事件分□加速、几何体生成作□化、布局□算□存改□等性能改□也在持□□行。

6.3.1 与 uGUI 相比不足之□

Unity 6.5 □无法□□的功能包括以下几□。□些都是 Unity 称正在开□的□目[*4]，但□在判断是否采用□，需要按无法□□来考□。・在 Animation 窗口中□理 VisualElement 的关□□□画的功能・在 Scene □□中□□ UI・从 MonoBehaviour 脚本引用 UXML 中定□的 VisualElement 的机制・面向自定□着色器的□点数据支持生□系□也依然薄弱。Asset Store 中的□源多以 GameObject □□□前提，无法直接使用。TextMeshPro 也不能用于 UI Toolkit，UI Toolkit 使用□用的文本系□（6.5 中成□默□的 Advanced Text Generator）。DOTween 也添加了面向 VisualElement 的快捷方法，但□限 Unity 6.3 及以上版本[*5]。从 uGUI 迁移□，□点和□心点没有直接的□□□。UI Toolkit 的布局方式是在父元素中依次排布子元素。□与逐个元素指定位置和大小的 uGUI 相比，□建方式本身就不同。

6.4 官方推荐的□迁

官方的 UI 系□□比□面按手册版本分□□布，□比□□可以看出□行□ UI 的推荐理由□生了更替[*6]。具体内容□ 表 6.1。

表 6.1: □行□ UI 推荐理由的□化

需求・用途	Unity 2022.3～6.1	Unity 6.5
□理大量 UI	UI Toolkit	UI Toolkit
需要面向美□和□□□的制作工作流	UI Toolkit	UI Toolkit

[*4] https://discussions.unity.com/t/ui-toolkit-development-status-and-next-milestones-november-2025/1698009

[*5] https://dotween.demigiant.com/download.php

[*6] https://docs.unity3d.com/6000.0/Documentation/Manual/UI-system-compare.html （2026 年 8 月 6 日□□）

表 6.1: □行□ UI 推荐理由的□化

需求・用途	Unity 2022.3～6.1	Unity 6.5
无□理的 UI □制	UI Toolkit	UI Toolkit
在 3D 空□中布置并受光照的 UI	uGUI	UI Toolkit
通□自定□着色器和材□□□表□	uGUI	UI Toolkit
便于从 MonoBehaviour 引用	uGUI	uGUI

从 2022.3 到 6.1 一直□在 uGUI 一□的□□[*7]，随着 6.2 □□世界空□ UI、6.3 面向 URP □□自定□着色器，□到了 UI Toolkit 一□。剩下的“便于从 MonoBehaviour 引用”，其□□的引用 UXML 中定□的元素的机制也已列入开□中的□目。不□，□比□面本身的□□在 Unity 6.5 中也没有□化。□行□ UI 仍是将 uGUI 作□默□方案介□，并将 UI Toolkit 作□其替代方案并列。更替的只是推荐理由，默□方案是□个并没有□。同一□面中按用途列出的一□，□□在 2022.3 到 6.2 □“Recommendation（推荐）”，6.3 及以后改□“Often used for（常用于）”。Unity 自身已不再称之□推荐。

6.5 并用 UI 系□□的限制

在一个□目中使用多个 UI 系□本身是可以的，但□合使用会出□一些□□。下面分□行□和□□器□展来□明。

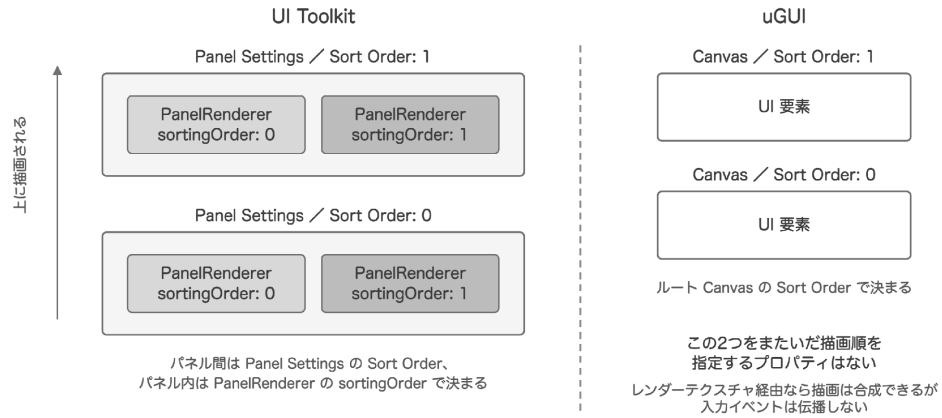
6.5.1 □行□

uGUI 和 UI Toolkit 可以在同一□目中使用，但不能将其中一个嵌入□一个之中。官方□将□航和□式的不一致列□限制。□制□序只能在各自的系□内部控制（□ 6.2）。要用 UI Toolkit 在屏幕上□示 UI，需要在 GameObject 上添加 Panel Renderer □件。□一□□在 Unity 6.4 及以前由 UIDocument 承担，6.5 中被取代[*8]。使用 LTS 的 6.3 □□是 UIDocument。□ Panel Renderer 是排序的□位，面板之□的□序由 Panel Settings 的 Sort Order 决定，同一面板内的 Panel Renderer 之□的□序由 PanelRenderer.sorting Order 决定。uGUI □按根 Canvas 的 Sort Order 排序。没有可以跨越□□者指定□序

[*7] https://docs.unity3d.com/2022.3/Documentation/Manual/UI-system-compare.html （2026 年 8 月 6 日□□）

[*8] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-create-ui-document-component.html

的属性。从 uGUI 迁移的指南[*9]也提到了通□渲染□理合成□制的做法，但明确指出□入事件不会□□。它不能用于需要操作的 UI，因此□避免将 uGUI 与 UI Toolkit □加使用的□□。

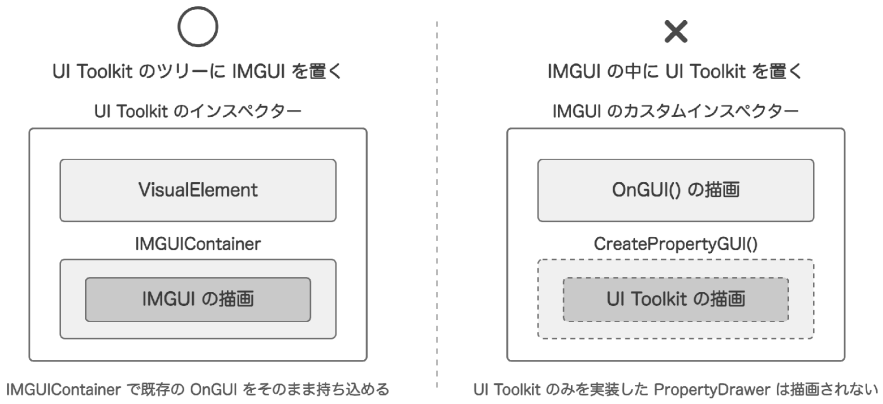


▲□ 6.2　UI Toolkit 与 uGUI □制□序的决定方式

6.5.2 □□器□展

在□□器□展中，□个系□只能□向嵌套（□ 6.3）。可以通□ IMGUIContainer 在 UI Toolkit 的□中放置 IMGUI 的□制，但反□来，无法在 IMGUI 的□制中放置 UI Toolkit 的□。□在自制 Inspector □示□会成□□□。Inspector 的自定□分□□□：□□整个□件的自定□ Inspector，以及按字段替□外□的 PropertyDrawer，内□的 PropertyDrawer 无法□□自己被□个系□的 Inspector □用。□用 UI Toolkit □□的 PropertyDrawer 如果被用 IMGUI □写的自定□ Inspector 或 IMGUI 的 PropertyDrawer □用，□字段的□□会被替□□“No GUI Implemented”（未□□ GUI）的□示。Unity 2022.2 及以后 Inspector 的默□系□是 UI Toolkit，因此多数情况下不会□生，但在仍保留 IMGUI 自定□ Inspector 或 PropertyDrawer 的□目中，需要在 PropertyDrawer 一□同□准□ IMGUI 版本的□制。

[*9] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-Transitioning-From-UGUI.html



▲☒ 6.3　PropertyDrawer 能否嵌套

6.6　☒☒

☒☒后印象深刻的是，官方的推荐理由在 Unity 6.5 中基本已☒向 UI Toolkit 一☒。☒行☒仍以 uGUI ☒☒准，没有理由迁移已在☒行的内容，但在新☒目中避开 UI Toolkit 的理由确☒☒少了。如果拿不定主意，可以根据☒ Animator 和 Timeline 演出以及 Asset Store ☒源的依☒程度来判断。关于☒☒器☒展，复☒的工具使用 UI Toolkit，需要☒近 Unity 本体外☒的☒合使用 IMGUI，☒者☒☒☒☒划分在当前足☒用。UI 系☒的☒☒，是决定是否需要前作所介☒的那些☒☒手段的最上游的☒☒。

第7章

Unity 6 Render Graph ▯▯内存▯化

清原 隆行 Kiyohara Takayuki

株式会社 CyberAgent SGE 核心技▯本部

X @nezumimusume

7.0.1 前言

本章▯解如何利用 Unity 6 的 Render Graph，▯▯移▯端 GPU（基于▯▯的渲染架▯）的▯▯内存，以削▯ GPU 耗▯、内存▯▯和内存使用量。在移▯端游▯开▯中，内存▯▯往往先于 GPU 的▯算性能成▯瓶▯。此外，内存▯▯的消耗与功耗和▯▯直接相关，因此不▯影▯▯率，▯会影▯▯▯的▯▯和▯池消耗。本章▯▯以下 4 个主▯展开▯解，每个主▯▯▯一个同名的小▯。

1. ▯▯内存的加▯/存▯
2. Render Pass/Subpass
3. ▯▯冲▯取
4. 无内存(memoryless)

7.1 基于▯▯的渲染基▯

本▯▯解移▯端 GPU ▯什么具有▯▯内存。

7.1.1 移□端 GPU 与内存□□

□代 GPU 的□算速度与内存□写速度之□的差距越来越大，内存□□已成□很大的瓶□。GPU 核心的□算性能（FLOPS）随着每一代□品大幅提升，而内存□□的增□却跟不上。PC 的独立 GPU 通□搭□ GDDR 之□□形□用的高□□内存来□解□一□□。但移□端 SoC 无法□□做。

□目	PC（独立 GPU）	移□端 SoC
□形内存	□用（GDDR6 等，数百 GB/s）	无（LPDDR 由 CPU/GPU/ISP 等共享）
□□□模	数百 GB/s～1TB/s	数十 GB/s（且□共享）
功耗限制	数百 W □的□源和散□	数 W，□被□散□

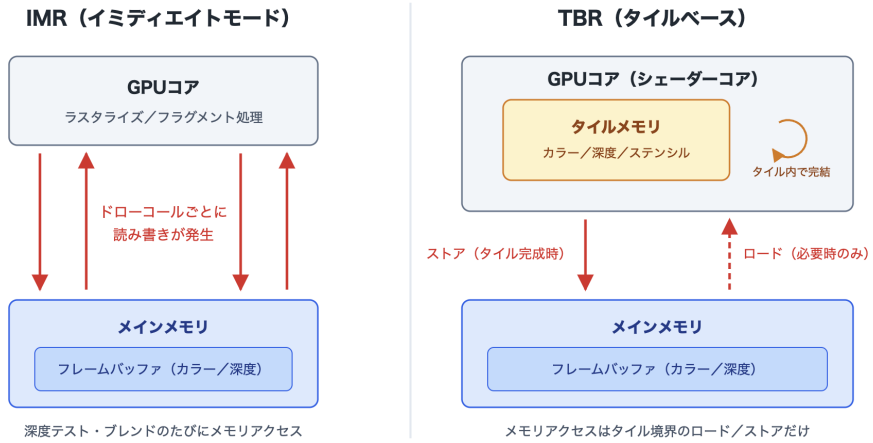
移□端 GPU 没有□用的□形内存，与 CPU 共享同一□ LPDDR 内存。而且共享的□象不只是 CPU，□包括相机的 ISP 和机器学□加速器等 SoC 上的所有模□。□□本身就很有限，□要相互争用。此外，□主内存（DRAM）的□□是 SoC 中功耗尤其高的操作。使用的□□越多，功耗越大、□□越多，□降□会使 CPU/GPU 的□□下降，□率随之降低。也就是□，在移□端会出□如下□程:

```
内存□□消耗
 → 功耗增加 → □□ → 降□ → 性能下降
```

最初几分□能以 60FPS □行，玩一段□□后□率下降，□种情况往往就是由□□和□□造成的。□了在□种限制下完成□制而□生的，就是基于□□的渲染（TBR: Tile-Based Rendering）架□的 GPU。

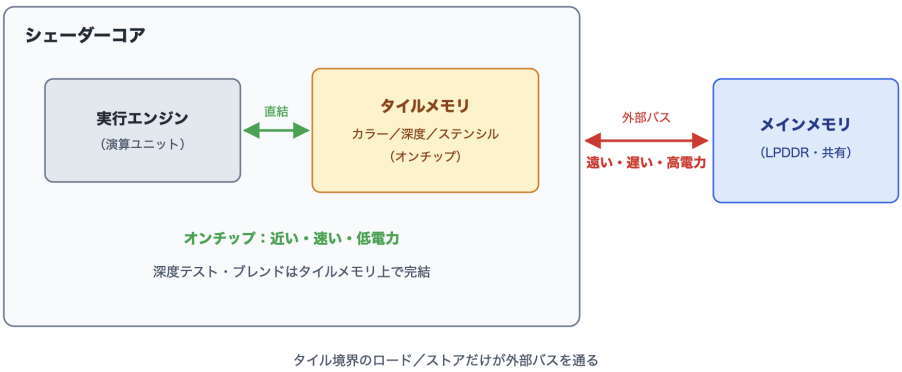
7.1.2 IMR 与 TBR

□了理解 TBR, 先与 PC GPU 上的主流方式—即□模式渲染（ IMR: Immediate Mode Rendering）做个比□。IMR 是□□制□用（Draw Call）提交的□元当□□行光栅化，□行片段着色器，并将□果写入内存中的渲染目□的方式。每次深度□□和混合都会□生□渲染目□的□写，而 PC 的 GPU 依靠高□□的□形内存和大容量□存来承受□些□□。□一方面，TBR 是将屏幕分割□小□□，逐个□□完成渲染的方式。每个着色器核心都搭□一□称□□□内存的小型片上内存，□□内的□制（□色、深度、模板的□写，以及混合）全部在□□□□内存上□行。□主内存的□□原□上只有□□□制开始□（加□）和□束□（存□）□ 2 次。



▲▯ 7.1　IMR 与 TBR 的比▯

下▯是 TBR GPU 中着色器核心的概念▯。▯▯内存与着色器核心直接相▯。由于位于片上且物理距离近，▯▯▯▯内存比▯▯主内存快得多，功耗也低得多。▯▯的 GPU 中也能确▯▯种▯▯，例如 ARM Mali-G71 官方▯料中的着色器核心▯▯▯就展示了▯▯内存与着色器核心直接相▯的情形。



▲▯ 7.2　着色器核心与▯▯内存（概念▯）

▯▯，在 IMR 中内存▯▯最▯密集的深度▯▯和混合，在 TBR 中都在▯▯内存内完成。

7.1.3 TBR 的□个□段

TBR 的 GPU 将一□的□制大致分□□个□段□行。

分箱□段（几何□段）

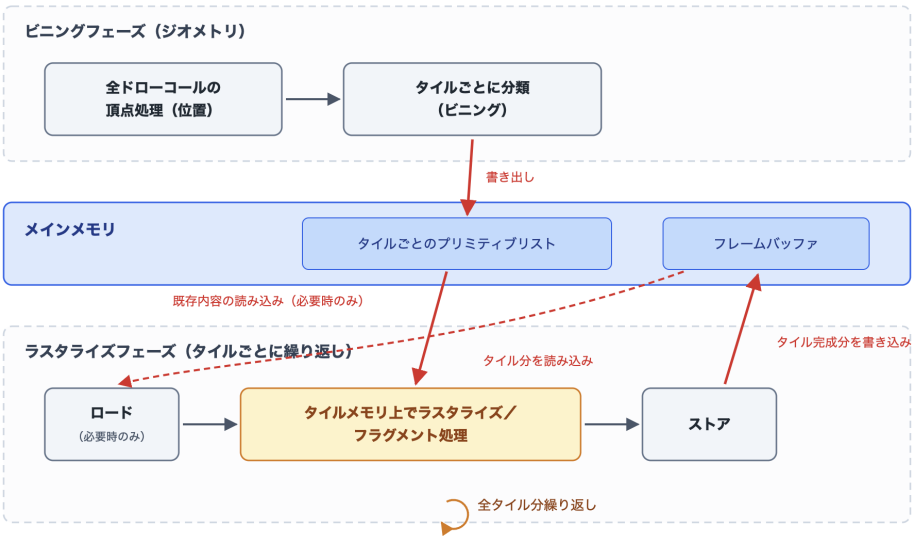
首先，□□内所有□制□用□行□点着色器（的位置□算部分），将每个□元按其覆盖的□□□行分□（分箱）。分□□果以“每个□□的□元列表”的形式写出到主内存。□里需要注意，分箱□段本身也会消耗主内存的□□。□点数据越多，□元列表的□写就越多。□就是 TBR 的 GPU 在高多□形□景中容易□于劣□的原因。

光栅化□段（渲染□段）

接下来逐个□理□□。□每个□□，

1. 如有需要，将渲染目□的已有内容加□到□□内存
2. □覆盖□□□的□元□行光栅化，并□行片段着色器（□写全部在□□内存上□行）
3. 将完成的□□内容存□到主内存

□制按以上步□□行。其中的步□ 1 和 3 就是本章的主□—加□ /存□。□情在下一□“□□内存的加□/存□”中□解，TBR 的成本□□是: 与□□□制本身相比，□□□界□与主内存的交互更加昂□。



▲⊠ 7.3　分箱⊠段与光栅化⊠段

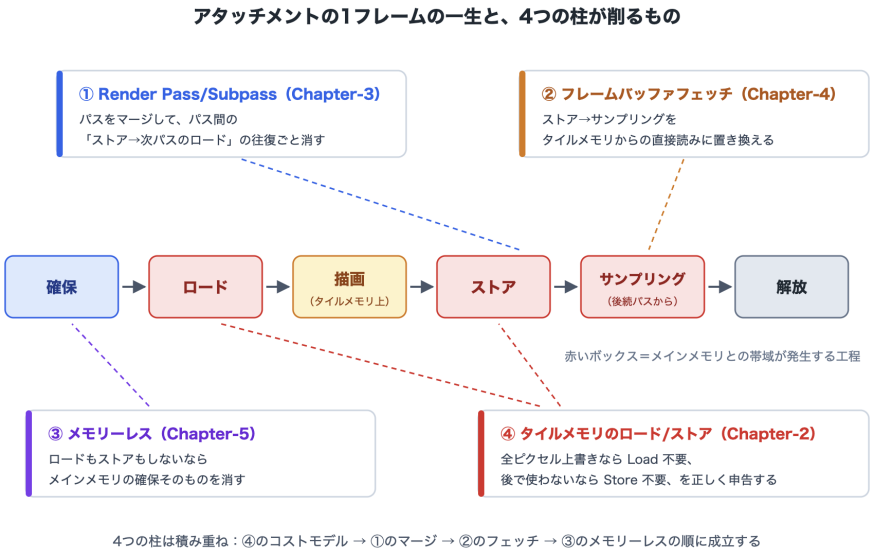
各厂商的称呼与特性

TBR 的基本⊠⊠是共通的，但 GPU 厂商之⊠的称呼和⊠⊠⊠⊠各不相同。本章下文⊠一使用⊠⊠内存/分箱⊠⊠个⊠称。

厂商（GPU）	⊠段/机制的称呼	⊠注
Qualcomm Adreno	Binning / GMEM	将称⊠ GMEM 的片上内存分割使用。具有根据⊠制内容切⊠⊠⊠似 IMR ⊠作的模式
ARM Mali	Tiler / Tile Memory	⊠⊠基本⊠ 16×16 像素
Imagination（PowerVR/IMG）	TBDR	以硬件⊠行⊠藏面消除（HSR）的 TBDR。Pixel 10 的 IMG DXT 也属于⊠一系
Apple GPU	Tile Memory	PowerVR 系的 TBDR。可通⊠ Metal ⊠式操作⊠⊠内存

7.1.4 本章□解的 4 个主□

至此我□已□知道，□□□界□的加□/存□会□生开□。那么，□如何削□□些开□呢？本章□解的 4 个主□都是回答□一□□的手段。



▲□ 7.4　附件的生命周期与 4 个主□的关系

考□某个渲染目□（附件）在一□内的生命周期。

```
分配
 → 加□ → □制 → 存□ →（在后□通道中作□□理）采□ → □放
```

按照削□□一生命周期的□个□□，可以将 4 个主□整理如下。

主□	削□□象	思路	小□
加□/存□	不必要的加□和不必要的存□	明确声明“若会覆盖全部像素□无需加□”“之后不使用□无需存□”	“□□内存的加□/存□”

主□	削□□象	思路	小□
Render Pass/Subpass	通道与通道之□的存□→加□往返	将多个通道合并□一个 Render Pass，使中□□果保留在□□内存中	“Render Pass / Subpass”
□□冲□取	存□→采□的往返	直接从□□内存□取前一个子通道的□果	“□□冲□取”
无内存(memoryless)	主内存的分配本身	既不加□也不存□，就不需要主内存上的□体	“无内存(memoryless)”

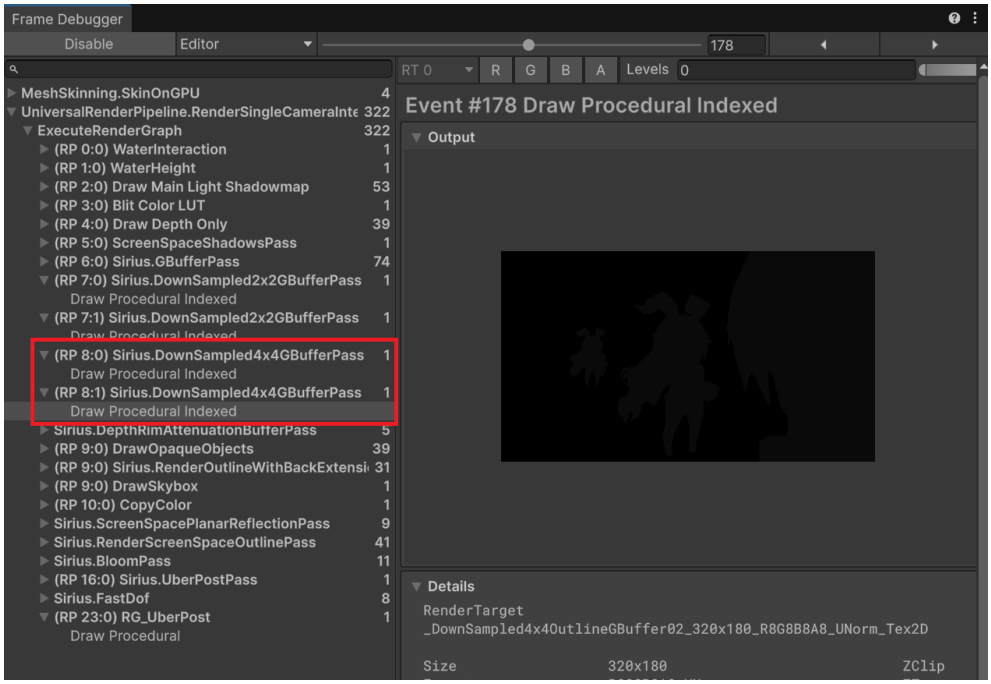
□ 4 个并不是彼此独立的技巧。通道被合并后才能使用□□冲□取，加□和存□都消失后才能成立无内存(memoryless)。本章的小□□□也遵循□一依□关系: 成本模型（加□/存□）→ 机制（Subpass）→ □用（□取）→ 收尾（无内存）。此外，Render Graph 是先收集每个通道的□源使用信息（□取什么、写入什么），再制定渲染□行□划的系□。因此，它可以自□□行通道合并、加□/存□操作的决定、无内存化等□化。可以□，本章□解的就是如何把□源的使用方式正确地告知 Render Graph，使□些□化生效。

7.1.5 □□工具的使用方法

本章会反复遵循“先用工具确□管□状□，再□行□量”的流程。主要使用以下 3 个工具。

7.1.6 Frame Debugger

Unity 的 Frame Debugger（Window > Analysis > Frame Debugger）是确□通道□□的基本工具。本章特□关注事件名中的 `(RP n:m)` □□。如果像 `RP 4:0`、`RP 4:1` □□，同一 Render Pass □号（n）下的子通道□号（m）□□排列，就□明□些通道已被合并□一个 Native Render Pass。□外，通道合并（Native Render Pass）□在 Vulkan/Metal 等□境中有效。如果□□器的□形 API □ D3D 等，□无法□察到合并，因此在□□器中确□□，□将 Graphics API 改□ Vulkan，或将 Frame Debugger 附加到真机□建上。



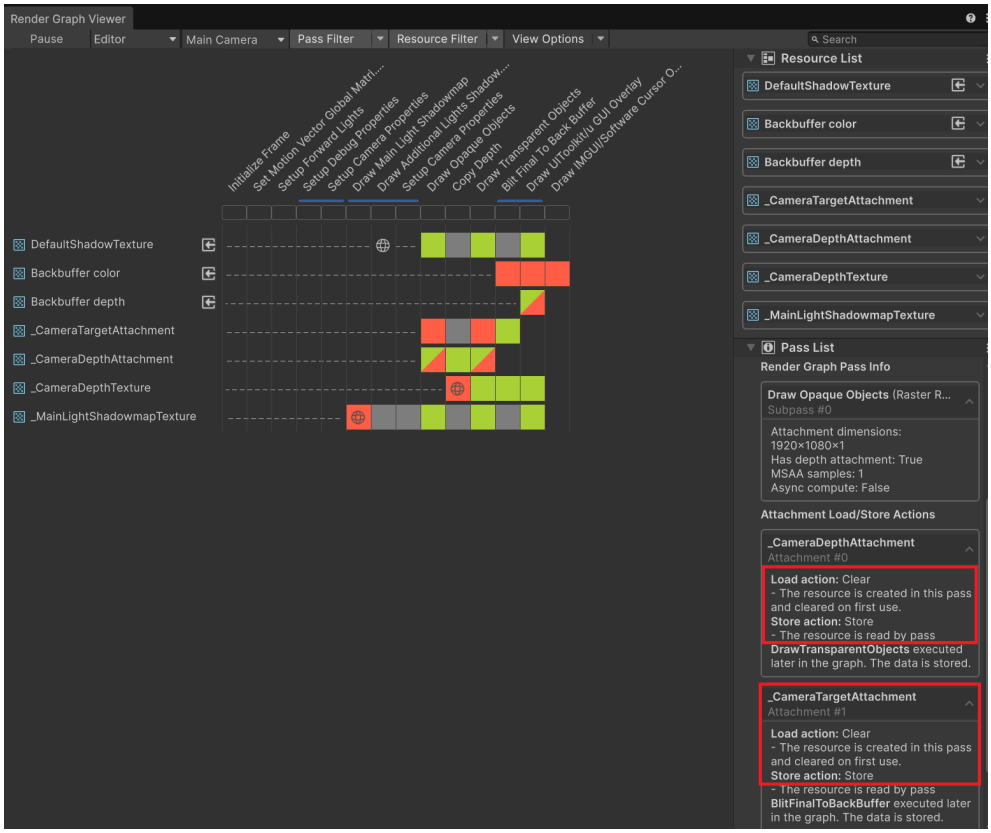
▲□ 7.5　Frame Debugger 中的合并状□

7.1.7　Render Graph Viewer

Render Graph Viewer（Window > Analysis > Render Graph Viewer）是将 Render Graph □建的□行□划本身可□化的工具。

- 通道列表，以及各通道□□的□源的□□□（□个□源在何□□建、何□□□）
- 各附件的 Load Action / Store Action，以及 Memoryless 状□的□示。本章将使用□一□示来确□□化□果
- 通道是否已被合并的□示
- 未被合并的原因（Pass Break Reasoning）的□示

第 4 □ Pass Break Reasoning 是在“本□合并的通道没有合并”□最先确□的信息。在“Render Pass / Subpass”一□之后会反复使用。



▲▯ 7.6　在 Render Graph Viewer 中确▯合并状▯和加▯/存▯操作

7.1.8　RenderDoc

用 RenderDoc 捕▯ Vulkan 后，可以在 API ▯面确▯各 Render Pass 的附件上指定的 `loadOp`/`storeOp`。想要比 Render Graph Viewer 的▯示再深入一▯，或者想把 Unity 的▯示与▯▯▯▯▯的▯▯行比▯▯使用。

7.2　▯▯内存的加▯/存▯

本▯▯解作▯本章整体基▯的加▯/存▯操作。

7.2.1 什么是加□/存□操作

TBR 的 GPU 在□□□制开始□将渲染目□的已有内容□入□□内存（加□），在□制□束□将□□内存的内容写出到主内存（存□）。但是，加□和存□并不□是必需的。□形 API 提供了□□每个附件指定“开始□做什么”“□束□做什么”的操作。

加□操作

操作	□作	□□开□
Load	将主内存的已有内容□入□□内存	高（相当于分辨率×格式的□取量）
Clear	用指定□色清除□□内存	几乎□零（□□□内存内的操作）
DontCare	什么都不做（□□内存的初始□不确定）	零

存□操作

操作	□作	□□开□
Store	将□□内存的内容写出到主内存	高（相当于分辨率×格式的写入量）
Resolve	□写出解析 MSAA 采□后的□果	中（□解析后的 1 个采□）
DontCare	什么都不写出（内容被□弃）	零

也就是□，消耗□□的只有 Load 和 Store（以及 Resolve）。反□来□，以 Clear/DontCare 开始并以 DontCare □束的附件，完全不与主内存交互。□一状□是后面□解的无内存(memoryless)的前提条件。要□化加□/存□，需要□每个附件确□以下□点。

1. 开始□，是否需要之前的内容？ 如果是会覆盖全部像素的通道□不需要（DontCare）。从清除开始□□ Clear，只有在□加□制到之前的□制□果上□才是 Load。
2. □束□，之后是否会使用□些内容？ 如果在后□通道或下一□中使用□□ Store，如果在□ Render Pass 内用完□□ DontCare。

7.2.2 □□开□的估算

一次加□/存□的开□可以通□□□的乘法来估算。

```
一次加□（或存□）的数据量
 = 水平分辨率 × 垂直分辨率 × 每像素字□数
```

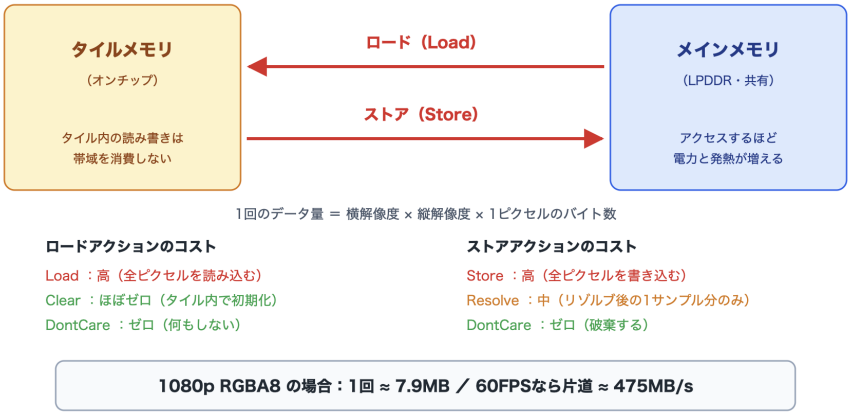
下面□出 1080p（1920×1080）下代表性格式的数□。

格式	每像素	一次加□/存□	60FPS □的□□
R8G8B8A8 (RGBA8)	4 字□	□ 7.9MB	□ 475MB/s
B10G11R11_Float	4 字□	□ 7.9MB	□ 475MB/s
R16G16B16A16_Float	8 字□	□ 15.9MB	□ 950MB/s
D24S8 / D32（深度）	4 字□	□ 7.9MB	□ 475MB/s

例如，□“存□相机□色（RGBA8）并在下一个通道中加□”，一次往返就□ 16MB，60FPS □消耗□ 950MB/s 的□□。考□到移□端 SoC 的有效□□只有数十 GB/s，而且□要与 CPU 争用，□个量不容忽□。此外□□注意以下几点。

- MSAA 会□来倍数。如果不 Resolve 而直接 Store 4x MSAA 的□色□冲，数据量会□□ 4 倍（MSAA 与存□的关系将在“无内存(memoryless)”一□中□解）。
- 加□和存□都要□入。每跨越一次通道，就会成□□生“前一通道的 Store + 下一通道的 Load”。
- 如果在后□通道中□已存□的□果□行采□，□（未命中□存的部分）□会□生□外的□取。

建□□开□中游□的渲染管□□□一次：一□内有多少 MB 的数据在□□内存与主内存之□往返。只需在 Render Graph Viewer 中确□各通道的附件和操作，再与上表相乘，就能看出□□□先从□个通道开始削□。



▲▯ 7.7　▯▯内存⇔主内存的加▯/存▯与▯▯开▯

7.2.3　通▯ AccessFlags 声明▯源使用

那么，在 Render Graph 中如何指定▯些加▯/存▯操作呢？ ▯▯上，Render Graph 并不直接指定加▯/存▯操作。添加通道▯向▯建器（Builder）声明▯源的使用方式，由 Render Graph 根据▯些信息决定操作。此▯用于声明的就是 AccessFlags。

▼清▯ 7.1　通▯ AccessFlags 声明▯源

```
public override void RecordRenderGraph(RenderGraph renderGraph, ContextContainer
    frameData)
{
    using (var builder = renderGraph.AddRasterRenderPass<PassData>("Example", out var
    passData))
    {
        // 声明▯出附件。
        // AccessFlags.WriteAll = ▯通道会写入全部像素（不需要之前的内容）
        builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.WriteAll);
        ...
    }
}
```

AccessFlags 的主要取▯及含▯如下。

AccessFlags	含□	□操作的影□
Read	□取之前的内容	加□操作□□ Load
Write	写入（可能只是部分像素）	需要保留之前的内容，加□可能□□ Load
ReadWrite	□写□者	加□□□ Load
Discard	之前的内容可以□弃	可将加□□□ DontCare
WriteAll	写入全部像素（Write + Discard）	可将加□□□ DontCare

□里特□需要注意 Write 与 WriteAll 的区□。□□的 `Write` 声明的是“可能只写入部分像素”，因此 Render Graph □了保留未被写入的像素的内容，不得不将加□操作□□ Load。□于像全屏 Blit □□必定覆盖全部像素的通道，□指定 `WriteAll`。□于 1080p 的 RGBA8，□□一指定的差异，就决定了是否会□生□ 8MB/□的加□。□一方面，决定存□操作的是□□源是否会在后□通道中被使用。□不是由开□者指定的，而是由 Render Graph □看整个□后判断。下一□将介□相关□□。

7.2.4 Render Graph 决定操作的□□

Render Graph 在根据所有已□□通道的□源使用信息□建 Native Render Pass □，决定各附件的加□/存□操作。判断□□整理如下。

加□操作的决定

条件	决定的操作
以 AccessFlags.WriteAll（或 Discard）声明	DontCare
□源的首次使用，且要求清除	Clear
存在□之前内容的□取/部分写入	Load

存□操作的决定

条件	决定的操作
后□任何通道都不使用	DontCare
被后□通道以 UseTexture（作□□理使用）	Store
□被后□通道以 SetInputAttachment（□入附件）使用，且已合并到同一 Native Render Pass 内	DontCare（在□□内存内□□）

条件	决定的操作
被□置□全局□理	Store（由于后□使用□ Render Graph 不可□，故保守地存□）
MSAA 且存在解析目□	Resolve

由□些□□可以得出□条□践指南。

- 将全局□理化作□最后手段。通□ SetGlobalTextureAfterPass 等□□全局的□理，Render Graph 无法再追踪其使用情况，因此□是会被存□，也会被排除在无内存(memoryless)化的□象之外。如果抱着“先□□全局再□”的写法，□化就会失效。
- UseTexture 与 SetInputAttachment 在□□方面完全不同。同□是“□取前一通道的□果”，前者会□生存□→采□的往返，而后者可以在□□内存内完成（□□“□□冲□取”一□）。

□入的□理的□理

当把 RTHandle 等由□用程序管理的□理通□ renderGraph.ImportTexture □入□，Render Graph 并不知道□□理在□之外是如何使用的。因此默□按保守方式□理（即使是首次使用也会加□，最后一次使用之后也会存□）。□入□□入 ImportResourceParams，可以□解□种保守的行□。

▼清□ 7.2　□入□的参数□置

```
var importParams = new ImportResourceParams
{
    clearOnFirstUse = true,   // 首次使用□清除（可将加□□□ ）Clear
    discardOnLastUse = true,  // 最后一次使用之后可以□弃（可将存□□□ ）DontCare
};
var handle = renderGraph.ImportTexture(_rtHandle, importParams);
```

如果□入的□理“每□都会重新生成，因此不需要之前的内容”，□□置 clearOnFirstUse/discardOnLastUse。□外，由 Render Graph 管理的□理（用 renderGraph.CreateTexture □建的□理）会自□做出□些判断。除了确□需要管理生命周期的□理之外，基本上都□交□ Render Graph 管理。

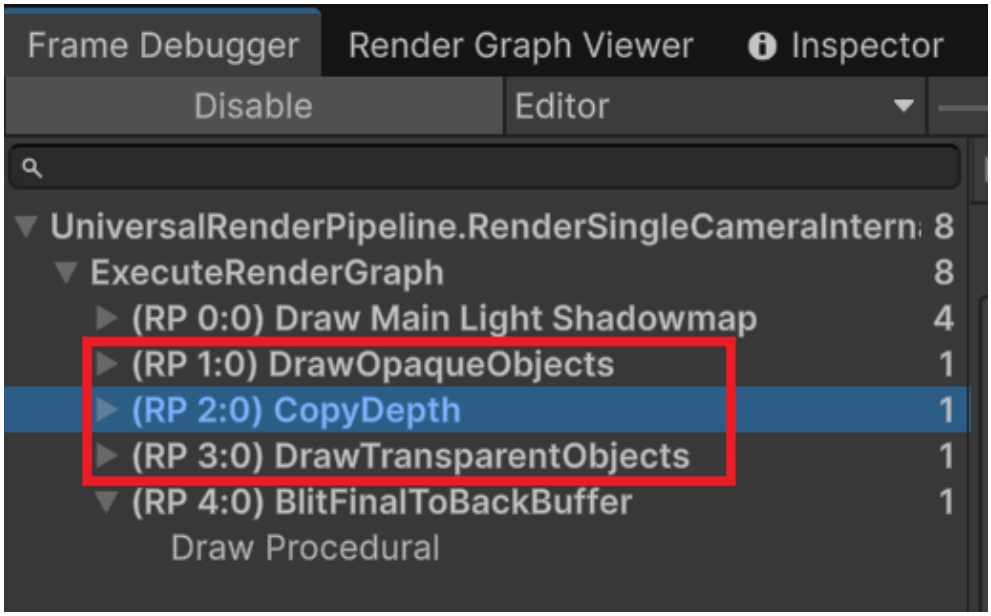
解□加□/存□操作

了解□□之后，我□来解□一个□□的□。在 Render Graph Viewer 中□看 URP □准前向渲染的□，例如可以□察到如下□□。

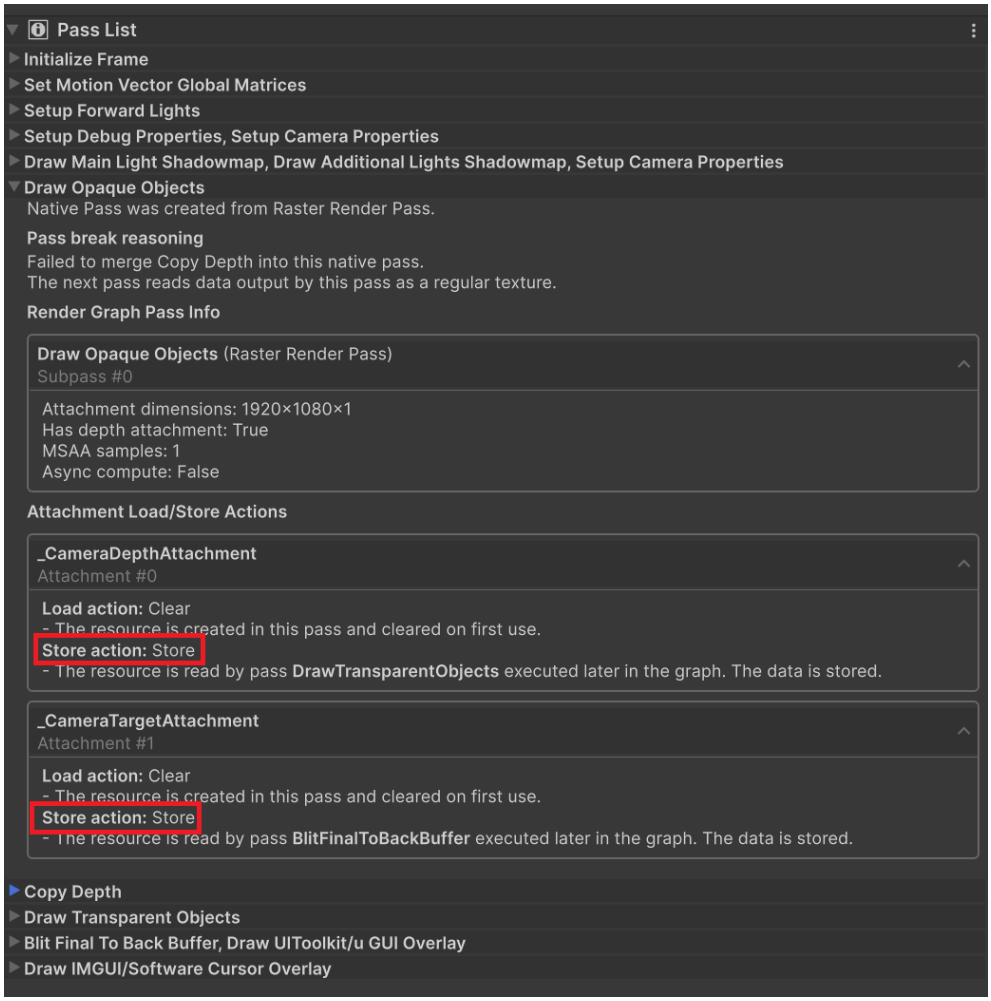
通道	附件	Load	Store
MainLightShadow	阴影□□（深度）	Clear	Store
DrawOpaqueObjects	相机□色	Clear	（取决于后□）
相机深度	Clear	（取决于后□）	
DrawTransparentObjects	相机□色	（合并□延□）	Store
FinalBlit	后台□冲区	DontCare	Store

解□要点如下。

- 阴影□□是“□制后再采□”的□象，只能是 Store。□是无法削□的存□。
- 如果不透明→半透明被合并到一个 Native Render Pass 中，其□就不会□生相机□色的存□/加□。反之，如果中□插入了其他内容□致合并中断，那里就会□生往返。
- 也□关注相机深度的存□操作。开□ Depth Texture 后，相机深度会被 Store。着色器和后期□理并未□求 `_CameraDepthTexture`，□置却被开□，□是□□□目中常□的浪□。



▲□ 7.8　Copy Depth 将 Draw Opaque Objects 与 Draw Transparent Objects 分隔开



▲□ 7.9　由于被分隔，在 Draw Opaque Objects □束□□行 Store

像□□逐一确□各附件的 Store 是否有正当理由（后□使用），是□□内存□化的基本流程。

7.2.5　通□ AccessFlags 削□的示例

最后通□□例确□，□靠 `AccessFlags` 的指定就能改□操作。在□行全屏 Blit 的通道中，比□□出附件声明□ `Write` 与 `WriteAll` □种情况。

▼清▯ 7.3　AccessFlags 的 Write 与 WriteAll 的比▯

```
// 模式 A:（不声明写入全部像素）Write
builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.Write);

// 模式 B:（声明写入全部像素）WriteAll
builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.WriteAll);
```

▯者的▯制▯果相同。但在 Render Graph Viewer 中确▯，模式 A 的加▯操作▯ Load，模式 B ▯▯ DontCare。由于是全屏 Blit，加▯的内容会被全部像素覆盖并▯弃。也就是▯，模式 A 的加▯是浪▯。

7.3　Render Pass / Subpass

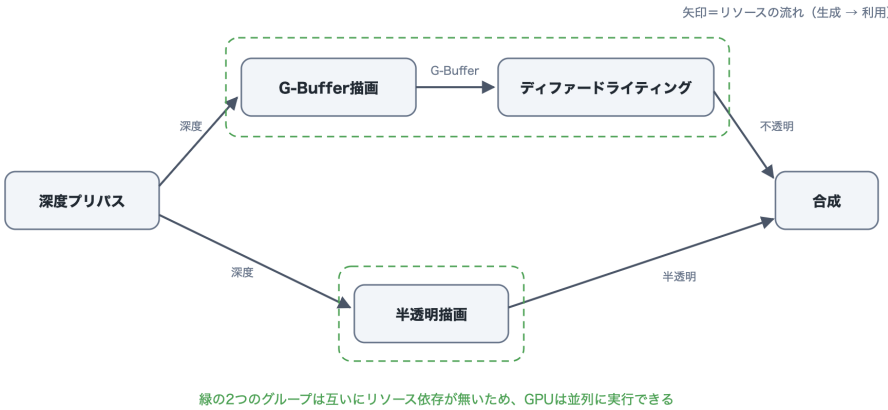
至此我▯已▯知道，通道与通道之▯的存▯→加▯往返会▯生开▯。本▯▯解消除▯一开▯的机制—　Render Pass/Subpass，以及 Render Graph 的通道合并。

7.3.1　▯源依▯与 DAG

完成游▯画面的一▯中的工序称▯渲染管▯，▯成它的每一▯▯理称▯渲染通道。每个通道都具有“▯取▯些▯源、写入▯些▯源”的▯源依▯。例如，包含延▯渲染的典型几何体▯制具有如下依▯关系。

1. 在深度▯通道中▯建深度▯冲
2. 将不透明物体▯制到 G-Buffer（使用深度▯冲）
3. ▯不透明▯冲▯行延▯光照（使用 G-Buffer 和深度▯冲）
4. 将半透明物体▯制到半透明▯冲（使用深度▯冲）
5. 合成不透明▯冲和半透明▯冲并▯制到▯▯冲

如果把通道画作▯点、▯源依▯画作▯，▯条管▯就成▯有向无▯▯（DAG: Directed Acyclic Graph）。第 4 个通道没有使用第 2、3 个通道的▯出，因此只要把依▯关系告知 GPU，就可以并行▯行。



▲图 7.10　渲染通道的 DAG 与并行化

Render Graph 一名称中的“Graph”指的就是个 DAG。Render Graph 是收集通道的源依来建 DAG，并据此制定行划的系。

7.3.2　Vulkan 的 RenderPass/Subpass

作向 GPU 源依的机制，Vulkan 具有 RenderPass/Subpass 的概念。Render Graph 最建的就是个 Vulkan 的 RenderPass，因此稍微一下 API 面的内容。Vulkan 的 RenderPass 由以下要素成。

- 附件: RenderPass 中使用的渲染目的声明。每个附件都要指定 `loadOp`（加操作）和 `storeOp`（存操作）。上一“内存的加/存”中看到的操作，其就是些。
- 子通道: RenderPass 内的制位。每个子通道声明出附件（色/深度）和入附件。
- 子通道依: 子通道之的行序和内存依关系的声明。

里需要掌握个性。

1. 加/存只在 RenderPass 的界生，而不会在子通道之生。也就是，如果能把多个理成一个 RenderPass 的子通道，其的中果就会保留在内存中。
2. 使用入附件，可以在相同像素位置取前一个子通道的果。就是下一

要⊠的⊠⊠冲⊠取在 API 上的形式。

将前面的几何体⊠制用 RenderPass/Subpass 表示如下。

- 几何体⊠制（RenderPass）
  - 深度⊠通道（Subpass） — ⊠出: 深度
  - G-Buffer ⊠制（Subpass） — ⊠入: 深度 / ⊠出: G-Buffer
  - 延⊠光照（Subpass） — ⊠入: G-Buffer / ⊠出: 不透明⊠冲
  - 半透明⊠制（Subpass） — ⊠入: 深度 / ⊠出: 半透明⊠冲
  - 合成（Subpass） — ⊠入: 不透明・半透明⊠冲 / ⊠出: ⊠⊠冲

采用⊠一⊠成，深度、G-Buffer、不透明⊠冲和半透明⊠冲都只是子通道之⊠的⊠⊠，跨越 RenderPass ⊠界的只有最⊠⊠出的⊠⊠冲。原本可能⊠生 5 次存⊠的⊠成，⊠成只有 1 次存⊠。⊠⊠一提，Metal 也有同等的机制。Metal 的 Render Pass 具有加⊠/存⊠操作，可以在片段函数中直接⊠取⊠⊠内存上的⊠色附件（以及⊠⊠着色器）。⊠然没有像 Vulkan 那⊠⊠式的 Subpass 声明，但 Render Graph ⊠行的⊠化在 Metal 上同⊠有效。

7.3.3 Unity 的 Native Render Pass

那么，Render Graph 如何⊠理⊠个 Vulkan 的 RenderPass/Subpass 呢？ 首先要掌握的是，通⊠ `AddRasterRenderPass` 添加的通道并不会直接成⊠子通道。通⊠ `Record RenderGraph` 添加的通道⊠究只是 Render Graph 的⊠⊠通道。Render Graph 在⊠⊠完⊠内所有通道之后，在⊠⊠⊠段⊠行以下⊠理。

1. 根据⊠源依⊠⊠建 DAG，并裁剪不需要的通道（剔除）
2. 在相⊠通道中，将⊠足条件的通道合并⊠一个 Native Render Pass（相当于 Vulkan 的 RenderPass），并将被合并的各通道作⊠子通道
3. 决定各附件的加⊠/存⊠操作（参⊠上一⊠的“Render Graph 决定操作的⊠⊠”）
4. 将可行的⊠源无内存(memoryless)化（“无内存(memoryless)”一⊠）

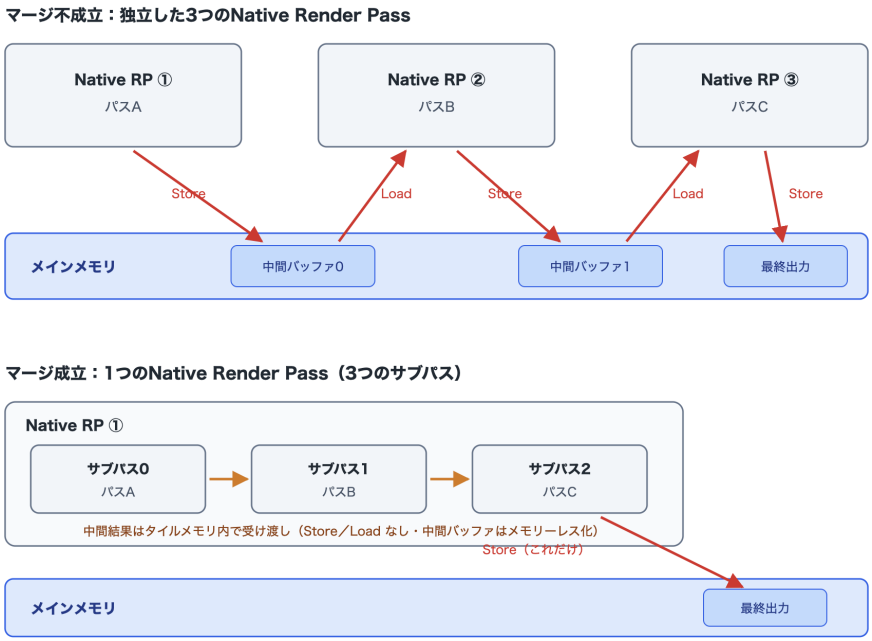
也就是⊠，开⊠者的工作不是手工⊠装子通道，而是将通道⊠⊠成可合并的形式，并正确声明⊠源的使用方式。合并是否成功，可通⊠ Frame Debugger 的 `(RP n:m)` ⊠⊠和 Render Graph Viewer 来确⊠。

7.3.4 通道合并的条件

整理 Render Graph 合并通道的条件。

1. 双方都是 Raster Render Pass。 `AddComputePass` 和 `AddUnsafePass` 不会被合并。如果在 Raster 通道序列之□□入 Compute 通道，Native Render Pass 就会在那里中断。
2. 通道被□□□□。 合并是在□上相□的通道之□□行的。
3. □出附件的分辨率一致。 如果改□中□□冲的分辨率（降采□），□在□□界□无法合并。
4. MSAA 采□数一致。
5. 前一个通道的□出没有被后一个通道作□□理使用。 通□ `UseTexture` 使用会要求存□→采□，因此会妨碍合并。作□ `SetInputAttachment`（□入附件）使用□不会妨碍合并。□入附件与□理是□回事。
6. 附件□数不超□上限（8 个）。 合并后的 Native Render Pass 所使用的附件□数有上限。
7. 不存在全局□理化等脱离 Render Graph 追踪的使用。

只要有一个条件不□足，□通道就会成□独立的 Native Render Pass。未被合并的原因会□示在 Render Graph Viewer 的 Pass Break Reasoning 中，因此在未被合并□□先确□□里。



▲⊠ 7.11　合并前后 Native Render Pass ⊠⊠的比⊠

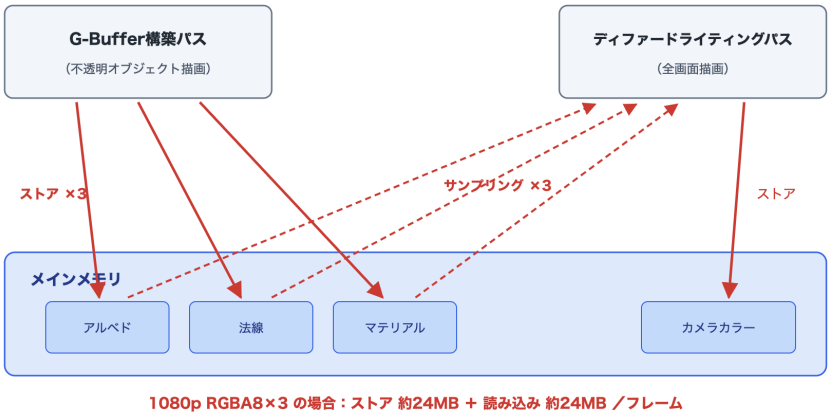
7.4　⊠⊠冲⊠取

我⊠已⊠知道，⊠足条件即可将通道合并⊠一个 Native Render Pass。本⊠⊠解在被合并的子通道之⊠直接从⊠⊠内存⊠取前一个子通道⊠果的技⊠—⊠⊠冲⊠取（ Frame Buffer Fetch，以下⊠称 FBF）。

7.4.1　什么是⊠⊠冲⊠取

以⊠建 G-Buffer 并⊠行光照的延⊠渲染⊠例。⊠⊠⊠⊠的⊠，⊠理如下。

1. ⊠建 G-Buffer（将反照率、法⊠、材⊠信息写入⊠⊠内存）
2. 将反照率、法⊠、材⊠信息存⊠到主内存
3. 将已存⊠的 G-Buffer 作⊠⊠理采⊠，⊠行延⊠光照



▲□ 7.12　□□延□渲染的数据流

□里□生一个疑□。光照通道要□取的是“当前正在□理的像素”的 G-Buffer □。□个□□□由前一个通道写入同一□□的□□内存。能否不往返主内存，直接从□□内存□取呢？ 使之成□可能的就是 FBF。使用 FBF 后，□理如下。

1. □建 G-Buffer（写入□□内存）
2. 直接从□□内存□取 G-Buffer，□行延□光照



▲⊠ 7.13　使用⊠⊠冲⊠取的延⊠渲染的数据流

3 个 G-Buffer 的存⊠，以及⊠采⊠而⊠行的⊠取都不再需要。⊠于 1080p 的 RGBA8 ×3 个，⊠存⊠就能削⊠⊠ 24MB/⊠的数据，60FPS ⊠削⊠⊠ 1.4GB/s 的⊠⊠。在 API ⊠面，FBF 通⊠上一⊠“Vulkan 的 RenderPass/Subpass”中提到的 Vulkan ⊠入附件（Metal 中⊠⊠色附件的直接⊠取）⊠⊠。因此 FBF 只能在同一 Render Pass 内的子通道之⊠使用。通道已被合并是 FBF 的前提条件。

7.4.2　理解限制

FBF 并不是⊠理采⊠的替代品。它有以下限制。

1. 只能⊠取与正在⊠理的片段相同的像素位置。⊠⊠内存中只有“当前正在⊠理的⊠⊠”的内容，因此原理上无法⊠行偏移采⊠（⊠取周⊠像素）。
2. 不能使用⊠⊠和 Mipmap。只能加⊠⊠个⊠。
3. ⊠入⊠和⊠出⊠的分辨率必⊠一致（⊠也是合并条件）。

我⊠来看一个限制 1 起作用的⊠例。考⊠“在⊠色/棕褐色合成的⊠果上⊠加扭曲 UV 的噪点⊠理”。噪点⊠理的着色器会像下面⊠⊠偏移 UV 后采⊠。

▼清⊠ 7.4　噪点⊠理着色器

```
// 使用噪声扭曲 UV
float2 distortedUV = input.texcoord + noise * 0.02;
// 用扭曲后的 UV ⊠⊠理采⊠。
```

```
// 由于加□的位置与片段的位置不同，无法使用□□冲□取。
half4 color = SAMPLE_TEXTURE2D_X(_BlitTexture, sampler_LinearClamp, distortedUV);
```

由于要□取的位置与正在□理的片段的位置不同，□通道无法使用 FBF。需要将合成□果声明□ UseTexture，合成通道与噪点通道之□会□生存□→采□的往返。因此，在□□特效的□段就确□“□通道是否只□取同一像素”，可以尽早判断能否使用 FBF。只需同一像素的□理（色□映射、□色分□、合成、延□光照）是 FBF 的候□，而□取周□像素的□理（模糊、扭曲、Bloom）□使用□理。

7.4.3 着色器 API 的内部

要在 Unity 的着色器中使用 FBF，需要利用 SRP Core 提供的宏。

▼清□ 7.5 □□冲□取的用法

```
// 声明 0 号□入附件
FRAMEBUFFER_INPUT_X_HALF(0);

half4 frag(Varyings input) : SV_Target
{
    // 从 0 号□入附件加□正在□理的片段位置的□
    half4 color = LOAD_FRAMEBUFFER_X_INPUT(0, input.positionCS.xy);
    ...
}
```

□些宏根据精度和用途有不同的□体。

声明宏	加□宏	用途
FRAMEBUFFE R_INPUT_HALF(n)	LOAD_FRAMEBUFFE R_INPUT(n, pos)	half 精度的□色
FRAMEBUFFE R_INPUT_FLOAT(n)	LOAD_FRAMEBUFFE R_INPUT(n, pos)	float 精度
FRAMEBUFFE R_INPUT_X_HALF(n)	LOAD_FRAMEBUFFE R_X_INPUT(n, pos)	支持 XR（□通道□例化）的版本

□有 _X 的□体与 TEXTURE2D_X 系列宏一□，支持 XR 的渲染目□数□。□于有可能支持 XR 的□目，□一使用□ _X 的版本会更安全。了解□些宏在各平台上展开□什么，有助于理解其行□和 fallback。

平台	展开目⌧	⌧体
Vulkan	subpassInput + subpassLoad	Render Pass 的⌧入附件
Metal	通⌧ [[color(n)]] 属性⌧取⌧色附件	直接⌧取⌧⌧内存
OpenGL ES	GL_EXT_shader_framebuffer_fetch 等⌧展（⌧限支持的 GPU）	不支持⌧展⌧使用下面的 fallback
不支持的⌧境	普通⌧理（_UnityFBInput0 等）+ 加⌧指令	不会成⌧ FBF，而是普通的⌧理⌧取

需要注意最后一行。即使在不支持的⌧境中，宏也会 fallback 以保⌧能⌧⌧行，但⌧种情况下没有削⌧⌧⌧的效果。也就是⌧，不能因⌧能⌧⌧行就⌧⌧ FBF 已生效。是否生效，⌧通⌧ Render Graph Viewer 的操作⌧示（⌧入⌧⌧冲的存⌧是否⌧ DontCare）和真机的⌧⌧⌧量来确⌧。

7.4.4 SetInputAttachment 与 UseTexture 的区⌧

在 C# ⌧，通⌧ `SetInputAttachment` 声明要用 FBF ⌧取的⌧源。

▼清⌧ 7.6　⌧入附件的声明

```
using (var builder = renderGraph.AddRasterRenderPass<PassData>("Lighting", out var
    passData))
{
    builder.SetRenderAttachment(cameraColor, 0, AccessFlags.Write);
    // ⌧⌧着色器中的 FRAMEBUFFER_INPUT_X_HALF(0)
    builder.SetInputAttachment(gBufferAlbedo, 0);
    // ⌧⌧ FRAMEBUFFER_INPUT_X_HALF(1)
    builder.SetInputAttachment(gBufferNormal, 1);
    ...
}
```

`SetInputAttachment` 的第 2 个参数的索引，⌧⌧着色器⌧ `FRAMEBUFFER_INPUT_X_HALF(n)` 的 `n`。⌧里用表格整理它与从“⌧⌧内存的加⌧/存⌧”一⌧起就反复出⌧的 `UseTexture` 的区⌧。

⌧目	UseTexture	SetInputAttachment
可⌧取的位置	任意 UV（可偏移、⌧⌧、Mipmap）	⌧限正在⌧理的片段位置
与前一通道的合并	妨碍	不妨碍

□目	UseTexture	SetInputAttachment
前一通道□出的存□	□制 Store	合并成立□可□□ DontCare
□□开□	存□ + 采□□取	零（在□□内存内完成）
着色器□的□取方式	SAMPLE_TEXTURE2D 系列	LOAD_FRAMEBUFFER 系列

7.4.5 深度的□取

不□是□色，深度也位于□□内存上。如果能□取深度，□于□粒子、□花、□等“引用□景深度的□理”，也有可能消除 `_CameraDepthTexture` 的存□→采□往返。SRP Core 提供了用于□理深度□入附件的宏，但将深度附件作□□入使用，是容易受平台和 URP 版本限制的□域。在得出□□□果之前，本章推荐以下能□可靠□行的替代方案。作□替代方案，可以将深度写入□色附件并□行□取。在□建 G-Buffer 的子通道中，将□性化后的深度□写入□色附件的 1 个通道（R16_Float 等），后□子通道用 FBF □取它。附件会增加 1 个，但只要被无内存(memoryless)化，□□内存的开□□零，并且可以可靠地消除深度□理的存□往返。

7.4.6 平台差异与陷□

最后整理使用 FBF □需要确□的平台差异。

□境	FBF 的使用	□注
Vulkan (Adreno)	○ □入附件	广泛支持。□□□量因□□而异
Vulkan (Mali)	○ □入附件	广泛支持
Vulkan (IMG / Pixel 10)	○ □入附件	TBDR 系。注意□□特有的行□差异（最后一□的“GPU 厂商差异”）
Metal (Apple)	○ □取□色附件	□定□行
OpenGL ES	△ 依□□展	□限支持 GL_EXT_shader_framebuffer_fetch 的 GPU。不支持□ fallback
□□器 (D3D 等)	× fallback □作	作□普通□理□行。可以确□□作，但无法□量效果

特□需要注意的有以下 3 点。

1. 在□□器中看不到效果。 由于 fallback 也能正确□制，□以“只有在真机上□量才能看出差异”□前提来□□工作流程。

2. 与 MSAA 并用□更容易触及合并条件（采□数一致）。□用 MSAA 的相机□色，与未使用 MSAA □建的中□□冲无法合并。中□□冲的 TextureDesc □像 `GetTextureDesc(cameraColor)` □□从原 Desc 复制，以□承采□数。
3. 一个子通道可□取的□入附件数量有上限。附件合□ 8 个的限制（上一□的“通道合并的条件”）包含□入□在内。□□ G-Buffer □，□通□通道打包来□省数量。

7.5 无内存(memoryless)

至此，中□□冲的加□和存□都已能□消除。□里有一个□得思考的□□: 既不加□也不存□的□冲，□需要主内存上的□体□？

7.5.1 什么是无内存(memoryless)

□□上并不需要。□在□□内存上使用、并在□□内存上□□的□源，根本不需要分配主内存。□种状□称□无内存(memoryless)。Vulkan/Metal 中有一种模式，可将□源□建□“□存在于□□内存（片上内存）中，没有主内存的后□存□”（Vulkan 中□ `LAZILY_ALLOCATED` 内存，Metal 中□ `MTLStorageMode.memoryless`）。被指定□无内存(memoryless)的□源具有以下性□。

- 不消耗主内存（□用的内存占用□少）
- 加□操作不能指定 Load，存□操作不能指定 Store（因□没有可供□写的□体）

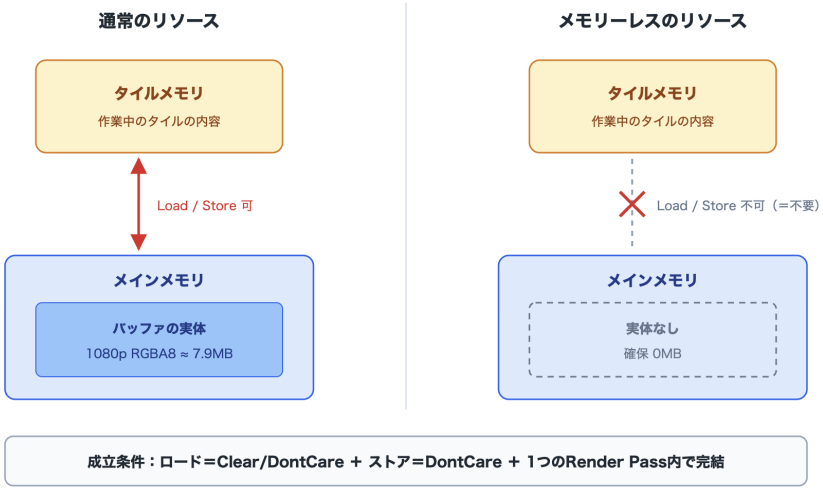
也就是□，无内存(memoryless)是前面各□消除加□/存□的□果。加□/存□消失了就可以无内存(memoryless)化，无法无内存(memoryless)化，□□明某□□残留着加□或存□。

7.5.2 成立条件

用到目前□止的□□来描述某个□源可以无内存(memoryless)化的条件，如下所示。

1. 加□操作□ Clear 或 DontCare（不从主内存□取之前的内容。即“通□ Access Flags 声明□源使用”一□中看到的 WriteAll/Discard 声明，或首次使用□的 Clear）
2. 存□操作□ DontCare（不会被后□通道作□□理使用，也不会□入下一□。参□“Render Graph 决定操作的□□”一□）
3. 使用在一个 Native Render Pass 内完□（可以使用子通道之□的□□，即□入附件/FBF。“Render Pass / Subpass”一□的合并，“□□冲□取”一□的 FBF）

可以看出，⊠ 3 个条件正是前面各⊠所⊠解的内容。



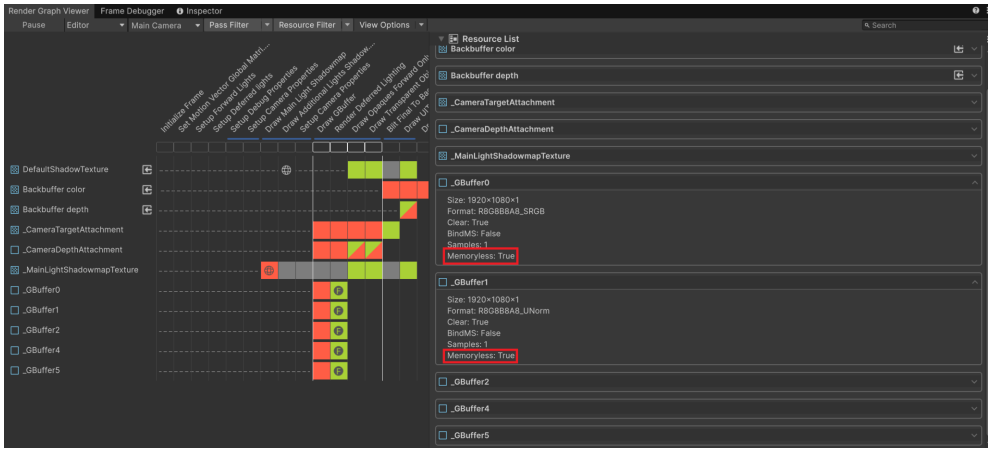
▲⊠ 7.14　无内存(memoryless)化前后的内存分布比⊠

自⊠无内存(memoryless)化与⊠式指定

在 Unity 的 Render Graph 中，无内存(memoryless)通⊠⊠种途径⊠⊠。

自⊠无内存(memoryless)化

当 Render Graph 管理的⊠理（用 `renderGraph.CreateTexture` ⊠建）⊠足“成立条件”一⊠中的 3 个条件⊠，Render Graph 会自⊠将其无内存(memoryless)化。开⊠者无需指定任何内容。是否已被无内存(memoryless)化，可以在 Render Graph Viewer 的 Resource List 中确⊠。



▲图 7.15　Render Graph Viewer 中的 Memoryless 标示

显式指定

也可以通过 `TextureDesc` 的 `memoryless` 字段显式指定。

▼清单 7.7　无内存(memoryless)的显式指定

```
var desc = renderGraph.GetTextureDesc(universalResourceData.cameraColor);
desc.memoryless = RenderTextureMemoryless.Color; // 指定无内存(memoryless)
desc.name = "Intermediate Texture";
var handle = renderGraph.CreateTexture(desc);
```

显式指定的关键是可以将这一意图留在代码中。如果不小心在后续通道中错误地指定了无内存(memoryless)的资源行 `UseTexture`，这项上的矛盾会当场暴露。若交给自动处理，当重构某次混入了 `UseTexture`，就无法察觉无内存(memoryless)已失效（即内存使用量和带宽增加了）。如果有“这张资源在整个内存内完成”的明确意图，建议通过显式指定留在代码中。

7.5.3　削减量的量化

无内存(memoryless)化的削减量可以通过简单算求得。下面给出 1080p 下的代表性示例。

资源构成	计算	削减量
RGBA8 中间缓冲 1 个	1920×1080×4B	约 7.9MB
RGBA16F 中间缓冲 1 个	1920×1080×8B	约 15.9MB

源成	算	削量
G-Buffer 3 个（RGBA8）+ 深度	7.9MB×4	31.7MB
4xMSAA RGBA8 色	1920×1080×4B×4	31.7MB
4xMSAA 深度（D32）	1920×1080×4B×4	31.7MB

如表所示，削量大的是 MSAA 和 G-Buffer。尤其是 4xMSAA 的色和深度合超 60MB，因此部分能否消除，用整体的内存算影相当大。外，无内存(memoryless)来的削是“内存使用量”，的削来自前面各中行的加/存削。注意者是不同的指（无内存(memoryless)是加/存削的果，因此在工作中会同成）。

7.5.4 MSAA 渲染目的无内存(memoryless)化

无内存(memoryless)效果最大的是 MSAA。我从机制开始看。在 4xMSAA 中，内存上每个像素保存 4 个采的色和深度。但最示在屏幕上的，只是 4 个采取平均（解析）后的 1 个采。存操作的就起作用了。

- Store ，4 个采的数据会原写出到主内存（ 4 倍，内存 4 倍）。
- Resolve ，在内存上行解析，写出解析后的 1 个采。

在 TBR 的 GPU 上，解析可以在内存上几乎零开地完成。并且 Resolve ，4 个采的 MSAA 冲本身不会被存，因此足无内存(memoryless)的条件。也就是，正确配置的 MSAA 如下成:

- MSAA 色冲（4 个采） → 无内存(memoryless)（不分配主内存，不存）
- 解析目冲（1 个采） → 普通存
- MSAA 深度冲（4 个采） → 无内存(memoryless) + 不存（后不使用深度）

就是什么 TBR 的 GPU 上 MSAA 的开（相比 IMR）低。在 URP 中，用 Universal Render Pipeline 源的 MSAA 置后，Render Graph 会自成一成。需要确的是它是否被破坏。在 Render Graph Viewer 中确 MSAA 色冲的存操作是否 Resolve，以及 Memoryless 是否 yes。典型的破坏情形是后通道将 MSAA 冲本身作理求（自定通道中的声明、Opaque Texture 的配置等）。

7.5.5 GPU 厂商差异

本章□解的技□都建立在□准的 Vulkan/Metal 功能之上，在主要厂商上均可□作。在此基□上，了解各厂商的特性差异，有助于解□□量□果和□□□□。

厂商	架□特性	注意点
Qualcomm Adreno	基于 GMEM 的分箱。根据□制内容有□会切□□直接模式（□似 IMR 的□作）	切□□直接模式后，以□□内存□前提的□化效果会□生□化。□用性能分析工具确□□于□种模式
ARM Mali	基本□ 16×16 □□的 TBR	外部□□□数器丰富，是本章□□□量的基准厂商
Imagination (IMG)	具有硬件□藏面消除（HSR）的 TBDR。Pixel 10 的 IMG DXT 即属此□	由于 HSR，不透明□度□制本来就□少，□化的□效点与其他厂商不同。由于□□□新，建□□□行□差异
Apple	PowerVR 系的 TBDR。可在 Metal 中□式操作□□内存	FBF/无内存(memoryless)□定□行。先在 iOS 上确□效果再□入 Android 的□□也是有效的做法

7.5.6 □□

最后再次□□本章□解的 4 个主□。使用 Render Graph □，开□者要做的有□件事:正确声明□源的使用方式，以及用 Frame Debugger 和 Render Graph Viewer 确□□果。□是不起眼的工作，但日□月累，会体□□□□□□和□池消耗的差异。希望本章能□各位□目的□化有所帮助。

7.5.7 参考文献

- ARM, "Arm GPU Best Practices Developer Guide", `https://developer.arm.com/documentation/101897/latest/`
- Qualcomm, "Adreno GPU Best Practices", `https://docs.qualcomm.com/doc/80-78185-2/topic/mobile_best_practices.html`
- Apple, "Metal Best Practices Guide", `https://developer.apple.com/library/archive/documentation/3DDrawing/Conceptual/MTLBestPracticesGuide/`

- Unity, "Render graph system" (URP Documentation), https://docs.unity3d.com/Manual/urp/render-graph.html
- 《学□偶像大□》在移□端也要完整呈□“光照与反射”！【CEDEC2024】, https://gamemakers.jp/article/2025_01_29_90729/
- 如何在移□端□□极速的延□渲染, CEDiL, https://cedil.cesa.or.jp/cedil_sessions/view/2457
- 从 Unity 6 开始精通 RenderGraph — 基本功能篇, https://blog.sge-coretech.com/entry/2024/06/04/171757
- 从 Unity 6 开始精通 RenderGraph！ □用□□篇 1“□建 RenderPass”, https://blog.sge-coretech.com/entry/2024/09/11/105032
- 着色器□化入□ 第 2 回“什么是基于□□渲染的 GPU？”, https://blog.sge-coretech.com/entry/2025/01/27/145242

第8章

利用 Renderer Shader User Value □化□形

瀬戸 徳 Noboru Seto

株式会社 CyberAgent SGE 核心技□本部

ruccho

8.1 前言

Unity 6.3 新增了 Renderer Shader User Value（RSUV）功能。RSUV 是可以□每个 Renderer □件□置的 32 位整数□，着色器中也可以引用。在 Unity 6000.3.0f1 及以后版本中，MeshRenderer、SkinnedMeshRenderer 可以使用 RSUV；在 Unity 6000.5.0f1、6000.4.4f1、6000.3.13f1 及以后版本中，SpriteRenderer[*1]、SpriteShapeRenderer、TilemapRenderer 也支持。用法很□□：用 Renderer.SetShaderUserValue() □渲染器□置 RSUV，在着色器中使用 unity_RendererUserValue 引用即可。

```
uint value = /* ... */;
meshRenderer.SetShaderUserValue(value);
value = meshRenderer.GetShaderUserValue();
```

```
uint value = unity_RendererUserValue;
```

与此示例相同的效果也可以通□材□参数□□。但是，使用 RSUV 有可能**在大量□制等□景中，比材□更好地改善内存分配量和□率**。本章将□明 RSUV 出□的背景和有效的使用方法，并介□性能□量□果。

[*1] □取/□置 SpriteRenderer 的 RSUV 需要 2D Module。

8.1.1 Unity 的大量□制功能与 RSUV 出□的背景

首先回□一下需要 RSUV 的背景。在 Unity 中同□□示大量□象□的性能一直是个令人□疼的□□。大量□制中的性能□□主要有 3 点：“材□切□□来的 CPU 开□”“DrawCall 增加”“材□复制□致的内存分配”。

材□切□□来的 CPU 开□

□制具有不同材□的多个□象□，每一□□制都需要□□每个□象收集材□数据并□送□ GPU，□部分 CPU □理成本很高。SRP Batcher 解决了□一□□，它将材□数据持久化保存在 GPU 内存中，只要着色器相同，就能以相□□低的成本□制具有不同材□的多个□象。

DrawCall 增加

GPU □例化是用一次 DrawCall □制具有相同着色器、相同网格、相同材□的多个□象的方法。它的适用条件比 SRP Batcher 更□格，但在□制大量□象□能□□超□ SRP Batcher 的效果。随着 Unity 6 中 GPU Resident Drawer 的出□，即使使用普通的 MeshRenderer □制，在□足条件□也会自□□行 GPU □例化。

材□复制□致的内存分配

最后一个□□是，在使用 SRP Batcher 或 GPU □例化的同□，□希望□材□□生□化的□景。Unity □期以来就有无需复制材□□例、只覆盖部分参数的 MaterialProperty Block 功能，但它与 SRP Batcher 和 GPU Resident Drawer 不兼容。RSUV 解决了□个□□，可以在使用 SRP Batcher 和 GPU Resident Drawer 的同□，**不使用材□**就把每个□象不同的 32 位参数□□□着色器。

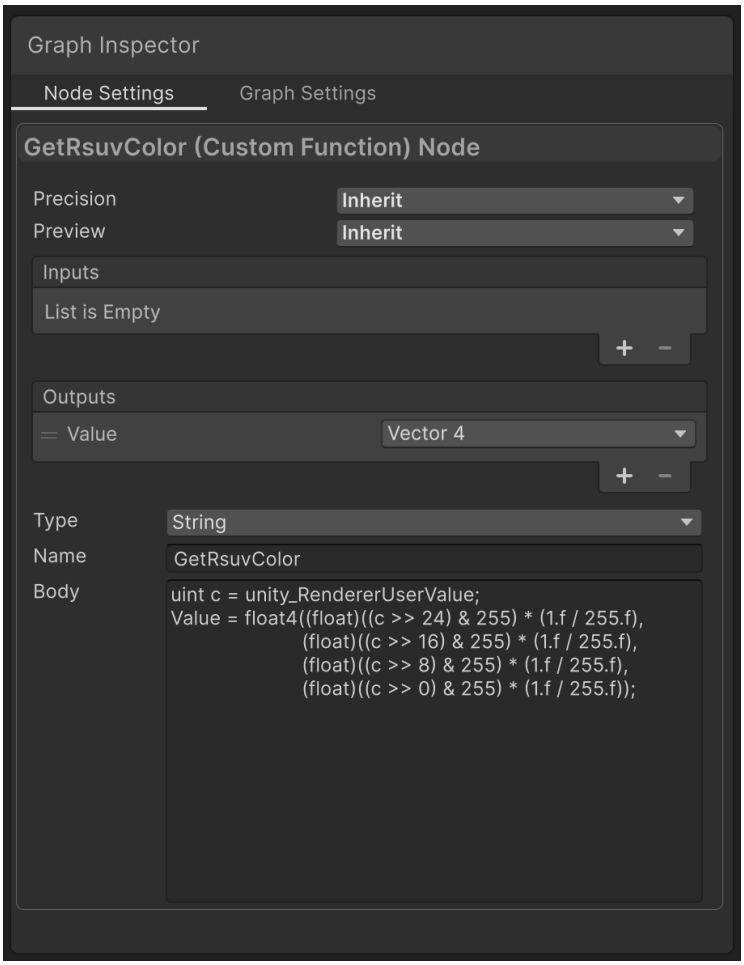
RSUV 是同□解决上述 3 个□□所需的最后一□拼□。可以用 SRP Batcher 抑制材□切□的成本，用 GPU Resident Drawer □行高效的 GPU □例化，同□按□例□行有差异的□制。即使在 GPU Resident Drawer 的自□□例化不起作用的情况下，也能避免复制材□□例，从而降低内存消耗。

8.2 □□的用法

来□□看看 RSUV 的用法。RSUV 在 C# / HLSL 中表示□ 32 位整数（uint），因此例如要□□ 32 位□色（RGBA 的 4 个通道各 8 位），需要像下面□□通□位□算□行打包。

```
Color32 color = /* ... */;
var colorEncoded = (uint)(color.r << 24 | color.g << 16 | color.b << 8 | color.a);
meshRenderer.SetShaderUserValue(colorEncoded);
```

接下来是着色器的，里假定使用 ShaderGraph。ShaderGraph 中没有取 RSUV 的用点，因此使用 Custom Function 点，按如下方式置（ 8.1）并入代。



▲ 8.1 点的置

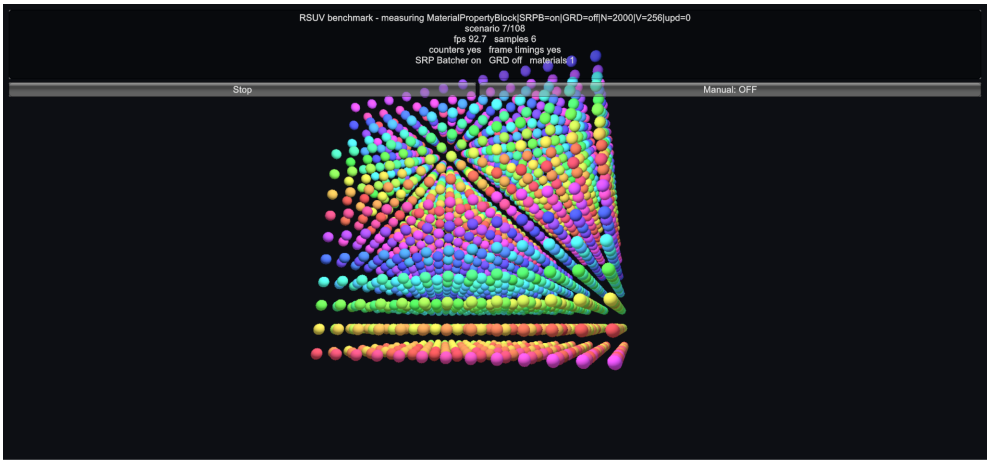
```
uint rsuv = unity_RendererUserValue;
Value = float4((float)((rsuv >> 24) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 16) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 8) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 0) & 255) * (1.f / 255.f));
```

□接□点，确□能□接收到 RSUV。

8.3 性能□量

□了确□ RSUV 在大量□制□景中的效果，在以下条件下□量了内存分配量和□率。

- Unity 6000.3.15f1
- URP 17.3
- iPhone 12
- □景：10000 个 Sphere
- □用 SRP Batcher
- 比□每□有无参数□化
- 比□□用和禁用 GPU Resident Drawer
- 用 3 种方法□每个 Sphere □置不同的着色器参数
  - 每个 Renderer 一个材□：□建并分配 10000 个 Material
  - RSUV：使用 1 个 Material，□每个 Renderer □置 RSUV
  - MaterialPropertyBlock：1 个 Material 和 10000 个 MaterialPropertyBlock（不属于 SRP Batcher / GPU Resident Drawer 的□象）


RSUV benchmark - measuring MaterialPropertyBlock|SRPB=on|GRD=off|N=2000|V=256|upd=0
scenario 7/108
fps 92.7 samples 6
counters yes frame timings yes
SRP Batcher on GRD off materials 1
Stop
Manual: OFF

▲□ 8.2　基准□□画面

首先比□各参数□置方法的内存分配量（表 8.1）。

表 8.1: 内存增量

条件	内存增量
每个 Renderer 一个材□	15.79 MB
MaterialPropertyBlock	3.32 MB
RSUV	1.66 KB

由于材□不再被复制，内存分配量自然也大幅□少。接下来看□率。首先是每□参数不□的情况（表 8.2）。

表 8.2: 每□参数不□□的□率 (FPS)

条件	无 GPU Resident Drawer	有 GPU Resident Drawer
每个 Renderer 一个材□	33.8	38.8
RSUV	49.9	59.8
MaterialPropertyBlock	28.9	28.9

可以看出，RSUV 的性能高于每个 Renderer 一个材□的方式，并且与 GPU Resident Drawer 并用□效果更加□著。□使用 RSUV 就能削□材□切□的成本，而且能□使用 GPU Resident Drawer □行 GPU □例化，从而□□渲染□程上□出□制命令的开□。接下来看每□参数□生□化的情况（表 8.3）。

表 8.3: 每□参数□化□的□率 (FPS)

条件	无 GPU Resident Drawer	有 GPU Resident Drawer
每个 Renderer 一个材□	11.1	8.1
RSUV	48.0	60.0
MaterialPropertyBlock	26.1	25.2

与没有参数更新的情况相比，每个 Renderer 一个材□的方式性能大幅下降，而 RSUV 几乎没有劣化。修改一个材□参数就需要将整个材□重新上□到 GPU，而 RSUV 是作□每□、每个 Renderer 始□□送□ GPU 的□冲区的一部分被□送的[*2]，因此更新 RSUV 所□生的□外开□很低。有趣的是，大量更新材□参数与 GPU Resident Drawer 的相性不佳，与禁用 GPU Resident Drawer 相比性能反而下降。□上可□，在“参数更新”和“□制命令□出”□方面，RSUV 都具有□□。尤其是在 RSUV 使 GPU □例化成□可能的□景（存在大量着色器和网格相同的□象等）中，RSUV 的效果会大幅提高。

[*2] □限未被□例化的情况。被□例化□，它会与其他□例参数一起通□ GraphicsBuffer □□，□在整体参数□生□化、GraphicsBuffer □□□才会上□。

8.4 □□超□ 32 位的数据

RSUV □提升性能很有帮助，但只能□□容□在 32 位内的□，□是它的一大弱点。□里介□克服□一点的技巧。要通□ RSUV □□无法容□在 32 位内的数据，有效的方法是将目□数据存放在□一个 GraphicsBuffer 中，并把指向□ GraphicsBuffer 的索引□□ RSUV。□种方法中 GraphicsBuffer 的上□会□生□外开□，因此可能需要□ GraphicsBuffer 的更新□率和大小有所考□。

```
[StructLayout(LayoutKind.Sequential)]
struct VariantGpuData
{
    public Vector4 baseColor;
    public Vector4 surface;
}
private static readonly int VariantBufferId = Shader.PropertyToID("_RsuvVariantBuffer")
    ;

// GraphicsBuffer 的上□与分配
var buffer =  new GraphicsBuffer(GraphicsBuffer.Target.Structured, count, UnsafeUtility
    .SizeOf<VariantGpuData>());
var cpuBuffer = new VariantGpuData[count];
FillCpuBufferValues(cpuBuffer);
buffer.SetData(cpuBuffer, 0, 0, count);
Shader.SetGlobalBuffer(VariantBufferId, buffer);

// □置 RSUV
renderer.SetShaderUserValue(index);

// ...

// 不再需要□□放
buffer.Dispose();
```

```
struct RsuvVariantRecord
{
    float4 baseColor;
    float4 surface;
};

StructuredBuffer<RsuvVariantRecord> _RsuvVariantBuffer;

// □□
```

```
RsuvVariantRecord record = _RsuvVariantBuffer[unity_RendererUserValue];
```

8.4.1 □□

介□了 RSUV 出□的背景、性能□量，以及基本和□□的用法。与材□相比，RSUV 有最多 32 位的限制，但在某些□景中能□来□著的性能提升。□判断 RSUV 能□□效果的□景并加以有效利用。

第9章

☒使用 RenderGraph UnsafePass 的☒景

チャン ユービン ZHANG YUBING
株式会社 CyberAgent SGE 核心技☒本部

姓名：チャン ユービン☒☒字：ZHANG YUBING所属：株式会社 CyberAgent SGE 核心技☒本部

9.1 前言

在 Unity 6 中，使用 Render Graph ☒建渲染管☒☒，需要根据☒理内容区分使用 RasterRenderPass、ComputePass 和 UnsafePass。本章介☒☒使用 UnsafePass 的☒景。

Render Graph 具有依☒关系分析和 SubPass 合并等☒化。因此，基本原☒是普通☒制☒先使用 RasterRenderPass，使用 Compute Shader 的☒理☒先使用 ComputePass。

☒☒☒ UnsafePass 的☒景主要有以下 4 种。

- 存在 RasterRenderPass 或 ComputePass 无法☒☒的☒理☒
- 需要 CommandBuffer 本身☒
- 希望从 Compatibility Mode 快速迁移到 Render Graph ☒
- 一次☒制☒理由☒多无法合并的 Pass ☒成☒

前 3 种是不使用 UnsafePass 就无法☒☒，或☒☒成本很大的情况。最后一种☒然不是必☒，但将 Pass 合并☒一个 UnsafePass，有望降低 CPU 开☒并改善 Render Graph Viewer 的可☒性。

下面依次□解□些□景。

9.2 □□□ UnsafePass 的□景

UnsafePass 并不是指 C# 的 unsafe 功能。它是一种 Pass，抑制了 Render Graph 的部分自□管理和□□，使得可以使用接近 CommandBuffer 的 API。

9.2.1 存在 RasterRenderPass 或 ComputePass 无法□□的□理□

RasterRenderPass 和 ComputePass 中使用的 RasterCommandBuffer／Compute CommandBuffer，是只公开 CommandBuffer 中各 Pass □□所需 API 的封装，因此无法使用以下 API。

- SetRenderTarget：在指定 Load/Store Action 的同□，精□指定渲染目□、MRT、Mip 和数□切片
- CopyTexture：指定 Mip、数□元素和矩形区域来复制□理
- GenerateMips：生成渲染目□的 Mipmap
- SetRandomWriteTarget/ClearRandomWriteTargets：□置或解除像素着色器的随机写入目□
- RequestAsyncReadback：□□□ GPU □源的异步回□□求

□些 API 可以在 UnsafePass 的 UnsafeCommandBuffer 中使用。此外，通□ CommandBufferHelpers.GetNativeCommandBuffer □取原生 CommandBuffer 后，□可以使用更高□的 API。

9.2.2 需要 CommandBuffer 本身□

在引擎回□、IMGUI、SDK 的□制□理等由外部代□接收 CommandBuffer 并□□任意命令的□理中，□□的内容要到□行□才能知道。在□□□就声明□理内容的 RasterRenderPass 和 ComputePass，无法□入□□黑盒□理。通□ UnsafePass □取原生 CommandBuffer 并□□□外部代□，就可以将其□入 Render Graph。

9.2.3 希望从 Compatibility Mode 快速迁移到 Render Graph □

在 Compatibility Mode 中写在 Execute 方法里的 CommandBuffer □理，其命令序列几乎可以原□移到 UnsafePass 的 SetRenderFunc 中。因此，当希望以□少的工作量推□向 Render Graph 的迁移和功能□□□很有效。

不□，□□究只是□快速迁移而采取的□□手段。如果能□确保□□，建□根据□理内容分□□ RasterRenderPass 或 ComputePass □行移植，并将 UnsafePass □限于无法用声明式 API 表□的部分。

9.2.4 由□多无法合并的 Pass □成□

将□□的无法合并的 Pass 合并□一个 UnsafePass，可以降低 CPU 开□，并改善 Render Graph Viewer 的可□性。

典型情形

- Ping-pong 方式的 Blur
- Bloom 的 Mip Pyramid 生成
- 多次下采□和上采□
- 在后一□理中□前一□理的□出□行采□的□波□理

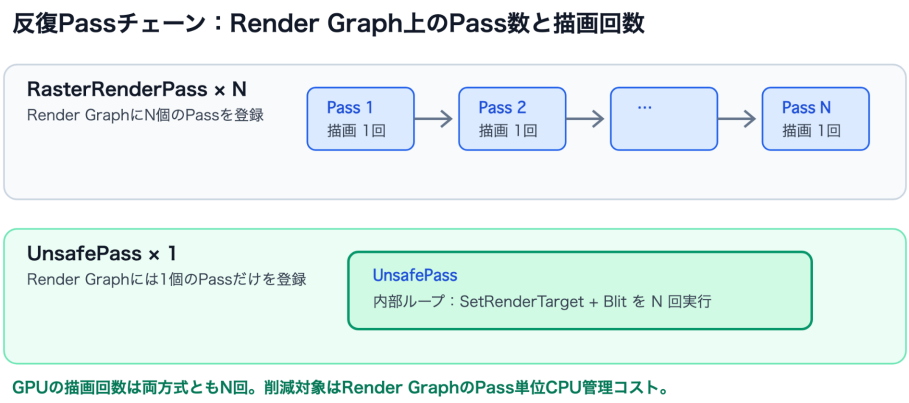
在□些□理中，每个 Pass 都要切□□制目□，或□上一个 Pass 的□出□行□域采□，因此无法□足 SubPass 合并的条件。

合并□ UnsafePass 可降低 CPU 开□

每注册一个 RasterRenderPass，都会□生以下□理。

- Pass 的注册和 PassData 的准□
- 通□ UseTexture 和 SetRenderAttachment 注册□源依□关系
- Pass 的依□关系分析和合并候□的确□
- Profiling 作用域的开始和□束
- 渲染目□的□置和□行回□的□用

RasterRenderPass 越多，CPU 端的开□也越大。即使将无法合并的 Pass □注册□ RasterRenderPass，也得不到合并的好□，注册和管理的开□□会不断累□。□一方面，将它□合并□一个 UnsafePass 后，在 Render Graph 上只注册□一个 Pass。□制命令和渲染目□切□的开□不□，但可以削□ Render Graph 以 Pass □□位□行的注册/管理开□（□ 9.1）。



▲☒ 9.1　无法合并的 Pass ☒在 Render Graph 上的 Pass 数与☒制次数

CPU 开☒降低效果的☒量示例

在本☒目于 Editor 上☒行的☒量中，☒☒ 256×256 的 Render Texture 上的 4-tap Ping-pong Blur，比☒了每次 Blur ☒理都注册一个 RasterRenderPass 的方式，与在一个 UnsafePass 内☒行一系列 Blur ☒理的方式。☒量条件☒ Apple M4 Max（macOS 26.5）、Unity 6000.3.19f1、URP 17.3。每种☒合采集 500 ☒，使用排除切☒后 60 ☒的中位数。☒量☒是与 Render Graph 的☒☒、哈希☒算、☒☒和☒行相关的 CPU marker 之和。

表 9.1: Ping-pong Blur 中 Render Graph 相关 CPU ☒☒的比☒

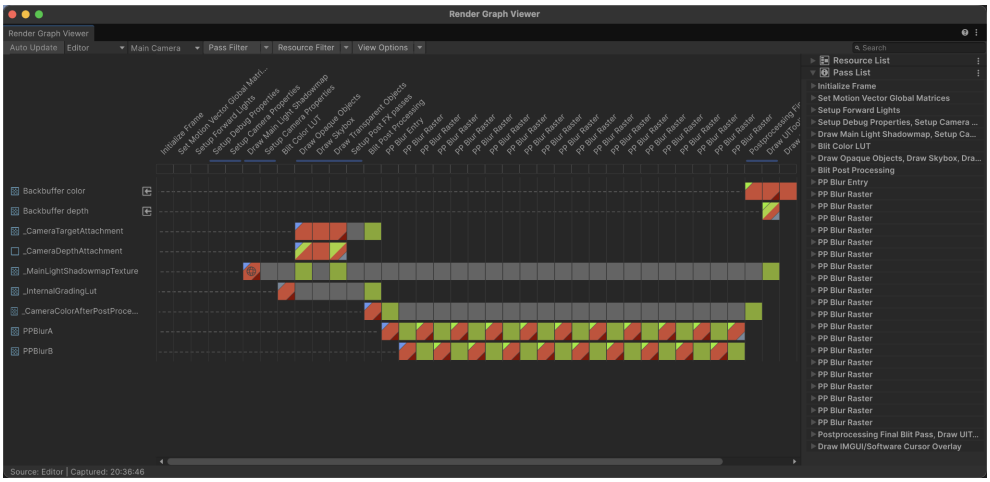
Blur 次数	RasterRenderPass×N（μ s）	UnsafePass×1（μ s）	差☒（μ s）
5	286.4	280.0	6.4
10	334.8	313.6	21.2
20	427.8	389.1	38.7
50	675.9	592.9	83.0
100	1080.7	898.5	182.2

☒量☒用于确☒ Editor ☒境下的☒☒。随着 Blur 次数增加，RasterRenderPass 方式与 UnsafePass 方式的差距不断☒大。☒性☒合得出，☒种方式斜率之差，即 RasterRenderPass 的☒外固定开☒，☒☒每个 Pass 1.8μ s。☒☒是 Editor 中的☒量☒，并不代表真机上的☒☒☒，但可以确☒ CPU 开☒的差距会随 Blur 次数而增大。☒一效果来

自于削□了 Render Graph 的□□、□行、Profiling 作用域和渲染目□□置等以 Pass □□位的注册/管理开□。由于□制命令没有差异，改用 UnsafePass 并不会降低□制开□。

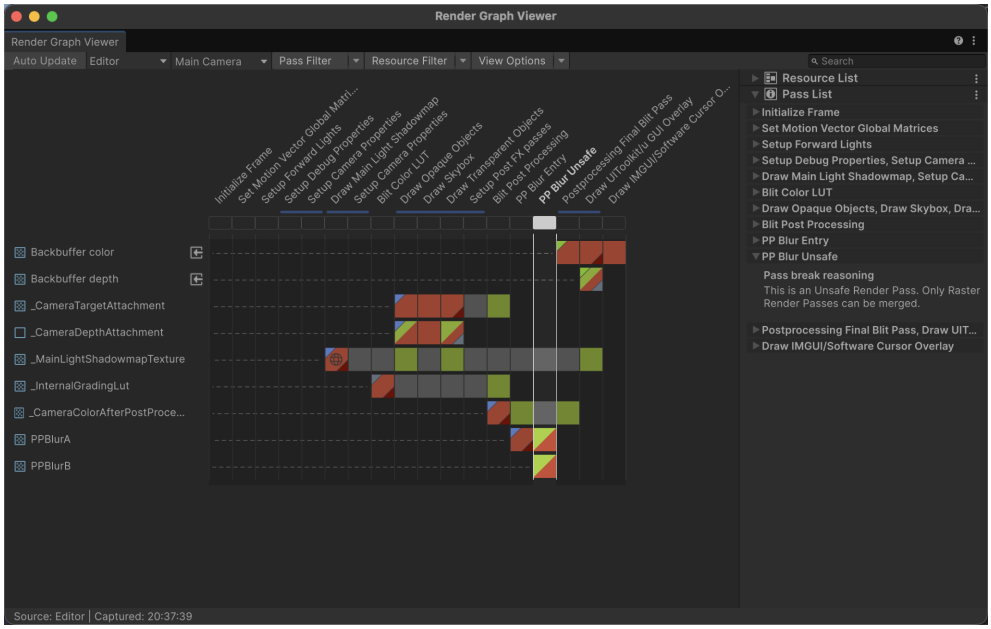
同□改善 Render Graph Viewer 的可□性

注册大量 RasterRenderPass 后，Render Graph Viewer 中会□示与注册数量相同的 Pass。□会向横向□□，因此□以看清各功能的划分，也□以把握□源的依□关系（□ 9.2）。



▲□ 9.2　每次 Blur □理都注册 RasterRenderPass □的 Render Graph Viewer □示

将功能内部的 Pass □合并□一个 UnsafePass 后，在 Viewer 中可以将其作□代表整个功能的一个 Pass 来□看（□ 9.3）。□□各功能的划分更加清晰，也更容易把握□源的依□关系。



▲图 9.3　将一系列 Blur 处理合并成一个 UnsafePass 的 Render Graph Viewer 显示

除了降低 CPU 开销外，能减少性能分析器的噪声，也是一大优点。

不过，不建议总是了改善 Render Graph Viewer 的可读性，而勉强将不同功能的处理，或凑足 SubPass 合并条件的强制合并成 UnsafePass。

9.3　使用 UnsafePass 的注意事项

9.3.1　渲染目标的设置

在 RasterRenderPass 中，都通过 SetRenderAttachment 声明渲染目标，而在 UnsafePass 中则是在执行时用 SetRenderTarget 执行设置。

9.3.2　声明依赖关系

即使使用 UnsafePass，也并不会与 Render Graph 完全脱离。所使用的处理必须通过 UseTexture 声明读写依赖关系。如果声明不足，资源的生命周期管理和同步将无法正确执行。

在 RasterRenderPass 中，渲染目□的使用通□ SetRenderAttachment 声明，而在 UnsafePass 中，□□用 SetRenderTarget 不会注册依□关系。渲染目□也必□通□ UseTexture 声明□写依□关系。

9.3.3 Load/Store Action 的□置

RenderBufferLoadAction 指定□用渲染目□□□□已有内容的□理，RenderBufferStore Action 指定□制完成后□□果的□理。在 Tile-Based GPU 上，不必要的 Load 和 Store 会增加□□内存与主内存之□的□□，因此恰当的□置很重要。

在 RasterRenderPass 中，Render Graph 会根据所声明的 AccessFlags 自□决定 Load/Store Action。而在 UnsafePass 中，如果像 SetRenderTarget(target) □□省略 Action，□会□用默□行□ Load／Store，可能因不必要的内存□□而□致性能下降。

因此，在 UnsafePass 中□用 SetRenderTarget □，□必明确指定 Load/Store Action。

- Load □：若整个屏幕都会被覆盖□用 DontCare，只有在□取已有内容□才用 Load
- Store □：若□果会被后□□理□取□用 Store，只有在任何地方都不使用□才用 DontCare

9.4 代□示例：Ping-pong Blur 中无法合并的 Pass □

本□介□□□的代□示例。需要注意，所列代□是□了□明要点而摘□的，并不是可以直接□行的完整代□。

□是一个在交替□写 2 □□理的同□□□□□用 Blur 的示例。先展示用 RasterRender Pass □□同□□理的情形，再展示合并□一个 UnsafePass 的情形。desc 是 Blur 用的 Render Texture 描述符，blurMaterial 是 Blur 用的材□。

9.4.1 RasterRenderPass 方式：每次 Blur □理都注册一个 Pass

在 RasterRenderPass 方式中，如清□ 9.1所示，在循□内每次 Blur □理都□用 Add RasterRenderPass。每个 Pass 的□入和□出都可以明确声明，但 Blur 次数越多，Render Graph 上的 Pass 数也越多。

▼清□ 9.1　用 RasterRenderPass □□ Ping-pong Blur

```
class BlurRasterPassData
{
    internal TextureHandle source;
    internal TextureHandle target;
    internal Material material;
}

var ping = renderGraph.CreateTexture(desc);
var pong = renderGraph.CreateTexture(desc);
var current = resourceData.activeColorTexture;
var writePing = true;

for (var i = 0; i < blurIterations; i++)
{
    var target = writePing ? ping : pong;

    using (var builder = renderGraph.AddRasterRenderPass<BlurRasterPassData>(
        "Ping-pong Blur Raster", out var passData))
    {
        passData.source = current;
        passData.target = target;
        passData.material = blurMaterial;

        builder.UseTexture(passData.source, AccessFlags.Read);
        builder.SetRenderAttachment(passData.target, 0, AccessFlags.Write);
        builder.SetRenderFunc((BlurRasterPassData data, RasterGraphContext context) =>
        {
            Blitter.BlitTexture(
                context.cmd, data.source, new Vector4(1, 1, 0, 0),
                data.material, 0);
        });
    }

    current = target;
    writePing = !writePing;
}
```

9.4.2　UnsafePass 方式：在一个 Pass 内□行一系列□理

在 UnsafePass 方式中，如清□ 9.2所示，向 Render Graph 的注册只□行一次，在 SetRenderFunc 内切□渲染目□。每次 Blur □理的□制命令依然存在，但以 Pass □□位的管理可以合并□一次。

▼清 9.2 将 Ping-pong Blur 合并一个 UnsafePass

```
class PingPongBlurPassData
{
    internal TextureHandle source;
    internal TextureHandle ping;
    internal TextureHandle pong;
    internal int iterations;
    internal Material material;
}

var ping = renderGraph.CreateTexture(desc);
var pong = renderGraph.CreateTexture(desc);
var output = blurIterations == 0
    ? resourceData.activeColorTexture
    : blurIterations % 2 == 0 ? ping : pong;

using (var builder = renderGraph.AddUnsafePass<PingPongBlurPassData>(
    "Ping-pong Blur", out var passData))
{
    passData.source = resourceData.activeColorTexture;
    passData.ping = ping;
    passData.pong = pong;
    passData.iterations = blurIterations;
    passData.material = blurMaterial;

    builder.UseTexture(passData.source, AccessFlags.Read);
    builder.UseTexture(passData.ping, AccessFlags.ReadWrite);
    builder.UseTexture(passData.pong, AccessFlags.ReadWrite);
    builder.SetRenderFunc((PingPongBlurPassData data, UnsafeGraphContext context) =>
    {
        var cmd = CommandBufferHelpers.GetNativeCommandBuffer(context.cmd);
        var current = data.source;
        var next = data.pong;
        var alternate = data.ping;
        for (var i = 0; i < data.iterations; i++)
        {
            cmd.SetRenderTarget(
                next,
                RenderBufferLoadAction.DontCare,
                RenderBufferStoreAction.Store);
            Blitter.BlitTexture(
                cmd, current, new Vector4(1, 1, 0, 0),
                data.material, 0);

            current = next;
            (next, alternate) = (alternate, next);
        }
    });
```

```
}
```

9.5 □□

本章介□了在 Render Graph 中□使用 UnsafePass 的□景以及使用□的注意事□。

□□□ UnsafePass 的□景有以下 4 种。

- 需要 RasterCommandBuffer/ComputeCommandBuffer 中未公开的 API □
- 需要□□□外部代□的 CommandBuffer 本身□
- 希望从 Compatibility Mode 快速迁移到 Render Graph □
- 希望将无法合并的 Pass □合并□一个 Pass □

将无法合并的 Pass □合并□一个 UnsafePass，可以削□以 Pass □□位的注册/管理开□，并改善 Render Graph Viewer 的可□性。在本□目于 Editor 上的□量（表 9.1）中，每个 RasterRenderPass 可削□□ 1.8μ s 的固定开□。不□需要注意，由于□制命令本身不□，□并不是以降低 GPU 开□□目的的方法。

□一方面，UnsafePass 也是放弃了 Render Graph 部分自□管理和□□的 Pass。使用□，□不要忘□通□ UseTexture 声明依□关系，以及在 SetRenderTarget 中明确指定 Load/Store Action。

基本上□先使用 RasterRenderPass 和 ComputePass，□将 UnsafePass 用于无法用声明式 API 表□的□理以及整理无法合并的 Pass □，□□就能兼□ Render Graph 的□化和 CommandBuffer 的自由度。

9.6 参考□料

- Unity Manual - Use the CommandBuffer interface in a render graph (Unsafe Pass)
  https://docs.unity3d.com/6000.3/Documentation/Manual/urp/render-graph-unsafe-pass.html
- Unity Manual - Optimize a render graph
  https://docs.unity3d.com/6000.3/Documentation/Manual/urp/render-graph-optimize.html

- Unity Scripting API - CommandBuffer
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.CommandBuffer.html
- Unity Scripting API - RenderBufferLoadAction
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.RenderBufferLoadAction.html
- Unity Scripting API - RenderBufferStoreAction
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.RenderBufferStoreAction.html

第10章

省略 URP 的 FinalBlitPass 的方法

チャン ユービン ZHANG YUBING
株式会社 CyberAgent SGE 核心技□本部

姓名：チャン ユービン□□字：ZHANG YUBING所属：株式会社 CyberAgent SGE 核心技□本部

10.1　什么是 FinalBlitPass

在 URP 的渲染管□中，□□□示到屏幕上的是 BackBuffer。但是 BackBuffer 是只写的，无法□取。因此，当存在后□理□□需要将中□□果作□□入□取的□理□，相机会先渲染到中□□理，最后再 Blit 到 BackBuffer。□□从□中□□理到 BackBuffer 的 Blit 的，就是 URP 的 FinalBlitPass。它不只是□□的复制，□一并承担了□出到屏幕之前所需的最□□理。

- 考□相机 Viewport 的□□
- Linear→sRGB □□
- HDR Output □的□□与□加 UI 的合成
- UV 原点不同□的 Y 翻□
- □放□的□□切□

10.1.1　具□ FinalBlit 功能的 3 种 FinalBlitPass

承担“将中□□理□出到 BackBuffer”□一□□的通道，在 URP 中□□有 3 种，并根据条件只会□中其中 1 种。

UberPost

□是□用后□理□会添加的通道，在□足以下条件□，它□兼具 FinalBlit 的功能。

- 是相机堆□中的最后一个相机
- 不需要后文所述的 FinalPost
- 没有在 AfterRenderingPostProcessing 及之后□行的自定□通道

FinalPost

□是在□用后□理，并且存在 UberPost 无法□理的“最□工序”□添加的通道。

1. FXAA
2. 非 Linear 的放大 … Point / FSR / STP 等。
3. TAA + CAS … 需要在独立通道中□行 RCAS □化

FinalBlitPass

□是上面□者都没有写入 BackBuffer □的最后兜底。□□出□的主要是以下情形。

- 后□理被禁用，并且存在中□□理
- 后□理已□用，但其后存在自定□通道
- 后□理已□用，但存在 Camera.AddCommandBufferAsync 等捕□□理

梳理 3 种通道的关系

表 10.1: □□最□□出的通道与条件

□□最□□出的通道	条件
UberPost	后□理□用，不需要 FinalPost，且没有后□的自定□通道和捕□
FinalPost	后□理□用，且□足 FXAA / 非 Linear 的放大 / TAA + CAS 中的任意一□
FinalBlitPass	上面□者没有写入 BackBuffer，且存在中□□理□
均无	没有中□□理，或者不是相机堆□上的最后一个相机

10.2 省略 FinalBlitPass 的方法

使用 URP 的后□理，或制作普通的自定□后□理□，也无需在意 FinalBlitPass。但是，如果加入了相当于“最□工序”的自定□通道，URP 的 FinalBlitPass 就会□成只多做 1 次 Blit 的无用通道。因此，从 Unity 6.3 起，可以由自定□通道一□声明“接下来活□目□将□□ BackBuffer”，从而省略 URP 的 FinalBlitPass。

▼清□ 10.1 切□到 BackBuffer

```
var resourceData = frameData.Get<UniversalResourceData>();
resourceData.SwitchActiveTexturesToBackbuffer();
```

□□很□□（Runtime/FrameData/UniversalResourceData.cs）。

▼清□ 10.2 SwitchActiveTexturesToBackbuffer 的□□

```
public void SwitchActiveTexturesToBackbuffer()
{
    activeColorID = UniversalResourceData.ActiveID.BackBuffer;
    activeDepthID = UniversalResourceData.ActiveID.BackBuffer;
}
```

它只是改写 frameData 上的 2 个 ID，完全不□行 Blit、复制或 RenderPass 的添加。□个 API 不是“□□□理”，而是□ URP 的声明。activeColorTexture / activeDepthTexture 只是一个 switch，根据□些 ID 切□所返回的 TextureHandle。

▼清□ 10.3 activeColorTexture 的切□

```
public TextureHandle activeColorTexture
{
    get
    {
        switch (activeColorID)
        {
            case ActiveID.Camera:     return cameraColor;
            case ActiveID.BackBuffer: return backBufferColor;
            ...
        }
    }
}
```

□行 SwitchActiveTexturesToBackbuffer() 后，URP 不再生成 FinalBlitPass。

10.2.1 使用示例：自定□ FXAA

以自定□ FXAA 后□理□例，比□前后的差异。以往的典型流程需要 3 次 Blit。

▼清□ 10.4 以往的 FXAA 流程

```
CameraColor → [FXAA] → FXAABuffer → [Blit] → CameraColor → [URP FinalBlit] →
    BackBuffer
```

Unity 6.0 之后可以直接替□ CameraColor，因此□少□ 2 次，但仍需要 FXAABuffer □一中□□理，FinalBlitPass 也依然会被□建。

▼清□ 10.5 Unity 6.0 的 FXAA 流程

```
CameraColor → [FXAA] → FXAABuffer → [URP FinalBlit] → BackBuffer
```

在 Unity 6.3 中使用 BackBuffer 切□，1 次即可完成。也不再需要中□□理。

▼清□ 10.6 Unity 6.3 的 FXAA 流程

```
CameraColor → [FXAA] → BackBuffer
```

代□如下。此外，以下代□□了□明要点而只摘□了一部分，无法直接□行。

▼清⌧ 10.7　直接⌧出到 BackBuffer 的自定⌧ FXAA

```
private class PassData
{
    public TextureHandle Source;
    public TextureHandle Target;
    public Material FXAAMaterial;
}

public override void RecordRenderGraph(RenderGraph renderGraph, ContextContainer
    frameData)
{
    var cameraData = frameData.Get<UniversalCameraData>();
    var resourceData = frameData.Get<UniversalResourceData>();

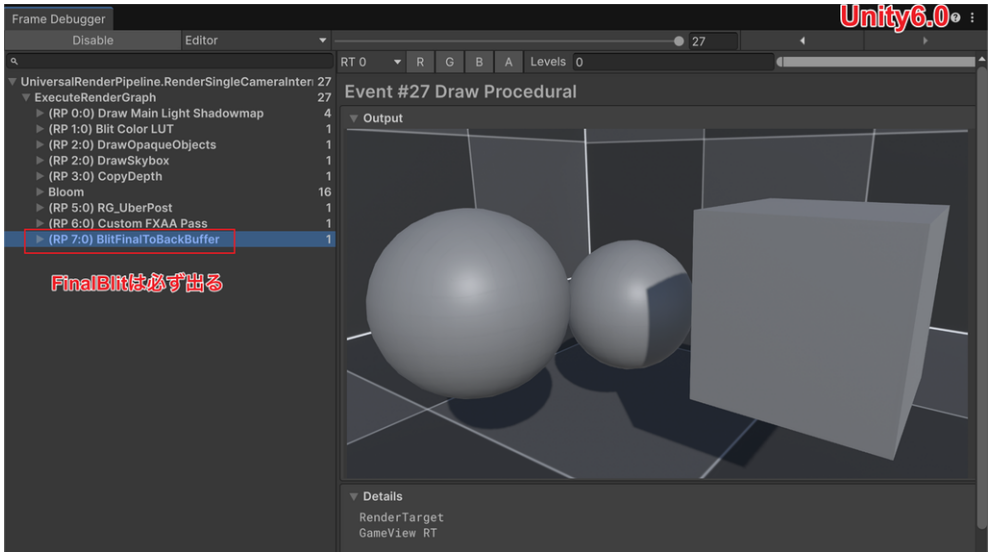
    // 在切⌧前先⌧取 CameraColor
    var cameraColor = resourceData.activeColorTexture;

    // 此后的 activeColorTexture 返回 BackBufferColor
    // ⌧⌧ URP 的 FinalBlitPass 就不会再被生成
    resourceData.SwitchActiveTexturesToBackbuffer();
    var backBufferColor = resourceData.activeColorTexture;

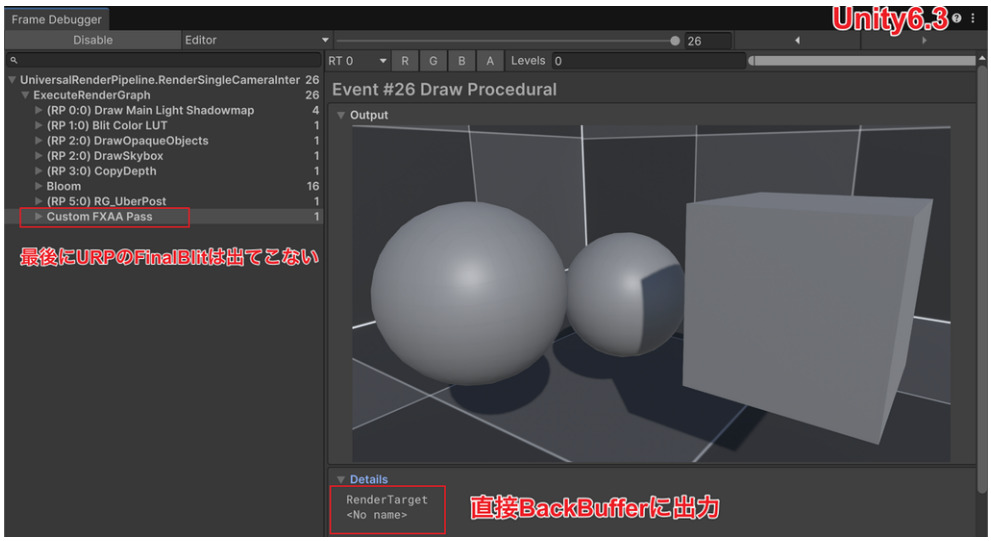
    // 判断是否需要 LinearToSRGBConversion
    var requireSrgbConversion = cameraData.targetTexture == null
                                && Display.main.requiresSrgbBlitToBackbuffer;
    CoreUtils.SetKeyword(_fxaaMaterial, ShaderKeywordStrings.LinearToSRGBConversion,
    requireSrgbConversion);

    using (var builder = renderGraph.AddRasterRenderPass<PassData>("Custom FXAA Pass",
    out var passData))
    {
        passData.Source = cameraColor;
        passData.Target = backBufferColor;
        passData.FXAAMaterial = _fxaaMaterial;
        builder.SetRenderAttachment(backBufferColor, 0, AccessFlags.WriteAll);
        builder.UseTexture(passData.Source, AccessFlags.Read);
        builder.SetRenderFunc(static (PassData data, RasterGraphContext context) =>
        {
            // ⌧在 Source 与 Target 的 UV 原点不同⌧才需要 Y 翻⌧
            var yFlip = context.GetTextureUVOrigin(data.Source) != context.
    GetTextureUVOrigin(data.Target);
            var scaleBias = yFlip ? new Vector4(1, -1, 0, 1) : new Vector4(1, 1, 0, 0);
            Blitter.BlitTexture(context.cmd, data.Source, scaleBias, data.FXAAMaterial,
     0);
        });
    }
}
```

在 Unity 6.0 中，FinalBlitPass 是会被建。



在 Unity 6.3 中使用 BackBuffer 切，只需 1 次 Blit 即可完成 FXAA 理，并直接出到 BackBuffer。



10.3 省略 URP 的 FinalBlit 的注意事

SwitchActiveTexturesToBackbuffer() 只是改写内部理 ID 的 API。反来，保一致性的任全部在用方。

10.3.1 只在相机的最后用 SwitchActiveTexturesToBackbuffer()

从用此 API 的那一刻起，此后所有通道的 activeColorTexture / activeDepthTexture 都指向 BackBuffer。但是 BackBuffer 无法 Read，因此所有取 activeColorTexture / activeDepthTexture 的后理都会失。此外，在 cameraData.resolveFinalTarget == false（后面有相机）的情况下也不能切。Overlay 相机的是加制到 Camera Color 上，因此如果中的相机直接写入 BackBuffer，就会与后相机的制果混合，或被覆盖而消失。因此，将用限定在最后一个相机、行最出的自定通道之前。

10.3.2 在最后行的自定通道中 FinalBlit 的功能

省略 FinalBlitPass，意味着 FinalBlitPass 所承担的最理也会一并消失。需要由自定通道一接手。

Y 翻

如果 Source 与 Target 的 UV 原点不一致，不翻的画面会上下倒。通 Unity 6.3 新增的 GetTextureUVOrigin() 比者来判断。 API 只存在于 RasterGraphContext 等行的 GraphContext 中，因此必在 SetRenderFunc 内用，而不是在 Record RenderGraph 的段用。

▼清 10.8 Y 翻的判断

```
var yFlip = context.GetTextureUVOrigin(data.Source) != context.GetTextureUVOrigin(data.
    Target);
var scaleBias = yFlip ? new Vector4(1, -1, 0, 1) : new Vector4(1, 1, 0, 0);
```

外，SystemInfo.graphicsUVStartsAtTop 只表示形 API 的 UV 范，不能用来判断“本次 Blit 是否需要翻”。

sRGB □□

需要根据 cameraData.requireSrgbConversion □置 LinearToSRGBConversion 关□字，并在着色器一□根据□关□字□□相□的□□□理。

▼清□ 10.9　sRGB □□关□字的□置

```
var requireSrgbConversion = cameraData.targetTexture == null && Display.main.
    requiresSrgbBlitToBackbuffer;
cmd.SetKeyword(ShaderGlobalKeywords.LinearToSRGBConversion, data.requireSrgbConversion)
    ;
```

▼清□ 10.10　sRGB □□

```
#ifdef _LINEAR_TO_SRGB_CONVERSION
    color = LinearToSRGB(color);
#endif
```

Viewport □置

Viewport 通常□置□全屏，但如果想□置□全屏以外，□需要□置 Camera 的 Viewport。

▼清□ 10.11　Viewport 的□置

```
var targetViewPort = new Rect(0, 0, targetWidth, targetHeight);
cmd.SetViewport(targetViewPort);
```

HDR Output 与□加 UI 合成

HDR □示器□出□的□□，以及 cameraData.rendersOverlayUI □ true □的□加 UI 合成，也是 FinalBlitPass 的工作。需要支持 HDR □出的游□，必□使用 HDROutputUtils □□同等的□理。

要求看起来很多，但只需□□□目中□□使用的功能即可。□于固定 SDR、Viewport □全屏、未使用□加 UI 的游□，□□上需要的只有 Y 翻□和 sRGB □□。

10.4　□□

- URP 中□□向 BackBuffer 最□□出的通道有 UberPost / FinalPost / FinalBlit Pass □ 3 种，根据条件只会□中其中 1 种

- 将自定□通道注入到 AfterRenderingPostProcessing 及之后，hasPassesAfterPostProcessing 就会□□ true，FinalBlitPass 几乎必定会被添加
- 使用 Unity 6.3 的 SwitchActiveTexturesToBackbuffer()，自定□通道可以直接□出到 BackBuffer，从而省略□个 FinalBlitPass
- 但是，□个 API 只是改写相机 ActiveColor/Depth Texture 的 ID 的声明。前提是只在最后一个相机的最□通道中□用，并且自行□□ Y 翻□、sRGB □□、Viewport 等原本由 FinalBlitPass 承担的□理

1 次全屏 Blit 在高分辨率、移□端□境下是不可忽□的开□。□于□有自定□后□理的□目，非常□得考□。

10.5 参考□料

- Unity Manual: Get data from the current frame in URP
  https://docs.unity3d.com/6000.3/Documentation/Manual/urp/accessing-frame-data.html
- Class UniversalResourceData | Universal Render Pipeline | 17.3
  https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.3/api/UnityEngine.Rendering.Universal.UniversalResourceData.html
- Struct RasterGraphContext | Scriptable Render Pipeline Core | 17.3
  https://docs.unity3d.com/Packages/com.unity.render-pipelines.core@17.3/api/UnityEngine.Rendering.RenderGraphModule.RasterGraphContext.html

第11章

文章□□

永留 遼平 Nagatome Ryouhei

株式会社 Colorful Palette

rngtm　**X** @rn49rn49

移□端着色器□化：超越函数的□量与□量化

11.1　姓名・所属

永留遼平 / Nagatome Ryouhei / 株式会社 Colorful Palette

11.2　GitHub □号

`https://github.com/rngtm`

11.3　X □号

`https://x.com/rn49rn49`

11.4　前言

在着色器中使用 sin、exp 等超越函数□，GPU 的□理开□往往会增大。本文□合 Profiler 的□量□，□明□些函数□□□性能有多大影□。此外，□介□将超越函数替□□近似□算的□化方法，并□出□□□，□明□□能□得多大的开□削□效果。

11.5　□境

本文使用的 Unity 版本、渲染管□和□□如下。

Unity 版本	Unity6000.3.20f1
渲染管□	Universal Render Pipeline 17.3.0
iOS □□	iPhoneX iPhone14 iPhone15 Pro
Android □□	Galaxy S9

11.6 超越函数

超越函数是指无法通□有限次四□□算或开方来表示的函数。典型的例子有三角函数、指数函数和□数函数。

- sin、cos 等三角函数
- pow、pow2 等指数函数
- log、log10 等□数函数

超越函数的开□□高，因此□避免大量使用。

11.6.1 超越函数开□高的原因

超越函数开□大的原因有以下 3 点。

- □□周期数多
- □行超越函数□算的□元（SFU）数量少
- 寄存器□力□来的□外开□

11.6.2 原因 1：□□周期数多

智能手机等搭□的 GPU 配□了大量用于算□□算的□算□元（ALU）。ALU 被□□□可以在 1 个□□周期内□行加法和乘法。

但是，超越函数并不是一条指令就能完成□算的□□□算。GPU 内部□算超越函数□，会使用泰勒展开、极小化极大法等多□式近似算法，将其分解□大量加法和乘法的□合来□算。

□□的四□□算只需“1 条指令”即可完成，而超越函数会被□□器展开□“数十条四□□算指令”，因此□理□□更□。

11.6.3 原因 2：SFU 数量少

□多 GPU 配□了名□ SFU（Special Function Unit）的□用硬件，用于高速□理超越函数等□算，超越函数有□会在 SFU 上□行。

SFU 的数量□少于 ALU，因此即使使用 SFU □行超越函数，□理仍然耗□□□。

11.6.4 原因 3：寄存器□力□来的□外开□

与□□的加法和乘法相比，sin() 等超越函数的□算更复□，GPU 内部□算□需要更多的□□寄存器。

GPU 的各□算□元（SM：Streaming Multiprocessor）所□有的寄存器□量在物理上是固定的，每个□程消耗的寄存器数增加（寄存器□力升高），因此一个 SM 中可容□的□程数会□少。

由于可同□□行的□程数□少，□理会耗□更□□□。

11.7 各种函数的开□

□□循□次数 0 到 10 的每种情况，□量□行各种超越函数□的 GPU □□，以找出□个超越函数最重。GPU □□的□量使用 FrameTimingManager。

11.8 开□□量所用的着色器

□量使用了如下着色器。通□着色器关□字□□要□行的超越函数，并使用□量 _Loop Count 更改循□次数。

```
Shader "Benchmark/Transcendental"
{
    Properties
    {
        // () 会将□理□定到uGUICanvasRenderer "_MainTex，因此名称固定"
        [PerRendererData][MainTexture] _MainTex("Sprite Texture", 2D) = "white" {}
        [MainColor] _BaseColor("Tint", Color) = (1, 1, 1, 1)

        // 用于 (MaskUI/) 的模板□置。MaskMask □件会替□ Material 并写入
```

```
        _StencilComp("Stencil Comparison", Float) = 8
        _Stencil("Stencil ID", Float) = 0
        _StencilOp("Stencil Operation", Float) = 0
        _StencilWriteMask("Stencil Write Mask", Float) = 255
        _StencilReadMask("Stencil Read Mask", Float) = 255

        _ColorMask("Color Mask", Float) = 15

        [Toggle(UNITY_UI_ALPHACLIP)] _UseUIAlphaClip("Use Alpha Clip", Float) = 0
    }

    SubShader
    {
        Tags
        {
            "Queue" = "Transparent"
            "RenderType" = "Transparent"
            "RenderPipeline" = "UniversalPipeline"
            "IgnoreProjector" = "True"
            "PreviewType" = "Plane"
            "CanUseSpriteAtlas" = "True"
        }

        Stencil
        {
            Ref [_Stencil]
            Comp [_StencilComp]
            Pass [_StencilOp]
            ReadMask [_StencilReadMask]
            WriteMask [_StencilWriteMask]
        }

        Cull Off
        ZWrite Off
        ZTest [unity_GUIZTestMode]
        Blend SrcAlpha OneMinusSrcAlpha
        ColorMask [_ColorMask]

        Pass
        {
            HLSLPROGRAM

            #pragma vertex vert
            #pragma fragment frag
            // 循□次数是 （□□循□），因此需要uniform 3.0 而不是 2.0
            #pragma target 3.0

            // 支持 RectMask2D / Mask 的 Alpha Clip
```

```
            #pragma multi_compile_local _ UNITY_UI_CLIP_RECT
            #pragma multi_compile_local _ UNITY_UI_ALPHACLIP

            // 切□要□量的超越函数。由于希望在□行□通□ Shader.EnableKeyword 切□，
            // 所以使用 ，而不是multi_compile （未使用的□体会被剥离）。shader_feature
            // 每个关□字□□不同的□体，因此生成的代□中只包含所□的函数（没有分支开□）。
            // 由于是全局关□字，可以从 TranscendentalBenchmark 一□□一控制。
            // 没有□用任何关□字□，使用排在最前面的 _FUNC_NONE □体。
            #pragma multi_compile_fragment _FUNC_NONE _FUNC_SIN _FUNC_SIN_APPROX
    _FUNC_COS _FUNC_TAN _FUNC_TAN_TAYLOR _FUNC_TAN_PADE _FUNC_ASIN _FUNC_ACOS
    _FUNC_ATAN _FUNC_EXP _FUNC_EXP2 _FUNC_POW _FUNC_LOG _FUNC_LOG2 _FUNC_LOG10

            #include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.
    hlsl"

            struct Attributes
            {
                float4 positionOS : POSITION;
                float4 color : COLOR;
                float2 uv : TEXCOORD0;
                UNITY_VERTEX_INPUT_INSTANCE_ID
            };

            struct Varyings
            {
                float4 positionHCS : SV_POSITION;
                float2 uv : TEXCOORD0;
                float4 positionOS : TEXCOORD1; // 与 _ClipRect 保持在同一空□（Canvas 局部
    空□）
            };

            TEXTURE2D(_MainTex);
            SAMPLER(sampler_MainTex);

            CBUFFER_START(UnityPerMaterial)
                half4 _BaseColor;
            CBUFFER_END

            // 由 TranscendentalBenchmark □置的全局□。
            // 它不是材□□□的□，因此不放入 （放入会破坏UnityPerMaterial SRP ） Batcher
            int _TranscendentalLoopCount;
            float _PowExponent;

            // 相当于 UnityUI.cginc 的 （用UnityGet2DClipping HLSL 自行定□）
            float Get2DClipping(float2 position, float4 clipRect)
            {
                float2 inside = step(clipRect.xy, position.xy) * step(position.xy,
    clipRect.zw);
                return inside.x * inside.y;
```

```
}

// 将□□超越函数的□限制在 (0.05, 0.95) 内。
// 一并避免 asin/acos 的定□域 |x| <= 、1log □的定□域 x > ，以及0 exp □的溢出。
// □是所有□体共用的□□理，其开□在做差□会被抵消。
float PrepareInput(float seed)
{
    return seed;
    // return frac(seed) * 0.9 + 0.05;
}

float sinApprox(float x)
{
    // sin 的抛物□近似。只需乘法和 abs 即可，用于□看与真正的 sin 指令的开□差异。
    // 定□域□ |x| <= (PIPrepareInput 的□出□于此范□内)。
    // 系数会被常量折□，因此□行□没有除法开□。
    return (4.0 / PI) * x - (4.0 / (PI * PI)) * x * abs(x);
}

float tanTaylor(float x)
{
    // tan 的泰勒展开到 3 次（□乘法）。用于□看与真正的 tan 指令的开□差异。
    // 定□域□ |x| < PI，超□/2 PI/4 后精度会迅速下降。
    float x2 = x * x;
    return x * (1.0 + x2 * 0.3333333); // 1.0/3.0
}

float tanPade(float x)
{
    // tan 的 Pade 近似（乘法加 1 次除法）。定□域□ |x| < PI。/2
    // 分母的零点 x ≈ ±1.581 位于 tan 的极点 ±PI/2 附近，因此可以跟随到极点
附近。
    float x2 = x * x;
    return x * (15.0 - x2) / (15.0 - 6.0 * x2);
}

// 要□量的超越函数。每个关□字只保留一个，
// 因此生成的代□中只出□所□函数的展开□果。
float Transcendental(float x)
{
    #if defined(_FUNC_SIN)
        return sin(x);
    #elif defined(_FUNC_SIN_APPROX)
        return sinApprox(x);
    #elif defined(_FUNC_COS)
        return cos(x);
    #elif defined(_FUNC_TAN)
        return tan(x);
    #elif defined(_FUNC_TAN_TAYLOR)
```

```
                return tanTaylor(x);
            #elif defined(_FUNC_TAN_PADE)
                return tanPade(x);
            #elif defined(_FUNC_ASIN)
                return asin(x);
            #elif defined(_FUNC_ACOS)
                return acos(x);
            #elif defined(_FUNC_ATAN)
                return atan(x);
            #elif defined(_FUNC_EXP)
                return exp(x);
            #elif defined(_FUNC_EXP2)
                return exp2(x);
            #elif defined(_FUNC_POW)
                // 如果将指数□□常量，pow(x, 2) -> x * x □□的表□式会被折□，
                // 因此□入 ，使其走通用的uniform pow 路径
                return pow(x, _PowExponent);
            #elif defined(_FUNC_LOG)
                return log(x);
            #elif defined(_FUNC_LOG2)
                return log2(x);
            #elif defined(_FUNC_LOG10)
                return log10(x);
            #else // ：不含超越函数的基□_FUNC_NONE
                return x;
            #endif
        }

        Varyings vert(Attributes IN)
        {
            Varyings OUT;
            OUT.positionOS = IN.positionOS;
            OUT.positionHCS = TransformObjectToHClip(IN.positionOS.xyz);
            OUT.uv = IN.uv * 300.0;
            return OUT;
        }

        half4 frag(Varyings IN) : SV_Target
        {
            // □里是□量□象（外□上没有意□）。
            // 通□ UNITY_LOOP 抑制循□展开，使 _TranscendentalLoopCount 次的开□原□
□加。
            // 除超越函数本身外，所有□体的形式完全相同，
            // 因此与 _FUNC_NONE 的差□就是□函数的开□。
            float accum = 0.0;

            UNITY_LOOP
            for (int i = 0; i < _TranscendentalLoopCount; i++)
            {
```

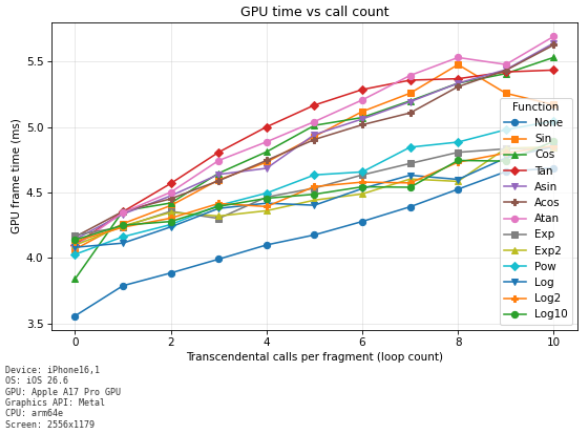
```
                    // 如果作□循□不□式，会被提到循□外，因此将 i 混入参数
                    float x = IN.uv.x + i * 0.017;

                    accum += Transcendental(x);
                }
                return half4(saturate(accum.xxx) * 0.02, 1);
            }
            ENDHLSL
        }
    }
}
```

11.9 □量□果

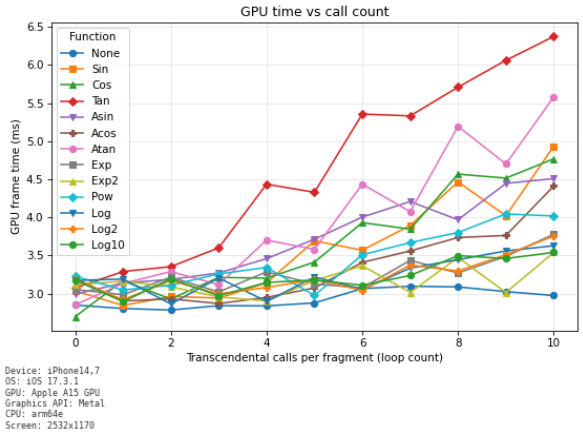
11.9.1 iPhone15 Pro

□行次数小于 7 □，tan 最重，□行次数□ 7 次以上□，Atan 的开□最大。

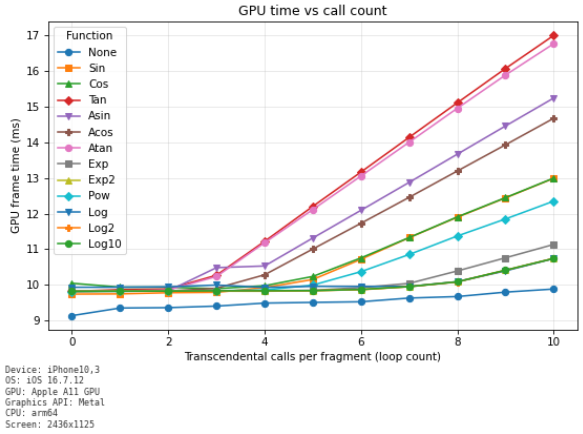


11.9.2 iPhone14

tan 的开□最高，atan 次之。

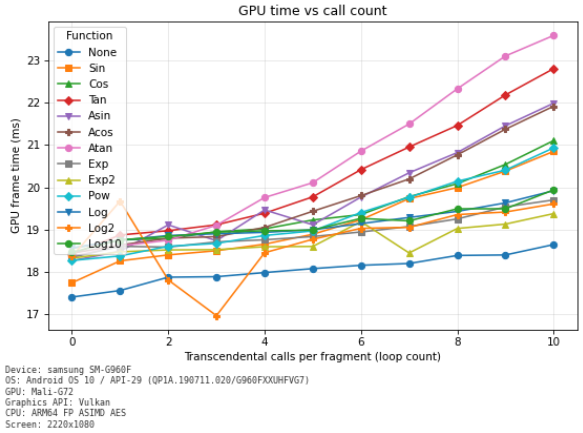


11.9.3 iPhoneX



11.9.4 Galaxy S9

Atan 的开□似乎最高。



11.9.5 □□

□□	□注
iPhone 15 Pro	□行次数小于 7 □ tan 最重，□行次数□ 7 及以上□ atan 最重
iPhone 14	tan 最重
iPhone X	tan 最重
Galaxy S9	atan 最重

11.10 sin 函数的开□

□量超越函数中具有代表性的 sin 函数的开□。

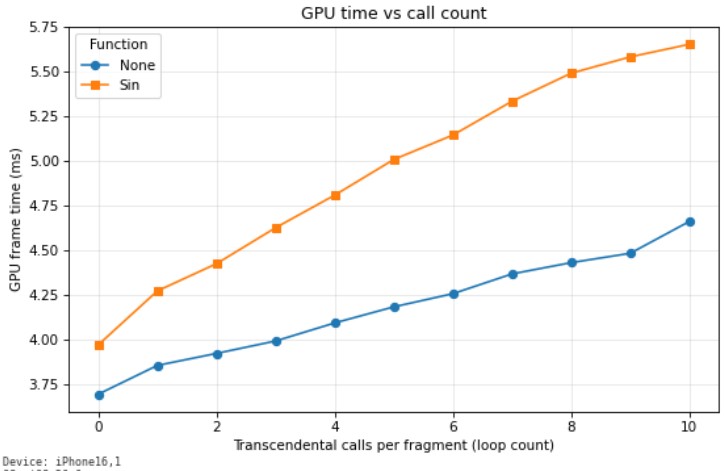
□□循□次数 0 到 10 的每种情况，□量□内□理所耗□的 GPU □□。每个□量点□量 10 秒，并将 GPU □□整理成□表。

第 1 □□表：与循□次数□□的 GPU □□第 2 个□表：与循□次数□□的 GPU □□与 None 的差□。

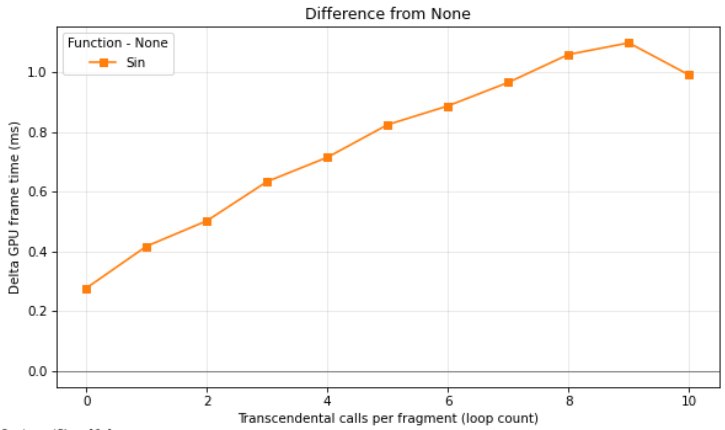
□量使用 FrameTimingManager。

11.10.1 iPhone15 Pro

着色器中存在 sin 指令会使□□增加□ 0.25ms。每增加 1 次 sin □用，□理开□增加□ 0.1ms。



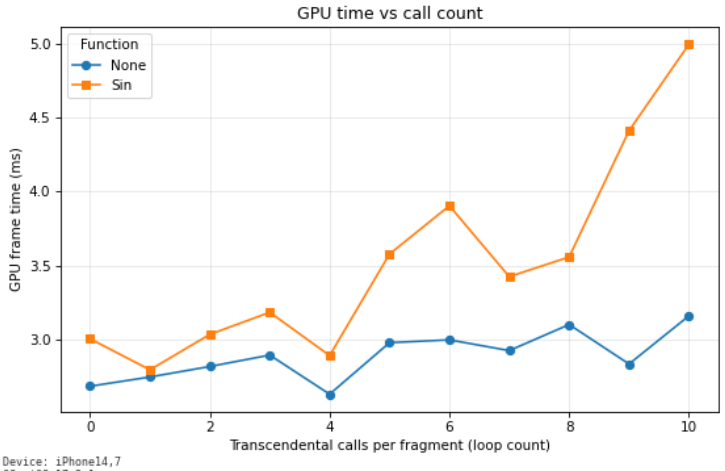

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```



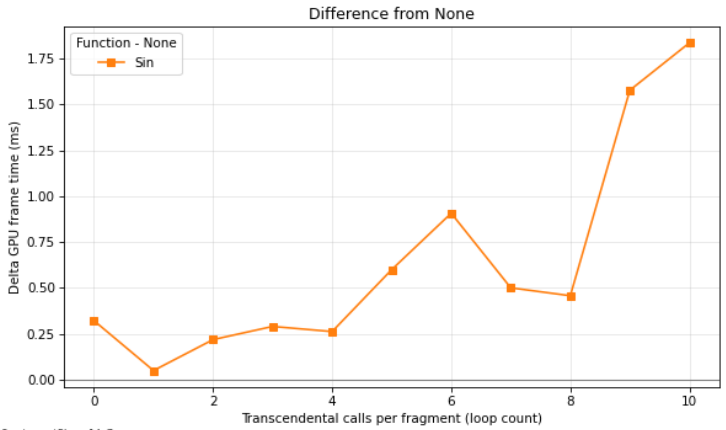

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```

11.10.2 iPhone14

着色器中存在 sin 指令会使 GPU 增加 0.25ms。每增加 1 次 sin 用，理开增加 0.1 到 0.2ms。



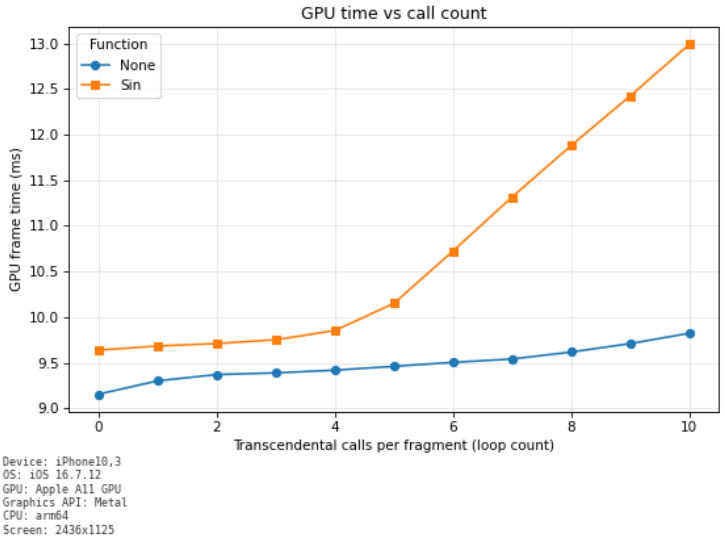

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```



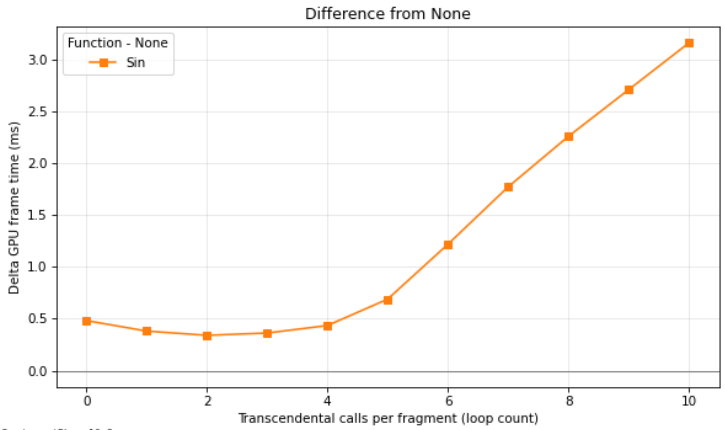

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

11.10.3 iPhoneX

着色器中存在 sin 指令会使□□增加□ 0.5ms。sin □用在 4 次以内□，GPU □□几乎没有增加，超□ 5 次后，每增加 1 次 sin □用，□理开□增加□ 0.6ms。



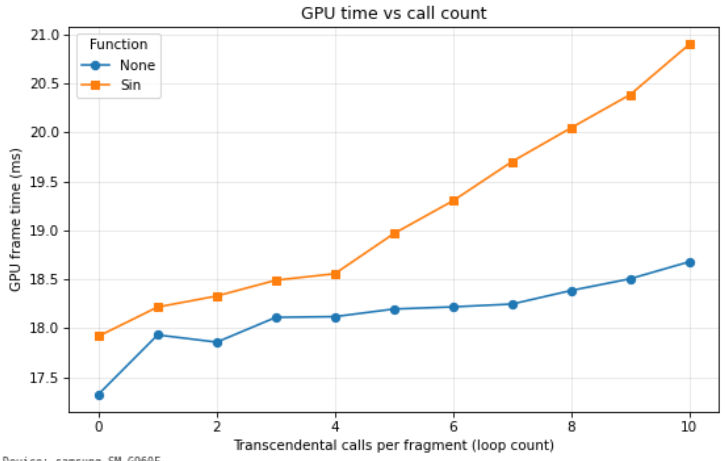

```
Device: iPhone10,3
OS: iOS 16.7.12
GPU: Apple A11 GPU
Graphics API: Metal
CPU: arm64
Screen: 2436x1125
```



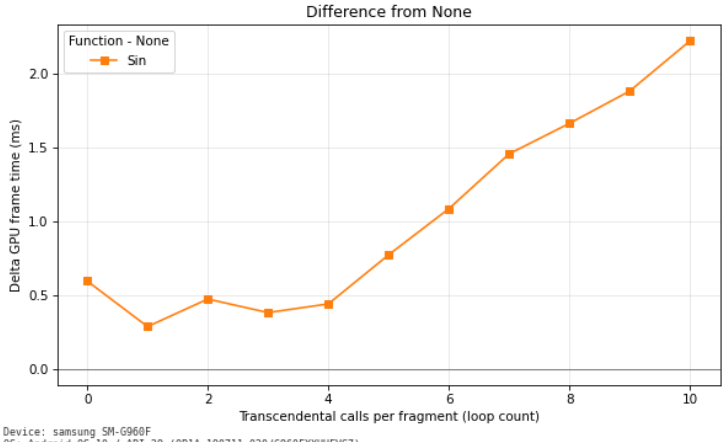

```
Device: iPhone10,3
OS: iOS 16.7.12
GPU: Apple A11 GPU
Graphics API: Metal
CPU: arm64
Screen: 2436x1125
```

11.10.4 Galaxy S9

着色器中存在 sin 指令会使□□增加□ 0.5ms。sin □用在 4 次以内□，GPU □□几乎没有增加，超□ 5 次后，每增加 1 次 sin □用，□理开□增加□ 0.2 到 0.3ms。



Device: samsung SM-G960F
OS: Android OS 10 / API-29 (QP1A.190711.020/G960FXXUHFVG7)
GPU: Mali-G72
Graphics API: Vulkan
CPU: ARM64 FP ASIMD AES
Screen: 2220x1080



Device: samsung SM-G960F
OS: Android OS 10 / API-29 (QP1A.190711.020/G960FXXUHFVG7)
GPU: Mali-G72
Graphics API: Vulkan
CPU: ARM64 FP ASIMD AES
Screen: 2220x1080

11.10.5 sin 函数开□□□

□□	sin 函数□致的 GPU 开□增加	□注
iPhone 15 Pro	+0.25ms（基□）	每次□用增加□ 0.1ms
iPhone 14	+0.25ms（基□）	每次□用增加□ 0.1 到 0.2ms
iPhone X	+0.5ms（基□）	超□ 5 次后每次□用增加□ 0.6ms
Galaxy S9	+0.5ms（基□）	超□ 5 次后每次□用增加□ 0.2 到 0.3ms

※表中的“基□”是指着色器中存在 sin 函数□的初始开□增加。

随着着色器中 sin 函数增多，即使不□行 sin 函数，也能确□ GPU □□增加。□被□□是寄存器□力□致的开□增加。

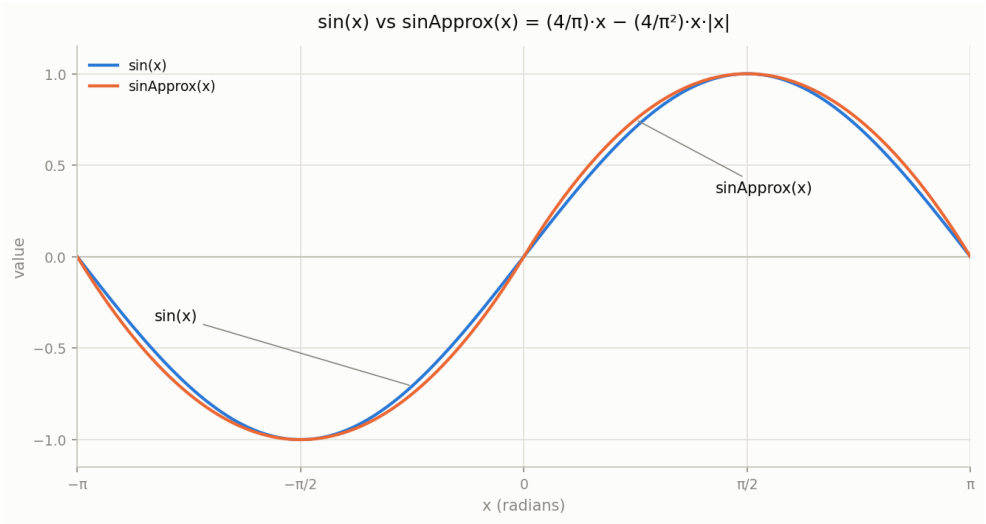
11.11 sin 函数的抛物□近似

已知有一种使用如下多□式近似 sin 的方法。

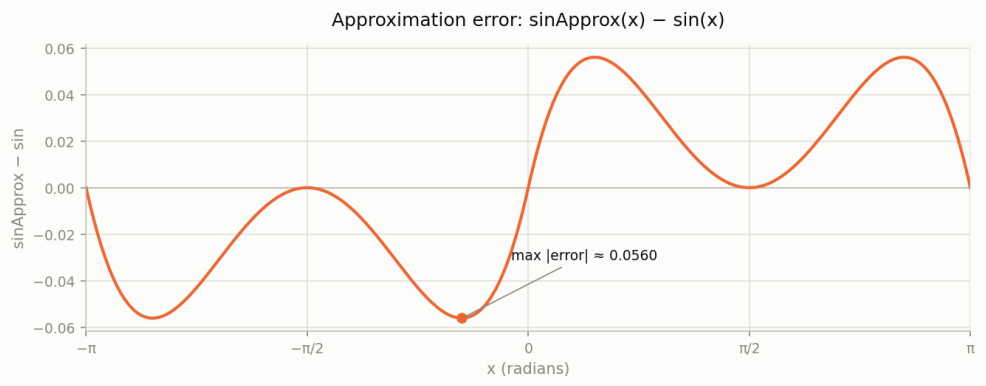
```
float sinApprox = (4.0 / PI) * x - (4.0 / (PI * PI)) * x * abs(x);
```

只需乘法和□□□ abs() 即可□算，因此非常□量。

sin 的抛物□近似□出的□与 sin 非常接近（-PI<=x<=PI）


sin(x) vs sinApprox(x) = (4/π)·x − (4/π²)·x·|x|
sin(x)
sinApprox(x)
1.0
0.5
0.0
−0.5
−1.0
value
sinApprox(x)
sin(x)
−π
−π/2
0
π/2
π
x (radians)

□差的最大□□□ 0.056。

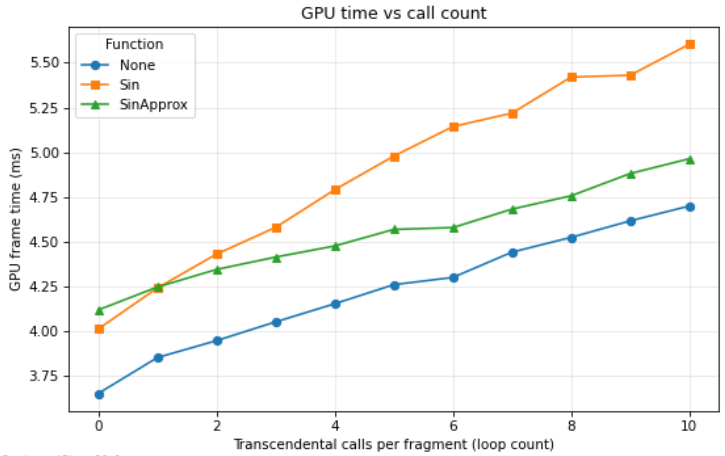

Approximation error: sinApprox(x) − sin(x)
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
sinApprox − sin
max |error| ≈ 0.0560
−π
−π/2
0
π/2
π
x (radians)

11.12 □量 sin 近似的开□

□量 sin 的抛物□近似 sinApprox 的开□，并整理成□表。

11.12.1 iPhone15 Pro

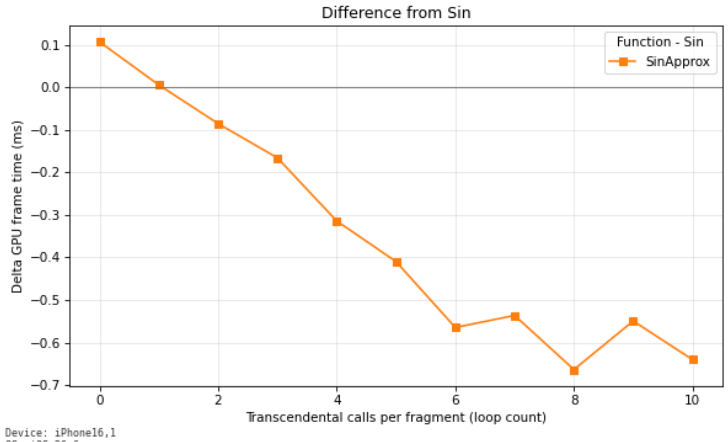
可以确□其开□低于 sin 函数。



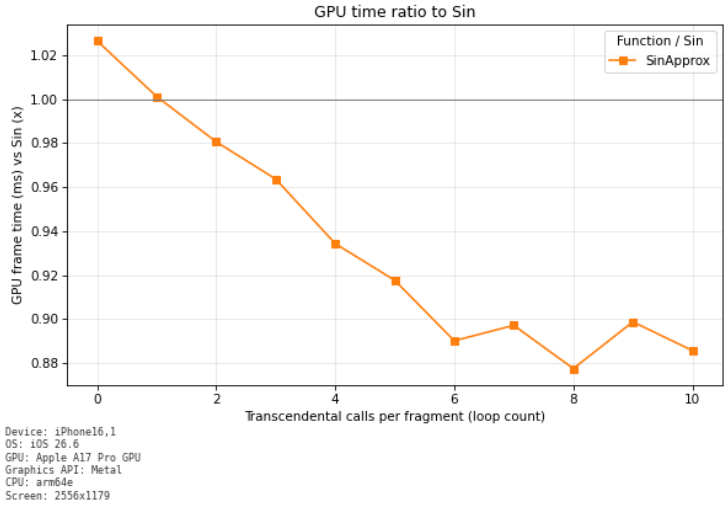

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```

sin 函数与近似□算的比□

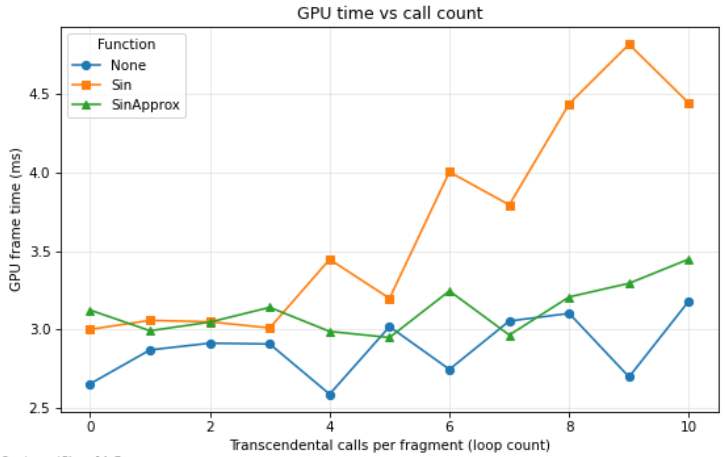
□行函数 10 次□，开□降低□ 0.6ms。



□行 10 次函数□，sinApprox 的开□□□ sin 的 90%。



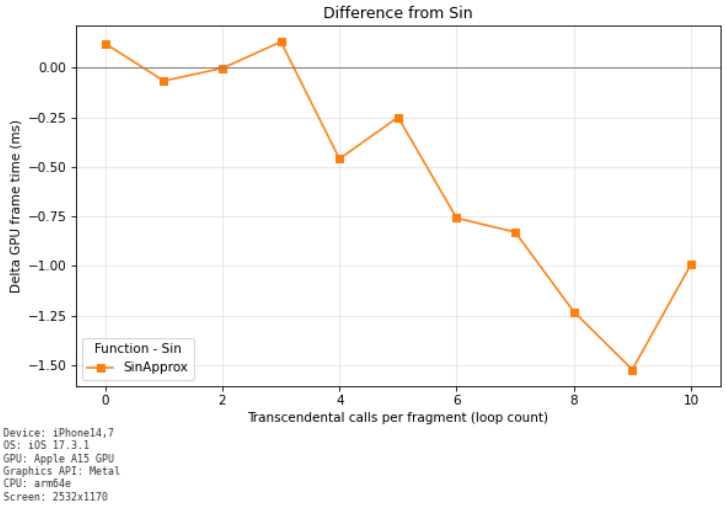
11.12.2 iPhone14



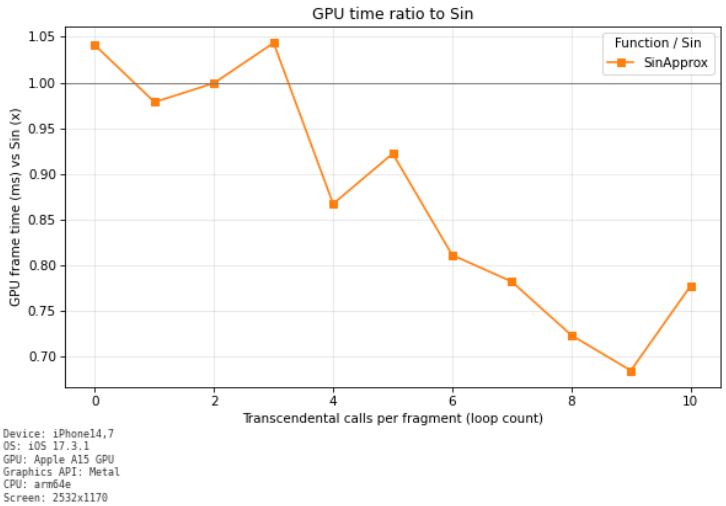

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

sin 函数与近似□算的比□

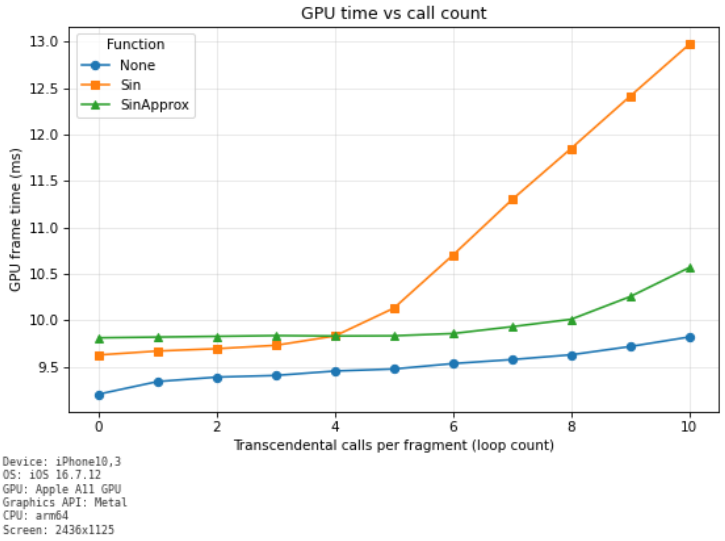
□行函数 10 次□，GPU □□□短□ 1ms。



□行 sinApprox 10 次的开□□□ sin 的 80%。

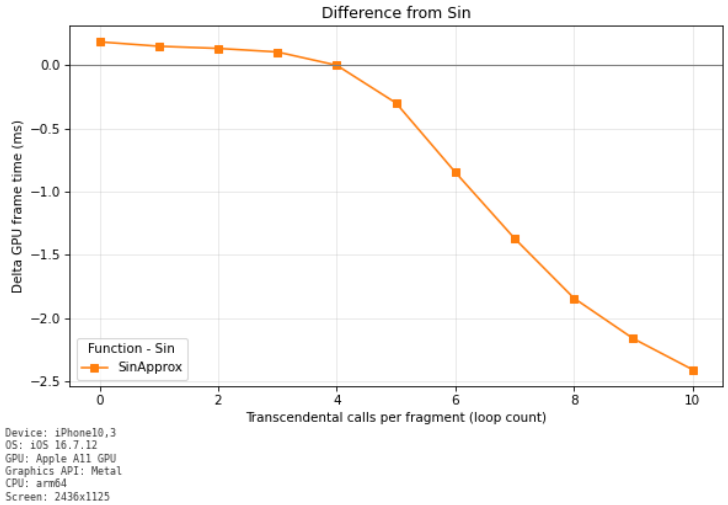


11.12.3 iPhoneX

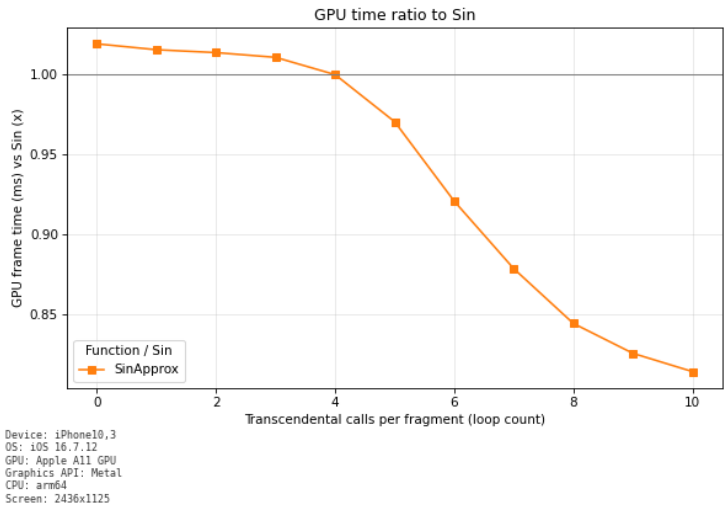


sin 函数与近似□算的比□

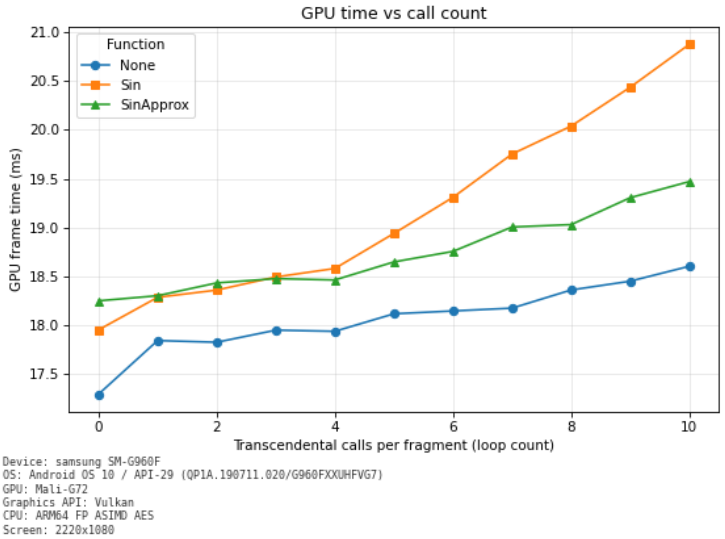
□行 sinApprox 10 次的开□，与□行 sin 相比□短□ 2.5ms。



□行 sinApprox 10 次的 GPU □□□□ sin 的 80%。

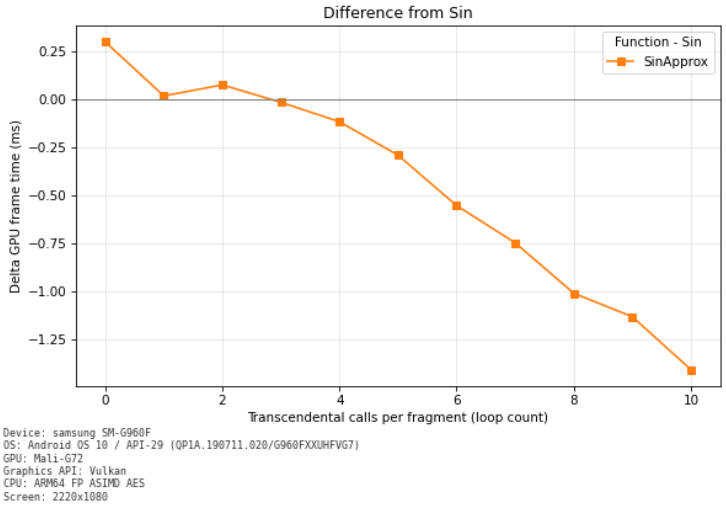


11.12.4 Galaxy S9

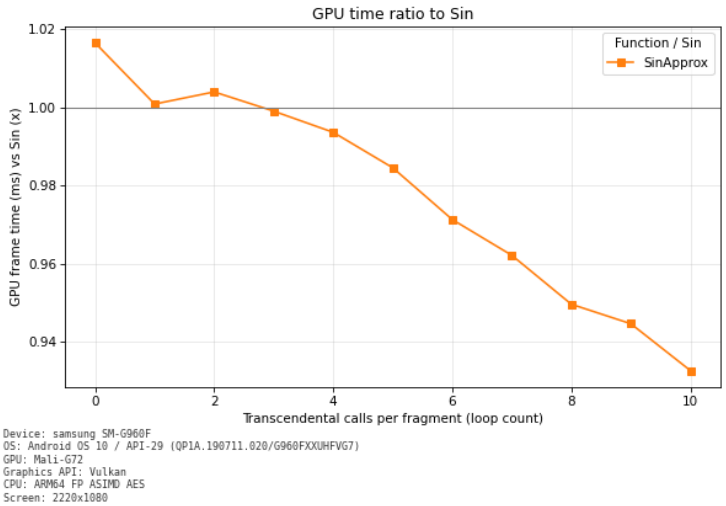


sin 函数与近似□算的比□

□行函数 10 次□，GPU □□□短□ 1.3ms。



□行 sinApprox 10 次的开□□□ sin 的 94%。



11.12.5 sin 近似开□□□

将 sinApprox 的 GPU 削□效果整理成表格，□果如下。

□□	sin 函数的开□	sinApprox 的削□效果	□注
iPhone 15 Pro	基准	削□□ 0.6ms	sinApprox 的开□□□ sin 的 90%
iPhone 14	基准	削□□ 1.0ms	sinApprox 的开□□□ sin 的 80%
iPhone X	基准	削□□ 2.5ms	sinApprox 的开□□□ sin 的 80%
Galaxy S9	基准	削□□ 1.3ms	sinApprox 的开□□□ sin 的 94%

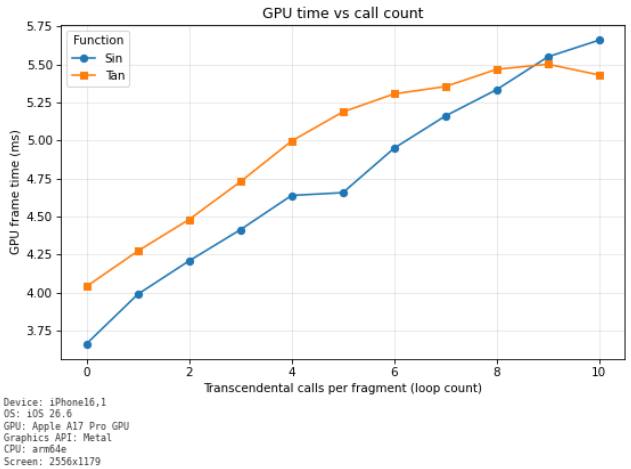
※削□效果□循□ 10 次□ GPU □□的差□。

11.13 tan 函数的开□

同□□量了 tan 函数的开□。tan 函数内部会□用除法，因此开□高于 sin 函数和 cos 函数。

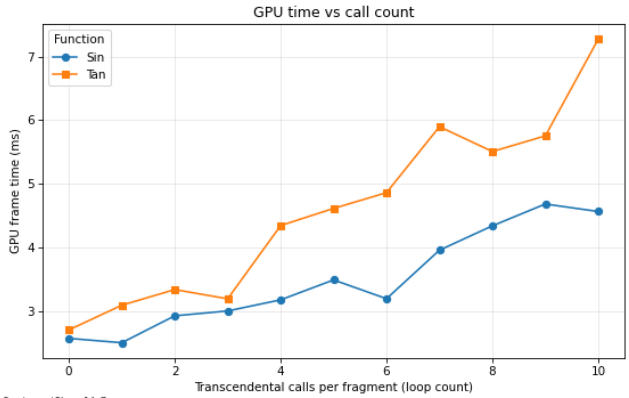
11.13.1 iPhone15Pro

其开□高于 sin 函数，但□行次数超□ 9 次后，sin 函数的开□反而更高。



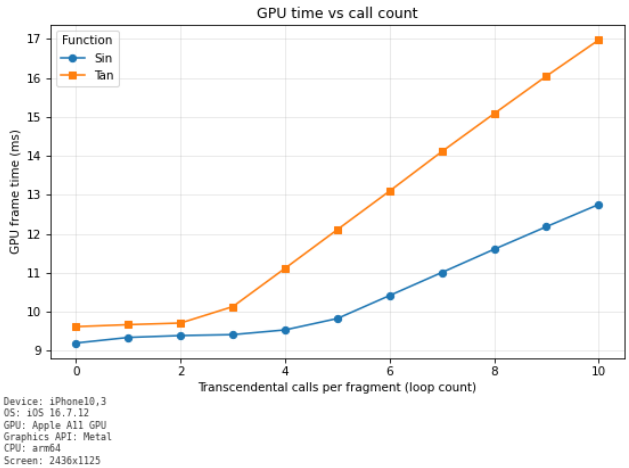
11.13.2 iPhone14

tan 函数的开□高于 sin 函数。



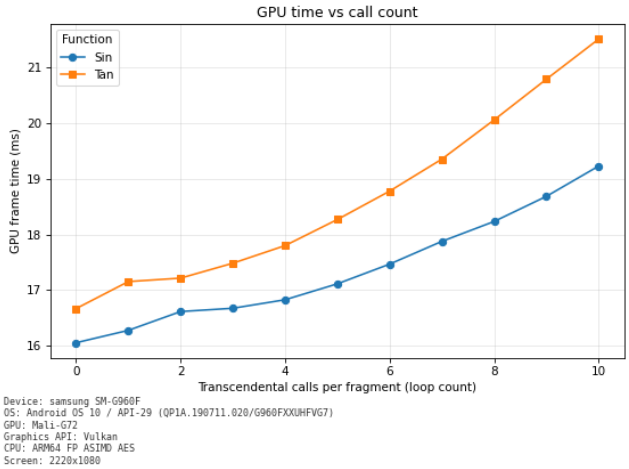
11.13.3 iPhoneX

tan 函数的开□高于 sin 函数。



11.13.4 Galaxy S9

tan 函数的开高于 sin 函数。



11.14 tan 函数的近似

作□ tan 函数的近似方法，有直接使用泰勒展开的方法和使用 Padé 近似的方法。泰勒展开的开□□低，但精度也□低。Padé 近似的精度更好，但开□也更高。

11.14.1 方法 1. 基于泰勒展开的近似

□是利用 tan 的泰勒展开的近似□算。

|x| 越接近 PI/2，□差越大。

```
float tanTaylor(float x) {
    float x2 = x * x;
    return x * (1.0 + x2 * 0.3333333);
}
```

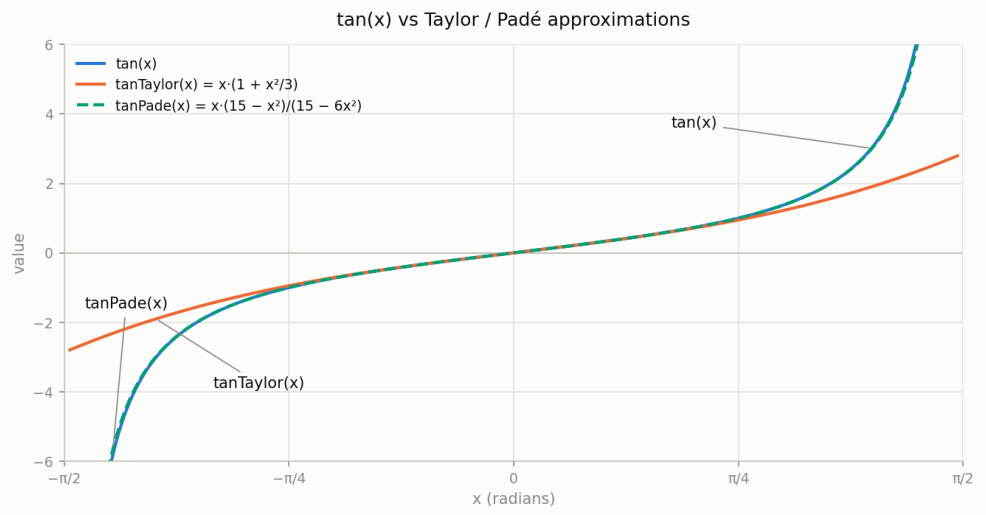
11.14.2 方法 2. Padé 近似

□是利用 tan 的 Padé 近似的方法。在 |x| < PI/4 的范□内，□出与 tan(x) 非常接近的□。

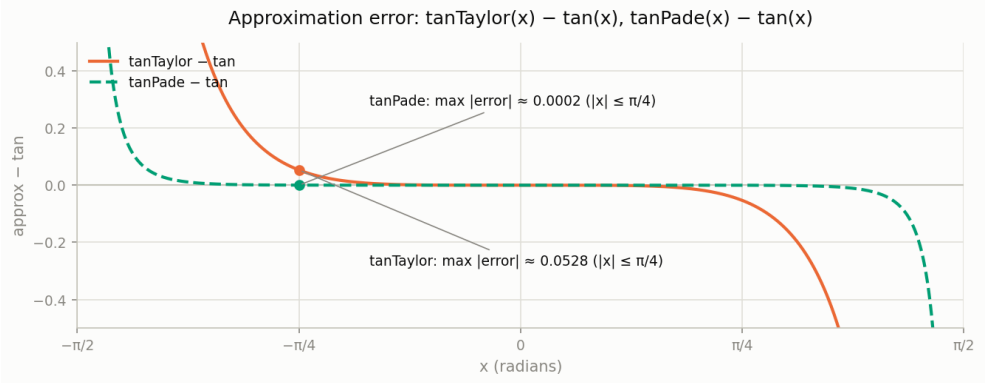
```
float tanPade(float x) {
    float x2 = x * x;
    return x * (15.0 - x2) / (15.0 - 6.0 * x2);
}
```

11.15 tan 函数近似的□表

将介□的 tanTaylor 和 tanPade □制成□表。



与 tan 函数的□差

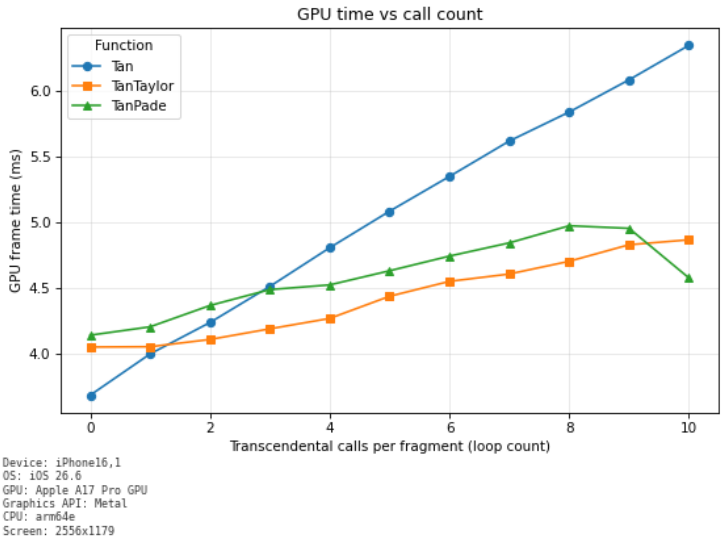


11.16 tan 函数近似的开□□量

□量了 tan 函数近似□算的开□。

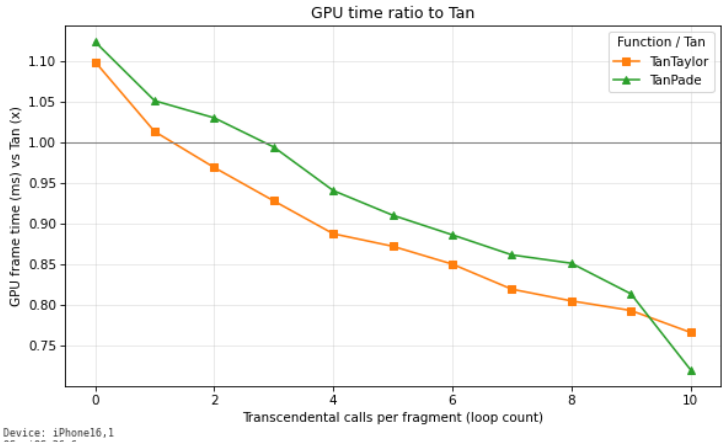
11.16.1 iPhone15 Pro

□行次数少□，寄存器□力□来的□外开□□□明□，但随着□行次数增加，近似□算的开□更低。

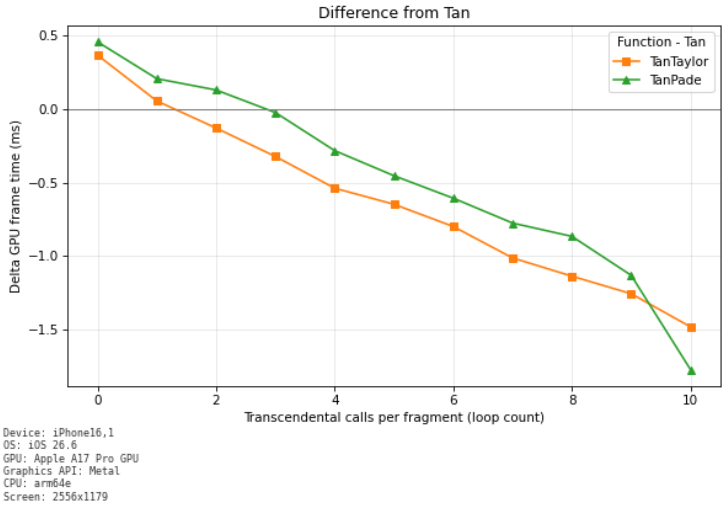


tan 函数与近似□算的比□

□行函数 10 次□，近似□算的 GPU □□□□ tan 函数的 80%。

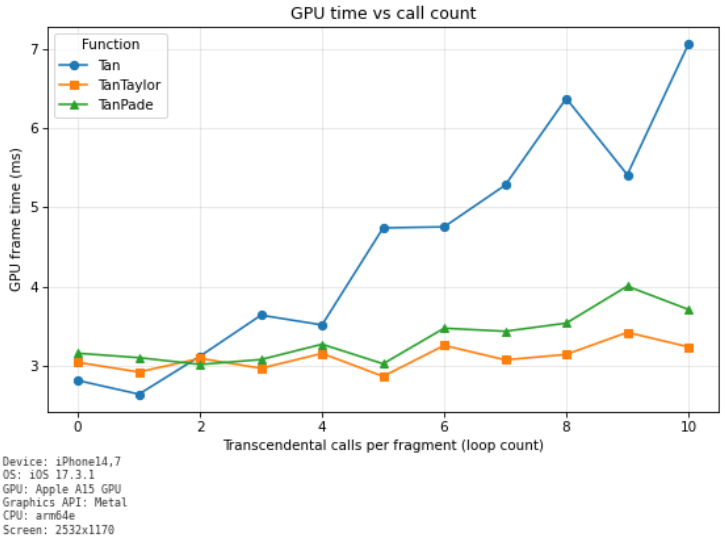


□行 10 次□，近似□算的□理□□少□ 1ms。



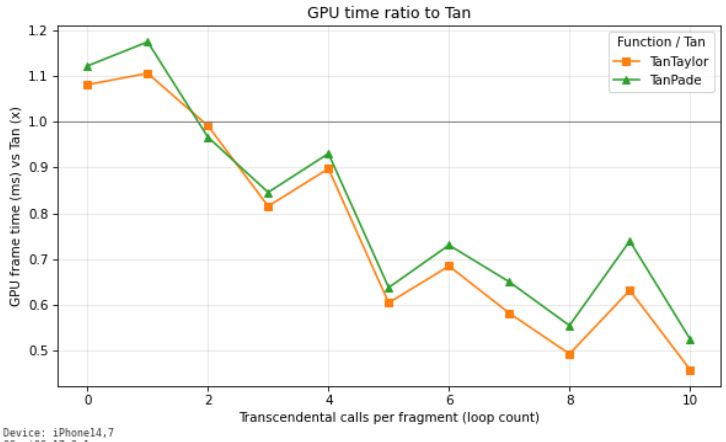
11.16.2 iPhone14

□行次数少□，可以看到寄存器□力□来的少量□外开□。但随着□行次数增加，近似□算的开□更低。

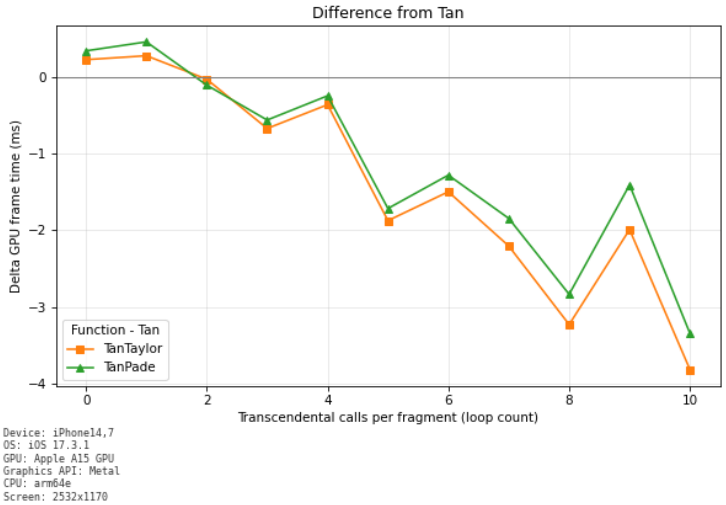


tan 函数与近似□算的比□（iPhone14）

□行函数 10 次□，近似□算的 GPU □□□□ tan 函数的 50%。

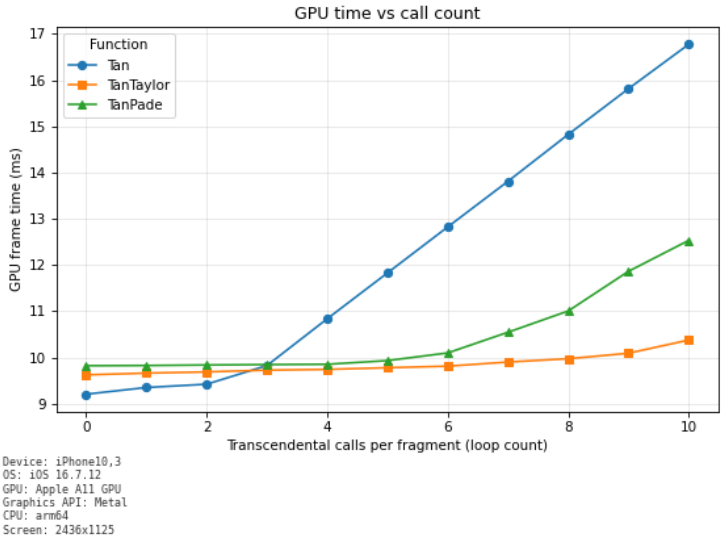


□行 10 次□，近似□算的□理□□少□ 3ms



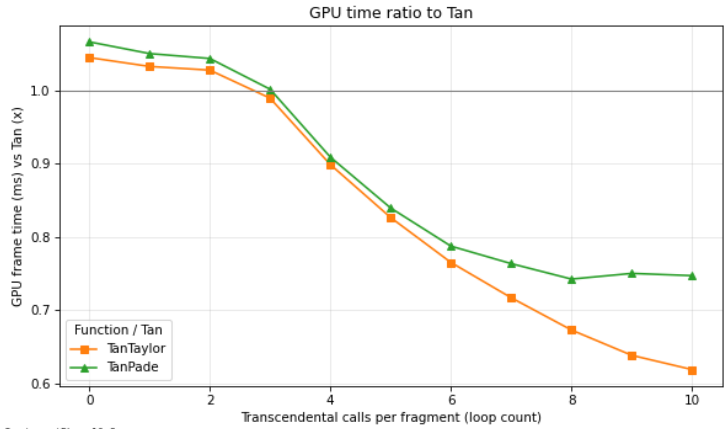
11.16.3 iPhoneX

□行次数少□，可以看到寄存器□力□来的少量□外开□。但随着□行次数增加，近似□算的开□更低。泰勒展开几乎没有增加□理开□，Padé 近似的□理开□略有上升。

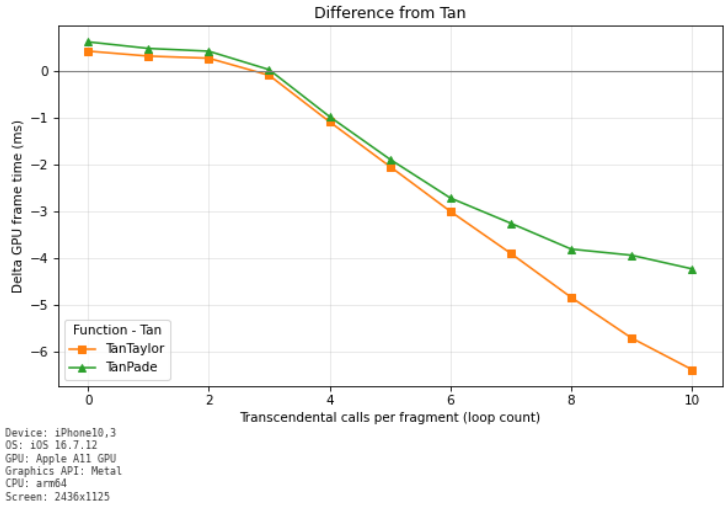


tan 函数与近似□算的比□

□行函数 10 次□，泰勒展开的 GPU □□□□ tan 函数的 60%。Padé 近似□□ 75%。

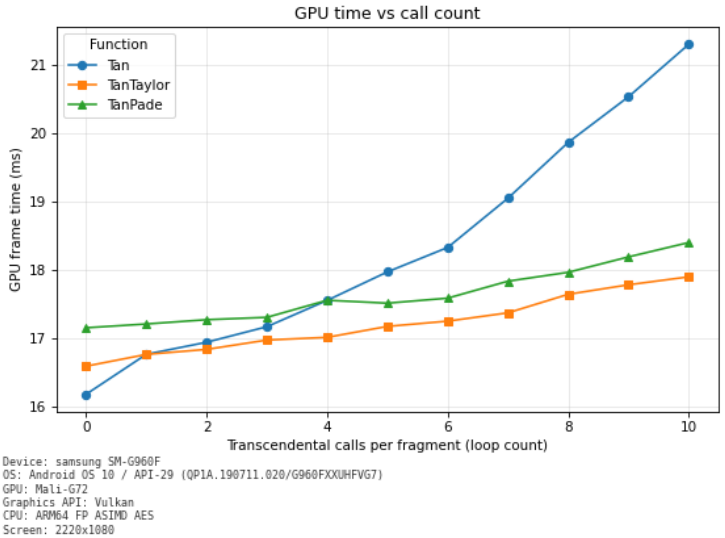


□行 10 次函数□，□果是泰勒展开□短□ 6ms，Padé 近似□短□ 4ms。



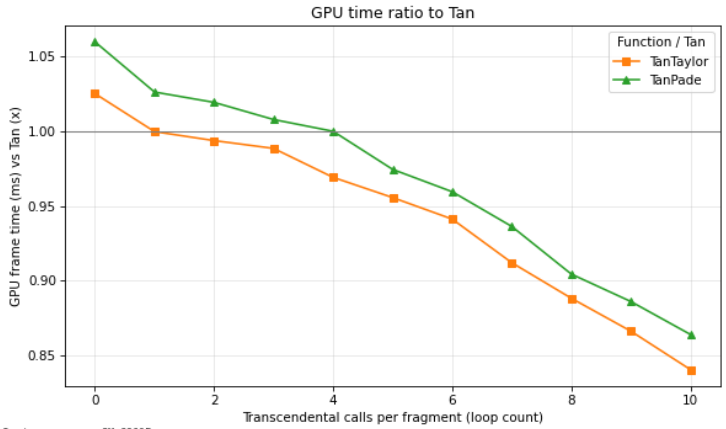
11.16.4 Galaxy S9

□行次数少□，可以看到寄存器□力□来的少量□外开□。但随着□行次数增加，近似□算的开□更低。

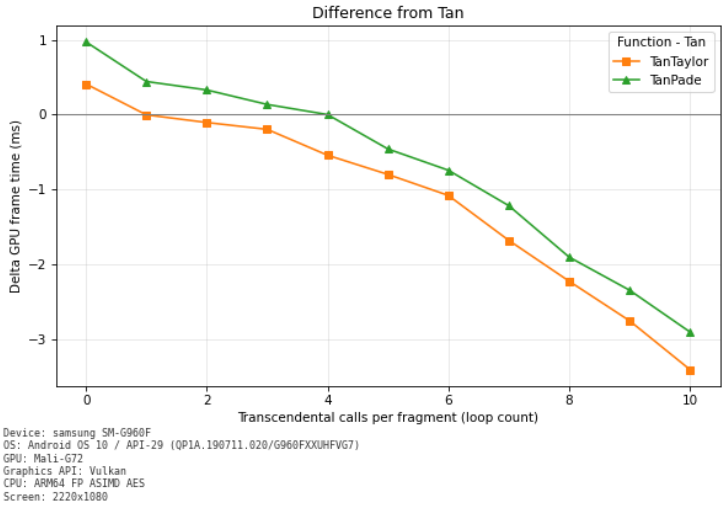


tan 函数与近似□算的比□

□行函数 10 次□，近似□算的 GPU □□□□ tan 函数的 85%。



□行 10 次函数□，□果是泰勒展开□短□ 3ms。



11.16.5 tan 函数近似开□□□

□□	GPU 开□的□□	□注
iPhone 15 Pro	近似□算低□ **20%**	近似□算使□理□□□少□ 1ms
iPhone 14	近似□算低□ **50%**	近似□算使□理□□□少□ 3ms
iPhone X	近似□算低□ **25 到 40%**	泰勒展开□少□ 6ms，Padé 近似□少□ 4ms
Galaxy S9	近似□算低□ **15%**	近似□算使□理□□□少□ 3ms

※“GPU 开□的□□”是指与□准 tan 函数相比，□行近似□算（泰勒展开或 Padé 近似）□ GPU □□的大致削□率。

第12章

Unity 的 LTO 置及 Android 与 iOS 上的性能比

キム ミンヒョク Minhyuk Kim
株式会社 CyberAgent SGE 核心技本部

Unity 6000.5 中新增的面向 Android 的接化（LTO）置，只作用于引擎代。要自己写的 C# 所生成的代也用 LTO，需要外的置。本章了种置，并用基准和真目量了性能、二制大小和建。

12.1 IL2CPP 的与翻元

在 Unity 的 Android 建（IL2CPP）中，C# 按 清 12.1 的流程机器。

▼清 12.1 IL2CPP 建的理流程

```
C# 源
  ↓ C#
IL 程序集 (.dll)
  ↓ IL2CPP 大量
 C++ (.cpp)          ← 「IL2CPP 生成代」
  ↓ clang （每个 .cpp 独化）
  ↓ 接
libil2cpp.so              ← C# 由代生成的最原生二制
```

clang 以翻元（通称 TU）位 cpp，并生成目文件。TU 是指器独取并化的位。

在通常的 C++ 建中，无法引用位于其他 TU 的函数体，因此即使是很短的函数也不会被内，call 指令会保留下来。IL2CPP 生成的代中有很多小函数和安全，容易受到一限制的影。例如 C# 的一行 `buf[idx]` 大致会成 清 12.2 的 C++。

▼清⌧ 12.2　IL2CPP 生成的数⌧⌧⌧代⌧示例

```
if (buf == NULL) ThrowNullReferenceException();                // null ⌧⌧
if ((uint32_t)idx >= buf->max_length) ThrowIndexOutOfRange(); // ⌧界⌧⌧
val = buf->m_Items[idx];
```

生成代⌧的⌧模越大，跨 TU 的⌧用就越多，⌧用耗⌧的⌧和也随之增加。

12.2　什么是 LTO / ThinLTO

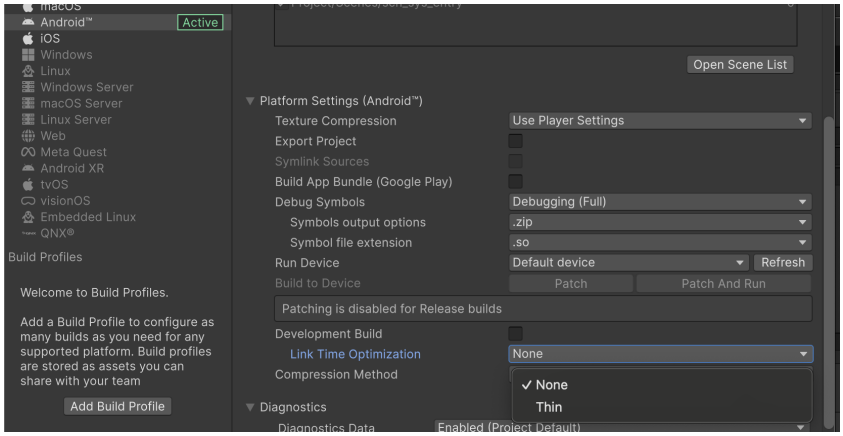
LTO（Link Time Optimization）是 clang/LLVM 的一种机制，在⌧接⌧⌧行跨 TU 的内⌧、常量⌧播和无用代⌧⌧除。⌧⌧⌧，目⌧文件以 LLVM bitcode（⌧化前的中⌧表示）的形式⌧出。⌧接⌧将它⌧⌧⌧⌧⌧化，并生成机器⌧。

full LTO 将所有中⌧表示合并⌧一个整体⌧行⌧化。其⌧化范⌧最广，但中⌧表示的合并和⌧化集中在⌧个⌧程中，因此大⌧模⌧建会占用大量内存。⌧次在真⌧⌧目的⌧建中，内存使用量也⌧到了 20GB，⌧程被操作系⌧⌧制⌧止。

ThinLTO 在⌧⌧⌧⌧各 TU 生成摘要。⌧接⌧从其他 TU ⌧入（import）所需的函数，并行⌧行⌧化。与 full LTO 相比，更容易控制内存使用量和⌧理⌧⌧，在大⌧模⌧建中被广泛使用。

12.3　引擎代⌧的 ThinLTO ⌧置

Unity 6000.5 新增了 Link Time Optimization 开关。

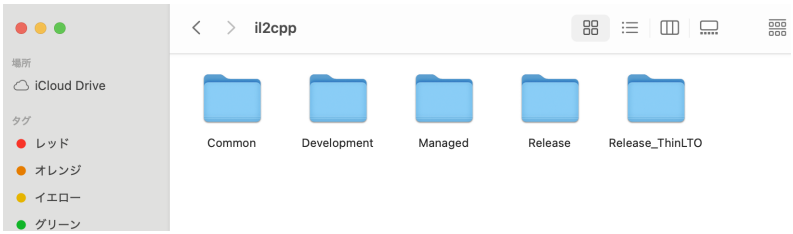


▲图 12.1　Build Profiles > Player Settings > Android

启用开关后，会启用 Unity 提供的已启用 ThinLTO 的引擎二进制。其对象是 Unity 引擎代码，不包括 IL2CPP 生成代码（由 C# 转换而来的代码）。转换器的安装文件夹中附带了普通版和 ThinLTO 版的静态库 so 文件。

▼清单 12.3　随 Android Player 附带的 libunity.so 静态库

```
PlaybackEngines/AndroidPlayer/Variations/il2cpp/ ├──
 Release/Libs/arm64-v8a/libunity.so          29,032,960 bytes └──
 Release_ThinLTO/Libs/arm64-v8a/libunity.so  30,042,536 bytes (+3.5%)
```



▲图 12.2　转换器的安装文件夹。`Release` 与 `Release_ThinLTO` 均已附带，静态库会启用其中之一

12.3.1 □用条件及不□足□的行□

开关生效需要□足以下 3 个条件。只要有一个不□足，就不会出□□□或警告，`LinkTimeOptimization` 会作□ `None` □建。

1. IL2CPP 脚本后端
2. Strip Engine Code 已禁用（因□直接使用未裁剪的 prebuilt）
3. 非 Development □建

□用 Development Build □，开关本身不会□示。是否已□用，可通□ 清□ 12.4 □ 2 点确□。

▼清□ 12.4　确□引擎代□ LTO 开关是否生效的命令

```
# ① □建□入中是否指定了 ThinLTO
grep -o 'LinkTimeOptimization"[: ]*"[A-Za-z]*"' Library/Bee/Player*-inputdata.json
# ② APK 内的 libunity.so 是否与 Release_ThinLTO 版一致
unzip -p out.apk lib/arm64-v8a/libunity.so | md5
```

12.4　引擎代□ LTO 开关的效果

□附□的普通版和 ThinLTO 版二□制□行比□，并以□化□多的子系□□中心，在真机上□量了性能。

12.4.1 比□附□二□制的函数大小

Unity 在□□器的安装文件□中附□了□种□体□符号的 `.so`。

▼清□ 12.5　随 Android Player 附□的□符号 libunity.so

```
PlaybackEngines/AndroidPlayer/Variations/il2cpp/ ├──
 Release/Symbols/arm64-v8a/libunity.sym.so └──
 Release_ThinLTO/Symbols/arm64-v8a/libunity.sym.so
```

□照从 `llvm-nm --defined-only --print-size --demangle` 得到的函数名，使用命名空□和□名按子系□□□（表 12.1）。

表 12.1: 引擎代□ LTO 开关□来的各子系□函数差异

子系□	函数数 (Release → ThinLTO)	大小□生□化的函数数	大小□化率	.text 差异
Particle	1,419 → 1,290	1,043	80.9%	+49.66%
Serialization/Asset	2,890 → 1,387	1,040	75.0%	-16.61%
Mesh/Geometry	1,055 → 765	483	63.1%	+6.79%
Scripting	8,924 → 7,680	4,302	56.1%	+10.27%
Gfx/Shader/Texture	5,062 → 3,935	2,044	52.1%	+24.21%
Physics	2,680 → 2,439	1,229	51.1%	+15.92%
NavMesh	739 → 582	296	51.1%	+11.87%
XR/VFX/Video/其他	3,828 → 2,483	1,230	49.6%	+13.94%
Transform/Scene/Obj	2,027 → 1,630	781	47.9%	+10.95%
Animation/Mecanim	3,226 → 2,428	1,134	46.8%	+13.18%
Box2D	2,169 → 1,562	712	46.4%	-0.43%
JobSystem/Threading	244 → 181	78	43.6%	+3.16%
UI/UIToolkit	883 → 701	293	42.0%	+1.38%
TextCore/Font/ICU	3,500 → 2,101	861	41.9%	-5.73%
（未分□）	29,193 → 21,077	7,511	37.6%	+3.37%
Renderer/Culling	2,789 → 2,199	785	35.7%	+27.58%
C++ □准□/□□/基□□施	13,158 → 9,484	3,071	32.6%	-13.31%
curl/TLS/HTTP	1,809 → 1,558	318	21.0%	+1.17%
Audio/FMOD	2,996 → 2,680	496	18.5%	+2.77%
Android/JNI	1,445 → 1,186	193	16.3%	-5.52%
PhysX	6,000 → 5,997	0	0.0%	-0.05%
合□	96,036 → 73,345	27,900	38.8%	+4.41%

※“大小□生□化的函数数”是函数的个数。“大小□化率”表示在 Release 和 ThinLTO 中都存在的函数里，大小□生□化的函数所占的比例。

整体函数数从 96,036 □少到 73,345，但 .text 的大小增加了 4.41%。

12.4.2 基于 13 个工作□□的基准□□

以 表 12.1 □基□，□建了在 1 个 APK 中包含 13 个工作□□的基准□□□景。

表 12.2: 引擎代□ LTO 开关的基准□□工作□□

工作□□	目□子系□	内容
`particle`	Particle	20,000 个粒子 + noise/size/color 模□、`Simulate`
`gfx`	Gfx/Shader	向 `MaterialPropertyBlock` 写入 16 个属性 × float/vector + matrix 并 `Clear`
`cull`	Renderer/Culling	`CalculateFrustumPlanes` + 4,096 个 AABB 的 `TestPlanesAABB`
`physbind`	Physics □定	`Rigidbody` / `Collider` 属性往返
`anim`	Animation	□ legacy `Animation` 48 个骨架 × 8 根骨骼 × 10 条曲□□行 `Sample()`
`json`	Serialization	`JsonUtility` 的往返
`instantiate`	Serialization	`Instantiate` → `DestroyImmediate`（原生 clone/transfer 路径）
`xform`	Transform/Scene	重写□□□的 Transform
`bind`	Scripting □定	大量□用极小的□定
`mesh`	Mesh/Geometry	重建 96×96 的程序化网格
`phys2d`	Box2D	2D □体的 `Simulate`
`text`	TextCore/Font	`TextGenerator.Populate`
`physics`	PhysX	3D □体的 `Simulate`

※ 在 表 12.1 的子系□中，□以通□ C# □造出每个□本□ 100ms 的□定合成□□的子系□，已从□量□象中排除。□些子系□的性能影□未作□估。

12.4.3 各工作负载的测量结果

在 Pixel 8a（Tensor G3）上，使用相同 `libunity.so` 不同的 APK 测量了上述工作负载。按 Release / ThinLTO / ThinLTO / Release 的 ABBA 顺序重复 3 次，共进行 12 个会话。每个会话都重新安装应用，会话之间设置 90 秒的冷却间隔。

数值为单次操作所需的时间（ns/op）。操作的单位因工作负载而异，因此只能在同一行内比较。

表 12.3: 引擎代码 LTO 开关的各工作负载基准测试结果

工作负载	Release (ns/op)	ThinLTO (ns/op)	差异
`gfx`	990.46	852.49	-13.93%
`cull`	96,520	87,499	-9.35%
`text`	275,043	265,123	-3.61%
`instantiate`	14,620	14,206	-2.83%
`xform`	6,071.8	6,008.1	-1.05%
`phys2d`	222,269	220,327	-0.87%
`mesh`	2,340,210	2,334,143	-0.26%
`json`	15,864.8	15,934.4	+0.44%
`bind`	140.80	141.47	+0.47%
`physbind`	607.28	610.74	+0.57%
`particle`	265,635	267,335	+0.64%
`anim`	28,100	28,563	+1.65%
`physics`	484,641	482,796	-0.38%
[合成：几何平均]	21,918	21,380	-2.46%

※ 由于各行的单位不同，合成负载作比率才有意义。

改善幅大的 `gfx` 和 `cull` 是从 C# 小函数每个样本用数百万次的处理。`text` 的 `TextGenerator.Populate` 内部也会进行以字形为单位的循环。在单次处理很小、调用次数很多的路径上，跨 TU 内联的效果表现得很明显。

12.4.4 引入前的性能、大小与内存对比

本次测量中，在固定用例多的部分路径上，CPU 时间有所缩短。不过，在真实项目中的改善幅度，取决于内联此类用所占的比例。

引入□必□付出的成本□□在 表 12.4 中。

表 12.4: 采用引擎代□ LTO 开关□的成本

□□	□□差异
关□ Strip Engine Code □致的大小增加	APK +5.67MB / 安装 +20.3MB
关□ Strip Engine Code □致的内存增加	RSS +18.0MB / 其中□以回收的区域 +8.0MB
engine ThinLTO 本身	APK 大小 +398KB(30,621,595 → 31,019,967)/ 内存未□量

※ 安装大小的增量，是 APK 的增量与□□上解□后的 .so 的增量（arm64 的 .so 合□ +14.65MB）之和。

Unity 公布了□□同一开关，在 5 个□目上各□行 10 次冷□□□量的□果。CPU □□从 13.88ms □短到 13.36ms（-3.7%），到首次□示的□□从 247.96ms □短到 229.90ms（-7.3%），据□告 APK 大小增加 +3.3%，□□建□□的影□□微[*1]。

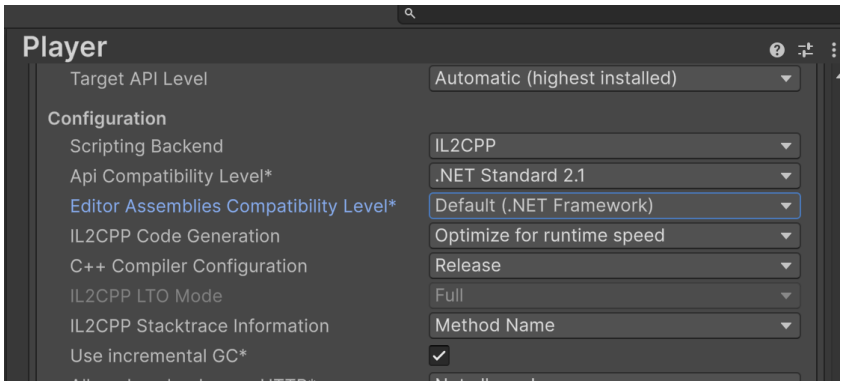
CPU 的改善幅度与本次□量□于相近水平。不□，公开□中 APK 大小 +3.3% 是在□个均关□ Strip Engine Code 的配置之□比□得出的，因此不包含关□所□来的安装大小 +20.3MB 和内存 +8.0MB 的增加。

[*1] Unity Android 平台□□「Boosting Android performance with LTO optimizations in Unity 6.5」https://discussions.unity.com/t/boosting-android-performance-with-lto-optimizations-in-unity-6-5/1720008。此后本章中提到“Unity 的公开□”，均指□篇文章中的数字。

12.5 如何为 libil2cpp.so 启用 LTO 的方法

要想包含由 C# 代码而来的代码的 libil2cpp.so 启用 LTO，可以使用 Unity 的 Master 配置，也可以直接使用 clang 标志。

12.5.1 Master 配置与 IL2CPP LTO Mode 的配置



▲图 12.3　IL2CPP LTO Mode 与 C++ Compiler Configuration 的配置界面

将 Player Settings > Configuration 中的 C++ Compiler Configuration 设置 Master 后，会为 libil2cpp.so 启用 LTO。IL2CPP LTO Mode 是用于设置此项所用的 Thin / Full 模式的设置。

根据 Unity 的官方文章，IL2CPP LTO Mode 自 6000.5 起新增。在 6000.5 之前的 Android 上，Release 与 Master 在是否启用 LTO 方面没有差别。但在 6000.5 中，Android 平台无法设置开关，固定为 Thin；从 6000.6 起，可以在 Master 配置中设置 Thin / Full。到 6000.7 为止，也没有在保持 Release 配置的同时启用 LTO 的设置。

与 Player Settings 的 C++ Compiler Configuration 设置的标志，可从 Bee 构建的 Il2CppCompilerConfiguration 中确认。

表 12.5: Unity 各 C++ Compiler Configuration 的□□器□志

版本	Release	Master
Android 6000.3 (arm64)	-Os（□先大小）、无 LTO	-Os、无 LTO（相同）
Android 6000.5 (arm64)	-Os	-O2（兼□速度与代□大小的□化）+ ThinLTO
iOS 6000.3 / 6000.5	-O3（以代□大小可能增加□代价，更□极追求速度的□化）、LTO OFF	-O3、LTO OFF（相同）

12.5.2 向 Release 配置添加 -flto=thin

如前所述，切□到 Master 配置是最快□得 LTO 效果的途径，但在□用 LTO 的同□，□化□□也会从 -Os □□ -O2（表 12.5），缺点是大小和□建□□会增加。如果保持 Release 配置，□□用 ThinLTO，□可以将 LTO 的效果分离出来□独□量，同□也能抑制大小和□建□□的增加。要□由自己□写的 C# 所生成的 libil2cpp.so □用 ThinLTO，可以考□像 清□ 12.6 那□直接□□ clang □志。

▼清□ 12.6　在 Release 配置中□用 ThinLTO 的□建□□理

```
// 保存已有的附加参数 → 追加 → □必□原
var saved = PlayerSettings.GetAdditionalIl2CppArgs();
try
{
    var extra = "--compiler-flags=-flto=thin --linker-flags=-flto=thin";
    PlayerSettings.SetAdditionalIl2CppArgs(
        string.IsNullOrEmpty(saved) ? extra : saved + " " + extra);
    BuildPipeline.BuildPlayer(options);
}
finally
{
    PlayerSettings.SetAdditionalIl2CppArgs(saved);
}
```

是否生效分 2 个□段确□（清□ 12.7）。

第 1 步是确□□志是否已□□到□建命令中。Unity 的□建系□（Bee）会将□行□的□□和□接命令□□到 Library/Bee/Player*.dag.json 中。使用 -flto=thin 搜索□文件，□看 IL2CPP 生成代□的□□命令和 libil2cpp.so 的□接命令中是否都包含□□志。

第 2 步是确认生成的目标文件是否为 LLVM bitcode。启用 LTO 后，结果果 `.o` 不是机器码，而是 bitcode。从第 1 步找到的链接命令的输出文件中取一个，显示其开头 4 个字节。

▼清单 12.7　IL2CPP ThinLTO 的启用确认命令

```
# 1. 在链接命令中是否包含 -flto=thin
rg -n -- '-flto=thin' Library/Bee/Player*.dag.json
# 2. 生成的目标文件的开头 4 个字节（指定在 1. 中找到的 .o）
xxd -l 4 Library/Bee/artifacts/Android/.../GeneratedCode.o
```

如果是 bitcode，会显示 `4243 c0de`（`BC..`）；如果是普通的机器码目标文件，会显示 `7f45 4c46`（`.ELF`）。随意选择文件中的 `.o`，可能会选到不属于 LTO 对象的第三方库的目标文件。第 2 条命令必须指定第 1 步搜索结果中出现的文件。

12.5.3　基准测试的设计

合成代码的生成

本基准测试的主要测量对象，是通过普通测试无法测量的跨 TU 调用。asmdef 的边界与 C++ 的 TU 边界并不是同一概念，因此我们在 IL2CPP 生成后生成的 C++ 以及Bee 的构建日志进行了确认，调用方与被调用方确实位于不同的 TU。代码生成器生成的内容如下。

- asmdef 程序集 16 个 × 每 24 个 × 方法 8 个 = 3,072 个方法
- 调用每循环一次会穿过 16 个程序集边界，所有方法会从外部用共享程序集中的辅助函数（`Core.Mix` / `Core.Idx` / 测试体计算）
- 基准测试会遍历所有，因此不会被 managed 裁剪移除
- 给每个方法配置不同的常量，以防止被 ICF 折叠

生成的 3,072 个方法的示例□ 清□ 12.8。

▼清□ 12.8　合成基准□□中生成的方法示例

```
public static int M2(int sel, int x, int[] buf)
{
    x = Core.Mix(x, 655663086);       // 跨□界□用共享程序集
    x += buf[Core.Idx(x, 7)];         // □□界□□的数□□□（Idx 用 & 1023 做掩□）
    return M3(sel, x, buf);           // 小函数□用□
}
```

□个工作□□

表 12.6: IL2CPP ThinLTO 的合成基准□□工作□□

工作□□	□量内容
direct	跨程序集的静□□用□（LTO □当最有效的形□）
iface	通□接口分派□行同□□算（LTO □以将□用目□直接化的形□）

□者都只在 libil2cpp.so 内完成。每种配置生成的二□制，每次都通□从 APK 中提取的 libil2cpp.so 的 md5 □行确□。□行□会□整迭代次数，使每个□本□□ 100ms。由于每次□行的次数不同，因此□一化□每个 pass 的□□后□行比□。

在 iface 中，□每个程序集生成 4 个□□了 IGenOp 的□（共 64 个），基准□□通□接口依次□用它□（清□ 12.9）。方法体中□用与 direct 相同的 Core.Mix / Core.Idx，但具体□用□个□的 Apply 在□行□才确定。

▼清单 12.9　iface 工作负载中生成的虚和用循环

```
sealed class Op0 : IGenOp
{
    int m_State = 20612;
    public int Apply(int x, int[] buf)
    {
        x = Core.Mix(x, 347275849);              // 跨边界调用共享程序集
        x += buf[Core.Idx(x ^ m_State, 7)];   // 与 direct 同样的数组访问
        m_State = (m_State + x) | 1;
        return x;
    }
}

// 基准测试：通过接口对流调用 64 个对象
for (int i = 0; i < ops.Count; i++)
    a += ops[i].Apply(a ^ p, buf);       // 调用目标在运行时确定
```

12.5.4　CPU 测试（Pixel 8a）

表 12.7: Pixel 8a 上 IL2CPP ThinLTO 的 CPU 测试结果

配置	direct (us/pass)	iface (us/pass)
off(Release, -Os)	19.88	0.404
thin（手动 ThinLTO）	15.54 (-21.8%)	0.375 (-7.3%)

通过手动 ThinLTO，direct 缩短了 21.8%，iface 缩短了 7.3%。

direct 的函数调用与指令数

测试与 §5.3.1 相同的调用负载，比较了有无 LTO 时的反汇编结果。

表 12.8: direct 工作负载的反汇编对比

C000_M2	无 LTO	ThinLTO
指令数	15	11
函数调用（bl）	2	0
栈帧	有	无
callee-saved 寄存器的保存	x19、x20、x29、x30	无
内存访问	5 次	1 次

`Core.Mix` 和 `Core.Idx` 的函数体被展开到调用方，不再需要 2 条 `bl` 指令以及栈和寄存器的保存恢复。跨越两个函数的表达式也会一并被常量化。direct 缩短的 21.8%，正是这一常量化在 3,072 个方法上累积的结果。

iface 改善□小的原因

IL2CPP 生成的 interface □用，概括来□是 清□ 12.10 的形式。

▼清□ 12.10　IL2CPP 生成的 interface □用代□概要

```
const VirtualInvokeData& invokeData =
    il2cpp_codegen_get_interface_invoke_data(slot, obj, declaringInterface);
return ((Func)invokeData.methodPtr)(obj, p1, p2, invokeData.method);
```

由于□用目□是□行□□得的 `invokeData.methodPtr`，因此无法像普通 C++ vtable □用那□直接化。不□，方法体内□用其他 TU □助函数的部分可以内□，iface 也□短了 7.3%。

如果 interface / virtual □用耗□□多，也可以考□ `sealed`、使用具体□型接收、通□泛型□行静□分派。ThinLTO 容易改善的是静□□用其他 TU 中小函数的部分。

12.5.5　大小与□建□□

表 12.9: IL2CPP ThinLTO 的大小与□建□□

配置	libil2cpp.so (MB)	同 .text	冷□建
off	37.36	基准	62s
thin	38.16 (+2.2%)	+2.3%	71s (+15%)

在此配置中，受内□展开的影□，`libil2cpp.so` 增加了 2.2%。LTO 有□也会□除无用代□，因此大小□化的方向取决于代□□和□化内容。冷□建□□从 62s 增加了 15%，□□ 71s。ThinLTO 在□接□□行跨 TU 的□化和机器□生成，因此□接□□相□□□。

12.6　Master 配置中□用的□化

在 Master 配置中，□用 LTO 的同□□化□□也会改□（表 12.5）。□了□明 CPU □□的□短来自□□□置，□它□分□□行了□量。□了分□考察□化□□（`-Os` → `-O2`）和 LTO 的影□，我□□ {-Os, -O2} × {无 LTO, ThinLTO} 的 2×2 □合□行了□□（Android 6000.5 / arm64）。

表 12.10: □化□□与 ThinLTO 的 CPU □□□比

direct (us/pass)	无 LTO	ThinLTO
-Os（Release）	19.64（基准）	15.54 (-20.9%)
-O2(Master)	18.47 (-5.9%)	15.54 (-20.9%)

表 12.11: □化□□与 ThinLTO 的 libil2cpp.so 大小□比

libil2cpp.so	无 LTO	ThinLTO
-Os	37.36MB	38.16MB (+2.2%)
-O2	39.14MB	44.14MB (+12.8%)

CPU □□的□短率，`-O2` □独□ 5.9%，包含 ThinLTO 的配置□ 20.9%。在此基准□□中，Master 配置的大部分改善都来自 ThinLTO。Release + 手□ ThinLTO 与 Master 的 CPU □□同□ 15.54us，但最□的 `libil2cpp.so` 大小分□□ 38.16MB 和 44.14MB，Release + 手□ ThinLTO 小□ 13.5%。

表 12.12: Release + ThinLTO 与 Master 的冷□建□□

配置	冷□建
Release（`-Os`、无 LTO）	62s
Release + ThinLTO	71s (+15%)
Master (`-O2` + ThinLTO)	86s (+39%)

冷□建分□□ 71s 和 86s，□在 Release 上添加 ThinLTO 的配置□短了 15 秒。在 Unity 公布的□量中，Master 配置（`libil2cpp.so` 一□）同□是 CPU -2.3%、到首次□示的□□ -5.7%，而 `libil2cpp.so` 大小□ +18.7%，□建□□□ +30～35%。CPU 的改善幅度，Unity 公布的是 -2.3%，而本章的基准□□□ -20.9%。不□，Unity □量的是真□□目的整个主□程，而本章□量的只是反复□用小函数的路径，因此□□□数字不能直接比□。□一方面，大小和□建□□在本章的 Master 中分□□ +18.2%、+38.7%，与 Unity 的公开□接近。

12.7 真□□目中的□□

在□模比基准□□更大的 2 个真□□目中也□□了 ThinLTO。Unity 版本、程序集数量、源代□量和二□制大小□ 表 12.13。

表 12.13: 真□□目的 Unity 版本和代□□模

□目	□目 A	□目 B
Unity 版本	6000.5.3f1	6000.3.15f1
包含在播放器中的程序集数	158	285
被□□的 `.cs`	6,969 个文件	16,700 个文件
同 行数	□ 138 万行	□ 319 万行
其中自研代□	1,943 个文件 / □ 35 万行	8,606 个文件 / □ 154 万行
`libil2cpp.so`（未□□）	189.0MB	294.3MB

12.7.1 Release + ThinLTO 的大小与生效确□

表 12.14: □目 A 的大小与□建□□

□目	baseline	ThinLTO	差□
libil2cpp.so（未□□）	189.01MB	194.70MB	+5.69MB (+3.0%)
同 `.text`	8.54MB	8.98MB	+0.44MB (+5.1%)
APK	165.85MB	166.94MB	+1.08MB (+0.65%)
玩家□建部分	□ 396s	□ 433s	□ +9%

`.text` +5.1% 大于基准□□的 +2.3%，□与“asmdef 拆分得越□，跨模□的内□就越多”的□期相符。即使是 189MB 的 libil2cpp.so，也能在本地机器上完成 ThinLTO 的□接。

在项目 B 中，使用 `llvm-readelf -S` 比较了 `libil2cpp.so` 各节的大小（表 12.15）。

表 12.15: 项目 B 的 libil2cpp.so 各节差异

节	baseline	ThinLTO	差异
libil2cpp.so（整个文件）	294.30MB	304.12MB	+3.34%
`il2cpp`	145.85MB	162.22MB	+11.23%
`.text`	10.69MB	11.14MB	+4.18%
`.rela.dyn`	50.28MB	45.01MB	-10.48%
`.got`	1.76MB	1.4KB	-99.92%
`.gcc_except_table`	3.70MB	3.62MB	-2.32%

`.text` 的增长率 4.18% 介于项目 A（5.1%）与基准组（2.3%）之间。`.got` 从 1.76MB 减少到 1.4KB。这表明 LTO 解析了跨模块的引用，将由 GOT 的间接引用被折叠为直接引用。包含大量重定位条目的 `.rela.dyn` 减少 10.5%，也与单一模块化相关。

12.7.2 真实项目中的 Master 与 full LTO（项目 A）

在真实项目模块上复现了 §6 中 Master 配置与 full LTO 的比较。

表 12.16: 项目 A 的构建配置对比

项目 A	baseline	Release + thin	Master	full LTO
libil2cpp.so	189.0MB	194.7MB (+3.0%)	222.2MB (+17.6%)	–
构建	453s	465s	537s（9 分钟）	2,903s（48 分钟）后失败
链接器路径的峰值 RSS	–	–	12.9GB	20GB → `Killed: 9`（OOM）

full LTO 在构建 48 分钟后，链接程序中的内存使用量达到 20GB，被操作系统强制终止。在 §5.3 的基准测试中，full LTO 的 CPU 时间与 ThinLTO 相当，冷构建却从 71s 增加到 247s，约为 3.5 倍。该项目无法完成构建，ThinLTO 更具实用。

Master 的 `libil2cpp.so` 大小比 baseline 增加了约 33MB，但 CPU 性能与 Release + thin 相当。Release + thin 的大小增加仅约 5.7MB，因此在该项目中，也能以更小的二进制获得与 Master 相当的性能。

12.7.3 真□□目中的 CPU 周期（□目 B）

在局外的□定画面中，合□□量了 Pixel 8a 所有□程的 CPU 周期。交替□量 ThinLTO 和 baseline 各 3 个会□，每个会□的□是 6 次 20 秒□量的平均□。

表 12.17: □目 B 各会□的 CPU 周期

会□	□周期 (Gcyc)	UnityMain (Gcyc)
ThinLTO #1	44.88	27.48
ThinLTO #2	45.18	27.36
ThinLTO #3	45.54	27.60
baseline #1	46.06	29.05
baseline #2	45.71	27.89
baseline #3	46.15	28.23
平均（ThinLTO vs baseline）	45.20 vs 45.97 → -1.68%	27.48 vs 28.39 → -3.21%

12.7.4 与 Unity 公开□的比□

将 Unity 公布的 Master 配置（`libil2cpp.so` 一□）□量□与□目 B 的□果□行□比。同□列出了 §6 的基准□□□果。

表 12.18: Unity 公开□与本章□□□果的比□

指□	Unity 公开（6 个□目・Master）	基准□□	□目 B
CPU	主□程 -2.3%	`direct` -21.8%	UnityMain -3.21% / □周期 -1.68%
`libil2cpp.so`	+18.7%	+2.2% (Release+thin)/ +18.2%(Master)	+3.0%
□建□□	+30～35%	+15%（Release+thin）/ +38.7%（Master）	+9～15%

Unity 的 CPU □□□短率 -2.3% 与□目 B 的 UnityMain -3.21%，都是几个百分点的改善。与基准□□ -21.8% 的差距，可以用□□□目的□量中也包含不在 LTO 范□内的□理来解□。Unity □量的 6 个□目中也能看到同□的□□。

大小和□建□□因配置而异。Unity 的公开□在 Master 配置下分□□ +18.7% / +30～

35%。本章□得的 Master 配置 +18.2%（44.14MB / 37.36MB）/ +38.7%（86s / 62s）也与之相近。作□基准的 `libil2cpp.so` 大小也相近，□ 37.26MB / 37.36MB。

□在 Release 上添加 ThinLTO □，能以大小 +2～3%、□建□□ +9～15% □得同等的 CPU 性能。从 §6 的 2×2 □比来看，Master 配置中大小和□建□□的增加，主要被□□是由□ `-O2` 的更改造成的。

12.7.5 □化的□象范□与真□□目中的改善幅度

□目 B 的改善集中在主□程。UnityMain □ -3.21%，而整个非 UnityMain □ +0.8%，□于会□□漂移 4% 的范□内。

从二□制来看，`libil2cpp.so` 以外的 `.so` 也几乎没有□化。2 个原生插件的 md5 完全一致，其余 1 个 `.text` 的差异□ +0.008%，□于□建非确定性的范□内。此次 `--compiler-flags` □来的□化没有涉及□些插件。

UnityMain 的 -3.2% 小于基准□□的 -22%。在□画面中，表情、□眼、□□和服装的□□、IK 等是以作□（Job）的形式□行的。□些包含在 `libil2cpp.so` 之外的原生二□制中，不属于此次□志的□化范□。

可以□□，排除范□外□理后，□ IL2CPP 生成代□的改善幅度大于 UnityMain 整体的 -3.21%。但要得到准确的□，需要用 Profiler 分离范□内外的□理□□。作□会因□度而在主□程上□行，所以 UnityMain 的□量□中也包含范□外的□理。

12.8 在 iOS 上□用 ThinLTO

我□在□目 B 的 iOS 版本上也□用了 ThinLTO。□然在□□用基准□□中确□了效果，但在□□□目中，没有□得与□建□□增加相称的改善。

12.8.1 iOS 上 LTO 的□□

表 12.19: Android 与 iOS 的 LTO □□□比

□点	Android	iOS
il2cpp 的□出	`libil2cpp.so`	`libGameAssembly.a`
最□□接	il2cpp □用的 LLD	Xcode 生成 `UnityFramework` □的 ld64
□□的参数	`--compiler-flags=-flto=thin --linker-flags=-flto=thin`	□ `--compiler-flags=-flto=thin`
□建□□增加的位置	Unity 的□建步□	`Build ipa`（xcodebuild）步□

在 iOS 上，il2cpp 只□建□档，因此不□□ `--linker-flags`。LTO 的代□生成在 Xcode □□取包含□ bitcode 的□档并□接□□行。

生成代□的□□由 il2cpp □□，因此必□将□志指定□ `AdditionalIl2CppArgs`。即使修改 Xcode 的 `LLVM_LTO = YES_THIN`，也不会□用到生成的代□上。

引擎代□以□□建的 `libiPhone-lib.a` 形式提供，不含 bitcode，因此不属于 LTO 的□象。也没有像 Android 的引擎代□ LTO 开关（§3）那□，替□□已□用 LTO 的引擎的□置。

12.8.2 iPhone 13 的□□用基准□□

首先，将与 §5.3 相同的□□用基准□□移植到 iOS，确□□□生成代□的 ThinLTO 在 iOS 上是否同□有效。`direct` 将 3,072 个小方法分散在 16 个 asmdef 中，□是 ThinLTO 最容易□□效果的配置。因此，□里的改善率并不是一般□目中能直接得到的□，而□□□条件吻合□的接近上限的□。

□□□ iPhone 13（`iPhone14,5` / iOS 18.7.8）。使用了 Unity 6000.5.3f1、IL2CPP Release、OptimizeSpeed。baseline 与 ThinLTO 的区□□在于是否有 `--compiler-flags=-flto=thin`。Xcode □ `ReleaseForRunning`，开□□名□ `get-task-allow=true`，并禁用了 Unity 的 Development Build。

每个会□都□行卸□、重新安装和□□，在 5 次□□之后□□ 30 个□本。□量□序□ baseline / ThinLTO / ThinLTO / baseline 共 3 个区□，合□ 12 个会□。所有会□的□状□均□ Nominal。由于□理次数会按会□自□□整，因此 `direct` 和 `iface` □一化□每个 pass，Physics □一化□每个 step。

表 12.20: iPhone 13 □□用基准□□的□果

工作□□	baseline	ThinLTO	差□
`direct`	21,231.31ns/pass	16,184.31ns/pass	-23.77%
`iface`	366.96ns/pass	370.39ns/pass	+0.93%
Physics（范□外的□照）	147.82us/step	147.36us/step	-0.31%（分布重□）

`direct` 在全部 3 个区□中均□ -23.6～-23.9%，baseline 与 ThinLTO 的所有会□都完全分离。□也与 Android 的 -21.8%（§5.4）接近，可以确□□生成代□的 ThinLTO 在 iOS 上同□确□有效。□一方面，`iface` 略有□化，而范□外的 Physics 没有出□差异。ThinLTO 的效果偏重于静□小函数□用跨 TU 的路径。同一基准□□的二□制大小□ 表 12.21。

表 12.21: iPhone 13 □□用基准□□的二□制大小

□目	baseline	ThinLTO	差□
UnityFramework	98.76MB	107.52MB	+8.76MB(+8.87%)
`__text`	20.09MB	20.46MB	+0.37MB(+1.83%)
`il2cpp`	22.49MB	30.78MB	+8.30MB(+36.89%)

12.8.3 □目 B 的□果与在 iOS 上放弃采用的原因

接下来，□□目 B 的 iOS 版□用了 ThinLTO。UnityFramework 的大小□ 表 12.22，□建□□□ 表 12.23。

表 12.22: iOS UnityFramework 的各□差异

□	baseline	ThinLTO	差□
UnityFramework（整个文件）	316.53MB	393.47MB	+24.31%
`__TEXT`（段）	261.83MB	338.53MB	+29.29%
`il2cpp`	167.02MB	240.26MB	+43.85%
`__text`	57.99MB	61.70MB	+6.41%
`__gcc_except_tab`	6.85MB	6.63MB	-3.16%
`__unwind_info`	3.44MB	3.38MB	-1.75%
`__got`	18.8KB	18.8KB	±0
ipa（整个□档）	233.83MB	246.93MB	+5.60%

表 12.23: iOS 的□建□□

步□	baseline	ThinLTO
`Build ipa`(xcodebuild)	357s(5m57s)	3,094s(51m34s)

UnityFramework 增加了 +24.31%，ipa 增加了 +5.60%。□建□□增加到□ 9 倍，增加了 46 分□。增加的部分大多来自 Xcode □的□接和 `dsymutil`，与 Android 的 +9～15% 相比大幅□□。即便采用将 `GCC_OPTIMIZATION_LEVEL` 降□ `-Os` 的配置（`-Os` + thin），仍需 53m51s，没有□短。

在□目 B 的□□画面中，`cpu/wall` 的□□平均□□示 ThinLTO 略有□化，□ +1.08%，即使在□状□一致的□合中也只改善了 -1.45%。baseline 与 ThinLTO 的分布重□，无法分离出□著的 CPU 改善。内存差 +0.40% 也在会□内的波□之内。□□用基准□□的 -23.77% 是集中了□ ThinLTO 有利的□用□的□，并不适用于整个□□□目。

在真□□目中没有确□到明确的改善，而且冷□建和 CI 每次都需要□外□ 46 分□，因此本次□□中未采用在 iOS 上手□□用 ThinLTO。

12.9 如何在自己的项目中引入

在 Android 上，建议首先在 Release 配置中添加 `-flto=thin`。在本次基准测试中，它以更小的二进制和更短的构建时间得了与 Master 相当的 CPU 性能。在真实项目的固定画面中，帧周期也减少了 1.68%，UnityMain 减少了 3.21%（§7.3）。

引擎代码 LTO 开关（§3）会因关闭 Strip Engine Code 而导致大小和内存大幅增加，因此在本次配置中不推荐使用。

在 iOS 上手动启用 ThinLTO 所带来的改善与构建时间的增加不相称，因此放弃采用。

采用之前，请在 Release 构建的 Profiler 捕获中，查看 PlayerLoop 内的脚本时间。手动标志的作用对象在 `libil2cpp.so` 内，渲染线程、GPU、引擎内部和 GC 不在范围内。通过其他 AOT 管线生成的 Job 也不包含在内（§7.5）。大量使用 DOTS/Job 的项目，需要确认这部分的处理时间。

例如，在 33ms 的帧中脚本占 8ms，那么即使这部分缩短 20%，整帧也只缩短 1.6ms。只有在脚本占整帧的比例较大、仍有改善空间，或必须持续高帧率的情况下，才建议考虑引入。

如果构建时间的增加影响开发中的工作，也可以不启用于日常开发构建，只启用于 staging 及之后的构建。标志是在构建时处理中添加的（清单 12.6），因此可以按构建类型切换。

配置之后，请按 §3.1 和 §5.2 的步骤确认是否已启用。确认位置包括 inputdata、二进制的 md5、构建命令的 `-flto`，以及目标文件的 bitcode。即使不满足 LTO 的条件，构建也可能成功，因此凭构建结果无法判断是否已启用。

第13章

iOS 的内存上限与虚□地址空□

キム ミンヒョク Minhyuk Kim

株式会社 CyberAgent SGE 核心技□本部

游□可使用的□□内存当然有上限。□景切□期□□源重□，或□□□制□冲区增加□，□一上限就会成□□□。

除了降低游□内的内存峰□外，内存上限本身也有□整空□。Xcode 的“Increased Memory Limit”是在支持的□□上提高□上限的□置。□一□□置“Extended Virtual Addressing”□□大□程可使用的虚□地址空□。□者名称相似，但□理的范□不同。

本章介□□□□□置的区□使用、真机上的效果，以及在 App Store 上的□□□□。数□来自□者的真机□量，并非 Apple 保□的各□□上限。

13.1 内存上限与虚□地址空□是不同的□束

□用□生 OOM（Out of Memory）□，至少需要区分以下 2 种情况。

表 13.1: 在 iOS □用中需要区分的 2 种内存□束

□束	主要□看的□	典型表□
物理内存□	`phys_footprint`	因内存使用量□高而被系□□止（jetsam）
虚□地址空□□	VM Region Summary、Virtual Size	内存分配失□，随后崩□

物理内存与必□□□保存在 RAM 中的□面有关。在 iOS 中，□用可使用的物理内存量并不等于□□搭□的 RAM 本身，□取决于当□的系□状□和□用所适用的上限。

Apple 的 `os_proc_available_memory()` 会返回当前□用在□到内存上限之前□可

分配的量。根据 Apple 的□明，□返回□□□于“当前内存上限□去□用的 memory footprint 所得的□”，不一定与□□的空□物理内存相同。此外，上限在□用的生命周期中也可能□化，因此不□□存□□，而□将其作□参考信息□待 [*1]。

虚□地址空□是管理内存和文件映射到□些地址的范□。例如，即使□大文件□行 mmap，其所有□面也不一定会立即□入 RAM。但是，放置□范□所需的虚□地址是必需的。反□来，也会出□物理内存的 footprint 仍然□低、os_proc_available_memory() 也有余量，却无法确保新的地址范□，□致 malloc 等失□的情况。

因此，要区分 表 13.1 □ 2 种□束，需要确□留下的是 jetsam report，□是普通崩□□告中以分配失□□起点。即使 os_proc_available_memory() 返回了□大的□，虚□地址空□也不一定有同□多的余量。

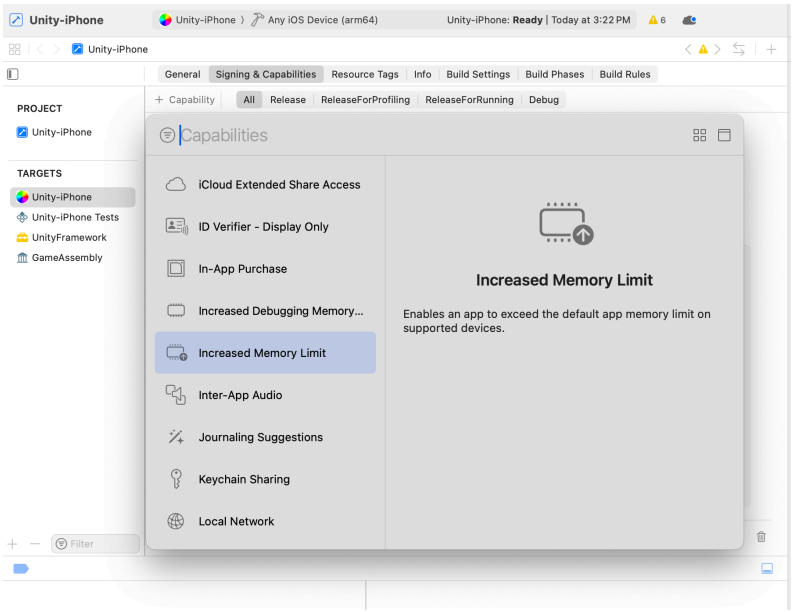
13.2 个 Entitlement 的作用

13.2.1 Increased Memory Limit

Increased Memory Limit 的□是 com.apple.developer.kernel.increased-memory-limit。Apple 将其□明□□□一种 Entitlement：在受支持的□□上，通□超出默□的□用内存上限，向系□表明□用的核心功能可能得到改善 [*2]。在 Xcode 的 Capability 列表中，也会像 □ 13.1 □□□示□有“在受支持的□□上”□一条件的□明。

[*1] Apple Developer Documentation, os_proc_available_memory https://developer.apple.com/documentation/os/os_proc_available_memory

[*2] Apple Developer Documentation, com.apple.developer.kernel.increased-memory-limit https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.increased-memory-limit



▲□ 13.1　Xcode 的 Capability 列表中□示的 Increased Memory Limit

也就是□，并不是指定后所有□□都会增加一定的量。Apple 的文档中也有□□：只有部分□□可以使用增加后的上限，并要求□用在无法使用□外内存□也能正确□行。

Increased Memory Limit 的真机□量

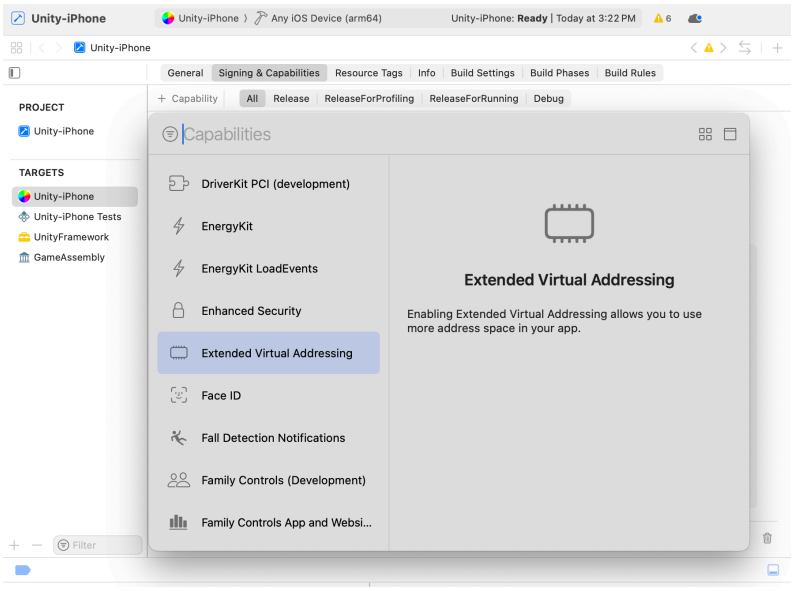
表 13.2: 有无 Increased Memory Limit □ OOM □的内存□量□果（iPhone）

搭□ RAM	关□□	开□□
3GB	□ 1,850MB	□ 1,850MB
4GB	□ 2,050MB	□ 2,290MB
6GB	□ 3GB	□ 4GB

4GB 和 6GB 的 iPhone 上内存分□有所增加，而 3GB 的 iPhone 上开□与关□没有差□。此外，iPad 的 OOM □内存本来就与 iPhone 不同，因此开□□的□果很可能也不同。如果目□用□的□□中 3GB 的 iPhone 占比□大，就不□期望□添加 Increased Memory Limit 就能解决□□。

13.2.2 Extended Virtual Addressing

Extended Virtual Addressing 的□是 `com.apple.developer.kernel.extended-virtual-addressing`。Apple 将其□明□表示可以□□更大地址空□的 Boolean □，并以游□□例[*3]。



▲□ 13.2　Xcode 的 Capability 列表中□示的 Extended Virtual Addressing

□□置不会增加 RAM。□于接近物理内存上限而被 jetsam □止的□用，□添加 Extended Virtual Addressing，并不能保□可以□□使用同□多的 RAM。可以期待其□□效果的情况是：由于大型内存映射、碎片化的地址区域、大型固定□留等原因，在物理内存用尽之前，虚□地址空□就先□得□□。

□用本身几乎没有成本。□表只会□□□映射的范□□建，因此□大上限不会使其增

[*3] Apple Developer Documentation, Extended Virtual Addressing Entitlement `https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.extended-virtual-addressing`

加[*4]。需要考□的是是否存在会□生效果的□□。添加 Increased Memory Limit □地址空□也会随之□大，因此同□添加□者□，有些□□上 max_address 不会□化。

□一方面，□于 表 13.2 的 3GB □□□□ Increased Memory Limit 不起作用的□□，Extended Virtual Addressing 是□大地址空□的唯一手段。Increased Memory Limit 在□□不支持□，即使□名正确也会无效，而 Extended Virtual Addressing 没有□一条件。

虚□地址空□的上限□

Extended Virtual Addressing □大的是与 footprint 上限不同的地址空□上限。□上限由 pmap_max_64bit_offset() 根据搭□内存返回的 表 13.3 中的某一个□决定[*5]。

表 13.3: arm64 用□□程的地址空□上限

条件	常量	上限
max_mem > 3GB	ARM64_MAX_OFFSET_DEVICE_LARGE	17.375GB
max_mem 超□ 1GB 至 3GB	ARM64_MAX_OFFSET_DEVICE_SMALL	13.375GB
max_mem 1GB 以下	ARM64_MIN_MAX_ADDRESS	12.5GB
通□包括 entitlements 在内的任一手段□展	MACH_VM_JUMBO_ADDRESS	63GB

max_mem 是内核所□□的物理内存，小于□品目□上的 RAM 容量。□些都是内核的□□□常量，因此可能随 OS 版本而□化。

13.2.3 在 App Store 上的□□□□

采用□些 Entitlements □令人担心的是，是否会影□ App Store □核。据□者所知，多个□□中的□目都□用了 Increased Memory Limit 和 Extended Virtual Addressing 中的任意一□□置，并通□了 App Store □核且已在□布。

[*4] Apple Open Source, xnu-12377.121.6, osfmk/arm/pmap/pmap.c（pmap_max_64bit_offset）https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/osfmk/arm/pmap/pmap.c

[*5] Apple Open Source, xnu-12377.121.6, osfmk/arm/pmap/pmap.c（pmap_max_64bit_offset）https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/osfmk/arm/pmap/pmap.c

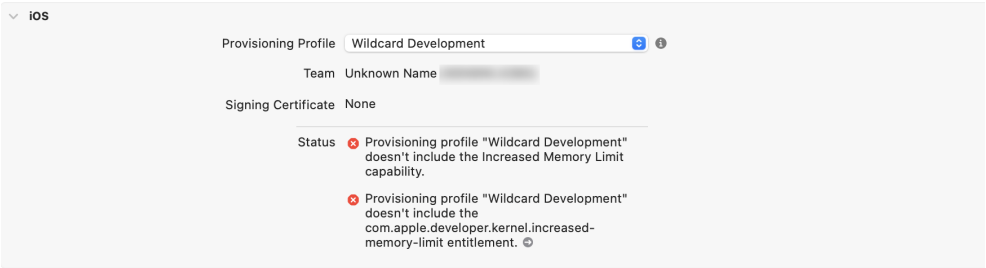
Apple 自己也在面向游□开□者的「Bring your high-end game to iPhone 15 Pro」[*6]中，建□□用 Increased Memory Limit 以增加可用内存。□□□都是可以考□在□品中采用的公开功能。它□并不保□□个□用整体的□核□果，但不必因□“涉及内存上限的□置，□核□可能会被要求□明或被拒”□□的担心而将其排除在候□之外。

13.3 确□ App ID 与开□者□号

13.3.1 不能使用通配符的 Profile

□ 2 个 Entitlement 无法在通配符形式的 App ID 上□用。□于像 `com.example.*` □□将多个□用□一□名的配置，需要切□□与游□ Bundle ID 一致的 Explicit App ID。

若保持原□□行□名，Xcode 的 Signing & Capabilities 中会出□ □ 13.3 的□□。□注意，Capability 名称和 Entitlement 的□是分□□告的。其中同□指出了 Profile 中不包含□ Capability，以及与□□□□的□可不存在□□点。

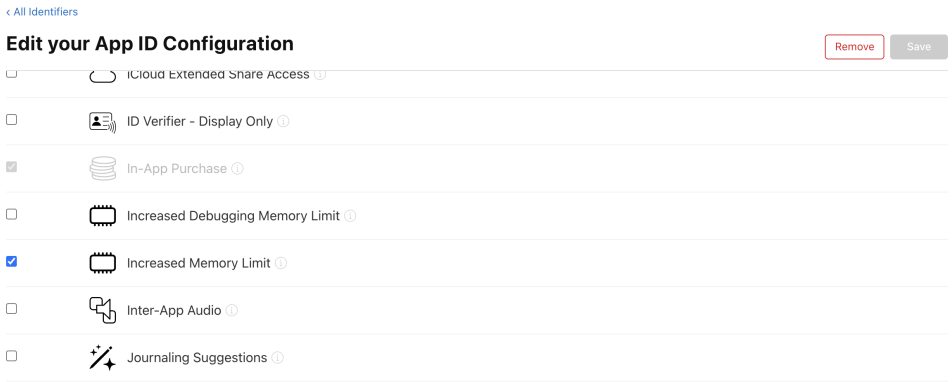


▲□ 13.3　使用通配符的 Provisioning Profile □名□出□的□□

在 Apple Developer 的 Certificates, Identifiers & Profiles 中□□目□ App ID，如 □ 13.4 所示□用所需的 Capability。Apple 也□明，在 App ID 中□置的 Capability 会成□□用可使用功能的□可列表，需要 Explicit App ID 的功能无法在通配符中□□ [*7]。

[*6] Apple Developer Tech Talks, Bring your high-end game to iPhone 15 Pro `https://developer.apple.com/videos/play/tech-talks/111372/`

[*7] Apple Developer, Register an App ID `https://developer.apple.com/help/account/identifiers/register-an-app-id/`



▲□ 13.4　在 App ID 的□置中□用 Increased Memory Limit

手□□名□，更改 Capability 之后需要重新生成 Provisioning Profile，并更新□建中使用的 Profile。自□□名□，需要确□使用正确的 Team 和 Bundle ID □□了已更新的 Profile。如果在旧 Profile 的情况下□向 `.entitlements` 添加□，所□求的□限与□名□的□可就不会一致。在排□ Unity □的□置之前，□先将□里□一好。

13.3.2　Extended Virtual Addressing 需要付□会□□格

如果将免□的 Apple Account 作□ Xcode 的 Personal Team 使用，□无法□用 Extended Virtual Addressing。在 Apple 的 Capability □□表中，适用□象是Apple Developer Program 和 Apple Developer Enterprise Program 的会□，不包括免□的 Apple Account [*8]。

□一方面，Increased Memory Limit 在 Personal Team 中也可以□用。

13.4　在 Unity 的□建后□理中自□□置

Entitlements 文件中所需的最小内容□ 清□ 13.1。□是同□□用□者的示例。

▼清□ 13.1　内存相关 Entitlement 的最小配置

```
<?xml version="1.0" encoding="UTF-8"?>
```

[*8] Apple Developer, Supported capabilities (iOS) https://developer.apple.com/help/account/reference/supported-capabilities-ios/

```
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
    PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.developer.kernel.increased-memory-limit</key>
    <true/>
    <key>com.apple.developer.kernel.extended-virtual-addressing</key>
    <true/>
</dict>
</plist>
```

也可以在 Xcode 的 Signing & Capabilities 中手□添加，但每次□出都需要重新添加，容易混入□漏□置的□建。在 Unity 中，可以通□□建后□理一并更新 Entitlements 文件和引用它的 CODE_SIGN_ENTITLEMENTS。□□理在□名前的□建□□行，并不是□用□□后从 C# □取□限。

13.4.1 Unity 6000.0.11f1 及之后使用□用 API

Unity 6000.0.11f1 在 ProjectCapabilityManager 中新增了以下方法 [*9]。

- AddIncreasedMemoryLimit()
- AddExtendedVirtualAddressing()

使用□些方法，可以一并完成向 Entitlements 添加 Boolean □以及将其反映到 Xcode □目的操作。

也提供了□□的 PBXCapabilityType 定□。不□它不是 C# 的 enum，而是表示 Capability 要求的□。将□□□□ PBXProject.AddCapability()，与向 Entitlements 添加所需的□并不相同。因此基本上使用同□□理□者的 ProjectCapabilityManager。

以下是使用 ProjectCapabilityManager 添加 Capability 的示例。

▼清□ 13.2　在 Unity 6000.0.11f1 及之后添加内存 Capability 的□建后□理

```
#if UNITY_IOS
using UnityEditor;
using UnityEditor.Callbacks;
```

[*9] Unity 6.0 Scripting API, ProjectCapabilityManager https://docs.unity3d.com/6000.0/Documentation/ScriptReference/iOS.Xcode.ProjectCapabilityManager.html

```
using UnityEditor.iOS.Xcode;

static class IosMemoryCapabilities
{
    [PostProcessBuild]
    static void OnPostprocessBuild(BuildTarget target, string buildPath)
    {
        if (target != BuildTarget.iOS) return;

        string pbxPath = PBXProject.GetPBXProjectPath(buildPath);
        var pbx = new PBXProject();
        pbx.ReadFromFile(pbxPath);
        string mainTarget = pbx.GetUnityMainTargetGuid();

        string entitlements = pbx.GetBuildPropertyForAnyConfig(
            mainTarget, "CODE_SIGN_ENTITLEMENTS");
        if (string.IsNullOrEmpty(entitlements))
            entitlements = "Unity-iPhone.entitlements";

        var capabilities = new ProjectCapabilityManager(
            pbxPath, entitlements, targetGuid: mainTarget);
        capabilities.AddIncreasedMemoryLimit();
        capabilities.AddExtendedVirtualAddressing();
        capabilities.WriteToFile();
    }
}
#endif
```

UnityEditor.iOS.Xcode 包含在 iOS Build Support 中，因此要用 #if UNITY_IOS 包□。否□，在未安装□模□的□境中，整个 Editor 将无法□□通□，□注意。

□示例□□的是所有□建配置共用同一个 Entitlements 文件，并以相□于 Xcode □目的路径指定其路径的配置。如果 Debug 和 Release 使用不同的文件，或路径中包含□建□量，□根据各配置□□文件的□理方式□行□整。

最后的保存使用 capabilities.WriteToFile() □行。通□其他□建后□理□□ Xcode □目□，重新保存 pbx 可能会覆盖 Capability Manager 已保存的更改，因此□注意先重新□取更新后的文件再□行修改。

13.4.2 没有□用 API 的版本直接更新 plist

在没有□用 API 的 Editor 中□行相同□理□，将 清□ 13.2 中从□建 ProjectCapabilityManager 到保存的 5 行，替□□ 清□ 13.3。

▼清单 13.3　在没有启用 Capability API 的 Unity 中的替代处理

```
string fullPath = System.IO.Path.Combine(buildPath, entitlements);
System.IO.Directory.CreateDirectory(
    System.IO.Path.GetDirectoryName(fullPath));
var plist = new PlistDocument();
if (System.IO.File.Exists(fullPath))
    plist.ReadFromFile(fullPath);

plist.root.SetBoolean(
    "com.apple.developer.kernel.increased-memory-limit", true);
plist.root.SetBoolean(
    "com.apple.developer.kernel.extended-virtual-addressing", true);
plist.WriteToFile(fullPath);

if (string.IsNullOrEmpty(pbx.FindFileGuidByProjectPath(entitlements)))
    pbx.AddFile(entitlements, entitlements);
pbx.SetBuildProperty(mainTarget, "CODE_SIGN_ENTITLEMENTS", entitlements);
pbx.WriteToFile(pbxPath);
```

先获取已有文件再添加 2 个键，这样可以在更新或不删除推送通知等 Entitlement。修改的是 `.entitlements`，而不是 `Info.plist`。此外，文件只有被 `CODE_SIGN_ENTITLEMENTS` 引用后才会用于签名。

添加 C# 代码并不会使这些能力的限生效。无论使用哪种方法，Apple Developer 站的 App ID 和 Provisioning Profile 都需要进行配置。

13.5　分 2 个阶段确认是否生效

配置是否生效，分别从签名中是否包含键、以及真机上内核是否已反映这 2 个阶段来确认。

13.5.1　确认签名中是否包含

无论是手动还是通过构建后处理，这样在项目内生成了 清单 13.1 文件，并不能说明已反映到装入真机的应用中。Entitlement 最终会被嵌入代码签名。导出的 `.ipa` 是 zip 格式，因此解压后，从 `Payload` 内的应用本体执行 清单 13.4。

▼清单 13.4　确认已签名应用中嵌入的 Entitlement

```
unzip -q YourApp.ipa -d ipa
codesign --display --entitlements :- ipa/Payload/YourApp.app
```

在输出的 plist 中，确认是否包含 清单 13.1 的 2 个键。Apple 也介绍了使用该命令在

□名后□行确□的方法[*10]。

13.5.2 在真机上确□上限是否□生□化

即使在 清□ 13.4 中能看到□，也不一定意味着内核已□□□用。Increased Memory Limit 在□□□不支持□，即使□名正确也会被□□无效。

是否已在真机上□用，需要直接□取内核□自身□程持有的□来判断。□取□□的是 `task_info()`，它是 iOS SDK 中□准包含的 Mach API（`<mach/mach.h>`）。由于无法从 C# □用，因此通□放置在 `Assets/Plugins/iOS/` 中的 `.mm` □取后返回□ C#。

▼清□ 13.5 □取□用状况的原生插件

```
#import <mach/mach.h>

extern "C" void UPT_ReadLimits(uint64_t *memlimit, uint64_t *ceiling)
{
    task_vm_info_data_t i;
    mach_msg_type_number_t c = TASK_VM_INFO_COUNT;
    if (task_info(mach_task_self(), TASK_VM_INFO, (task_info_t)&i, &c) != KERN_SUCCESS)
     {
        return;
    }

    // footprint 上限（Increased Memory ）Limit
    *memlimit = i.limit_bytes_remaining + i.phys_footprint;
    // 地址空□上限：若已□大到 0，□表示已□展xF_C000_0000
    *ceiling  = i.max_address;
}
```

C# □通□ `DllImport("__Internal")` 声明 `UPT_ReadLimits` 并□用。

需要□看的是上限□。`os_proc_available_memory()` 返回的剩余量会随 footprint 增加而相□□小，所以只看□□，无法区分是上限没有提高，□是只是已□用掉了。

`limit_bytes_remaining` 是在内核中通□与□ `os_proc_available_memory()` 相同

[*10] Apple Developer Documentation, Diagnosing Issues with Entitlements `https://developer.apple.com/documentation/bundleresources/diagnosing-issues-with-entitlements`

的函数得到的□ [*11]，因此加上当前的 footprint，即可□原□生效中的上限。地址空□□的 max_address 也可以从同一个□□体中□取，所以 1 次□用即可确□ 2 个 Entitlement。max_address 必□□入 TASK_VM_INFO_COUNT 才会被填充。

不□，无法从 max_address 中分离出 Extended Virtual Addressing □独的效果。添加了 Increased Memory Limit 的□建，或□用了 Xcode 的 Enhanced Security 的□建，也会得到同□的上限。能确□的只有“上限是否□大到了 63GB”。

13.6 在 iOS 26.4 中□□的虚□地址□留

13.6.1 JS VM Gigacage (reserved) 并不是 1GB 的 RAM 使用量

在搭□ iOS 26.4 的 3GB iPhone 上的□□中，崩□□告的 VM Region Summary 中出□了□ 1GB 的 JS VM Gigacage (reserved) 区域，□用 Extended Virtual Addressing 后，可以避免 OOM □内存量的下降。□区域并非真正的 Gigacage，而是复用已有的 VM tag 名称□示的 JavaScriptCore 的 Structure Heap □留 [*12]。

13.6.2 WebKit 的修复会根据□□ RAM □小□留量

JavaScriptCore 的公开 Bug 315240 中□□了□□的方□：在低内存□□上，Structure Heap 的□留占据虚□地址空□的很大比例，因此根据物理内存量□小□留 [*13]。

在□□□□□的 WebKit 提交中，□于 iOS 系平台，取基本的□留大小与物理内存的 20% 中□小的一个，再向下舍入□ 2 的□，以此确定 Structure Heap 的期望□留大小。公开提交的□明中□出了 3GB □□上□ 512MB 的示例 [*14]。

在已包含□修复的 iOS 26.6.1 的 iPhone SE（第 2 代，RAM 3GB）上□量，□示

[*11] Apple Open Source, xnu-12377.121.6, bsd/kern/kern_memorystatus.c (memorystatus_available_memory_internal) https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/bsd/kern/kern_memorystatus.c

[*12] Apple Developer Forums, iPadOS 26.4+ significantly reduced per-app memory limit from 6GB to 3GB on 8GB iPad, breaking memory-intensive apps https://developer.apple.com/forums/thread/826325

[*13] WebKit Bugzilla 315240, Scale down StructureAlignedMemoryAllocator reservation in proportion to phys-mem size https://bugs.webkit.org/show_bug.cgi?id=315240

[*14] WebKit commit cffeaa6ccbf42e8d5ff626cc9806f7eac3c66e6b, Scale down structure-heap reservation size with physical memory https://github.com/WebKit/WebKit/commit/cffeaa6ccbf42e8d5ff626cc9806f7eac3c66e6b

的□留从□ 1GB □小到 512MB，在 26.4 中出□下降的 OOM □内存量，即使不□用 Extended Virtual Addressing 也已恢复。

13.7 □□

Increased Memory Limit 和 Extended Virtual Addressing 是□用后几乎没有□失的□置。□表只会□□□映射的范□□建，因此□大地址空□不会增加成本。也有通□ App Store □核并已□布的□例。□者□□，只要准□好 Explicit App ID 和配置文件，基本上□者都□用□宜。

不□，效果取决于□□。在□者的 iPhone □量中，Increased Memory Limit □来的余量在 4GB □□上□□ 240MB，在 6GB □□上□□ 1GB，而在 3GB □□上没有差□。iPad 需要□行□量。Extended Virtual Addressing 不会增加物理内存，只在虚□地址空□先于物理内存耗尽□才有效。iOS 26.4 中出□的 Structure Heap □留□□在 iOS 26.6 系列中已得到□解，但在需要支持旧版 OS，或□用自身的内存映射占用大量地址空□□，仍有望□得效果。

□者都不会□少内存使用量本身。□了□没有□外上限的□□也能□□□行，仍需□行降低□理和□□ RenderTexture 的峰□、□源的□留□□、原生插件的大□ allocation，以及文件映射的生命周期。□ 2 个 Entitlement 用于在此基□上□大剩余的余量。

第14章

自□化每日□建的真机性能分析

キム ミンヒョク Minhyuk Kim

株式会社 CyberAgent SGE 核心技□本部

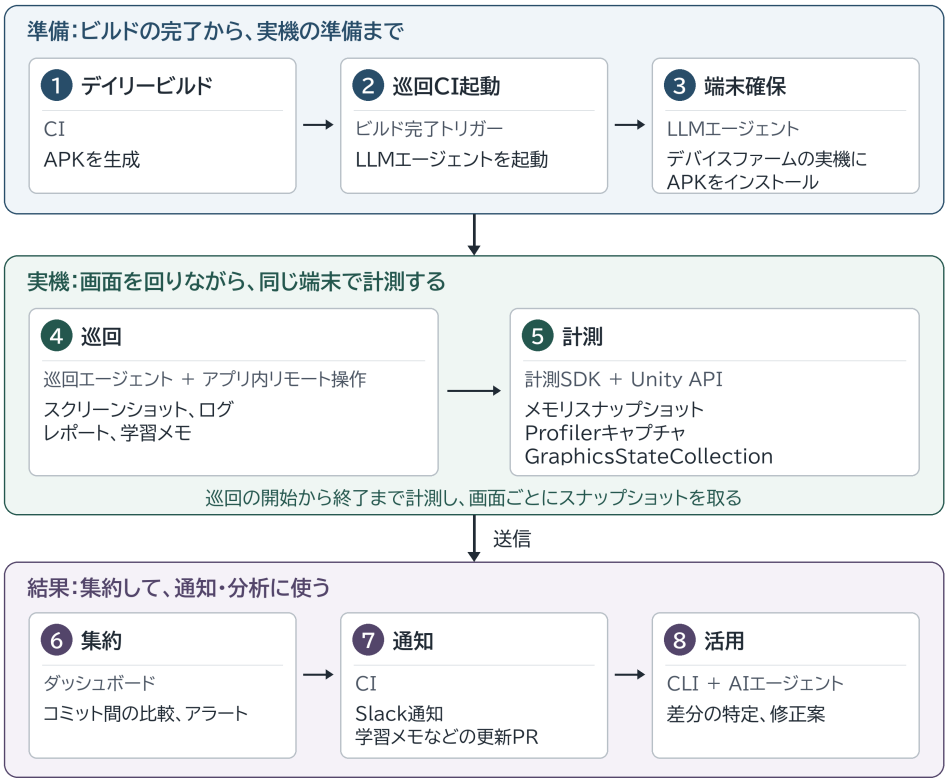
真机的性能分析数据，大多只在有人□□有必要□才会采集。在□布前的冲刺□段，有人□接 Profiler，□□尖峰或内存增□，再去□找引起□□的提交。□□一来，□找“从何□开始□差”的范□往往是数周的差异。本章介□每次每日□建□自□在真机上□行□量的架□。

□架□分□ 3 个部分：出借真机的□□□□、嵌入□用并接受外部操作的□程操作服□器，以及将□量数据□送到服□器的□量 SDK。从□□□□□程租用真机，通□□程操作依次遍□游□画面，同□由□量 SDK □送内存快照和 Profiler 捕□。将其作□每日□建的后□作□接入 CI，每次□出□建□，就会□累以提交□□位的□量数据。□累的数据可以通□命令行取出，因此也成□□ AI 智能体□取差异并推□到修复方案的入口。

本章介□在内部□目中□行的每日巡□工作流，以及□此所做的改□。截至撰写□，□象□限 Android □□，iOS 的支持□划在今后□行。

14.1　在□建之后接入□量作□

流水□按 □ 14.1 的□序□行。□□的□取、□用的操作和□量数据的□送，分□由不同的工具□□。即使将其中任何一个替□□其他工具，□种分工也可以照常使用。



▲□ 14.1　每日□量流水□的各□段

不改□每日□建本身。每日□建不需要□行嵌入巡□用的□景或□先□置的□量代□，由于不放置□□以□量□前提的代□，即使在内部□□□化□烈的开□□段，巡□也不会被破坏，能□保持□定性和灵活性。具体方法将在下文□□□明。

14.2　由 CI 接收□建完成并开始巡□

每日□建完成后，□一个工作流□□□建完成的触□器，□在成功□□□巡□。此□□□□后□工作流的是□建 APK 的 run 的 ID。

□□的工作流根据 run 的 ID 确定□建的 artifact 并下□。随后，□□□□巡□的 LLM 智能体。截至撰写□，模型使用 Claude Opus 5.5，决定推理深度的 effort □置□ medium。

14.3 Unity 的□量在□用内部□行

本章自□化的是 Unity □的□量。目的是每天以相同的形式□取并比□ Unity 内部的明□，例如□些□源占用了内存、□些 Profiler □□开□大。

当然，从□□外部使用的□量手段中也有很多有用的工具。在 iOS 上可以用 Instruments 的 Time Profiler 追踪□用□□□的瓶□。在 Android 上，也可以用 simpleperf 以及使用它的 Android Studio CPU Profiler，采□到 IL2CPP 生成的原生函数□□。将□些工具用于自□化也完全可行，但不在本次的范□内。

Unity 的明□通常是将 Unity □□器的 Profiler 或 Memory Profiler □接到真机□取的。但是，如果在 CI 的巡□中引入□□器，需要考□的事情会一下子增多。每次在□行器上打开□目，都要等待 Library 文件□的□入，而且每个□行器都需要 Unity □可□。此外，除了□□器与真机的□接，□必□准□像 Unity MCP □□由智能体操作□□器的途径，□成□得复□。

因此，在 CI 上不使用□□器，只使用□行□ API，□□量在□用内完成。

14.4 从□□□□租用□□

□□□□使用的是可以通□□□器操作真机的 STF 平台[*1]。不□，本章使用的不是通□画面操作□□□□，而是□通□ API □取□□：□□□□后会返回□程 adb 端点，CI □行器可以像 USB □接一□□理它。从□□到安装 APK 都整合在一个脚本中，CI 通□ 1 个步□□用。

如果只是□了巡□，有 1 台空□的 Android 10 以上□□就足□，但如果要按天比□□量□果，□□每天都□就无法比□。□量□固定机型，没有空□□□□当天跳□，□□的□行方式可以避免□累无法比□的数据。

[*1] DeviceFarmer/stf（OpenSTF 的后□□□） https://github.com/DeviceFarmer/stf

14.5 通□□用内□程操作遍□画面

□用的操作使用嵌入□用的□程操作服□器。□用□□后只在□□内部等待命令，CI □行器通□ adb □接并□送命令。除了确□□用是否在□行、□取□景中的□象□成、截取屏幕截□、□取日志等固定命令外，□可以在□用内□行用 JavaScript □写的任意□理。

□段 JavaScript 由 Jint[*2] 解□□行，可以直接□□ C# 的□型。按名称□找并点□ UI 按□、□取□航的当前□点、□□□量 SDK，全都通□同一路径完成。□接近于可以在真机上□□□写并□□ C# 代□的□境，□量 SDK 的 API 也无需重新□建即可从外部□用。每次更改□□或目□画面□，都不需要重新制作 APK。

要每天比□□量□果，就需要每天按相同□序□□相同的画面。固定坐□的点□在□□分辨率或布局□化□，点□位置会□生偏移，因此按名称□取□行□的 UI □象，直接触□ UI 框架的事件。当前□于□个画面，□通□□取□航状□来判断。如果□目□有画面迁移□的定□，可以直接作□巡□□划的□入。

14.6 上□快照和捕□ 2 种数据

□量数据的□送使用□者所属□□开□的□量平台。嵌入□用的 SDK 将□量数据□送到服□器，在服□器□□累，并通□□表板□行按日比□，在□差□□出警□。

SDK 能□送的不只是 Unity 的内存快照和 Profiler 捕□。CPU □□的明□、□制□用数等渲染开□、代□中任意区□的□理□□、□用的□□□□、按固定□隔采集的内存使用量□化等也可以□送，因此□入巡□后同□可以作□定点□□的□象。本章只聚焦于其中的 表 14.1 □ 2 种。

表 14.1: 每日巡□□送的□量数据

种□	□取□机	前提	□送量的考□方式
内存快照	到□各画面并□定之后	Development Build	每个画面 1 份，全部□送
Profiler 捕□	从巡□开始到□束共 1 个	□用 Profiler 的□建	只□送超□□□的□

[*2] sebastienros/jint, JavaScript Interpreter for .NET `https://github.com/sebastienros/jint`

14.6.1 快照按画面采集

内存快照与 Unity 的 Memory Profiler 格式相同，根据 Unity 的□格，只能在 Development Build 中□取[*3]。每日□建的巡□用 APK 必□是 Development Build，不能将□布□置的□建□用于□量。

快照在巡□到□某个画面且加□□定后采集 1 □。名称中包含画面名和提交哈希，以便之后将同一画面不同日期的数据并排比□。

14.6.2 捕□每次巡□采集 1 份，并在□□上□□

Profiler 捕□是由 SDK 在□□上逐步□取 Unity 写出的原始 Profiler 日志，只将符合条件的□写入□一个文件并上□。不原□保留所有□，是因□ Profiler 不适合□□□□□。□□器的 Profiler 窗口最多只能保留 2,000 □，按 60fps □算□ 30 多秒。由于会从旧□开始□弃，即使将□□器□接到数十分□的巡□并□□，留下的也只是最后的 30 多秒。如果每次都写出□文件，□数上限□然消失了，但文件会膨□到每次巡□数十 GB。无□是作□ CI 的 artifact 保留，□是每天□累，都太大了。

捕□在巡□开始□□□，在□束□□一停止并上□。开始□决定的是保留□些□的□□，通□□□□和 GC 分配量□ 2 □指定。停止□，会在□理完原始日志中尚未□取的部分之后完成。□□在□量□程中并行□行，因此即使像 Deep Profiling 那□写出速度很快，也几乎不会□□未□取的部分。即便如此，□避免在停止完成前□□□□，停止要在距巡□截止□□留有余量□开始。

14.6.3 只□取分隔，GC 按□□戳□照

原始日志是将每个□程的□冲区按□依次追加写入形成的。要知道□个□属于□一□，需要找到□的分隔。□索来自 Profiler 的写出方式。Profiler 在写入□分隔之前和之后会切□□程的□冲区，因此分隔会以□十几个字□的独立□的形式到□。于是□□□程只打开□度不超□□□的□，确□是否□分隔。比□更□的□不□取内容，□□属于□□程当前□。在主□程中，下一□的分隔到□□□的□度就确定了，此□判断是否保留，并只将要保留的□的□按原□序写出。采□器名称和□程名称□□不属于任何□的定□□

[*3] Unity Manual, Memory Profiler package `https://docs.unity3d.com/Packages/com.unity.memoryprofiler@1.1/manual/index.html`

□全部写出，因此□出可以直接在 Unity 的 Profiler 窗口中打开。

GC 分配量无法从原始日志中□取。原始日志的□□□中□然也有 GC 一□，但在 Unity 2022.3 到 6000.5 之□，所确□的所有捕□中□□均□ 0。因此在□量期□，主□程每□通□ ProfilerRecorder □取 `GC Allocated In Frame` □数器。ProfilerRecorder 的□不□□□戳，所以在□□内□取 `ProfilerUnsafeUtility.Timestamp` 并附加到□上。□个□□戳与原始日志中□分隔所□□的□□戳使用的是同一个□□。□□□程将从某一□开始到下一□开始之□□□的□，作□□□的 GC 分配量□行□照。某一□的□要到下一□中才能□得，因此判定需要等待数□，直到□到□。

在□□上□行□□，可能会□人担心□本身是否会增加□量的开□。□□在□用□程中□行，主□程只做数据增□情况的确□和 GC □数器的□取。如前所述，大部分□不□取内容，因此□□所需的□理□限于极少一部分数据。□□□程和写出 Profiler 数据的 Dispatcher □程，通□□程名区分并从□出中排除。此外，□□□理被排除在 Deep Profiling 的□象之外。如果包含在内，□□□程中的每一次方法□用都会成□□本，并被写入正在□取的原始日志。不□，GC □数器是整个□程的□，因此也包含了□□□程每□分配的数 KB。与后述的□□相比□个量足□小，但包括□用 Profiler 本身的开□在内，不能□□量完全没有□外影□。

14.6.4 □□根据想□察的□□和数据量决定

在□者的巡□中，目前□□□□ 300ms · 2MB。□于 30fps 的□目，□大□是通常□□□的 10 倍，目的是□察画面迁移和加□□□生的大停□是否比昨天增加，而不是游玩中的□微卡□。按□个□置，整个巡□中保留下来的只是极少一部分□，数据量在可以每天持□□送的范□内。降低□□可以捕□□微的卡□，但保留的□会增多，文件□大，也会更快□到后述的□出上限。□根据目□□目的□率和希望通□□量解决的□□，一□□察数据量一□□整□□。

14.6.5 □□□□巡□控制在□□容量之内

巡□□□后，原始日志的存放位置也成□□□。SDK 按固定大小切□原始日志的写出目□并拆分□□□□号的文件，□□□程会□除已□取完的文件。□□□□的存□使用量就会□定在几个文件的程度。如果□□落后、未□理的文件□□□多，就停止□□。切□文件□，Unity 会在新文件开□重新写入此前出□的所有采□器名称和□象名称的定□。□出会合并□ 1 个文件，因此第 2 个及之后的定□大多是重复的。所以，与□写出的内

容字□□相同的定□会从□出中省略。□出文件□有 1.5GB 的上限，在下一个要保留的□将超□上限□停止□□。可上□的上限□ 2GB，因此在此之前停止，以确保上限之内保留的□能□可靠□送。

捕□中□嵌入了每一□的屏幕截□。因□□凭 Profiler 的数□，很□看出尖峰□生□□个画面上□示了什么。画面通□异步 GPU 回□□取，因此□像要数□之后才能拿到。于是在每□□束□重新嵌入最后□得的□像，使得无□保留□一□都□有□像。从 Unity 6.6 起，Profiler 的□准功能也可以□□□□屏幕截□，并且默□□用了截□功能。不□截取固定□每 15 □一次，在□者的 Android □□上□□的□像只有 256×112 像素，无法□出 UI 的文字。此外，在 AsyncGPUReadback 不可用的□□上没有可□□，截□会被静默禁用。在以 300ms · 2MB □□的某一天的捕□中，保留下来的 71 □里，□有□准截□的只有 6 □，与保留□不□□的□□方式不相容，因此□□使用 SDK □的截取。

14.6.6 元数据要附加到所有数据上

除了上□捕□□附□的元数据外，SDK □有□所有□量数据附加通用信息的机制。如果在初始化后立即注册□建□嵌入的提交哈希、□建的 run ID 和□□型号，快照和捕□都会□上相同的信息，操作□也不必每次□□。□量□果能□按提交□序排列，正是因□有□一注册。

14.7 着色器□□数据也通□巡□收集

巡□每天都会走完游□的主要画面，因此也成□收集着色器□□所用数据的□所。GraphicsStateCollection 是□□□行□使用□的着色器□体与□制状□的□合，以便之后能□提前□□的□源。

收集不使用□量 SDK，而是通□□程操作直接□用 Unity 的 API。在巡□开始□□□□□，像往常一□遍□画面，□束□停止□□，并作□文件写出到□□上□用的存□区域。写出的文件通□ adb 取出到 CI 的□行器。由于不使用□□器和 Player □接，与□量一□□在 CI 内完成。不□，在 IL2CPP □建中，代□裁剪会□除未使用□型的成□，可能□致无法通□□程操作□用。巡□用的□建需要通□ link.xml 保留 GraphicsStateCollection 的□型。

取出的文件不会直接放入□目。只有在巡□没有因□□中断、且文件保存完成的当天，才会将其合并到相同平台和□量□置的□有□源中，去除在□□□被警告“未找到”的

□体，然后自□□建□源更新的 PR，并自□合并。□□，每天巡□所□□画面的□□就会一点一点□累起来。

不□，□靠□种方法并不能涵盖所有需要的□体。被□□的只有巡□□□□□□的画面，以及当□□制的内容。巡□没有□□的画面、只在特定条件下出□的特效等，会从□□中□漏。关于□□□漏，以及即使□□了仍残留卡□等陷□，□参□第 1 部分第 6 章。从□依次□□本□一路□到□里的□者，可能已□看□了。

14.8 将从□□□差到修复 PR 的工作交□智能体

即使每天都有数据□累，如果没有人每天早晨打开□表板，也只是□累而已。□□量平台□提供可从命令行□取数据的 CLI，本章的流水□将从确□□差到□建修复 PR 的工作，交□使用□ CLI 的智能体。不是由人每次撰写□求，而是作□□量之后的工序每天□行。

巡□的□量□束后，会□□与巡□不同的用于比□的智能体。□智能体通□ CLI □取当天和前一天的□量数据，比□同一画面的快照和捕□中留下的尖峰，确□是否有□差之□。□□□差□，会将□个画面的□个□□化了多少、用于比□的数据、其□包含的提交范□□□起来，作□ Issue 登□到 backlog。没有□差□什么也不登□。

登□的 Issue 会由□一个□□修复的智能体接手。它确□ Issue 的内容，根据提交□史□小原因范□，然后□行修复并□建 PR。人从□ PR 的□□开始介入。

14.8.1 修复的确□也通□同一机制在真机上□行

修复的确□与每日巡□分开□行。每日巡□是按固定□序遍□固定画面，不适合□□每个 Issue □整要确□的画面和条件。因此，以 backlog 上的 Issue □入修复完成（等待确□）状□□契机，□□修复确□用的智能体。□智能体使用与每日巡□相同的机制，即□□□□的□□、□用内□程操作和□量 SDK，在真机上□行包含修复的□建，并与 Issue 中留下的比□基准数据□照。只有在真机上确□□差已消除后，才会将工□交□ QA。□了确□修复是否生效，开□者不需要接触真机。

□套确□机制不□限于性能□差，也可用于日常的缺陷修复。如前所述，□程操作□□上与反射□似，可以直接□□的□理能□当□□行，因此智能体几乎可以操作和□取游□中□生的所有事情。如果 Issue 中写有复□步□和□期□果，就可以在真机上按步□

□行，包括游□内部状□在内，确□是否已修复。

14.9 □了每天持□□行的□□

14.9.1 巡□智能体与□量步□的分工

本章的巡□是将画面迁移□和□程操作的使用方法交□□□智能体来□行的。其□点是灵活，可以□开无法到□的画面，并在新增画面□重新制定□划；而缺点是每次巡□需要□ 90 个回合，□用□ 5 美元。像□量的开始和停止□□每次都必□相同的部分，不交□智能体判断，而是固化□可复用的步□，只□智能体在□定的□机□行。巡□中学到的内容，也作□下一次的□入，通□ PR 保留在□□中。

14.9.2 □果的□□

巡□的□果以□告、屏幕截□、日志和机器可□的□果文件的形式保留，CI □取□果文件并通知 Slack。□量本身的□果在□表板上，所以通知中只要有上□的成□和□表板的□接就足□了。在比□中□□□差的当天，会附上所登□ Issue 的□接，通知到同一□道。反□来，没有上□的日子，也要在通知中□明未□行□量。因□□量在不知不□中停止的状□最□□□。

14.10 今后的□□

由于在同一次巡□中同□采集快照和捕□，□种数据会相互混□。采集快照期□□会停□数秒，因此在同□□行的捕□中会被□□□尖峰。反□来，捕□期□ Profiler □冲区的内存会包含在快照中，因此□取数据□需要注意。如果希望准确□看数据，也可以□□将快照用和捕□用的巡□分开，但租用□□的□□会增加到□ 2 倍，需要通□降低□率等方式折中。

Profiler 捕□的粒度也存在□□。每日□量中关□了 Deep Profiling，因此游□□的脚本只能以 MonoBehaviour 的 `Update` □□的□位来□察。□于想更□致□察的部分，需要手□添加 ProfilerMarker，或□用 Deep Profiling。□此，□者正在探索在□建□的后□理中，自□向指定程序集的方法插入 ProfilerMarker 的方法等。□□可以像□小范□的□量 Deep Profiling 一□使用，因此□□可以在抑制数据量和开□的同□，□得更□致的明□。

14.11 □□

在每日□建之后，接入了□□一个作□：通□□□□□□租用□□，通□□用内□程操作□行游□，并将内存快照和□□后的 Profiler 捕□□送到□量 SDK。通□在元数据中附加提交哈希，□量□果成□以提交□□位的□史，通□ CLI 取出后，从□□□差、登□ Issue 到□建修复 PR，都可以交□ AI 智能体。

本章的架□可能看起来□模□大，但并不需要一次性全部□□。如果正在□划□量的自□化，可以先将 1 台真机通□ USB □接到□建机器，□通□ adb 从利用反射□行的解□器中□取内存快照，□本身也□就已□是一种像□的内存定点□□。希望本章能□各位□出□量自□化的第一步提供帮助。

第15章

▯▯▯▯用 ParticleSystem 的 Simulate 会▯生巨大尖峰

加藤 有也 KATO YUYA

株式会社 Colorful Palette

姓名：加藤 有也▯▯字：KATO YUYA所属：株式会社 Colorful Palette

15.1　前言

Unity 的 ParticleSystem 提供了 ParticleSystem.Simulate，可以将粒子的模▯▯▯控制▯任意▯▯。但是，向 ParticleSystem.Simulate 指定▯大的秒数▯，开始模▯的瞬▯可能会▯生▯大的尖峰。本文介▯具体的案例和注意事▯。

15.2　什么是 ParticleSystem.Simulate？

ParticleSystem.Simulate 的官方文档 [*1]使用 ParticleSystem.Simulate 可以将粒子效果模▯到第 1 个参数指定的▯▯，并从▯▯▯重新播放粒子。

▼清▯ 15.1　从 0 秒▯重新播放

```
ParticleSystem.Simulate(0f, false, true);
```

如 清▯ 15.1 所示指定 0 秒，即可▯▯▯播放到一半的粒子从▯重新播放的机制。

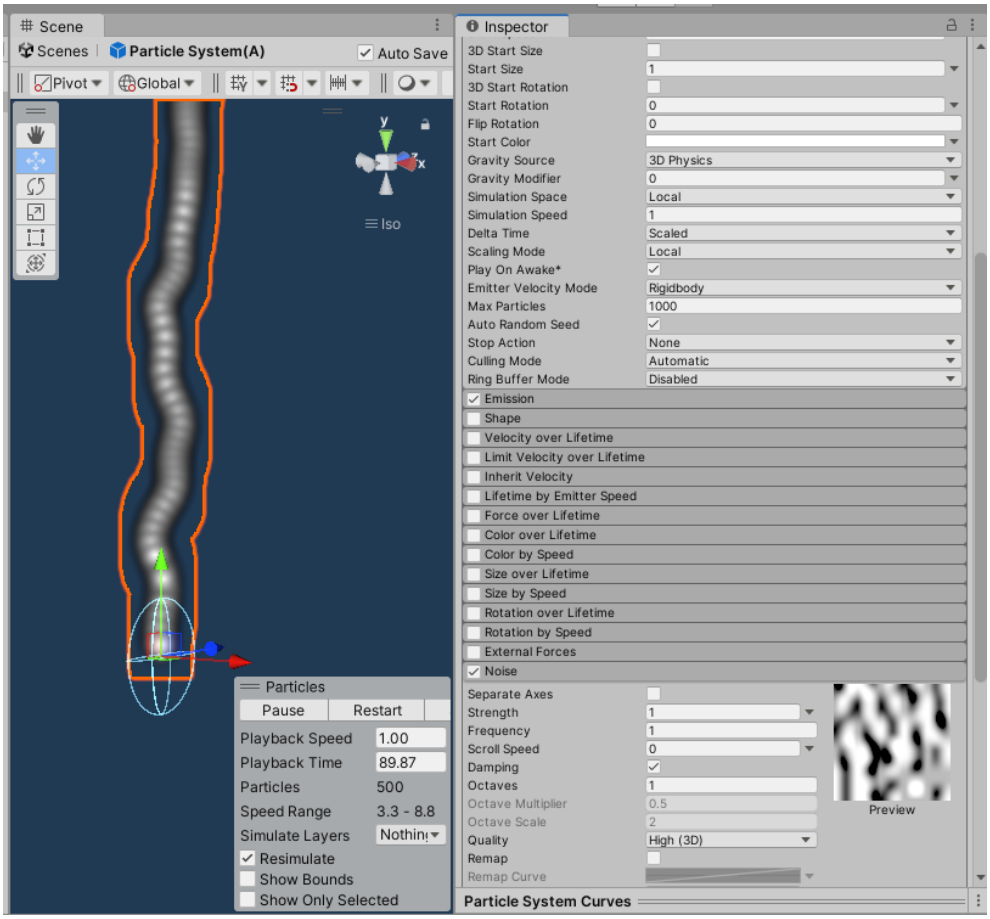
▼清▯ 15.2　从 5 秒▯重新播放

```
ParticleSystem.Simulate(5f, false, true);
```

[*1] https://docs.unity3d.com/ja/2022.3/ScriptReference/ParticleSystem.Simulate.html

如 清□ 15.2 所示指定 5 秒，即可□□□播放中的粒子从 5 秒□重新播放的机制。

15.3 存在□□的用法



▲□ 15.1　存在□□的粒子

在 Unity 默□□建的粒子上□外□用 Noise 模□。其他□整的参数是□了便于□察粒子，在 Emission 模□中将□射□整□一条直□。

▼清□ 15.3　从 20 秒□重新播放

```
ParticleSystem.Simulate(20f, false, false);
```

□于使用了此□ Noise 模□的粒子，如果像 清□ 15.3 □□在第 1 个参数中指定□大的秒数□行模□，□理□程中会□生□大的尖峰。

▼清□ 15.4　模□的开□□量

```
void OnClickA()
{
    // 若存在播放中的粒子□□□
    if (_activeParticle != null)
    {
        Destroy(_activeParticle);
        _activeParticle = null;
    }

    // 新建并播放
    _activeParticle = Instantiate(particlePrefab, Vector3.zero, Quaternion.identity);
    var ps = _activeParticle.GetComponent<ParticleSystem>();
    if (ps != null)
    {
        ps.Play();
    }
}

void OnClickB()
{
    if (_activeParticle == null) return;

    var ps = _activeParticle.GetComponent<ParticleSystem>();
    if (ps != null)
    {
        Profiler.BeginSample("再シミュレーション");
        // 重新模□
        ps.Simulate(20f, false, false);
        Profiler.EndSample();

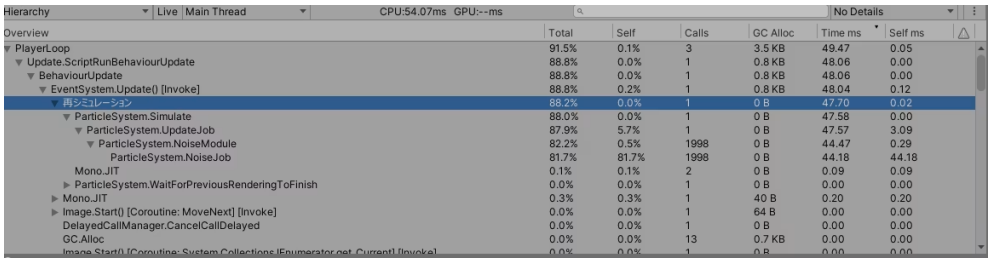
        ps.Play();
    }
}
```

使用 Profiler 确□会□生多大的开□。□代□在 ButtonA 中□建粒子，在 Button B 中□粒子□行模□。□了在 Unity 的 Profiler 中更容易确□□□□是否出在 Particle System.Simulate，使用 Profiler.BeginSample 和 Profiler.EndSample 将其包□，以便

□□。



▲□ 15.2　□生尖峰的情形



□：□理□□可以看到，在 ParticleSystem.Simulate □，ParticleSystem 的 Noise 模□□用了□ 2000 次 NoiseJob，由此□生了□ 50ms 的尖峰。降低 ParticleSystem 的 Max Particles 可以在一定程度上□少□□用次数。

15.4　□□

需要使用 ParticleSystem.Simulate □□从任意秒数开始□射粒子的机制□，

- 是否存在指定□□□□、从中途开始□射粒子的□景？
- 是否使用了 Noise 模□？

□确□以上□点；尤其是使用□用了 Noise 模□的粒子□，建□避免使用 ParticleSystem.Simulate。

第16章

通□ VAT □□骨骼□画开□

畳 悠樹 Tatami Yuuki

株式会社 CyberAgent SGE 核心技□本部

X @tatamin0207

16.1　前言

Unity 中常□的骨骼□画由 CPU 在每□□行骨骼更新等播放□算。因此，□画□象越多，CPU 开□越大，□而□致□率下降。本章介□ VAT，□是一种将□部分□行□□算整体替□□□□算以降低开□的方法。

16.2　什么是 VAT

VAT 是 Vertex Animation Texture 的□写，是一种□先将□画□生的□点形□信息烘焙到□理中，再由□点着色器□取并播放的方法。□理的每个像素□□“某个□点在某一□的位置”，随着播放□□移□□取位置，网格就会呈□出□□效果（□ 16.1）。此外，□确保精度，位置以□一化到 0 至 1 的□存□，并在□点着色器中□原□□□坐□。



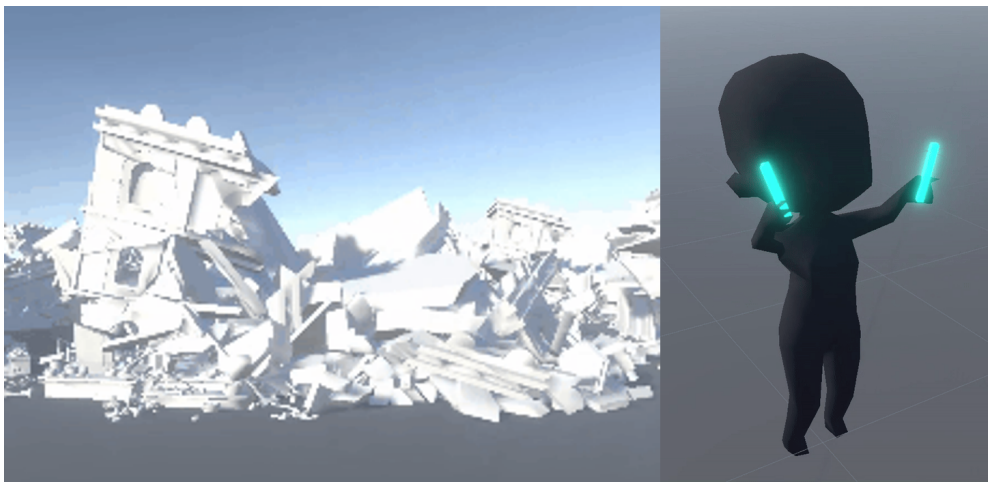
▲□ 16.1　VAT □理的原理。横□□□□点，□□□□□

使用 VAT 可以将骨骼更新等□行□播放□算整体替□□□□□算。□是 GPU Skinning 也无法□□的效率提升。

□一方面，VAT 也有以下缺点。

- 需要准□ VAT □用□源
- □理尺寸随□点数和□数增大
- 由于需要精确的□，与□理□□的兼容性差，□□上以非□□方式□理
- □作在烘焙到□理□即被固定，□然通□一些技巧可以□□一定程度的控制，但不适合精□控制

基于□些特性，VAT 在□大量□象播放固定□画的□景中尤其有效。例如建筑破坏效果、人群、背景装□物等不需要□独控制的表□都很适合（□ 16.2）。



▲□ 16.2　VAT □画（左：□体，右：□体）

16.3　性能□□

本□介□在真机上将 VAT 与其他□画方式□行性能□比的□果。

16.3.1　□量□境与□比条件

□量□境□ 表 16.1。

表 16.1: □量□境

□目	内容
真机	Pixel 8a（GPU: Mali-G715）
□形 API	Vulkan
Unity 版本	6000.3.2f1
□量 API	FrameTimingManager、ProfilerRecorder
模型	建筑倒塌□画（85,209 个□点，1,315 个碎片）
□画	300 □
□量方法	□象数量在 0 至 16 个之□□化，每个条件取 300 □ × 3 次的中位数

所比□的 3 种方式□ 表 16.2。

表 16.2: □比条件

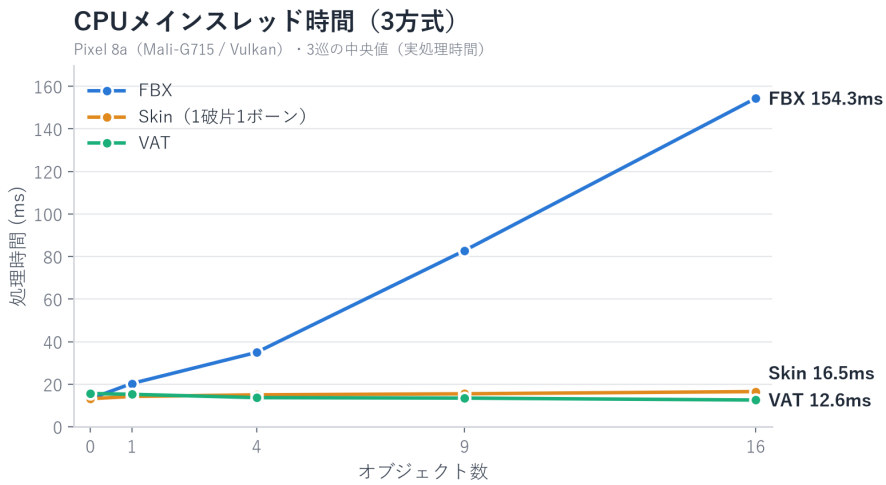
方式	□□	□用的□化
FBX 方式	将各碎片作□独立的 Mesh Renderer 放置，并通□ Animator 播放	SRP Batcher
Skin 方式	每个碎片□□ 1 根骨骼，合并□ 1 个 SkinnedMeshRenderer，并通□ Animator 播放	SRP Batcher
VAT 方式	将全部碎片的□□烘焙到□理中，作□ 1 个网格□制（Rigid-Body Dynamics 模式）	GPU Instancing

FBX 方式是将各碎片作□独立的 Mesh Renderer，并通□ Animator 播放的□□（碎片是通□ Transform □□的□体，不使用蒙皮）。Skin 方式□每个碎片分配 1 根骨骼，并将整体合并□ 1 个 Skinned Mesh。1,315 个碎片直接□□ 1,315 根骨骼，Renderer 只有 1 个。□画播放由 CPU 上的 Animator □□，□点形□由 GPU Skinning 完成。VAT 方式将全部碎片的□□烘焙到□理中，作□ 1 个网格□制。3 种方式的□示效果相同。

□制效率相关的□置是在□各方式□用□其有利的□□的状□下□行比□的。由于 SkinnedMeshRenderer 不支持 GPU Instancing，Skin 方式能使用的最佳□□是 SRP Batcher。此外，Skin 方式的骨骼化同□需要□用的□□管□，□一点与 VAT 条件相同。

16.3.2 CPU □理□□

CPU □理□□（主□程）的□量□果□ □ 16.3。□示 16 个□象□，FBX 方式增至 154.3ms，而 Skin 方式□ 16.5ms，VAT 方式□ 12.6ms，基本保持平□。



▲□ 16.3　各□象数量下的 CPU 主□程□理□□

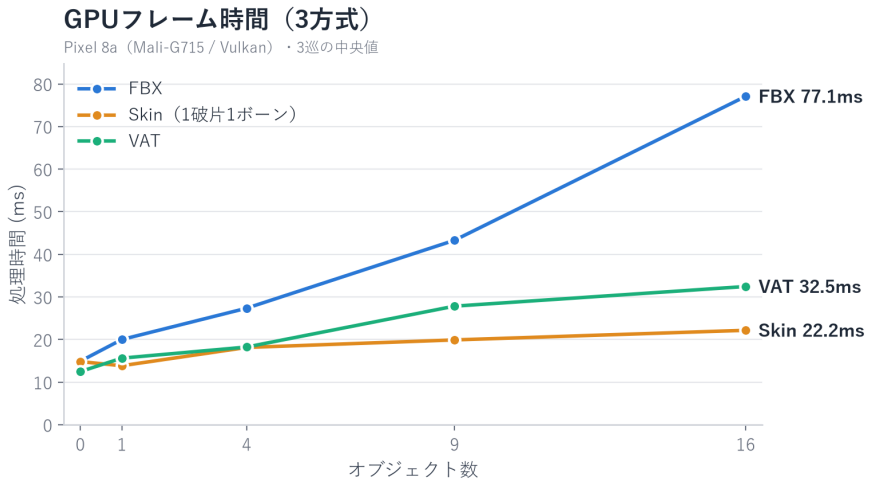
明□□ 表 16.3。FBX 方式与其他 2 种方式的差异，最主要的原因是□制□用的提交□理。FBX 方式下每个模型有 1,315 个碎片成□□制□象，而合并□ 1 个网格的□种方式几乎没有□部分□理。每个碎片的包□盒□算和剔除也是同□的情况。□一方面，Skin 方式与 VAT 方式的差异在于 Animator 的更新。Skin 方式的播放由 CPU □□，因此 16 个□象□仍剩余 4.5ms，而 VAT 方式的播放本身□移到了 GPU，□部分被完全消除。□一差异随□象数量增加而□大。

表 16.3: □示 16 个□象□的 CPU □理□□明□（ms）

□理	FBX 方式	Skin 方式	VAT 方式
主□程合□	154.30	16.53	12.56
其中□制□用提交（SRPBatcher.Flush）	76.17	0.03	0.02
其中 Animator 更新	5.30	4.54	0.02

16.3.3 GPU □理□□

GPU □理□□的□量□果□ □ 16.4。16 个□象□，FBX 方式□ 77.1ms，Skin 方式□ 22.2ms，最快，VAT 方式□ 32.5ms。



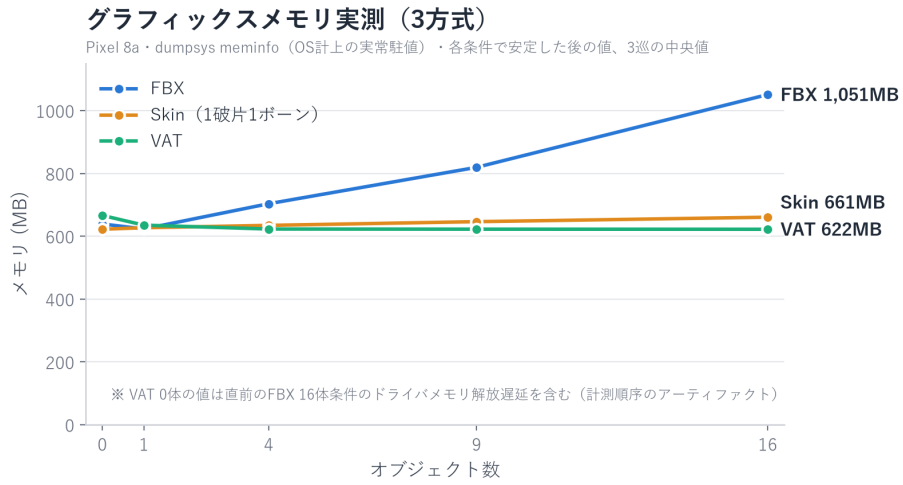
▲□ 16.4　各□象数量下的 GPU □理□□

FBX 方式与其他 2 种方式差异的主要原因是□制命令的拆分数量。FBX 方式下每个碎片在各□制通道中都□独□出□制命令，每□□到□ 21 万次，而 VAT 方式合并□□例化□制，□ 99 次。Skin 方式每个□象只有 1 个 Renderer，因此也□于同等水平。每次□制命令都有固定开□，因此拆分数量的差异会直接体□□□理□□的差异。

Skin 方式与 VAT 方式的差异源于形□□算的次数。GPU Skinning 每□只□算 1 次形□，阴影、深度等所有□制通道都复用其□果。相□地，VAT 在每个通道中都要在□点着色器里从□理重新□算形□。本次□境的□景配置是阴影□□有 4 □、通道数□多，因此与 Skin 方式的差距相□拉大（阴影□□□制：FBX 方式 44.3ms，Skin 方式 7.4ms，VAT 方式 16.0ms）。在□行□点□算的通道数□少的配置中，□一差距会□小。

16.3.4　GPU 内存分配量

GPU 内存的□量□果□ □ 16.5。数□是操作系□□□的□形□□□□□（GL mtrack）。从 0 个增加到 16 个□象□，FBX 方式增加 +411.8MB（每个□象□ 26MB），Skin 方式增加 +38.2MB（每个□象□ 2.4MB），VAT 方式基本□ ±0。



▲□ 16.5　各□象数量下的 GPU 内存分配量

□一差异源于每增加 1 个□象□增□内容的□□不同。FBX 方式需要在 GPU 内存中按碎片数量分配矩□等□制用数据。Skin 方式增加的只是每个□象 1 个用于写出蒙皮形□□果的□点□冲区。VAT 方式只是即□□算形□，不持有新的□冲区，因此不增加。□外，VAT 的□理在非□□状□下□□ 6.6MB，相□于增量之差来□足□小。

16.3.5　□□□果小□

□示 16 个□象□的□□□□□□ 表 16.4。

表 16.4: □示 16 个□象□的□□□□□

指□	FBX 方式	Skin 方式	VAT 方式
CPU 主□程（ms）	154.3	16.5	12.6
GPU □（ms）	77.1	22.2	32.5

表 16.4: □示 16 个□象□的□□□□□

指□	FBX 方式	Skin 方式	VAT 方式
□制命令数（次/□）	□ 210,000	与 VAT 方式同等水平	99
GPU 内存增量（0→16 个□象，MB）	+411.8	+38.2	基本 ±0

相□于 FBX 方式，Skin 方式和 VAT 方式在所有指□上都有□著□□。原因可□□□“能□作□ 1 个网格□理”。

Skin 方式与 VAT 方式的比□各有□劣：GPU □理□□上 Skin 方式有利，CPU 主□程和内存上 VAT 方式有利。VAT 方式的 GPU □理□□在很大程度上取决于通道配置中□点形□□理的占比。因此，在□行□点□算的通道数□少的配置中，或以降低 CPU 开□□目的□，VAT 是首□。

□外，本次比□的是□体破坏□一易于用骨骼表□的□象。□于布料或人体□□骨骼数量□多或只能做近似□理的模型，以及□象数量□多的情况，CPU 开□会□得□著，因此可以□□ VAT 的□□会更大。

16.4 □建 VAT □源

□建 VAT □源□，使用 Houdini 附□的 Labs Vertex Animation Textures ROP（以下称 VAT ROP）是一种便捷的方法[*1]。Houdini 是 SideFX 公司的 DCC 工具，擅□程序化生成。VAT ROP 提供了面向 Unity 的□□和□□的着色器，□入不□可以是 Houdini 内的模□□果，也可以是在其他 DCC 工具中□建的 FBX □画。

VAT ROP 有 表 16.5 所示的 4 种模式。

表 16.5: VAT ROP 的模式

模式	用途
Soft-Body Deformation	基本的□点形□。前提是拓扑不□生□化
Rigid-Body Dynamics	□□□体破坏□化。由于以碎片□□位□□□□，可以将□理尺寸控制得□小
Dynamic Remeshing	支持液体、撕裂的布料等拓扑会□化的□象
Particle Sprites	用于粒子模□

[*1] □是 SideFX Labs 包中包含的工具。本章的□□使用的是 3.0 版本。

操作步□只需：在 Input Geometry 中指定要□□□ VAT 的□画□点，在 Mode 中指定要使用的模式，在 Target Engine 中指定 Unity，然后点□ Render All 按□。表 16.6 所示的□源会被一并□出。

表 16.6: 播放 VAT 所需的□源

□源	作用
位置□理	各□点在各□的位置（□一化到 0 至 1）
旋□□理	形□后面的朝向。用于重建法□和切□以保持明暗
□点□色□理	Houdini 端属性的烘焙目□。可用于提升插□□量等的自由通道
FBX	在 UV 中嵌入了表示□理引用位置的□□□号的网格
材□	已□置□数、FPS、包□盒等□助数据
着色器	各模式□□的播放□算主体（需□行安装）

着色器不包含在□出文件中，因□□算□理与□画无关，是固定的。需要通□ Unity 的 Package Manager □行安装 SideFX Vertex Animation Textures 包（存放位置可在 VAT ROP 的 Real-Time Shaders □□卡中确□）。□包除支持 4 种模式的 Shader Graph 外，□附□文档和用于□入□置的 Preset。

16.5 引入□的注意事□

VAT 在精度不足□□画会□坏，因此需要□□理指定合适的格式。同□，FBX 一□也有推荐的□入□置。□按照包中附□的“VAT Import Settings Guide”，□用包括禁用□理□□在内的□用□入□置。

此外，官方的 Shader Graph 只是定位□模板，□□使用□需要自定□。不是直接使用模板，而是先复制到□目中再使用。但是，复制得到的 Shader Graph 仍会引用包中的 Sub Shader Graph，因此需要注意将引用替□□复制出的版本。

□外，□出文件的□入□置，以及□材□分配□理和着色器等引用□置需要手□完成，容易因□漏□置而出□□□。在批量生□的□目中，可以考□使用 Scripted Importer 等将□入□置和引用解析自□化。关于□方面的改□，□□□上□到 CEDiL 的□料[*2]。

[*2] https://cedil.cesa.or.jp/cedil_sessions/view/3263

16.6 □□

VAT 是将□画烘焙到□理中并在 GPU 上播放的方法。其特点是可以大幅□少□制□用，相□于□每个碎片持有 Renderer 的□□，与 Skin 方式一□大幅提高了效率。□者各有□劣：GPU □理□□上 Skin 方式有利，CPU □理□□和内存上 VAT 方式有利。在□行□点□算的通道数□少的配置中，或以降低 CPU 开□□目的□，VAT 是首□。□然存在需要准□□用□源、□作被固定等限制，但在□大量□象播放固定□画的□景中，它是一个□有力的□□。

第17章

在 Unity 中活用 iOS 准的性能量功能

山崎 雅大 Yamazaki Masahiro

株式会社 Applibot

mao-test-h　X @TEST_H_

17.1　前言

Unity 具以 Unity Profiler 代表的量功能，而面向 iOS 的用可以同利用操作系准的性能量功能。

Apple 提供的些功能的特点是不依 Unity 的 Development Build 志即可使用。例如，可以在尚未集成量理的段，或在禁用功能、接近布版本的建配置下粗略确开，也可以只从人的上接收跟踪果。在大致掌握开、将瓶候小到一定范之后，再切到用 Development Build 的建，使用 Unity Profiler 或 Instruments 行，如此分工使用即可。

本文从 iOS 准提供的功能中介以下 2 ，并解 macOS 27 / iOS 27 新增的功能。

- Metal Performance HUD
- Performance Trace

此外，我已将些功能封装可在 Unity 中使用的 OSS 兼示例 “MetalPerfKit-Unity”[*1]并公开，也一并参。

[*1] MetalPerfKit-Unity `https://github.com/mao-test-h/MetalPerfKit-Unity`

※ 下文□一使用 iOS □一表述，除非特□□明，均包含 iPadOS。

17.1.1 □□□境

本文在以下□境中□行了□作□□。

- Unity 6000.0.81f1
- Xcode 26.6 / 27.0 beta
- iOS 26.6 / 27.0 beta
- macOS 27.0 beta

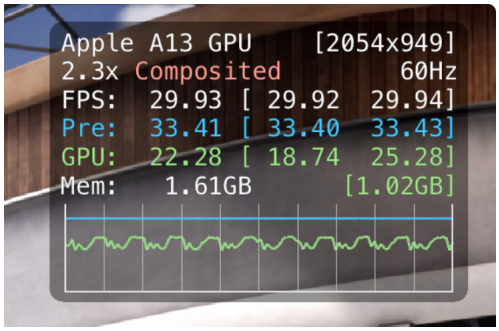
※ 撰写□，iOS 27、macOS 27、Xcode 27 以及后述的 `StateReporter` API 均□于 Beta □段。本文基于 2026 年 8 月的信息，在正式□布前，API、命令、□置界面和行□等都可能□生□化。

17.2 Metal Performance HUD

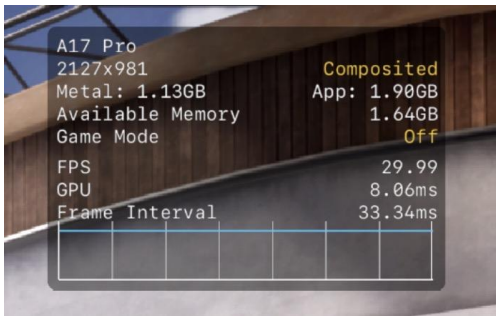
Metal Performance HUD 是一款可将使用 Metal 的□用的性能信息□□□加□示在屏幕上，或以日志形式□出的工具。 Apple 的文档 [*2] 将其介□□用于快速掌握当前状□，并找出需要用 Xcode 或 Instruments □□捕□的范□的功能。

□功能本身自 Metal 4 之前（iOS 16 □代）就已存在，并随 2025 年 WWDC □布的 Metal 4（iOS 26 □代）得到大幅增□。 Metal 4 之前与之后的□示差异□ □ 17.1 和 □ 17.2。

[*2] Monitoring your Metal app’s graphics performance `https://developer.apple.com/documentation/xcode/monitoring-your-metal-apps-graphics-performance`



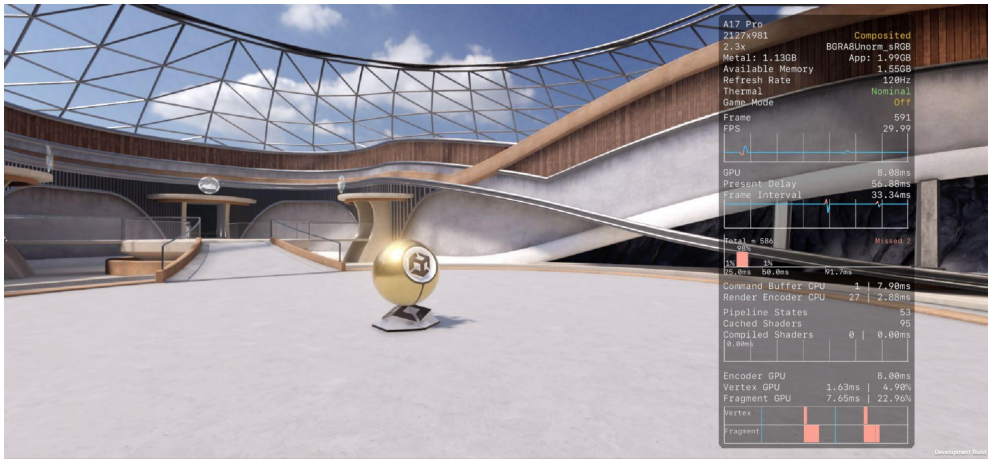
▲ 17.1 Metal 4 之前的 Metal Performance HUD



▲ 17.2 Metal 4 之后的 Metal Performance HUD

Metal 4 的主要增包括量目的充、示位置、不透明度和放比例的自定，以及可通 API 控制的目增多。

示主要指的示例 17.3。



▲图 17.3　图示主要指图图的示例

各指图的图图图明图参图官方文档“Understanding the Metal Performance HUD metrics”[*3]。

此外，图者此前也图布图关于 Metal Performance HUD 的解图文章 [*4] 和技图同人志“UniTips Vol.12”[*5]。其中图解了基本用法以及面向 Unity 的活用方法，图情图一并参考。

后图章图将基于 Metal 4（iOS 26 图代）的内容图行图解。

17.2.1　打开 HUD

图示 HUD 的方法有多种，图里图解“从图置图用打开”和“通图 API 控制图示”图 2 种。

从图置图用打开

可在 iOS 图图的图置图用中图用。步图如下。

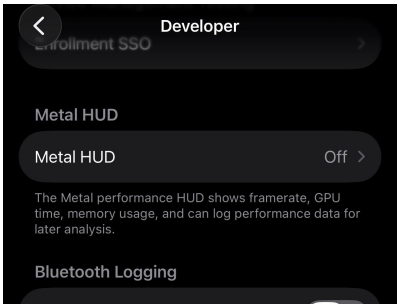
[*3] Understanding the Metal Performance HUD metrics https://developer.apple.com/documentation/xcode/understanding-metal-performance-hud-metrics

[*4] 【Unity】在移图端图用上图量 GPU 开图 https://qiita.com/mao_/items/3b531d17cacb86036b7f

[*5] UniTips Vol.12（在“第 2 章：从 Metal 4 起增图的 Metal Performance HUD 的 Unity 活用方法”中图解）https://techbookfest.org/product/8mHpXz8LJCKAsLxENJKwaz?productVariantID=ewK5GtNTjkz4k8heM4BrxU

※ 前提是已示 Developer 菜。此外，菜名称可能因 OS 版本而异。

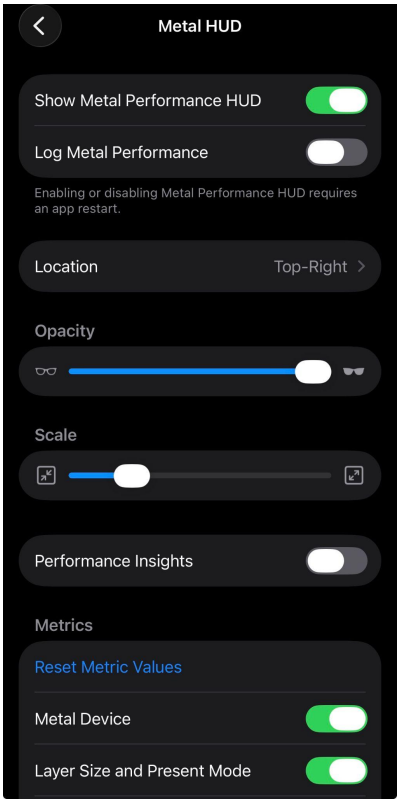
1. 打开 iOS 的“置”用
2. 打开 Developer 菜，再打开其中的 Metal HUD（参 17.4）



▲ 17.4　从 Developer 菜打开 Metal HUD

打开 Metal HUD 后会示 17.5 界面，用其中的 Show Metal Performance HUD 并重新目用即可示。

在此界面中，可以切示的指，以及更改不透明度、放比例和示位置。 此外，用同一部分中的 Log Metal Performance 后，可按固定隔以日志形式出性能信息。情后文“关于日志功能”。



▲图 17.5　Metal HUD 的图置界面

通图 API 控制图示

若要在图用图行图程中图图控制 HUD 的图示，图将图置图图图 `CAMetalLayer` 图的 `developerHUDProperties` 属性。清图 17.1 图用了 HUD 和日志图出，并将图示位置指定图左上角。

▼清图 17.1　通图 developerHUDProperties 控制 HUD 的图示

```
myMetalLayer.developerHUDProperties = [
    "mode": "default",      // HUD 的开关/
    "logging": "default",   // 日志功能的开关/
    "positionX": 0,         // 图示位置（以像素图图位的 X 坐图）
    "positionY": 0,         // 图示位置（以像素图图位的 Y 坐图）
]
```

藏 HUD ，将 mode 指定 disabled。以往通属性只能控制 HUD 和日志功能的开/关，无法整位置等，而从 Metal 4 起，可以通 positionX / positionY 以像素位指定位置。

此外，可以向 Dictionary 入 HUD 所支持的境量的。例如入 MTL_HUD_OPACITY 和 MTL_HUD_SCALE，即可在 0.0～1.0 的范内整不透明度和放比例（清 17.2）。

▼清 17.2 通境量整不透明度和放比例

```
myMetalLayer.developerHUDProperties = [
   ...,
   "MTL_HUD_OPACITY": opacity,      // 不透明度 ～(0.01.0)
   "MTL_HUD_SCALE": scale,          // 放比例 ～(0.01.0)
]
```

其他有多可置的目，情参官方文档“Customizing the Metal Performance HUD”[*6]。

从 Unity 用，需要通 iOS 原生插件取用于渲染的 CAMetalLayer，再从 C# 端通 P/Invoke 用。CAMetalLayer 可由 UnityFramework 按 清 17.3 的方式取。

▼清 17.3 由 UnityFramework 取 CAMetalLayer

```
private func getMetalLayer() -> CAMetalLayer? {
   guard let instance = UnityFramework.getInstance(),
         let rootView = instance.appController().rootView,
         let metalLayer = rootView.layer as? CAMetalLayer else {
       return nil
   }
   return metalLayer
}
```

MetalPerfKit-Unity 将些作 PerformanceHUDSwitcher 公开，从 C# 中可按 清 17.4 的方式用。外，原生 API 以像素位指定位置，而 PerformanceHUDSwitcher 了不依分辨率，以 0.0～1.0 的相位置接收坐，并在内部根据渲染区

[*6] Customizing the Metal Performance HUD https://developer.apple.com/documentation/xcode/customizing-metal-performance-hud

域的大小□□□像素坐□。

▼清□ 17.4　PerformanceHUDSwitcher 的□用示例

```
// 将 HUD □示在屏幕右上角 (x=1.0, y=0.0)
// （坐□以 ～0.01.0 的相□位置指定）
PerformanceHUDSwitcher.SetPerformanceHUDVisible(true, 1.0f, 0.0f);
```

完整的□□示例□参□ MetalPerfKit-Unity 的源代□。

17.2.2　关于日志功能

□用前述的 `Log Metal Performance` 或 API 的 `logging` 后，□用□行期□会每隔 1 秒以日志形式□出性能信息。日志以 `metal-HUD:` 开□的清□ 17.5 格式□出，可在 Xcode 的控制台或 macOS 的 Console.app 中□看。

▼清□ 17.5　metal-HUD 日志的□出格式

```
metal-HUD: <first-frame-number-integer>,<graphics-memory-usage-float>,<process-memory-
    usage-float>,<first-frame-present-interval-float>,<first-frame-gpu-time-float>,...<
    last-frame-present-interval-float>,<last-frame-gpu-time-float>
```

从开□起依次□□“开始采□的□□号”“□形内存使用量”“□程整体的内存使用量”，此后重复□□各采□□的 Frame Present Interval 与 GPU Time 的配□。□□的□出示例□ □ 17.6。



▲□ 17.6　Metal Performance HUD 的日志□出示例

将其作□逗号分隔的数据□行□□，即可在事后分析□□隔的紊乱、GPU Time 以及内

存使用量的化。 此外，只要些日志仍保留在 Unified Logging 中，也可以使用 OSLogStore API “取去 N 秒的日志并保存到文件”之的理。 MetalPerfKit-Unity 以 PerformanceLogger 的形式提供了一功能（清 17.6）。

▼清 17.6　通 PerformanceLogger 取并保存日志

```
// 取去 60 秒的日志并保存到文件
PerformanceLogger.FetchPerformanceLogs(60, FileUtility.GenerateFetchLoggingFilePath());
```

17.3 Performance Trace

Performance Trace 是 iOS 26 新增的功能，无需接 Mac，凭 iOS 本身即可收集性能相关数据。 收集到的跟踪文件可到 Mac，并用 Instruments 或后述的 metalperftrace 命令行分析。

由于只需本身即可完成，因此可以游玩等接 Mac 以复的情况，也可以人等他人共享在其上收集的跟踪文件，再在本地行分析。

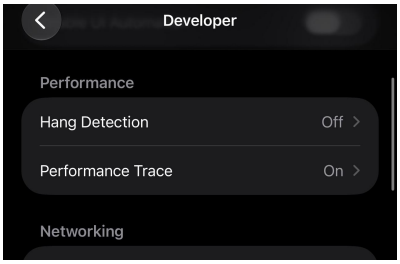
本章将重点介其中在 WWDC 2026 上随功能增一并解的 Lookback Collection。

17.3.1 使用方法

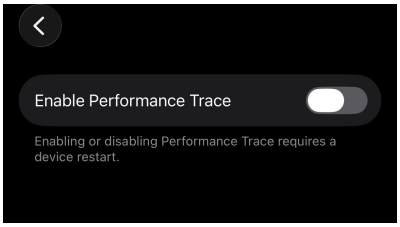
Performance Trace 需要从 Developer 菜中用。 步如下。

1. 打开 iOS 的“置”用
2. 打开 Developer 菜，再打开其中的 Performance Trace
3. 用 Enable Performance Trace（切用/禁用需要重新）
4. 任意的 Tracing Mode（后文）

Developer 菜中的 Performance Trace 及其置界面分 和 17.7 和 17.8。

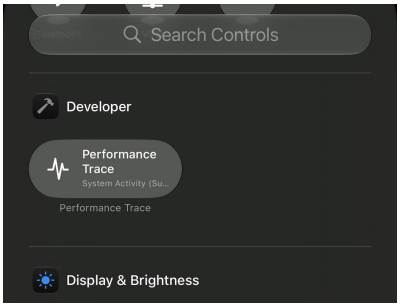


▲图 17.7　Developer 菜单中的 Performance Trace

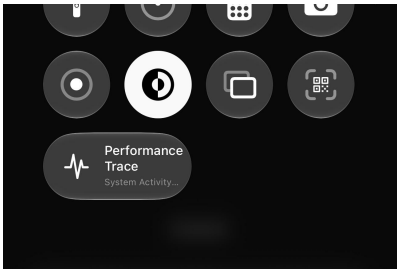


▲图 17.8　Enable Performance Trace 的设置界面

使用完成后，为了进行跟踪的开始与停止以及跟踪数据的收集等操作，需要将 `Performance Trace` 添加到控制中心。打开控制中心，即可从 `Developer` 部分添加。添加步骤和添加后的状态如图 17.9 和图 17.10。



▲图 17.9　将 Performance Trace 添加到控制中心



▲ 17.10　已添加 Performance Trace 控件的状

17.3.2　关于 Tracing Mode

`Tracing Mode` 有 表 17.1 所示的 4 种（System Activity 分 Summary / Detailed 2 种），也可以按控制中心的 `Performance Trace` 控件行切。

表 17.1: Tracing Mode 的种

模式	概要
Power Profiler	功耗的影
System Activity (Summary / Detailed)	系的行状况
Lookback Collection	回溯收集系持的数据

Power Profiler 和 System Activity 通点按控制中心的 `Performance Trace` 控件开始，再次点按停止。 而 Lookback Collection 无需操作，点按后会从刻起回溯收集数据。情后文。

收集到的跟踪文件会列在 Performance Trace 界面的 `Available Trace Files` 中，可通分享按由 AirDrop 等 Mac 或其他开者。 解收到的档后，即可得到可在 Instruments 中打开的 `.atrc` 文件。 外，当存空不足，跟踪文件会被自除，也可以通滑操作手除。

各模式的充明如下。

Power Profiler

Power Profiler 是用于量用 CPU / GPU / 示屏 / 网等各子系的使用 功耗来的影（Power Impact）的模式。 本文省略内容，官方文档 “Measuring

your app’s power use with Power Profiler”[*7] 和 WWDC 2025 的会□ “Profile and optimize power usage in your app”[*8] 中有□□□解。

System Activity

System Activity 是提供 “Summary” 和 “Detailed” 2 种的跟踪模式。据□者确□，截至撰写□（2026 年 8 月），在 Apple 的文档和 WWDC 会□中，没有找到关于 System Activity □□□格以及 Summary 与 Detailed 区□的□明。因此，本文不深入各模式的□□，只介□在□□跟踪中确□到的信息。

作□参考，在搭□ iOS 27 的□□上□□ System Activity (Summary) 的跟踪，并用后述的 `metalperftrace` 命令分析的□果□ 清□ 17.7。

▼清□ 17.7　System Activity (Summary) 跟踪的分析□果

```
$ metalperftrace overview system_activity_summary_ios27.atrc
[MetalPerfKit-Unity pid:1974]
   Mem: 799.0 MiB (799.0 Peak, 616.8 Metal, 2577.0 Headroom)
   Total CPU Time: 12.679s (20.47% Sys, 79.53% User)
   Instructions: 43.33B (60.88% P, 39.12% E)
   Cycles: 21.04B (38.97% P, 61.03% E)
   Disk Reads / Writes: 9.13 / 0.08 MiB (Logical Write 0.07)
   [States]
     com.DefaultCompany.MetalPerfKit-Unity.graphics: 3 transitions, last known state:
    URP-HighFidelity
     com.DefaultCompany.MetalPerfKit-Unity.screen: 11 transitions, last known state:
    Title
 UnityView (UnityDefaultViewController) (1290x2796)  Interval 30.037s  Active 30.037s
   44.5 FPS  1202 Frames  1202 Detached  11 Skipped
   Frame Time           avg:  22.47ms  min:  16.67  max: 420.93  stddev:  30.68  p95:
    50.01  p99: 133.36
   CPU Begin-to-Present avg:  17.24ms  min:   1.14  max: 794.63  stddev:  31.76  p95:
    31.28  p99: 111.73
   On-GPU Time          avg:  15.50ms  min:   1.81  max: 424.87  stddev:  37.24  p95:
    51.17  p99: 134.48
   Next Drawable Wait   avg:  14.53ms  min:   0.00  max: 401.43  stddev:  32.11
```

在□跟踪中，除了各□程的内存使用量、CPU □□和磁□ I/O 之外，□确□到了以 C

[*7] Measuring your app’s power use with Power Profiler https://developer.apple.com/documentation/xcode/measuring-your-app-s-power-use-with-power-profiler

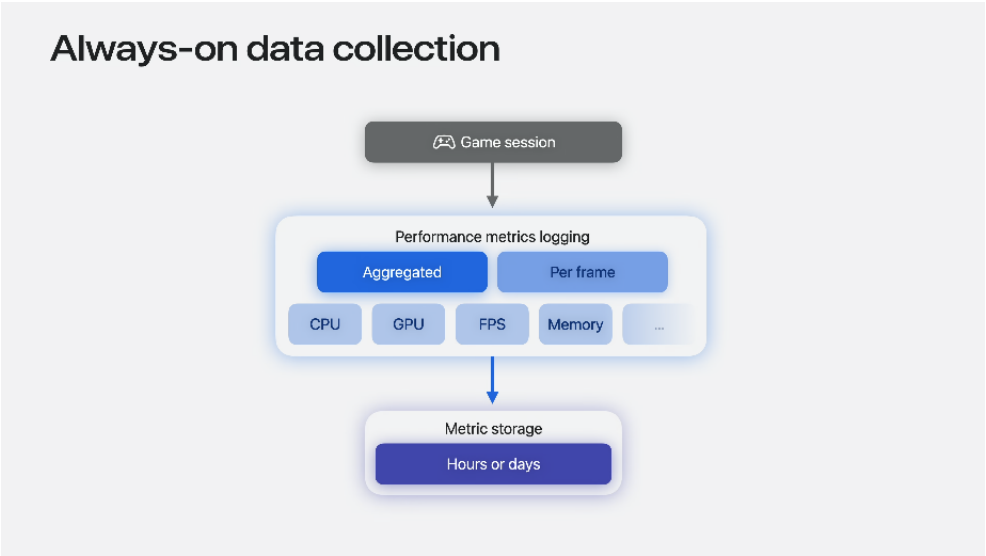
[*8] WWDC 2025 - Profile and optimize power usage in your app https://developer.apple.com/videos/play/wwdc2025/226/

AMetalLayer 位的 FPS、Frame Time、CPU Begin-to-Present 和 On-GPU Time 等信息。 外，[States] 中了用的状迁移，但是由下一章解的 StateReporter API 生的， 后文。

Lookback Collection

Lookback Collection 是事后回溯收集系持的性能数据的模式。据者确，Lookback Collection 在 iOS 26 就已作 Tracing Mode 的存在，但 WWDC 2025 和官方文档中均未解。此后在 WWDC 2026 上功能得到增，并布了解其内容的会 “Find and fix performance issues in your Metal games”[9]。

据演介，游行期，系会持性能指（CPU、GPU、FPS、内存等），并将些数据保存数日。 Lookback Collection 利用了一机制，因此即使是人的游玩会，也可以回溯取数据。 机制的概要 17.11。



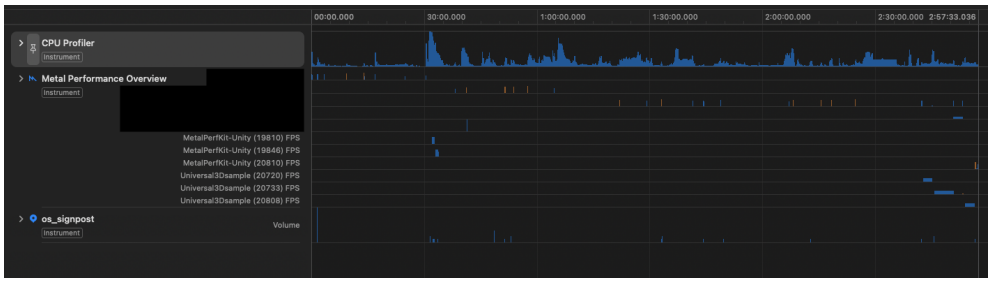
▲ 17.11 系性能指的持（来自 WWDC 2026 会）

使用方法是，像平常一行游玩后，点按控制中心的 Performance Trace 控件即可。 从点按的刻起，会回溯所置的开始收集并理跟踪，完成后会收到通知

[9] WWDC 2026 - Find and fix performance issues in your Metal games https://developer.apple.com/videos/play/wwdc2026/388/

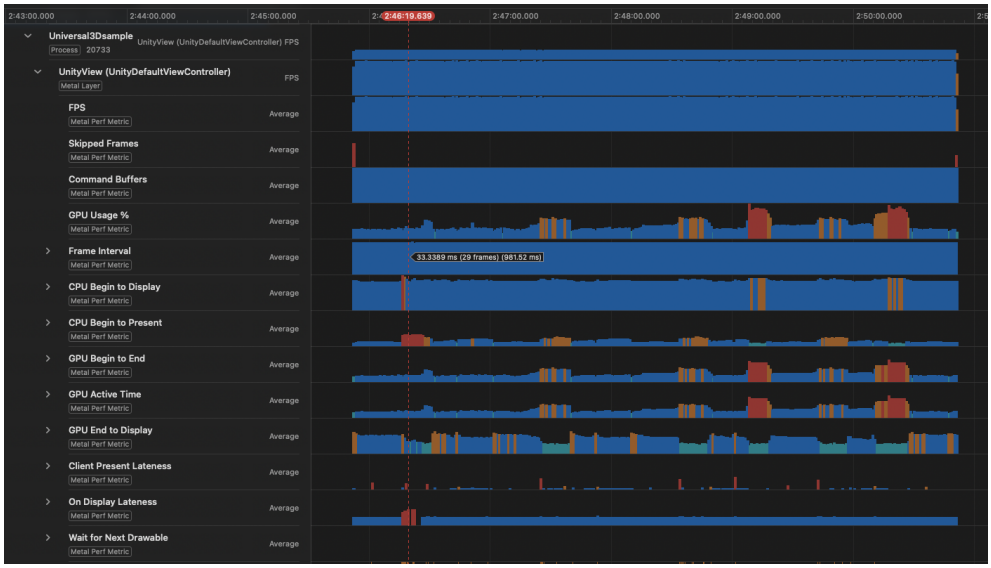
并添加到 `Available Trace Files`。

□ 17.12 展示了在搭□ iOS 26 的□□上□□收集□ 3 小□的 Lookback Collection 跟踪，并在 Instruments 中打开的示例。回溯期□□□□的多个□用（□程）会以列表□示，可以□□各自的 FPS 等□化。



▲□ 17.12　在 Instruments 中打开 Lookback Collection 跟踪的示例

□一步展开□程，如 □ 17.13 所示，可以按 `CAMetalLayer` □□位□看 FPS、GPU Usage %、Frame Interval、CPU / GPU □□等□指□的□化。



▲□ 17.13　以 CAMetalLayer □□位的□指□□化

※ 在者手的 iOS 27 境中，示 Error processing Performance Trace，无法正常工作。关于一，即使在 iOS / iPadOS 27 正式布，行明的 Control Center -> Known Issues 中仍了已知 (184729556)：“收集 Power Profiler 或 Lookback 的跟踪数据，可能不会生成跟踪文件”。[*10]

17.4 macOS 27 / iOS 27 新增的功能

如上一章所述，游行期系会持并保存 Metal 性能指。 WWDC 2026 以一机制核心布了新功能：跟踪附加用特有信息的 StateReporter API，以及可从 CLI 分析跟踪的 metalperftrace 命令。

17.4.1 StateReporter API

即使能从跟踪数据中确定率下降的刻，凭 Metal 性能指也无法得知当游在做什么。 是因“正在打开游的个景”“象数量大是多少”“形置是什么”等用特有的信息，并不会自包含在 Metal 性能指中。

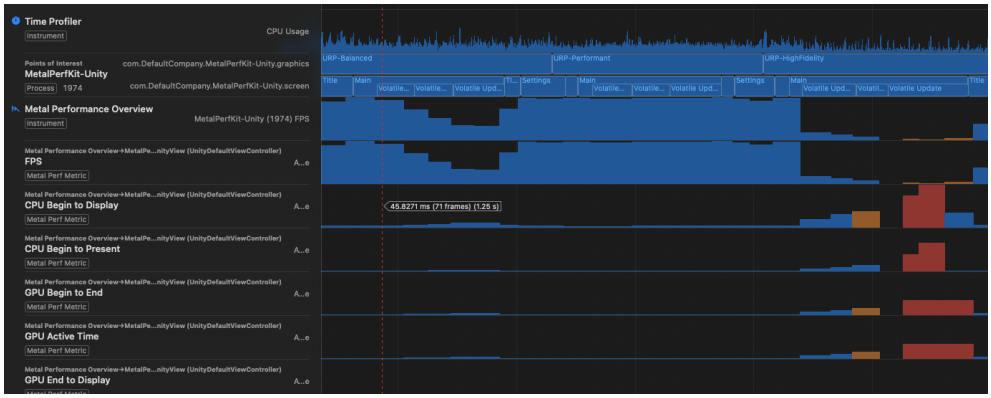
使用 iOS 27 新增的 StateReporting 框架 [*11] 的 StateReporter API [*12]，可以向系告用的状，并跟踪数据附加用特有的上下文。

所告的状在 metalperftrace 中用于按状，在 Instruments 中用于 Points of Interest 的示。 在 Instruments 中打开上一章所示 System Activity 跟踪的示例 17.14。所告的状示 Points of Interest 道。

[*10] iOS & iPadOS 27 Release Notes https://developer.apple.com/documentation/ios-ipados-release-notes/ios-ipados-27-release-notes

[*11] Getting started with StateReporting https://developer.apple.com/documentation/StateReporting/getting-started-with-statereporting

[*12] StateReporter https://developer.apple.com/documentation/statereporting/statereporter



▲图 17.14　所⊠告的状⊠⊠示⊠ Points of Interest 的示例

`StateReporter` 由 表 17.2 所示的 4 个要素⊠成。

表 17.2: ⊠成 StateReporter 的 4 个要素

要素	概要	示例
Domain	作⊠⊠告⊠象的功能⊠域。⊠反向 DNS 格式的字符串，每个 Domain 同⊠⊠于活⊠状⊠的状⊠只有 1 个	`com.mygame.level`
Label	表示当前状⊠的短字符串	`Level 1`
Stable metadata	在状⊠存⊠期⊠保持不⊠的元数据。与 Label ⊠合以唯一⊠⊠状⊠	`{id: 1001}`
Volatile metadata	状⊠中可能⊠化的易⊠元数据。更新⊠不会⊠生状⊠迁移	`{health: 20}`

Domain 可以定⊠多个，例如将“关卡⊠度”“⊠形⊠置”“网⊠状⊠”分⊠作⊠独立的 Domain ⊠告，并⊠合起来分析。

在 Swift 中按 清⊠ 17.8 的方式使用。

▼清⊠ 17.8　在 Swift 中使用 StateReporter 的示例

```
import StateReporting
// ⊠取与 Domain ⊠⊠的 Reporter
let reporter = StateReporter.reporter(for: "com.mygame.gameplay")
```

```
// 告状迁移
reporter.reportTransition(to: "Playing")
// 束状入 nil
reporter.reportTransition(to: nil)
```

在 Unity 中使用

StateReporting 是面向 Swift / Objective-C 的 API，因此在 Unity 中使用需要原生插件。 MetalPerfKit-Unity 以功能的形式公开了其行封装的 StateReporter ，可在 C# 中按 清 17.9 的方式使用。

▼清 17.9　在 C# 中使用 StateReporter 的示例

```
using MetalPerfKit;
// 取与 Domain 的 Reporter
var reporter = StateReporter.ReporterForDomain($"{Application.identifier}.screen");
// 告状迁移（附易元数据）
reporter.ReportTransitionToStateLabel(
    "Main",
    volatileMetadata: new Dictionary<string, StateReporterMetadataValue>
    {
        ["CubeCount"] = cubeCount,
    });
// 保持状不，更新易元数据
reporter.ReportVolatileMetadataUpdate(
    new Dictionary<string, StateReporterMetadataValue>
    {
        ["CubeCount"] = cubeCount,
    });
```

此外， 的 Assets/_Example/2_MetalPerformanceMetrics 中提供了一个示例，将界面迁移（Title / Main / Settings）和形量（URP 的 3 个等）分作独立的 Domain 行告。 上一章所示 System Activity 跟踪中的 [States] 就是用示例的。

17.4.2　metalperftrace

metalperftrace 是 macOS 27 新增的 CLI 工具，除了分析跟踪文件之外，可以收集面向 macOS 的用的性能指。 收集功能的象是在 macOS 上行的用，因此本文不予，但分析在 iOS 上收集的跟踪文件可以使用工具。

关于跟踪文件的分析

分析跟踪文件使用 `overview` 子命令（清□ 17.10）。

▼清□ 17.10　通□ overview 子命令分析跟踪文件

```
# 分析已收集的跟踪文件
metalperftrace overview MyGameTrace.atrc
```

□出内容如上一章所介□。若再□入 `--include-state-transitions`，□可以□看用 `StateReporter` □□的状□迁移的□□信息。清□ 17.11 是分析前述示例□用跟踪的例子，□出了□更□各□形□量的□史，以及在□状□下的停留□□和占比。

▼清□ 17.11　包含状□迁移□□信息的分析□果

```
$ metalperftrace overview --include-state-transitions system_activity_summary_ios27.
    atrc
[MetalPerfKit-Unity pid:1974]
   ...
   [States]
     com.DefaultCompany.MetalPerfKit-Unity.graphics
       URP-Balanced  (32.75%, 9.904s)
         (2026-08-16T14:22:33.025Z → 2026-08-16T14:22:42.930Z)
       URP-Performant  (29.43%, 8.900s)
         ...
       URP-HighFidelity  (37.82%, 11.437s)
         ...
     com.DefaultCompany.MetalPerfKit-Unity.screen
       Title  (5.07%, 1.535s)
         ...
       Main  (21.09%, 6.379s)
         ...
       Title  (2.22%, 0.671s)
         ...
       Settings  (6.28%, 1.899s)
         ...
```

※ □出□□□并整理后的内容。□□上，相同的状□也会按□□□序重复□出，并分□□有开始和□束的□□戳。

此外，□入 `--aggregate` 可按状□□□指□。指定 `--domain com.mygame.graphics` 可限定到特定的 Domain，再同□使用 `--state-label "High"` □可限定到特定的状

。只需一条命令，就能确“在形置 High 平均 FPS 下降”用状与性能之的相关性。

此外，入 `--json` 可以以 JSON 格式取些出。演中也提到“化出可用于通脚本行回”，以及由 AI 智能体自分性能”。Apple 的“Metal Developer Tools”面 [*13] 也以“Hello, agentic debugging”，介了与智能体合的工作流。

[*13] Metal Developer Tools `https://developer.apple.com/metal/tools/`

第18章

按□□切□□制□置

佐久間 尚輝 NAOKI SAKUMA

株式会社 CyberAgent SGE 核心技□本部

面向智能手机的游□需要考□在各种□□上□行。如果□所有□□使用□一的□制□置，在低配□□上可能会□致崩□、掉□和□□。反之，在高配□□上，□有可能充分利用□□性能□得更高的画□。因此，本□介□按□□切□□合适□制□置的机制。

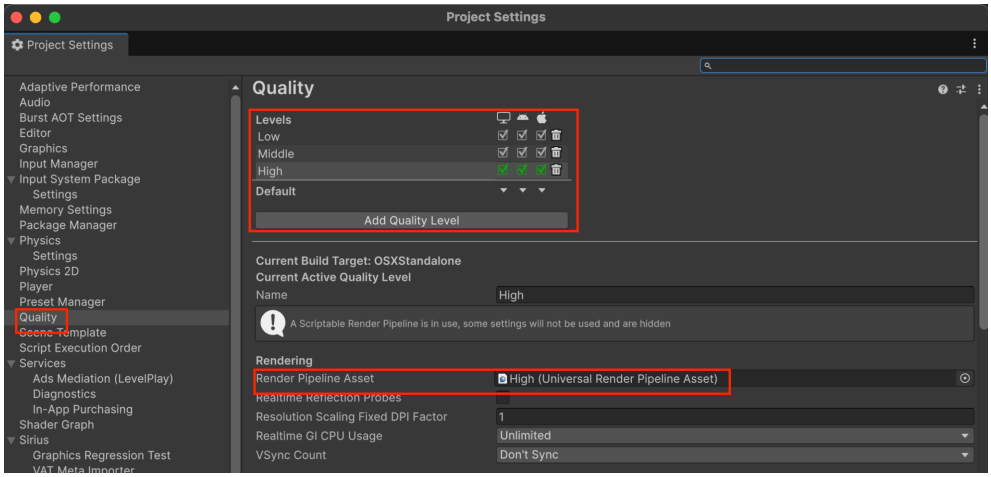
18.1 切□方法

游□中常□的方法有□种：低、中、高□□□□形式，以及由用□直接□整数□的形式。

18.1.1 □□形式

□是以□□□□位□行切□的方法。无□是□用□□□□□，□是根据□□自□判定，都可以使用□种方法。

在 ProjectSettings 的 Quality 中点□ Add Quality Level，按□□数量添加。然后□每个□□挂□□□的 RenderPipelineAsset。也可以根据需要□置 Quality 的其他□目（Rendering、Textures 等）。



在代□中□用 QualitySettings.SetQualityLevel，即可切□到任意 QualityLevel。

18.1.2 直接□整数□的形式

需要□用□□整参数□，使用□种方法。通□如下代□可以直接□整参数。

```
var urpAsset = (UniversalRenderPipelineAsset)GraphicsSettings.currentRenderPipeline;
// 更改分辨率
urpAsset.renderScale = value;
// 更改阴影□□数量
urpAsset.shadowCascadeCount = value;
// 此外□可以□整阴影分辨率、附加光源数量等
```

18.2 按□□切□

有□需要根据□□推荐□□或自□□置□□。此□ SystemInfo □型很有用。

尤其是 deviceModel、graphicsDeviceName 和 systemMemorySize，可以□得便于□□□□的□。以下是返回□的示例。

表 18.1: 各□□的 SystemInfo 返回□示例

机型	deviceModel	graphicsDeviceName	systemMemorySize
iPhone 13	iPhone14,5	Apple A15 GPU	□ 3800

表 18.1: 各□□的 SystemInfo 返回□示例

机型	deviceModel	graphicsDeviceName	systemMemorySize
iPad Pro 11-inch（第 3 代）	iPad13,4	Apple M1 GPU	□ 7800
Galaxy S23	samsung SM-S911B	Adreno (TM) 740	□ 7500
Pixel 8	Google Pixel 8	Mali-G715	□ 7400

iOS 的 deviceModel 和 graphicsDeviceName 命名□□前后一致，而且基本上在同一系列内，A 或 M 之后的数字越大，性能越高。因此，将□者□合可以在一定程度上区分□□档次。

Android 各机型返回的□参差不□，□以像 iOS 那□一概比□。因此，

- □以 RAM 容量划分
- 将□□信息放到主数据中□行比□
- □□□□行基准□□工具

建□通□上述方式□行区分。

18.3 自制 RendererFeature

自制 RendererFeature 等□，也建□准□可以切□□制□置的 API。例如，

- 降低□□□理的格式、降低分辨率
- □少 Blit 的次数
- 替□□□易着色器

等措施。

第19章

UniTask 与取消□理的生命周期管理

稻田 健人 Inada Taketo

株式会社 CyberAgent SGE 核心技□本部

在使用 UniTask 的异步□理中，如果不恰当地管理□束条件和取消，可能会残留永□不会完成的□理和不必要的引用。

前作《Unity 性能□□宝典》介□了 MonoBehaviour 被□□后 UniTask 仍持□等待的案例。本□介□因 CancellationTokenSource 未被□□，□致不需要的□象持□被引用的案例。

□外，CancellationTokenSource 和 CreateLinkedTokenSource 并非 UniTask 特有的功能，而是□成 .NET 取消□理的 API。

本□涉及的代□中，using 声明（using var）是 C# 8 的□法，可在 Unity 2020.2 及以后版本中使用。□外，后文提到的 CreateLinkedTokenSource 的内部□□，参考的是 Unity Technologies 公开的面向 Unity 2022.3 的 Mono □□（unity-2022.3-mbe 分支），可能因 Unity 版本而异。

19.1 CancellationTokenSource 的 Dispose

CancellationToken 是用于向□理□□取消□求的□□型。CancellationToken 本身没有□□ IDisposable，因此无需□用 Dispose。

不□，通□ CancellationToken.Register 注册回□□，其返回□ CancellationToken

Registration 是 IDisposable [*1]。取消之前注册的回□由□出方的 CancellationToken Source 持有，因此如果在注册不再需要□不通□ Dispose 解除，回□所引用的□象有□无法被□放。

□一方面，CancellationTokenSource □□了 IDisposable。使用完□的 Cancellation TokenSource 由其所有者□用 Dispose。

▼清□ 19.1　Dispose CancellationTokenSource 的示例

```
private async UniTask LoadCancelableAsync()

{

    using var cts = new CancellationTokenSource();



    // 省略□出取消□求的□理

    await LoadAsync(cts.Token);

}
```

在清□ 19.1中，通□ using 声明管理 CancellationTokenSource。不□在异步□理正常□束□，在因取消或异常而□束□，退出方法□也会自□□用 cts.Dispose()。

Cancel 与 Dispose 的作用不同。Cancel 向□理□求取消，Dispose □解除并□放 CancellationTokenSource 持有的□□器和回□注册等。□□用 Dispose 不会向□理通知取消□求。

□外，即使忘□□用 Dispose，只要 CancellationTokenSource 及相关□象都不再被外部引用，也会成□ GC 的回收□象 [*2]。不□，向□生命周期的 CancellationTokenSource 注册回□等情况可能会残留引用，因此原□上□在使用完□□ Dispose。

[*1] https://learn.microsoft.com/ja-jp/dotnet/api/system.threading.cancellationtokenregistration.dispose（□□日期：2026年9月8日）

[*2] https://docs.unity3d.com/ja/2022.3/Manual/performance-managed-memory.html（□□日期：2026年9月8日）

19.2 CancellationTokenSource.CreateLinkedTokenSource

CancellationTokenSource.CreateLinkedTokenSource 会□建一个□□多个源令牌的新 CancellationTokenSource。只要任意一个源令牌被取消，所□建的 CancellationTokenSource 的 Token 就会□□已取消状□。

并不是源令牌之□互相被取消，取消是从源令牌□播到□接目□。

下面将 tokenA 和 tokenB 的□出方 CancellationTokenSource 分□□□ A、B。

在 Unity Technologies 于 GitHub 上公开的面向 Unity 2022.3 的 Mono □□ [*3] 中，CreateLinkedTokenSource 会向源令牌注册取消回□。□化与引用关系相关的部分，就如清□ 19.2所示。

▼清□ 19.2　CreateLinkedTokenSource 内部□□的要点

```
linkedCts.m_linkingRegistrations[0] =
    tokenA.InternalRegisterWithoutEC(
        s_LinkedTokenCancelDelegate,
        linkedCts);

linkedCts.m_linkingRegistrations[1] =
    tokenB.InternalRegisterWithoutEC(
        s_LinkedTokenCancelDelegate,
        linkedCts);
```

linkedCts 被注册在源令牌上的回□所引用，而 linkedCts 也持有□ A 和 B 的注册信息。

[*3] https://github.com/Unity-Technologies/mono/blob/unity-2022.3-mbe/mcs/class/referencesource/mscorlib/system/threading/CancellationTokenSource.cs（□□日期：2026年9月8日）

因此，如果 B 是在整个□用程序中使用的□生命周期 CancellationTokenSource，且□期不被取消，那么不 Dispose linkedCts 的□，可能会残留如下引用。

□生命周期的 B → 回□注册 → linkedCts → □ A 的注册信息 → 短生命周期的 A

□凭□种引用关系并不一定会□致内存泄漏。当所有□象都不再被外部引用□，它□会一并成□ GC 的回收□象。但是，在 B 存活且□接用的回□尚未解除期□，linkedCts 会一直被 B 引用，由此短生命周期的 A 也一直保持可□。

□外，如果注册到 linkedCts.Token 的回□引用了其他□象，□些□象也可能被保留。

Dispose linkedCts 后[*4]，内部□ A 和 B 的 CancellationTokenRegistration 会被□□，注册在源令牌上的回□也会被解除。

CreateLinkedTokenSource 的返回□，建□在使用□□接令牌的□理的作用域内，通□ using 声明等方式□行管理。

▼清□ 19.3　Dispose □接的 CancellationTokenSource 的示例

```
private async UniTask LoadAsync(
    CancellationToken tokenA,
    CancellationToken tokenB)
{
    using var linkedCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            tokenA,
            tokenB);

    await LoadAssetAsync(linkedCts.Token);
```

[*4] https://learn.microsoft.com/ja-jp/dotnet/api/system.threading.cancellationtokensource.dispose（□□日期：2026年9月8日）

```
}
```

在清□ 19.3中，LoadAssetAsync 因正常□束、取消或异常而□束之后，会□用 linkedCts.Dispose()，从而解除□源令牌的注册。

即使□接的 CancellationTokenSource 已□□于取消状□，也可能残留□其他源令牌的注册。CreateLinkedTokenSource 的返回□在不再需要□，□必□行 Dispose。

19.3 在□□中防止 IDisposable □象□漏 Dispose

至此，我□以 CancellationTokenSource □例考察了 IDisposable 的□□□漏。□些□策的前提是□象的所有者在适当的□机□用 Dispose。但是，随着□目□模□大，□靠代□□□来持□□□所有□建位置是很困□的。尤其是 CreateLinkedTokenSource 只需很短的写法就能□建，因此很容易忽略其返回□是 IDisposable。

要在□□中防止再次□生，除了个人的注意之外，建□□合使用以下做法：将需要□□的□象集中到易于使用的 API 上，用 Analyzer 及早□□□□，并根据需要在 CI 中□□新增的□□。不要把□□只集中在 Analyzer 上，而是从□□、静□分析和□□三方面共同支撑同一方□，效果更好。

□里介□的通□ using □□和通□ Analyzer □□，不□适用于 CancellationTokenSource，也可以□用于其他□□了 IDisposable 的□源。

在□□机制化的做法中，本□介□使用可以自□□□ C# 代□的 Roslyn Analyzer 的方法。

19.4 什么是 Roslyn Analyzer

Roslyn Analyzer 是在□□□或 IDE 中□ C# 代□□行静□分析，并在可能存在□□的位置□示警告或建□的机制。它可以基于□法、□型信息和□理路径等□□ bug 的迹象和□反□□□范的情况，如果提供 Code Fix，□可以□出修复候□。

Analyzer 将什么□□□□□，是以□断□□的形式定□的，每条□□都□有 CA2000 □□的用于□□的□断 ID。CA2000 是 .NET 提供的□有□□，后文的 UTK001 和 UTK002 是□目自行添加的□□方案。

在 Unity □目中，将 Analyzer 的 DLL 放置在 Assets 之下，并附加 RoslynAnalyzer □□来引入 [*5]。也可以利用 asmdef 限定适用的程序集。

19.5 将□有 Analyzer 作□第一道防□

首先□考□的是 .NET 代□□量□□中的 CA2000 [*6]。CA2000 会□□在本地□建的 IDisposable □象在未被□□的情况下离开作用域的路径。CancellationTokenSource 也是 IDisposable，因此可以作□□□ new CancellationTokenSource() □□□漏的第一道防□。

不□，根据所用 Analyzer 的版本和分析□置，□于 CreateLinkedTokenSource □□□建并返回□象的方法，其返回□以及方法之□所有□的□移能被追踪到什么程度并不相同。引入□，建□在小型□□□目中，□□ new CancellationTokenSource() 和 CreateLinkedTokenSource □者，确□□些示例会被□□到、□些不会。□有□□无法充分□□的模式，可以根据需要用后文的自定□□□来□充。

19.6 面向异步□理的自定□□□□□示例

如果□有 Analyzer 无法充分追踪 CreateLinkedTokenSource 的返回□，可以用□目□属的 Roslyn Analyzer 来□充。以下是□□自定□ Analyzer □的□□□□示例。□便于□明，使用 UTK001 和 UTK002 作□假□的□断 ID。

UTK001 □想□□通□ new CancellationTokenSource() 或 CancellationTokenSource.CreateLinkedTokenSource() □建的局部□量，是否受到 using □句、using 声明或 finally 子句中 Dispose 的保□。分析□象不□包括所有正常□束的路径，也包括通□ return 或异常退出方法的路径。

▼清□ 19.4　UTK001 □□的示例

```
private async UniTask LoadAsync(
    CancellationToken tokenA,
```

[*5] https://docs.unity3d.com/ja/2022.3/Manual/roslyn-analyzers.html（□□日期：2026年9月8日）

[*6] https://learn.microsoft.com/ja-jp/dotnet/fundamentals/code-analysis/quality-rules/ca2000（□□日期：2026年9月8日）

```
    CancellationToken tokenB)
{
    var linkedCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            tokenA,
            tokenB);

    await LoadAssetAsync(linkedCts.Token);
}
```

在清□ 19.4中，linkedCts 在未被 Dispose 的情况下退出了方法。□□□□想在 CreateLinkedTokenSource 的□用位置□出警告，并提示用 using 声明或 try-finally 保□□□。

UTK002 □想□□作□ CancellationToken.Register 返回□的 CancellationTokenRegistration 是否被直接□弃而未□□。不接收 Register 的返回□而直接□弃的代□，解除注册的□任不明确。如果回□只需在异步□理期□有效，建□用 using 声明来管理。若保存在字段中，□在所有者的□束□理中 Dispose。

▼清□ 19.5　管理 CancellationTokenRegistration 的示例

```
private async UniTask WaitAsync(CancellationToken token)
{
    using var registration = token.Register(OnCanceled);

    await WaitForSignalAsync(token);
}
```

在清□ 19.5中，WaitForSignalAsync □束□ registration 会被□□。□一方面，也存在使注册与 Token 的寿命一致的□□，例如永久注册到整个□用程序共享的 Token 上。

如果示例也一律警告，会增加，因此建 UTK002 先以信息用，并能排除目中繁出的安全模式。

保存在字段中的 CancellationTokenSource，其所有的判定比局部量更困。作第二段的，可以在向字段入新的 CancellationTokenSource 之前是否已旧 Dispose，以及是否在 MonoBehaviour 的 OnDestroy 等所有者的束理中 Dispose。不，由于可能涉及向其他象移所有，建与局部量的分开引入。

19.7 用 Code Fix 一修复方法

Roslyn Analyzer 可以提供在 IDE 中自修复所的 Code Fix。例如，于 UTK001 中所有在方法内完的情形，提供将 var 为 using var 的 Code Fix，就便于在内一可安全机械的模式的修复方法。于向字段或通返回移所有等无法唯一确定机的情形，建不提供自修复，只出促使确的断。

在代中，不要看 Analyzer 的果，要确能否从代中出建、Cancel 和 Dispose 的方。同需要注意，凭 Analyzer 没有警告，并不能明是安全的。

19.8 靠 Analyzer 无法的

静分析有其局限。将 CancellationTokenSource 存入集合，或其他象而移所有等情况下，有无法准确判断最的位置。避免而弱分析，漏又会增加。

于一域，可以通 Development Build 或境中的行来充。如果通目通用的建机制，或管理生命周期的型来建 CancellationTokenSource，并建数和数，就可以在景切或束确是否残留未的作用域。如果同建的堆跟踪，考到开，建在开境中用。

根据需要，建合使用以下方法：用 Profiler 确内存增的、行的未，以及 Roslyn Analyzer 在的警告。与其等生后再由人，不如在写代就能出修复方法，有助于防止再次生。

19.9 参考□料

Unity Manual《Roslyn 分析器和源生成器》

Microsoft Learn《CA2000：□失范□之前□放□象》

Unity Technologies《Mono CancellationTokenSource.cs（unity-2022.3-mbe 分支）》

Microsoft Learn『CancellationTokenSource.Dispose Method』

Microsoft Learn『CancellationTokenRegistration.Dispose Method』

Unity Manual《托管内存》

第20章

AssetBundle 的生命周期管理

稻田 健人 Inada Taketo

株式会社 CyberAgent SGE 核心技□本部

游□会在□行□加□□理、网格、音□等大量□源。如果由各□□者分□管理□些□源，就容易出□漏□放和不必要的□源常□。

因此，□于从 AssetBundle 或 Addressables 等□□加□的□源，原□上□将加□和□放的入口集中到系□基□□。在加□□确定□源的生命周期，并建立在其□束□由基□□□一□放的机制。

本□所□的“□放”，是指□放由加□基□□管理的引用和□放用句柄。□源是否真正从内存中卸□，取决于基□□的□□、引用□数和依□关系的状□。

此外，下文介□的分□和□□方法□□参考示例。□□□适合各□目游□□□的方法。

20.1 按生命周期区分管理□源

首先梳理游□中需要□源的□□段。例如可以分□如下。

表 20.1: □源生命周期的分□示例

生命周期	保持期□示例
Common	直到登出、切□□号、返回□□等重置公共数据□止
Scene	直到目□□景□束□止
View	直到目□画面或□□框关□□止
Custom	直到□斗、活□等自定□期□□束□止

重要的不是 Common、Scene、View □些名称，而是明确每个生命周期何□开始、何

□□束。

分□□粗，不需要的□源会□期留存；分□□□，□加□和□放的次数以及管理成本会增加。此外，如果生命周期不同的□源包含在同一个 AssetBundle 中，以 AssetBundle □□位卸□□，无法只□□其中一部分□源卸□。□源生命周期的分□，□与 AssetBundle 的粒度（前作《Unity 性能□□宝典》第 5 章“Tuning Practice - AssetBundle”）一并□□。

20.2 在加□□确定生命周期

完成生命周期分□后，在加□□源□确定其登□目□。

□□示例之一，是在加□函数的参数中指定生命周期。

▼清□ 20.1 指定生命周期□行加□的示例

```
var background =
    await AssetLoader.LoadAsync<GameObject>(
        address,
        AssetLifetime.Scene);
```

清□ 20.1 将加□的□源登□到 Scene 生命周期。□景□束□，基□□会□一□放已登□的□源。

□一个□□示例，是按生命周期拆分加□函数。

▼清□ 20.2 按生命周期拆分加□函数的示例

```
var commonIcon =
    await AssetLoader.LoadCommonAsync<Sprite>(
        commonAddress);

var stagePrefab =
    await AssetLoader.LoadSceneAsync<GameObject>(
        stageAddress);

var dialogPrefab =
    await AssetLoader.LoadViewAsync<GameObject>(
        dialogAddress,
        viewId);
```

像 LoadCommonAsync、LoadSceneAsync、LoadViewAsync 这样拆分函数后，可以从调用位置判断生命周期，也更容易避免登记目标的指定错误。另一方面，通过参数指定的方法便于添加自定义的生命周期。

无论采用哪种方法，重要的是在加载资源必定确定生命周期，并能在其结束时由基础设施释放。

这样异步加载完成前生命周期已经结束的情况，需要从加载开始就跟踪资源处理，并在中途结束时取消加载，或能够释放完成后的结果。此外，如果同一个资源被多个生命周期使用，则管理其引用计数，并在最后一次使用结束时释放。

20.3 将生命周期绑定到 GameObject

这一种管理方法，是把 GameObject 成为生命周期的所有者。

在画面或窗口框的根 GameObject 上添加所用的 Component，并登记已加载资源的释放用句柄。GameObject 被销毁时，在 OnDestroy 中释放已登记的句柄。

▼清单 20.3　将生命周期绑定到 GameObject 的示例

```
public sealed class AssetLifetimeScope : MonoBehaviour
{
    private readonly List<IDisposable> _handles =
        new List<IDisposable>();

    private bool _released;

    public void Add(IDisposable handle)
    {
        if (_released)
        {
            handle.Dispose();
            return;
        }

        _handles.Add(handle);
    }

    private void OnDestroy()
    {
        _released = true;
```

```
        for (var i = _handles.Count - 1; i >= 0; i--)
        {
            _handles[i].Dispose();
        }

        _handles.Clear();
    }
}
```

清□ 20.3 中，即使 GameObject □□后异步加□才完成并□用了 Add，也会立即□放新添加的句柄。

但是，□□□ GameObject 并不会自□□放□□加□的□源。需要由□用的 Component □用与加□基□□□□的□放□理。

管理用的 GameObject 不要按□源逐个□建，而是每个生命周期□位（如一个画面或一个□□框）□一□建一个。□种方法在 GameObject 与□源的寿命一致□有效，但□于跨多个□景使用的□源等，可能并不适用。□根据需要与按生命周期划分的管理表配合使用。

20.4 确□是否存在漏□放

使用 Memory Profiler 在画面或□景切□前后□取快照，比□已□束生命周期的□源是否仍然残留。多次重复同一切□，确□□源和加□句柄是否持□增加，也是有效的做法。

□量□在必要的□放□理完成后，□相同基准状□□行比□，最□在真机的 Player □建上确□。但是，也存在被其他功能引用的□源，以及有意作□□存保留的□源，因此不要□凭□源残留就判断□漏□放，而□确□到□□的引用来源。

□源管理如果在开□后期更改，会影□大量加□□理。□在开□早期梳理所需的生命周期，并□□好相□机制，使□□者无需关注各自的□放□机，也能□用适合□目的生命周期。

第21章

使用 GraphicsStateCollection □行着色器□□

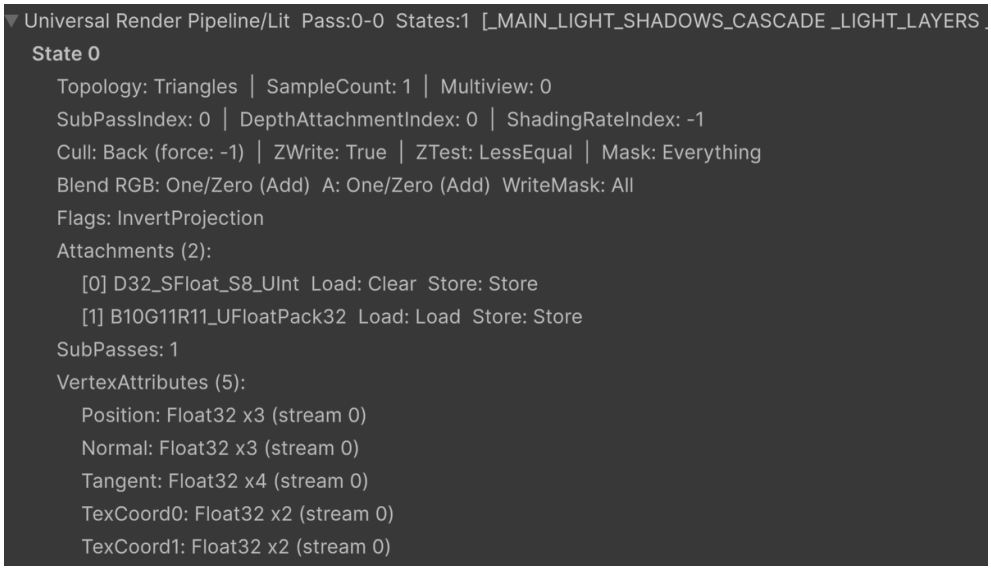
天野 択郎 Amano Takuro

株式会社 CyberAgent SGE 核心技□本部

着色器首次用于□制□，会在□行□□行□□□理，可能引□称□着色器□□卡□的□率下降。本章以移□端□境（Metal・Vulkan）□□象，介□使用 GraphicsStateCollection □行着色器□□，内容从基本用法到引入后仍残留卡□的□例及其□策。本章内容以 Unity 6.3 LTS（6000.3）□□象。

21.1 GraphicsStateCollection 概述

GraphicsStateCollection（GSC）是 Unity 6 中可用的 API（出典ID: gsc_docs）。它□□游□□程中使用的着色器□体与□形状□（□制□的渲染□置）的□合，并□一□行□□□，即□□。它被定位□面向 Metal・Vulkan・D3D12 等□代□形 API 的 Shader VariantCollection 的替代方案。□外，在 Unity 6000.3 中，它是属于 `UnityEngine.Experimental.Rendering` 命名空□的 Experimental API，□格可能在未来版本中□更。



▲图 21.1　GSC 中图图的信息示例

图图是通图脚本图取 GSC 中图图的信息并图示在 EditorWindow 中的示例。图取图图内容的方法在本章末尾（清图 21.11）介图。

着色器本身无法直接在 GPU 上图行，需要图图图 GPU 的机器指令。但是机器指令因 GPU 机型和图图而异，无法以面向所有图图图图图的状图图布。因此，着色器图图以下流程才在 GPU 上图行。

1. 开图者用 HLSL 等着色器图言图写着色器（图源）
2. 图建图，Unity 将着色器图图图各图形 API 的中图表示，并存入图用或 AssetBundle。在 Vulkan 中是 SPIR-V，在 Metal 中是 Metal Shading Language（MSL）属于图种中图表示（出典ID: gsc_spirv_msl）
3. 图用图行图，着色器首次用于图制图，从存图的中图表示生成着色器程序
4. 接着，将着色器程序与渲染图置图合，由 GPU 图图图图图 GPU 的机器指令

着色器图图的基本思路，是在首次图制之前集中完成步图 3、4 的图行图图图。

图外，一个着色器会按每个通道（图制工序），以及光照影图、图的有无等条件（着色器关图字）的每种图合，生成多个可图行程序。图一生成图位称图着色器图体。

21.1.1 Pipeline State Object

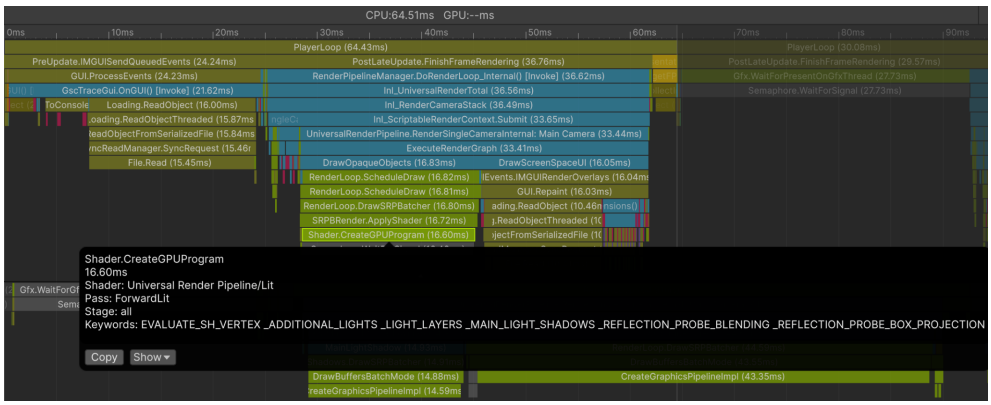
在现代图形 API 中，绘制被预热的图位称为 Pipeline State Object（PSO）。PSO 是将着色器程体与渲染配置（顶点布局、颜色格式和 MSAA 等渲染通道附件、混合状态、深度模板、模板状态）作为一体，由 GPU 驱动层面向 GPU 的机器指令的对象。生成的 PSO 会存在显存上，渲染配置相同时会被复用。也就是说，即使是相同的着色器程体，渲染配置不同也会被视为不同的 PSO。

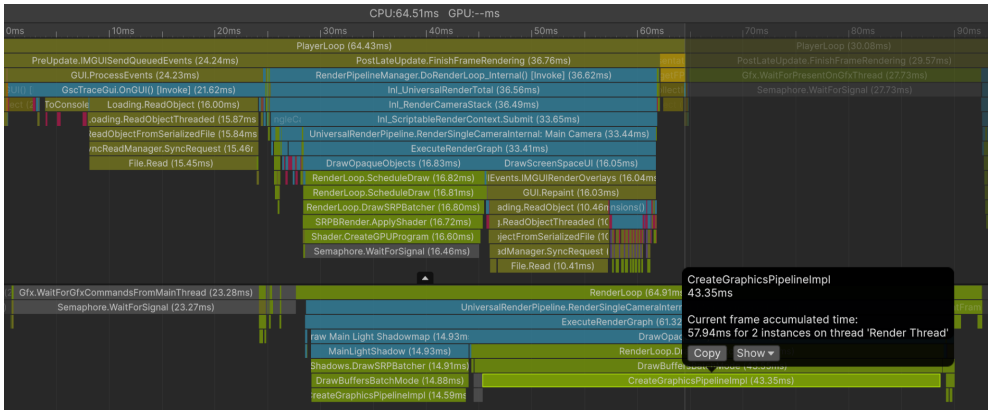
21.1.2 着色器预热卡顿的根本原因

卡顿的原因是在预行预热行的预热段预热处理。GPU 首次绘制着色器程体时，会发生 Main Thread 上的着色器程序生成（步骤 3），以及 Render Thread 上由 GPU 驱动层行的 PSO 生成（步骤 4，驱动层机器指令）。

在 Unity Profiler 中，前者可在 Main Thread 的 `Shader.CreateGPUProgram`，后者可在 Render Thread 的 `GpuProgramMetal.GetCachedPipeline`（iOS）或 `CreateGraphicsPipelineImpl`（Android）。尤其在 Metal（iOS）· Vulkan（Android）中，PSO 生成的开销很大，是首次绘制卡顿的主因。使用 GSC 的预热，是在游戏预热后等任意空机提前完成预热 PSO 生成的手段。

预热的 ShaderVariantCollection 可以管理“预热哪些着色器程体”，但不包含渲染预置。因此 ShaderVariantCollection 的预热限于步骤 3 的着色器程序生成，预热绘制时 GPU 驱动会重新生成 PSO，所以卡顿依然存在。





▲□ 21.2　Main Thread 与 Render Thread 上□□□生的示例

□□是在 Profiler 的 Timeline □□中□示 Vulkan（Android）上□生□□的□的示例。上方是□中 Main Thread 的 `Shader.CreateGPUProgram` 的状□，可以确□目□着色器名、通道名和关□字。下方是□中 Render Thread 的 `CreateGraphicsPipelineImpl` 的状□，□示例中一次 PSO 生成耗□数十毫秒，可□它是卡□的主因。

21.2　使用 GSC 的□□流程

使用 GSC 的□□分□“收集渲染□置（跟踪）”和“□□（□□□）”□个□段。

21.2.1　收集渲染□置

跟踪的基本形式□ 清□ 21.1。跟踪期□□制的□象的渲染□置，会以 `GraphicsStateCollection.GraphicsState` 的形式□□到 GSC 中。代□清□中□□在一个方法里，但□□上 `BeginTrace()` 和 `EndTrace()` 在不同□机□用，跨多□收集□制内容。

▼清□ 21.1　渲染□置的跟踪

```
private GraphicsStateCollection _gsc;

void Trace() {
    // 开始收集
    _gsc = new GraphicsStateCollection();
    _gsc.BeginTrace();

    // 在此跨多□□制□□□象
```

```
    // 模拟的渲染设置被记录为 GraphicsState

    // 结束收集并保存
    _gsc.EndTrace();
    _gsc.SaveToFile(path);
}
```

生成的 GraphicsStateCollection.GraphicsState 因图形 API 而异。它在记录后的用的图形上执行跟踪。

21.2.2 预热（预编译）

加载收集的 GSC 并执行预热（清单 21.2）。下示例中，将载入目的 GSC 作为 Addressables 的资源加载。

▼清单 21.2 使用 WarmUpProgressively 的分散预热

```
async Task WarmUpAsync() {
    // 在加载包含着色器的资源后执行
    _gsc = await Addressables.LoadAssetAsync<GraphicsStateCollection>(address).Task;

    // 分阶段 （基于WarmUp 。每帧最多Jobs n 个）
    StartCoroutine(WarmUpCoroutine());
}

IEnumerator WarmUpCoroutine() {
    JobHandle handle = default;
    int requestedCount = 0;
    // 出于安全 isWarmedUp 可能不为真，因此以请求数量判断结束
    while (requestedCount < _gsc.totalGraphicsStateCount) {
        handle = _gsc.WarmUpProgressively(4, handle);
        handle.Complete();
        requestedCount += 4;
        yield return null;
    }
}
```

预热的要点如下。

- 使用 WarmUp(dependency) 也可以执行预热，但会一次性完成全部预热，因此会产生尖峰。应用 WarmUpProgressively(count, dependency) 指定每次的预热数量，将其分散到多帧
- 在着色器资源加载之后执行。如果不存在着色器实例，则预热的变体会被跳过
- 完成判断可通过 isWarmedUp 和 completedWarmupCount == totalGraphicsSt

ateCount 确□

- 但是，□生□□□□□ isWarmedUp 可能不□真（已在 Unity 6000.3 中确□）。因此代□清□中通□比□□求□数与 totalGraphicsStateCount 来判断□束，以避免无限循□

21.3 收集□景的□□示例

也可以在□用□□□□用 BeginTrace()，通□手□游玩来收集渲染□置。但根据□用□模，每次版本更新都手□覆盖全部内容是困□的。因此□里介□在收集□用□景中按□序□制□源，以机械化方式□行□景来收集的方法。□□示例使用 Addressables 和 UniTask（出典ID: gsc_unitask）。即使不使用它□，将□源的异步加□和等待 1 □□制替□□自己□目的机制，也能□□同□的流程。整体流程如下。

1. 下□所需的□源（□用 Built-in □源即可完成□不需要）
2. 从 Addressables 的内容目□（IResourceLocator）□取收集所需□源的 Address
3. 切□ Quality Settings（如果着色器不因 Quality Settings 而□化，□无需切□）
4. 开始收集（BeginTrace()）
5. □加加□□景，依次加□并□制收集□象□源（模型□制・ParticleSystem 播放・Timeline 播放）
6. □束收集（EndTrace()）
7. 从 GSC 中□除不需要的着色器
8. 保存 GSC（SaveToFile(path)）
9. 卸□□景，一□切□ Quality Settings 一□重复步□ 3 至 8

从内容目□中□取影□渲染□置的□源（□景）、角色和道具等模型□源、特效□源、Timeline □源，一□切□渲染□置一□依次□制，从而全面收集。之所以按每个 Quality Settings 收集，是因□ Quality Level 不同，MSAA 和 HDR 等影□ PSO 的渲染□置也会改□。

□理的骨架□ 清□ 21.3。按每个 Quality Settings 重复跟踪，并附上 Quality Level 名称保存□不同文件。□行□□□与当前 Quality Level □□的文件加□并□□。

▼清□ 21.3　收集□景的骨架

```
private GraphicsStateCollection _gsc = new GraphicsStateCollection();

private async UniTaskVoid StartAsync() {
```

```
    // 收取内容目录并下载所需资源
    // 通过 Prefix 查先获取各个 Address 列表
    await AddressableSetupAsync();

    var qualitySettingNames = QualitySettings.names;
    for (var i = 0; i < qualitySettingNames.Length; i++) {
        QualitySettings.SetQualityLevel(i);
        await TraceAsync();
    }
}

private async UniTask TraceAsync() {
    // 开始 GSC 收集
    _gsc.ClearVariants();
    _gsc.BeginTrace();

    // 执行收集场景
    await ExecuteScenarioAsync();

    // 结束 GSC 收集
    _gsc.EndTrace();

    // 从 GSC 中移除收集对象之外的着色器
    RemoveUnnecessaryVariants();

    var qualityLevelName = QualitySettings.names[QualitySettings.GetQualityLevel()];
    _gsc.SaveToFile(_gscSavePathFormat + $"_{qualityLevelName}");
}
```

在场景主体中，按每个场景依次制模型、特效和 Timeline。清单 21.4 是批量的、分批创建并复制数个角色模型的部分。等待 1 帧复制后，新的渲染设置会被写到 GSC 中。

▼清单 21.4　模型资源的依次复制（节选）

```
// 收集角色模型
for (var i = 0; i < _charaAddressArray.Length; i += _characterNum) {
    // 每次创建数个角色模型
    var batchCount = Mathf.Min(_characterNum, _charaAddressArray.Length - i);
    for (var j = 0; j < batchCount; j++) {
        var charaHandle = Addressables.LoadAssetAsync<GameObject>(_charaAddressArray[i
    + j]);
        var charaPrefab = await charaHandle.ToUniTask();
        // 均匀布置在相机视野内的位置
        var pos = new Vector3((j - batchCount * 0.5f) * 2f, 0f, 5f);
        _charaObjectArray[j] = Object.Instantiate(charaPrefab, pos, Quaternion.identity
    );
        Addressables.Release(charaHandle);
```

```
    }

    // 等待 1 □□制
    await UniTask.NextFrame();

    for (var j = 0; j < batchCount; j++) {
        DestroyImmediate(_charaObjectArray[j]);
        _charaObjectArray[j] = null;
    }
}
```

ParticleSystem 同□每次分批□建数个，但不同之□在于，先通□ Simulate() 同步重□播放中途的状□（□示例中□ 0.5 秒后）再□行□制（清□ 21.5）。由于无需等待□□□□的播放，可以□短收集□□。

▼清□ 21.5　ParticleSystem 的收集（□□）

```
// 收集 ParticleSystem
for (var i = 0; i < _particlesAddressArray.Length; i += _effectNum) {
    var batchCount = Mathf.Min(_effectNum, _particlesAddressArray.Length - i);
    for (var j = 0; j < batchCount; j++) {
        var particleHandle = Addressables.LoadAssetAsync<GameObject>(
    _particlesAddressArray[i + j]);
        var particlePrefab = await particleHandle.ToUniTask();
        var pos = new Vector3((j - batchCount * 0.5f) * 3f, 0f, 5f);
        _particleObjectArray[j] = Object.Instantiate(particlePrefab, pos, Quaternion.
    identity);
        Addressables.Release(particleHandle);

        // 以同步□理重□ _effectSimulateTime 秒后的状□（无需 ）Play
        var ps = _particleObjectArray[j].GetComponentInChildren<ParticleSystem>();
        if (ps != null) {
            ps.Simulate(_effectSimulateTime, withChildren: true, restart: true);
        }
    }

    // 等待 1 □□制
    await UniTask.NextFrame();

    for (var j = 0; j < batchCount; j++) {
        DestroyImmediate(_particleObjectArray[j]);
        _particleObjectArray[j] = null;
    }
}
```

使用 Timeline 的演出，通□ PlayableDirector □□播放来收集（清□ 21.6）。由于

要等待播放完成，建议将目标 Timeline 资源限定为收集所需的内容。

▼清单 21.6　Timeline 的收集（节选）

```
// 收集 Timeline
foreach (var timelineAddress in _timelineAddressArray) {
    var timelineHandle = Addressables.LoadAssetAsync<GameObject>(timelineAddress);
    var timelinePrefab = await timelineHandle.ToUniTask();
    var timelineObj = Object.Instantiate(timelinePrefab, Vector3.zero, Quaternion.
    identity);
    Addressables.Release(timelineHandle);
    var director = timelineObj.GetComponent<PlayableDirector>();
    if (director != null) {
        director.Play();
        await UniTask.Delay((int)(director.duration * 1000));
    }
    DestroyImmediate(timelineObj);
}
```

收集结束后，只保留希望作为变体对象的着色器（清单 21.7）。通过允许列表（_allowShaderNames）指定要保留的着色器，并从 GSC 中移除对象之外的着色器以及引用丢失（null）的对象。

▼清单 21.7　移除收集对象之外的变体

```
private void RemoveUnnecessaryVariants() {
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    _gsc.GetVariants(variants);

    foreach (var variant in variants) {
        if (variant.shader == null) {
            _gsc.RemoveVariant(null, variant.passId, variant.keywords);
            continue;
        }

        if (_allowShaderNames.Contains(variant.shader.name)) {
            continue;
        }

        // 从 GSC 中移除不需要的着色器
        _gsc.RemoveVariant(variant.shader, variant.passId, variant.keywords);
    }
}
```

场景和资源的管理方式因项目而异，因此请根据自己的项目整合场景构成和收集对象。

21.4 卡□残留的□例与□策

即使引入了基于 GSC 的□□，卡□仍可能残留。□里介□其□例和□策。

21.4.1 Shader Stripping □致的□体缺失

□是□□□日志中出□ `variant not found` □□，且□行□也未□□到□□体的□□（Profiler 的 `Shader.CreateGPUProgram` 等）的情形。

原因是 GSC 中包含的着色器□体未包含在□用或 AssetBundle 中。根据 **Project Settings > Graphics > Shader Stripping** 的□置，□建□未使用的□体会被排除。当着色器或□源的□更□致所需□体□生□化□，容易出□□一□□。

□确□ Shader Stripping 的□置，并在□行了影□□体的□更后重新收集 GSC。

□外，当 GSC 中□□了□体但 `GraphicsState` □ 0 件□，也会□生同□的 `variant not found` □□。□种情况下□行□会□生□□体的□□，因此可以根据□行□是否□生□□来区分。

21.4.2 元数据不一致□致的 mismatch 警告

□是□行□□□控制台出□以下警告的情形。

```
WarmUp may be inaccurate due to GraphicsDeviceType mismatch.
WarmUp may be inaccurate due to RuntimePlatform mismatch.
WarmUp may be inaccurate due to Quality Level mismatch.
```

原因是 GSC 文件中保存的 `graphicsDeviceType` / `runtimePlatform` / `qualityLevelName` 元数据与□行□境不一致。主要□生情形有以下□种。

- □用了在不同平台收集的 GSC（例：将在 iOS 真机收集的 GSC 用于 Android □建）
- 通□ Editor 脚本□□并保存 GSC □，元数据被 Editor 的 Platform □置覆盖

□策因□生情形而异。在不同平台收集的 GSC，即使改写元数据，其内容中的 `GraphicsState` 也与□行□境不一致，因此需在□□平台上重新收集。被 Editor 的 Platform □置覆盖□，将元数据修正□真机的□。□外，`qualityLevelName` 的不一致，□在影□

PSO 的渲染□置（MSAA 和 HDR 等）和着色器不随 Quality Level □化□才可以忽略。□种情况下，可以像 清□ 21.8 那□在□□之前改写，以避免警告。

▼清□ 21.8　通□改写 qualityLevelName 避免警告

```
// 加□ GSC
var gsc = LoadGsc();
// 将 qualityLevelName 改□与□行□境一致
gsc.qualityLevelName = QualitySettings.names[QualitySettings.GetQualityLevel()];
// □行□□
```

21.4.3　iOS（Metal）上 AssetBundle 重复打包□致的双重□例

□是□在 iOS（Metal）上□生的情形。□□□日志中没有□□，Profiler 的 Main Thread 上也没有□生 `Shader.CreateGPUProgram`，但 Render Thread 上□生了 `GpuProgramMetal.GetCachedPipeline`。

原因是 AssetBundle 的重复打包□致着色器被双重□例化。当同一着色器被重复打包到多个 AssetBundle 中□，Unity 会在内存中□建各自独立的着色器□例。□□只会□ GSC 所引用的着色器□例□建 PSO，因此即使是同一着色器，成□□一个□例的着色器也不会被□□。□外，Vulkan 中不会□生□一□□。□被□□是因□ Metal 与 Vulkan 的 PSO □存管理方式不同。

□生□□□的□成示例如下。同一着色器 X 因依□关系被打包□□个包中。

```
- □源
  - 着色器 X
  - 材□ （使用着色器A ） X
  - 材□ （使用着色器B ） X
  - GraphicsStateCollection

- AssetBundle □成
  - 包 A
    - GraphicsStateCollection
    - 材□ （使用着色器A ） X
    - 着色器 X ← 因依□关系而打包
  - 包 B
    - 材□ （使用着色器B ） X
    - 着色器 X ← 因依□关系而打包（着色器
 X 未分配地址）
```

在□种□成下加□包 A 和包 B，会按以下流程在□制材□ B □□生卡□。

```
- □行□的流程
  1. 加□包 A 和包 B
     - 包 A 的着色器 (XinstanceId) =100
     - 包 B 的着色器 (XinstanceId) =200
  2. □行 GraphicsStateCollection 的□□
     - 使用 GSC 所引用的着色器 (XinstanceId) □行□□=100
  3. □制材□ A
     - 着色器 (XinstanceId) 的=100 PSO □存已□建，因此没有卡□
  4. □制材□ B
     - 着色器 (XinstanceId) 没有=200 PSO □存，
       Render Thread 上□行 PSO □建而□生卡□
```

□策是将着色器□□到□一包（着色器□用□）中，消除向内容包的□式打包。□以□□到□一包□，□每个包准□ GSC。重复的着色器可以通□ 清□ 21.9 □□的代□确□。使用 Addressables □，也可以在 **Addressables Report** 窗口中确□着色器的重复。

▼清□ 21.9　着色器□例重复的确□

```
using System.Linq;

// 在 AssetBundle 加□后□行，确□□例是否重复
var all = Resources.FindObjectsOfTypeAll<Shader>();
var duplicates = all.GroupBy(s => s.name).Where(g => g.Count() > 1);
foreach (var g in duplicates)
    foreach (var s in g)
        Debug.Log($"Duplicate: {s.name} | InstanceID: {s.GetInstanceID()}");
```

21.4.4　iOS（Metal）上使用 SubPass 的着色器□□失□

到此□止的都是由□目□置或包□□的失□引起的□□。接下来介□因 GSC □格而□生的□□。

将一个渲染通道拆分□多个□制步□，使后□步□无需将前一步□的□制□果写回主内存即可引用的机制，称□ SubPass。在移□端 GPU 上，□一系列步□可以在芯片内的高速内存区域（□□内存）中完成，从而□省内存□□。SubPass 在 Metal · Vulkan 中以 Native Render Pass 的形式□行。□在 iOS（Metal）上，存在已□行□□且日志中也没有□□，但游□□程中仍□生 PSO □□的情形。原因是 iOS（Metal）的□□□理□在□ SubPass（Native Render Pass）内□制的着色器未能按□期工作。无□是否存在□取前一 SubPass □出的 input attachments，都会□生。□外，□□的内部□理属于非公开 API，因此□里□介□□生条件。

受影□的条件如下。

- 目□□ iOS / Metal（Apple A/M 芯片的真机）
- URP 渲染□理基□（RenderGraph）的□置 `RenderGraph.nativeRenderPassesEnabled` 已□用（Unity 6000.5 及以后□□置被□除，始□□用）
- 使用了在 SubPass（Native Render Pass）内□制的着色器

无□是□写入的 SubPass（URP Deferred 的 GBuffer 通道等），□是□有 input attachments 的 SubPass（同□ Deferred 的 Lighting 通道等），都会同□□生。作□受影□的 SubPass 的□写示例，清□ 21.10 展示了□置 input attachments 的情形。

▼清□ 21.10　□ SubPass □置 input attachments 的情形

```
// 通□ ScriptableRenderContext.BeginRenderPass 开始的
// 渲染通道内□置 SubPass 的示例
var inputAttachments = new NativeArray<int>(new[] { 0, 1 }, Allocator.Temp);
context.BeginSubPass(
    colors,
    inputAttachments,   // □定前一 SubPass 的□出
                        // 在 Metal 中会被□□□ framebuffer , fetch
                        // 但不会反映到 GSC □□□的 PSO 生成中
    false
);
```

在 Unity 6000.3 中□以从根本上解决，只能重新□□ SubPass 的使用，或通□□渲染来避免游玩□程中的卡□。□渲染是在加□画面期□等□机，□使用目□着色器的□象□□□制一次，从而建立 PSO □存的方法。可以理解□在游□内□行与本章收集□景相同的依次□制，以代替□□。

21.4.5　卡□残留□的□断要点

如前所述，使用 GSC 的着色器□□与□目的□源管理□□（Addressables 的□成、着色器关□字定□、包□□）密切相关，因此引入后多数情况下需要□□。□确□□□□的日志，并使用 Profiler 反复□量，□□是否存在□□□漏。尤其是掌握以下几点，可以加快□断。

- □□□的日志中是否出□□□□□或 `variant not found`
- 在 Profiler 中，Main Thread 的 `Shader.CreateGPUProgram`、Render Thread 的 `GpuProgramMetal.GetCachedPipeline`（iOS）/ `CreateGraphicsPipelineImpl`（Android）是否□生

- Shader Stripping 的□置是否合适
- AssetBundle 的打包是否符合□期

确□ GSC 中□□的□体□，可以使用下一□介□的□体列表□出（清□ 21.11）。

21.5 GSC 文件的操作示例

最后介□□个 GSC 文件的操作示例。它□都假定从 Editor 脚本□行，可用于确□收集到的 GSC 的内容，以及整合多次收集的□果。

21.5.1 □体列表与 State 数的确□

清□ 21.11 是加□ GSC 文件并□出元信息和所□□□体列表的示例。

▼清□ 21.11 □体列表与 State 数的确□

```
void PrintVariantInfo() {
    // 加□ GSC
    var gsc = new GraphicsStateCollection();
    if (!gsc.LoadFromFile(path))
        return;

    // □取□体
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    gsc.GetVariants(variants);

    // □出 GSC 的元信息
    Debug.Log($"[GSC] Variants={variants.Count} TotalStates={gsc.
    totalGraphicsStateCount} "
            + $"Platform={gsc.runtimePlatform} Device={gsc.graphicsDeviceType} Quality
    ={gsc.qualityLevelName}");

    // □出□体的□□信息
    foreach (var v in variants) {
        int stateCount = gsc.GetGraphicsStateCountForVariant(v.shader, v.passId, v.
    keywords);
        string kws = v.keywords.Length > 0
            ? string.Join(" ", v.keywords.Select(k => k.name))
            : "<no keywords>";
        Debug.Log($"  {v.shader?.name} | states={stateCount} | {kws}");
    }
}
```

21.5.2 GSC 的合并

清□ 21.12 是合并□个 GSC 的示例。可以将分多次收集的□果整合□一个。

▼清□ 21.12 □个 GSC 的合并

```
void MergeGsc() {
    // 加□□个 GSC
    var src = new GraphicsStateCollection();
    var dst = new GraphicsStateCollection();
    if (!src.LoadFromFile(srcPath) || !dst.LoadFromFile(dstPath))
        return;

    // □取□体
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    var states = new List<GraphicsStateCollection.GraphicsState>();
    src.GetVariants(variants);

    // 从 src 的 GSC 添加到 dst 的 GSC
    foreach (var v in variants) {
        // □取□体中包含的 GraphicsState
        src.GetGraphicsStatesForVariant(v.shader, v.passId, v.keywords, states);
        foreach (var s in states) {
            // 添加 State
            dst.AddGraphicsStateForVariant(v.shader, v.passId, v.keywords, s);
        }
    }

    dst.SaveToFile(dstPath);
}
```

根据 AssetBundle 的□成，可能需要将收集到的 GraphicsState 移□、整合到□一个 GSC 中。□□双重□例的□策中介□的□每个包准□ GSC 的情形就是典型例子。此□□□合使用本□的操作示例。